PhantomSDR-Plus

Complete Documentation

18 documents · generated 2026-11-10

Contents

1. README 8

- Key Features 8
- Supported Hardware 10
- Quick Start 11
- Performance Benchmarks 12
- Usage 12
- Configuration 12
- Waterfall Floor — waterfall.sh 13
- Customization 14
- Documentation 16
- Online WebSDR List 16
- Troubleshooting 16
- Contributing 17
- License 17
- Authors & Credits 17
- Links 17
- Support 17
  - Performance Tips 17

2. Installation Guide 19

- Table of Contents 19
- System Requirements 19
- Pre-Installation Preparation 20
- Installing Dependencies 21
- Installing Node.js and npm 22
- Installing OpenCL (Optional but Recommended) 23
- Installing PhantomSDR-Plus 25
- Installing Opus Audio Codec 31
- Autorun Spot Reporter (FT8/FT4/WSPR) 32
- KiwiSDR Client Emulation (optional) 34
- Configuration 34
- SDR Device-Specific Setup 37
- Testing and Verification 40
- Setting Up Autostart 41
- CPU Thermal Protection 43
- Troubleshooting 45
- Performance Optimization 49
- Updating PhantomSDR-Plus 50
- Backup and Restore 52
- Security Considerations 53
- Getting Help 53
- Appendix A: Complete Dependency List 54
- Appendix B: Configuration Examples 54

3. Admin Panel Setup 56

- Overview 56
- What the admin panel gives you 56

Requirements 57
Step 1 — Files 57
Step 2 — Run the setup script 58
Step 3 — Start / Stop / Restart 59
Step 4 — Open firewall port 61
Step 5 — Kick Users feature 61
Configuration — admin_config.json 62
The sdr_host setting 63
proxy.py — what it is and when you need it 63
Changing ports after setup 63
Useful manual commands 64
Log files 64
Access summary 66
Chat message deletion 66
Waterfall broadcast messages 67
System Graphs 68
Thermal Guard 69
Spot Reporting (Autorun FT8/FT4/JS8/WSPR) 71
System Health 76
Visitor statistics 76
Alerts (email / Telegram) 77
Receivers 77
Settings — ports, start/stop and HTTPS 79
Backup & restore of the station settings 80
On a phone or tablet 80
Themes 80
Troubleshooting 83

4. Thermal Guard 86

Contents 86
1. What it does 86
2. Quick start 87
3. How it decides 87
4. The four modes 88
5. Choosing a mode 90
6. Configuration reference 91
7. Running it without the admin panel 91
8. Enabling the throttle stage without root 92
9. Unattended operation 93
10. Testing it before you trust it 94
11. Reading the log 94
12. Lockout and the rate limit 95
13. Troubleshooting 96
14. Turning it off or removing it 96
See also 96

5. Connection Limits 97

Contents 97
1. What you are actually defending against 97
2. The three layers 98
3. Per-IP limits 98

4. Idle connections 99
5. The kernel guard 100
6. Configuration reference 101
7. What a refused listener sees 102
8. Reading the logs and counters 103
9. Choosing your numbers 103
10. Traps worth knowing about 104
11. Turning it all off 104
6. Several Receivers 105
Contents 105
1. How it works 106
2. Two ways to publish the receivers 106
3. Adding a second receiver 107
4. receivers.toml 108
5. Starting and stopping 109
6. What listeners see 109
7. The S-meter above 30 MHz 110
8. Security 111
9. How many receivers fit 111
10. Traps worth knowing about 111
7. Secure Access (HTTPS) 112
What you need 112
Turning it on 112
How it works 113
What changes for listeners 113
Receive Diversity over https 114
The local network only — --lan 114
A web server is already running on this computer 114
Troubleshooting 115
8. Statistics Server 116
Overview 116
Prerequisites 116
Installation Methods 116
Configure Your PhantomSDR Application 119
Testing the Installation 121
Service Management 122
Manual Installation (Option 2) 122
Changing the Port After Installation 124
Troubleshooting 124
What You'll See 127
Temperature Detection 127
Features Summary 128
Security Notes 128
Support & Resources 128
9. Decoders 131
Table of Contents 131
1. How to Start a Decoder 131
2. FT8 136

3. FT4-FT2 137
4. JS8 138
5. CW — Morse Code 139
6. QRSS Grabber 141
7. WSPR 144
8. HF FAX / WEFAX 145
9. NAVTEX 146
10. FLDIGI — ALE 2G, RTTY, PSK31, Olivia, FST4W, MFSK, MSK144, DominoEX, THOR, THROB, Hellschreiber, MT63, Packet and APRS 148
11. SSTV 159
12. SERVICES 161
13. DRM 164
14. General Tips 165

10. RADE README 167

Installing it — the short way 167
Table of Contents 167
How It Works 168
Architecture Overview 169
RADEL vs RADEU 169
RADE V2 (beta) — RADE2 169
Prerequisites 171
Installing by hand 171
The Patched Files 171
Sidecar control 172
Port 8074 173
Using RADE in the Browser 173
Verification and Debugging 174
Environment Variables 176
Files Changed 177
Signal Flow Summary 177
Troubleshooting 178
Measuring Concurrency on Your Own Hardware 180
Updating RADE v1 182

11. RADE General Install (Linux) 185

Step 1 — System packages 185
Step 2 — Python packages 186
Step 3 — Clone and build the radae repository 187
Step 4 — Verify the decode pipeline 188
Step 5 — Build the PhantomSDR-Plus frontend 189
Step 6 — Sidecar control 189
Step 7 — Open port 8074 189
Step 8 — Start the server 190
Step 9 — Verify RADE 191
Step 10 — Using RADE in the browser 191
CPU note 192
Optional — RADE V2 (beta) with rade_c 192
Updating RADE in future 192

12. Editing Variants 194

1. What the Variants Are 194
2. Which File Does What 195
3. Making a Change That Affects Only Some Variants 197
4. Where the Variant Switches Are 197
5. Other Knobs Worth Knowing 198
6. After You Edit — Rebuilding 198
7. How to Check Your Edit Before Trusting It 199

13. User Guide 201

Table of Contents 201
Introduction 201
Getting Started 202
Interface Overview 203
Basic Operations 205
Advanced Features 209
Demodulation Modes 215
Digital Decoders 219
Keyboard Shortcuts 223
Bookmarks 224
Mobile Usage 225
Tips and Best Practices 226
Troubleshooting 227
Frequently Asked Questions 229
Resources 229
Appendix: Common Frequencies 230
Glossary 231

14. KiwiSDR Client Emulation 232

Table of Contents 232
1. What a Kiwi client gets 232
2. Installing the bridge 233
3. Switching it on 233
4. Configuration reference 234
5. Connecting a client 234
6. Audio level 234
7. The S-meter 235
8. Waterfall and spectrum 236
9. If something looks wrong 237
10. How far this has been verified 237

15. Receive Diversity 239

What it does, and what it does not 239
Quick start 240
Using a WebSDR 241
Reading the panel 242
Remote SNR trim 244
Choosing a second receiver 244
Troubleshooting 245
Limitations 245

16. Rig Control 247

What you need 247

Supported receivers 247
What it does, and what it does not 248
Quick start 248
Choosing how to reach the rig 248
Serial settings 249
Sync 250
Modes 251
Filter width 251
Mute while transmitting 252
Frequency offset 252
The Rig menu 252
Linux 253
Windows 253
TCI-CAT on the receiver page 253
For receiver operators 261
For developers: the page interface 262
Troubleshooting 263
Known limitations 263
17. Project Structure 264
Table of Contents 264
Directory Tree 265
Root Directory 280
Source Code (src/) 283
Frontend (frontend/) 284
Frequency Lists (frequencylist/) 290
Configuration Files 291
Build System 293
File Dependencies 293
Data Flow 294
File Modification Guide 294
Important Paths 295
Common File Operations 295
Version Control 296
18. Appendix 297
Installing and setting up the station 297
Receiver drivers 298
Starting and stopping the receiver 299
Starting at boot 300
Building and updating 301
Look of the page 302
Admin panel and proxy 303
Thermal guard 304
Connections, firewall and HTTPS 304
Optional services 305
Rig control (TCI bridge) 306
Checking and troubleshooting 306
Internal scripts (not run by hand) 306

CHAPTER 1

README

README.md

PhantomSDR-Plus

An enhanced open-source WebSDR server with advanced features and futuristic design

PhantomSDR-Plus is a fork of PhantomSDR that provides a high-performance Software Defined Radio (SDR) web server capable of handling hundreds of simultaneous users. It features an enhanced user interface, multiple decoder support, band plan visualization, and compatibility with various SDR hardware platforms.

Key Features

Performance & Scalability

- **Multi-user support**: Handle hundreds of concurrent users depending on hardware
- **High sample rate**: Support for SDRs up to 70 MSPS (real) / 35 MSPS (IQ)
- **Hardware acceleration**: OpenCL and CUDA support for GPU-accelerated processing
- **Optimized streaming**: Low-latency FLAC and Opus audio compression. The browser decodes each FLAC packet the moment it arrives, and its playback buffer adjusts itself to the listener's connection, keeping only as much audio in hand as needed — typically 100–150 ms from receiver to loudspeaker on a phone over Wi-Fi

User Interface

- **Futuristic design**: Modern, responsive web interface
- **Mobile-optimized**: Enhanced mobile GUI for on-the-go listening
- **Band plan visualization**: Interactive waterfall with frequency band overlays
- **Customizable colormaps**: Multiple waterfall color schemes
- **Dual S-meter**: Choice between analog or digital signal meter display
- **One-touch decoder buttons**: A Decoders row sits on the main panel directly under the Modes (FT8, FT4, FT2, JS8, CW, WSPR, FAX, SSTV, NAVTX, FLDIGI), with RADE-v2U, RADE-v1L and RADE-v1U beside the Modes itself and repeated in the Modes and Bands pop-ups. One press starts the decoder and opens its window, a second press stops it. The old Bandwidth row has been removed.
- **Channel scanner**: sweeps the band, or exactly what the waterfall is showing, and stops on the first channel rising a chosen number of dB above the band's own noise floor, then carries on by itself once that channel goes quiet. The step follows the mode and stops land on the channel grid (1 kHz SSB, 100 Hz CW, 5 kHz shortwave AM, 9/10 kHz medium wave, 9 kHz long wave); a *Skip empty* mode reads the waterfall spectrum and jumps straight to the signals; a lockout button drops an always-busy channel out of the sweep. Everything is local to the listener's browser, so scanning never moves the receiver for anyone else (see User Guide)
- **Receive diversity**: pairs the receiver with a second one somewhere else and plays whichever of the two currently has the better signal, so a fade at one site is covered by the other. The partner can be another PhantomSDR-Plus, a KiwiSDR, an UberSDR or a WebSDR; the first three need nothing beyond the listener's browser, and only a WebSDR needs a small relay on your own server. Alignment takes about fifteen seconds, and a remote SNR trim balances the two sites against each other. Each

listener keeps a named list of second receivers in their own browser, in whatever order they choose, and can export it to a file and import it back (see Receive Diversity)

- **Several receivers, one page**: one computer can run two or more receivers at once — for example an RX-888 for HF and an RTL-SDR for 2 m — and listeners switch between them with buttons in the page header. Each receiver keeps its own waterfall, chat, markers, listener list and station details, and the S-meter reads VHF S-units (S9 = −93 dBm) above 30 MHz; with **Show All** the other receivers run live under the waterfall, a click opens one tuned there, and a frequency only another receiver covers opens that receiver by itself (see Several Receivers)

Signal Processing

- **Multiple demodulation modes**: AM, FM, USB, LSB, CW, and more. A version 1 RADE decoder has also been implemented. RADE V2 (beta), the new waveform of FreeDV 3.0, is decoded too, with the RADE-v2U button.
- **Synchronous AM detection**: Improved AM reception quality
- **Noise reduction**: Advanced NR, NC (Noise Cancel), and NB (Noise Blanker)
- **AGC options**: Multiple Automatic Gain Control modes
- **Auto squelch**: Automatic noise-based squelch threshold
- **CTCSS tone squelch**: FM audio passes only while the chosen subtone — or any standard one — is received; the tone is picked from a small window beside the CTCSS button, which then shows it
- **AI noise reduction**: RNNoise neural network in the listener's browser, removes band noise from speech (voice modes only; see User Guide)

Advanced Features

- **Digital decoders**: FT8, FT4, FT2, JS8, CW (with CW Browser), QRSS Grabber, WSPR, HF FAX, SSTV, NAVTEX, the FLDIGI modes (ALE 2G HAM, RTTY, PSK31/63/125 (BPSK/QPSK), Olivia, Contestia, PSK Browser, Olivia Browser, FST4W, MFSK16/32/64, MSK144, DominoEX, THOR, THROB, Hellschreiber, MT63, Packet (AX.25), APRS with a live map of the stations heard) and FreeDV RADE, each running on its own background thread so decoding never interrupts the audio (see Decoders)
- **Decoder ID**: names the digital mode in the passband and offers the matching decoder in one click, from measured bandwidth, tone spacing, symbol rate and burst timing, with the frequency taken into account as supporting evidence — which is also what separates **FT8 from JS8**, two modes whose signals are identical. It only suggests, never switches by itself, and stays quiet rather than guessing when the signal is too weak. Fading does not silence it: when QSB breaks a two-minute WSPR transmission into fragments it still names the mode, reading the cadence from the whole listening history, and lowers the confidence it shows to say that the evidence is weaker. Between about 400 Hz and 2700 Hz the position in the passband makes no difference; outside that an amber **Recentre & retry** button appears, which moves the dial once and starts a fresh measurement. When you do accept a suggestion, the receiver is set up to work that mode: the tuned frequency moves to the middle of the waterfall, the view becomes about 100 kHz wide around it, and the sideband and passband become the ones the mode is worked with — the 3 kHz sub-band for the FT8 family, 1350–1650 Hz for WSPR, ±250 Hz and CW for Morse, and so on. Off by default, and about 0.5% of one core while running (see Decoders)
- **Digital mode presets**: pressing FT8, FT4, FT2, JS8 or WSPR sets both the sideband and the passband
- **JS8 conversation panel**: multi-frame messages are reassembled into whole sentences
- **Statistics dashboard**: Real-time server and user statistics
- **Connected users list**: everyone listening, with a tune-to button on each row. Your own session is marked **you** — both in the pop-up window and when `users.html` is opened as a page of its own. Each row also shows the decoder the listener is running (FT8, DRM, HFDL…).

- **Automatic spot reporting**: an autorun daemon decodes FT8, FT4 and WSPR on the server and uploads spots to PSK Reporter and WSPRnet, with two counters in the admin panel — per-decoder tiles for the current run, and an all-time total beside each band/mode checkbox
- **Server graphs**: an admin **Graphs** page plotting CPU frequency, CPU load, CPU temperature and users online over the last 15 minutes to 24 hours (see Admin Panel)
- **Admin alerts, backup, visitors and themes**: email/Telegram alerts when a receiver goes off the air, restarts or crashes, a one-zip backup and restore of the station's settings, private visitor statistics, a System Health card, and six themes with a THEME selector in the top bar (see Admin Panel)
- **Thermal protection**: a guard in the admin panel stops the server if the CPU overheats and restarts it once cool, with thresholds derived from your own CPU's critical trip point and no assumptions about how you start or stop the server; it ships in log-only mode, so it does nothing until you enable it (see the Thermal Guard manual)
- **KiwiSDR client emulation**: an optional bridge that answers the KiwiSDR protocol on the same host and port, so Kiwi software such as **AetherSDR** and `kiwiclient` connects to the receiver directly — with real retuning, waterfall and an S-meter on the same scale as the web one. Off until `[kiwi_emulation] enabled = true` is added to your config (see KiwiSDR Client Emulation)
- **Transceiver control (CAT)**: keeps your own rig and your receiver window on the same frequency, mode and filter, in either direction or both — through the Rig menu of Desktop PhantomSDR+ (Icom, Yaesu, Kenwood, Elecraft, FlexRadio and QRP Labs directly, every other rig through Hamlib, or flrig) or the CATsync Tool for WebSDRs in a browser. Pages expose `catsync_*` functions for frequency, mode, filter width and mute. It works with PhantomSDR-Plus, KiwiSDR, PA3FWM WebSDR and UberSDR receivers (see Rig Control)
- **Bookmark system**: Import/export frequency bookmarks
- **Keyboard shortcuts**: Efficient navigation and control
- **Mouse wheel support**: Intuitive frequency tuning
- **WebSDR directory**: Integration with https://sdr-list.xyz

Supported Hardware

PhantomSDR-Plus supports a wide range of SDR receivers:

Device	Sample Rate	Format	Interface
RX888 MK2	Up to 64 MHz	16-bit	Native
RTL-SDR	Up to 3.2 MHz	8-bit	rtl_sdr
HackRF One	Up to 20 MHz	8-bit	hackrf_transfer
Airspy HF+ / Discovery	Up to 912 kHz	16-bit	SoapySDR (SoapyAirspyHF) + rx_sdr
SDRplay RSP1A	Up to 10 MHz	16-bit	SoapySDR (libmirisdr-5 + SoapyMiri, or the SDRplay API) + rx_sdr
RigExpert Fobos SDR	50 MHz (HF direct sampling) / up to 20 MHz (RF)	16-bit / 32-bit float	SoapySDR (SoapyFobosSDR) + rx_sdr
Other devices	Variable	Various	SoapySDR/rx_tools

Quick Start

System Requirements

Minimum:

- Ubuntu 22.04 LTS (recommended) or Fedora
- 2-core CPU
- 4 GB RAM
- 10 GB disk space

Recommended:

- Ubuntu 24.04 LTS
- 4+ core CPU (Ryzen 5 2600 or Intel i5-6500T or better)
- 8 GB RAM
- GPU with OpenCL support (optional but highly recommended)
- SSD storage

Installation

```
git clone https://github.com/sv1btl/PhantomSDR-Plus
cd PhantomSDR-Plus
bash install.sh
```

The installer asks every question first — the receiver, what to receive, your station, your address on the internet and the extras — then installs everything on its own, starts the receiver, and shows the one port to open on your router (9000). No `chmod`, and no terminal to restart. The station questions are described in INSTALLATION.md.

> **Already running PhantomSDR-Plus?** Do not install it again — update it. Fetch the updater once and run it; it shows what would change and asks before writing anything, and your configuration, markers, admin password, frequency list and chat history are never touched:
>
> `bash cd ~/PhantomSDR-Plus curl -fLO https://raw.githubusercontent.com/sv1btl/PhantomSDR-Plus/main/update.sh bash update.sh`
>
> The *Updating PhantomSDR-Plus* chapter has the details.

Use `install_fedora.sh` on Fedora, `install_arch.sh` on Arch or `install_opensuse.sh` on openSUSE Tumbleweed — they do the same work with a different package manager. Every Debian and Ubuntu release is covered by `install.sh` itself.

Nothing has to be installed by hand first. The script installs the build dependencies and Node.js, builds the backend, builds the driver for the receiver you choose (and the RX888 udev rules with it), opens `site_information.json` for you to fill in, builds the frontend, installs OpenCL when the hardware supports it, then sets up the **admin panel**, the **FreeDV RADE decoder** and the **statistics server** — all three by default, answer `n` to skip any — and finishes with a full `recompile.sh`. It is interactive throughout and can take from twenty minutes to over an hour.

For the step-by-step description, see INSTALLATION.md → What the installer does.

Performance Benchmarks

Hardware	Sample Rate	CPU Usage	User Capacity
Ryzen 5 2600 (all cores)	64 MHz (32 MHz IQ)	38-40%	100+ users
AMD RX 580 (GPU)	64 MHz (32 MHz IQ)	28-35%	100+ users
Intel i5-6500T (with OpenCL)	60 MHz (30 MHz IQ)	10-12%	100+ users

Note: Per-user CPU overhead is minimal (<1% per user) when hardware acceleration is enabled.

Usage

Basic Operation

1. **Configure your SDR**: Edit the appropriate config file (e.g., `config-rtl.toml`)
2. **Update site information**: Edit `frontend/site_information.json`
3. **Start the server**: Run the appropriate start script: `bash ./start-rtl.sh # For RTL-SDR ./start-rsp1a.sh # For SDRplay RSP1A ./start-airspyhf.sh # For Airspy HF+ ./start-fobos-hf.sh # For RigExpert Fobos SDR, HF1/HF2 (0-25 MHz) ./start-fobos.sh # For RigExpert Fobos SDR, RF (25-6000 MHz) ./start-hackrf.sh # For HackRF One ./start-rx888mk2.sh # For RX888 MK2` The RSP1A, Airspy HF+ and Fobos run through SoapySDR and `rx_sdr`; the installer sets up their driver when you pick them (options 3, 6 and 5), or run `./setup-rsp1a.sh`, `./setup-airspyhf.sh` or `./setup-fobos.sh` on an installed station. See INSTALLATION.md. The HackRF One (option 7) needs only the distribution's `hackrf` package: `./setup-hackrf.sh`. Each start script is **self-contained**: it stops any running instance, launches the receiver + `spectrumserver`, **detaches into the background** (survives closing the terminal), and runs a **watchdog** that auto-restarts the chain if it dies. Progress is logged to `logwebsdr.txt` — watch it with `tail -f logwebsdr.txt`. Re-running a start script is a clean restart, and a `flock` lock ensures only one receiver runs at a time. Optional: `SPECTRUM_CORES=0-3` to pin CPUs, `RX_ARGS="…"` to override receiver args without editing the file (`RX888_ARGS` for RX888), `RX_DRIVER=miri|sdrplay` to choose the RSP1A driver.
4. **Access the interface**: Open your browser to `http://localhost:PORT` (default varies by config)

Stopping the Server

One shared stop script works for **every** receiver — it stops the watchdog first (so it can't auto-restart), then the receiver and `spectrumserver`:

```
./stop-websdr.sh
```

Configuration

Essential Configuration Files

1. **`config-[device].toml`** - Server and SDR configuration
2. Server settings (port, threads, HTML root)
3. Input settings (sample rate, FFT size, frequency)
4. WebSDR registration options
5. Audio compression settings

6. After modifying it, restart the server.
7. `frontend/site_information.json` - Public site information
8. Operator details (callsign, email, location)
9. Hardware and antenna information
10. Region and bandwidth settings
11. Chat enable/disable
12. After modifying it, run recompile.sh in a terminal.
13. `markers.json` - Frequency markers and band plan
14. `frontend/src/bands-config.js` - Band plan configuration This file is located in frontend/src/bands-config.js and defines the bands the SysOp creates.
15. You will see something like the following, and you are free to edit it as you like: ```
16. const bands = ...
17. { ITU: 1, name: '40m', min: -30, max: 110, initFreq: '7120000', publishBand: '1', startFreq: 7000000, endFreq: 7200000, stepi: 1000, color: 'rgba(50, 168, 72, 0.6)', modes: [{ mode: MODES.CW, startFreq: 7000000, endFreq: 7040000 }, { mode: MODES.LSB, startFreq: 7040000, endFreq: 7200000 }] }, { ITU: 2, name: '40m', min: -30, max: 110, initFreq: '', publishBand: '1', startFreq: 7000000, endFreq: 7300000, stepi: 1000, color: 'rgba(50, 168, 72, 0.6)', modes: [{ mode: MODES.CW, startFreq: 7000000, endFreq: 7050000 }, { mode: MODES.LSB, startFreq: 7050000, endFreq: 7300000 }] }, { ITU: 3, name: '40m', min: -30, max: 110, initFreq: '', publishBand: '1', startFreq: 7000000, endFreq: 7200000, stepi: 1000, color: 'rgba(50, 168, 72, 0.6)', modes: [{ mode: MODES.CW, startFreq: 7000000, endFreq: 7040000 }, { mode: MODES.LSB, startFreq: 7040000, endFreq: 7200000 }] ..... etc ``` **Where:**
18. **ITU** is the region in which the server is located,
19. **min & max** are the waterfall brightness limits for the given band,
20. **initFreq** is the initial frequency to tune to when the band is selected,
21. **publishBand** sets whether the band is shown: 1 for amateur bands, 2 for broadcast bands,
22. **startFreq & endFreq** define the band limits,
23. **stepi** is the default mouse-wheel step for the band, and **modes** is the preferred default mode for the band.
24. After making changes, **run recompile.sh in a terminal** from the PhantomSDR-Plus folder.

For configuration examples, see the included sample files:

- `config-rtl.toml` - RTL-SDR configuration
- `config-rsp1a.toml` - SDRplay RSP1A configuration
- `config-airspyhf.toml` - Airspy HF+ configuration
- `config-fobos-hf.toml` / `config-fobos.toml` - RigExpert Fobos SDR configuration (HF / RF)
- `config-hackrf.toml` - HackRF One
- `config-rx888mk2.toml` - RX888 MK2 configuration

Waterfall Floor — `waterfall.sh`

`waterfall.sh` (in the repository root) changes the **default minimum waterfall level** in dB — the floor the waterfall starts from on a fresh visit.

That value lives in two source files at five different places (`frontend/src/waterfall.js` → `this.minWaterfall`, and `frontend/src/App.svelte` → the initial `let min_waterfall`, the two "min" reset cases and the bookmark default). Editing them by hand is easy to get wrong, so the script does it for you. It never uses hard-coded line numbers — every spot is located by pattern, so it keeps working after the sources are updated.

- A **higher** value (e.g. `-15`) gives a **darker** waterfall.
- A **lower** value (e.g. `-45`) gives a **brighter** waterfall.

```
./waterfall.sh              # interactive — shows the current values and asks for the new one
./waterfall.sh -v -15       # set the value without being asked
./waterfall.sh -v -15 -y    # ... and skip every confirmation
./waterfall.sh -s           # show the current values only, change nothing
```

Before editing, it timestamps a backup of both files (`*.bak-YYYYmmdd-HHMMSS`) and prints the restore command. Only literals equal to the value currently in use are touched, so unrelated numbers in those files can never be rewritten by accident. If the new value does not show up afterwards, the script restores the backups by itself and exits with an error. It also warns you when the two files are out of sync and sets both.

- After the change the frontend must be rebuilt — the script offers to run `recompile.sh` for you (with `-y` it skips the rebuild, so run `./recompile.sh` yourself when you are ready).

Customization

Changing the Background Image

Replace `frontend/src/assets/background.jpg` with your preferred image (keep the same filename).

- After modifying it, run recompile.sh in a terminal.

Audio Codec Selection

Choose between FLAC and Opus in your `.toml` configuration:

```
[input]
audio_compression="opus"  # or "flac"
```

- After modifying it, restart the server.

S-meter calibration

The two meters are deliberately on **separate chains**, so they are tuned with two different knobs.

The two knobs

Both live in the `[input]` section of the config the server is started with (`config.toml`, `config-rx888mk2.toml`, etc.):

```
[input]
smeter_offset=5         # digital bar only
analog_smeter_offset=5  # analog needle only
```

They are read in `src/websocket.cpp` and sent to the browser in the first `basic_info` frame, so **a server restart is enough** — no frontend rebuild is required.

Analog chain

```
displayed dBm = -130 + (rawDb + analog_smeter_offset + 130) x 1.1
```

- The visual gain of 1.1 is applied *after* the offset, pivoting at -130 dBm. So one unit of `analog_smeter_offset` moves the reading by 1.1 dB. To shift the display by X dB, use `analog_smeter_offset ~= 0.91 x X`. The TOML value is parsed as an integer, so only whole steps are available.
- The needle law (`powerFromDbm` in `frontend/src/lib/SMeterAnalog.svelte`) puts **S9 at -73 dBm**, at 60 of 100 needle units, with a `pow(...,0.6)` curve from -130 to -73 below S9 and a `pow(...,0.8)` curve from -73 to -13 (S9+60) above it.
- It is **not** a linear 6 dB per S-unit scale, so S9 is the meaningful calibration anchor — the marks below S9 will not track a signal generator in 6 dB steps.

Digital chain

```
value    = (rawDb / 150) x 100 + smeter_offset   -> clamped to [-100, 0]
segments = round((value + 100) x 35 / 100) + DIGITAL_BAR_TRIM
```

- Note the `/150 x 100`: the digital scale is **compressed to 2/3**, and it never sees `analog_smeter_offset` or the 1.1 visual gain. That is why the bar and the needle never agree exactly — it is intentional.
- 35 segments over 100 units means one segment is 2.86 offset units, roughly 4.3 raw dB. So `smeter_offset=3` is about one segment.
- `DIGITAL_BAR_TRIM` in `frontend/src/lib/SMeterDigital.svelte` (currently 0) shifts the bar by whole segments. Changing it is a source edit and **does** require a frontend rebuild.

The LED windows are analog-driven in both meters

The dBm / dBµV / SNR / NF windows in **both** meters read the analog-calibrated value and add a fixed `VISUAL_DBM_OFFSET` of 5, hardcoded in `SMeterAnalog.svelte` and `SMeterDigital.svelte`. Therefore:

- `smeter_offset` does **not** change any number in the windows, only the bar length.
- dBµV = dBm + 107 (50 ohm), and NF = dBm - SNR by construction, so NF follows automatically.
- To correct the **numbers** without touching the needle position, `VISUAL_DBM_OFFSET` is the knob — but it is a source constant, needs a rebuild, and must be changed in **both** files to stay consistent.

Procedure

1. Feed a known level into the antenna port (signal generator at -73 dBm = S9), in SSB/CW with a stable passband. Readings depend on bandwidth and AGC, so fix the mode and bandwidth first.
2. Note what the analog dBm window shows, then set `analog_smeter_offset ~= 0.91 x (-73 - shown)`. Restart the server and recheck. One iteration should be enough; the needle should land on S9.
3. Switch to the digital meter and set `smeter_offset` so the bar length places S9 where you want it — roughly 3 units per segment. This step is cosmetic matching only and has no effect on the readouts.
4. Without a signal generator, a quiet band with a known-level reference beacon is a reasonable substitute, or simply anchor the noise floor to a plausible value (e.g. -120 dBm on 20 m with a decent antenna) — accepting that this calibrates the whole chain including antenna gain, not the receiver alone.

GUI Variants

All four GUI variants (analog/digital S-meter x v1/v2 layout) live in the one build. Visitors switch between them with the ⚙menu at the top right of the page — nothing reloads, so audio, the waterfall and any running decoder carry on — and each visitor's choice is remembered in their own browser.

- Run recompile.sh in a terminal to set the *starting* variant, the one a first-time visitor sees.
- **EDITING_VARIANTS.md** - How to edit the frontend variants (S-meter and layout) and rebuild them

---

Documentation

- **INSTALLATION.md** - Complete installation guide for system operators
- **ADMIN_PANEL_SETUP.md** - Installation guide for admin system operators
- **USER_GUIDE.md** - End-user guide for operating the WebSDR
- **THERMAL_GUARD.md** - Sysop manual for the CPU over-temperature guard: the four modes and what to do with each
- **CONNECTION_LIMITS.md** - Sysop manual for connection limits: protecting a public receiver from connection floods
- **MULTI_RECEIVER.md** - Sysop manual for running several receivers on one computer: instances, `receivers.toml`, the receiver picker and `start-all.sh`
- **HTTPS.md** - Secure access: serving the receiver over https:// with Caddy and a free Let's Encrypt certificate, `setup-https.sh`, the local-network mode, and an existing nginx or Apache
- **PROJECT_STRUCTURE.md** - Directory structure and code organization
- **EDITING_VARIANTS.md** - How to edit the frontend variants (S-meter and layout) and rebuild them

---

Online WebSDR List

Register your WebSDR on the official directory: https://sdr-list.xyz

Set `register_online=true` in your `.toml` configuration file to automatically register.

Troubleshooting

Common Issues

OpenCL Not Working

- Ensure drivers are properly installed
- Check with `clinfo` command
- See INSTALLATION.md for detailed OpenCL setup

Audio Latency Issues

- Keep FLAC as the codec: it is the lower-latency one, while Opus holds back up to 20 ms per packet
- Open the page with `?diag=1` added to its address (for example `/mobile?diag=1`) to see the delay split into packet, buffer and output; the buffer shrinks by itself within a minute or two of a network hiccup
- If the audio breaks up, move the buffer control one step higher (see the User Guide, *Audio Buffer and Delay*)
- Ensure adequate CPU/GPU resources

Build Failures

- Verify all dependencies are installed
- Try cleaning build directory: `rm -rf build && meson setup build`
- Check for conflicting library versions

Contributing

Contributions are welcome! This is an independent fork with additional features. Please:

1. Fork the repository
2. Create a feature branch
3. Commit your changes
4. Push to the branch
5. Create a Pull Request

License

This project is licensed under the GNU General Public License v3.0 - see the LICENSE file for details.

Authors & Credits

- **SV1BTL, 9A7AOF, F1NSK & SV2AMK** - PhantomSDR-Plus development and enhancements
- Based on the original PhantomSDR project

Links

- **Live Demo**: http://phantomsdr.no-ip.org:8900/
- **WebSDR Directory**: https://sdr-list.xyz
- **GitHub Repository**: https://github.com/sv1btl/PhantomSDR-Plus

---

Support

- **Issues**: Report bugs via GitHub Issues
- **Email**: contact sv1btl@otenet.gr

Performance Tips

1. **Enable OpenCL/CUDA** for GPU acceleration - dramatically reduces CPU usage
2. **Use SSD storage** for better I/O performance
3. **Allocate sufficient RAM** - at least 8 GB recommended
4. **Optimize FFT size** - larger FFT = better resolution but more CPU usage
5. **Consider dedicated GPU** - AMD or NVIDIA with OpenCL support

73 de SV1BTL, 9A7AOF, F1NSK & SV2AMK

For detailed setup instructions, see INSTALLATION.md For end-user operation guide, see USER_GUIDE.md For other translations in various languages, please refer to the folders in this section

CHAPTER 2

Installation Guide

INSTALLATION.md

PhantomSDR-Plus Installation Guide for System Operators

This comprehensive guide will walk you through installing and configuring PhantomSDR-Plus on your server.

Table of Contents

1. System Requirements
2. Pre-Installation Preparation
3. Installing PhantomSDR-Plus — the installer, and what it does
4. Autorun Spot Reporter (FT8/FT4/WSPR)
5. KiwiSDR Client Emulation (optional)
6. Configuration
7. SDR Device-Specific Setup
8. Testing and Verification
9. Setting Up Autostart
10. CPU Thermal Protection
11. Troubleshooting

Reference only — the installer already does all of this for you. Read these sections when you are on a distribution none of the installers covers, or when you need to repair one piece by hand:

- Installing Dependencies
- Installing Node.js and npm
- Installing OpenCL
- Building by hand
- Installing Opus Audio Codec

System Requirements

Supported Operating Systems

Primary (Recommended):

- Ubuntu 22.04 LTS (Jammy), 24.04 LTS (Noble) and 26.04 LTS (Resolute) — 24.04 recommended
- Debian 12 (Bookworm) and Debian 13 (Trixie)

Alternative:

- Fedora (latest stable release)
- Arch Linux (rolling)

- openSUSE Tumbleweed (not Leap — see the note further down)

Hardware Requirements

Minimum Configuration:

- CPU: Dual-core processor (2+ GHz)
- RAM: 4 GB
- Storage: 10 GB free space
- Network: 100 Mbps connection

Recommended Configuration:

- CPU: Quad-core or better (Ryzen 5 2600, Intel i5-6500T or better)
- RAM: 8 GB or more
- Storage: 20 GB+ SSD
- GPU: AMD/NVIDIA with OpenCL support (highly recommended)
- Network: 1 Gbps connection

High-Performance Configuration:

- CPU: 6+ cores (Ryzen 7, Intel i7 or better)
- RAM: 16 GB or more
- Storage: NVMe SSD
- GPU: Dedicated GPU with OpenCL/CUDA support
- Network: 1 Gbps or better

Pre-Installation Preparation

1. Update Your System

```
# Ubuntu/Debian
sudo apt update
sudo apt upgrade -y
sudo reboot

# Fedora
sudo dnf update -y
sudo reboot
```

2. Verify Ubuntu Version

```
lsb_release -a
```

Expected output should show: Ubuntu 24.04 LTS

3. Check Available Disk Space

```
df -h
```

Ensure you have at least 10 GB free in your home directory.

4. Check CPU Information

```
lscpu
```

Note the number of cores/threads for configuration optimization.

Installing Dependencies

> **IMPORTANT**
>
> **You do not need this section for a normal installation.** `./install.sh` — or the installer for your distribution — does all of it for you; see What the installer does. What follows is a reference for a manual setup, for a distribution none of the installers covers, or for repairing one piece by hand.

Ubuntu 24.04 LTS

```
sudo apt install -y \
  build-essential \
  cmake \
  pkg-config \
  meson \
  libfftw3-dev \
  libwebsocketpp-dev \
  libflac++-dev \
  zlib1g-dev \
  libzstd-dev \
  libboost-all-dev \
  libopus-dev \
  libliquid-dev \
  git \
  psmisc \
  wget \
  curl
```

Fedora

```
sudo dnf install -y \
  g++ \
  meson \
  cmake \
  fftw3-devel \
  websocketpp-devel \
  flac-devel \
  zlib-devel \
  boost-devel \
  libzstd-devel \
  opus-devel \
  liquid-dsp-devel \
  git \
  psmisc \
  wget \
  curl
```

Verify Installation

```
# Check if key dependencies are installed
pkg-config --modversion fftw3
cmake --version
meson --version
```

Installing Node.js and npm

> **IMPORTANT**
>
> **You do not need this section for a normal installation.** `./install.sh` — or the installer for your distribution — does all of it for you; see What the installer does. What follows is a reference for a manual setup, for a distribution none of the installers covers, or for repairing one piece by hand.

PhantomSDR-Plus requires Node.js for building the frontend. We'll use NVM (Node Version Manager) for installation.

1. Install NVM

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.4/install.sh | bash
```

2. Load NVM

IMPORTANT: Close and reopen your terminal, or run:

```
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
```

3. Verify NVM Installation

```
nvm --version
```

Expected output: `0.40.4` or similar

4. Install Node.js

```
# Install Node.js 22 - the version every install script requires
nvm install 22

# Make it the default, so systemd units and cron see it too
nvm alias default 22

# Verify installation
node --version
npm --version
```

5. Optional: Install Additional Node Versions

```
# Install latest version
nvm install node

# Install specific version (if needed)
nvm install 22.22.3

# List installed versions
nvm list

# Use specific version
nvm use 22
```

Installing OpenCL (Optional but Recommended)

> **IMPORTANT**
>
> **You do not need this section for a normal installation.** `./install.sh` — or the installer for your distribution — does all of it for you; see What the installer does. What follows is a reference for a manual setup, for a distribution none of the installers covers, or for repairing one piece by hand.

OpenCL dramatically improves performance by offloading FFT calculations to the GPU. This section covers Intel integrated graphics. For AMD/NVIDIA GPUs, refer to manufacturer documentation.

Intel CPU with Integrated Graphics

1. Install Base OpenCL Components

```
sudo apt install -y \
  build-essential \
  cmake \
  pkg-config \
  meson \
  libfftw3-dev \
  libwebsocketpp-dev \
  libflac++-dev \
  zlib1g-dev \
  libzstd-dev \
  libboost-all-dev \
  libopus-dev \
  libliquid-dev \
  libclfft-dev \
  ocl-icd-opencl-dev \
  clinfo
```

2. Download Intel Compute Runtime

```
mkdir -p ~/neo
cd ~/neo

wget https://github.com/intel/compute-runtime/releases/download/19.07.12410/intel-
gmmlib_18.4.1_amd64.deb
wget https://github.com/intel/compute-runtime/releases/download/19.07.12410/intel-igc-
core_18.50.1270_amd64.deb
wget https://github.com/intel/compute-runtime/releases/download/19.07.12410/intel-igc-
opencl_18.50.1270_amd64.deb
wget https://github.com/intel/compute-runtime/releases/download/19.07.12410/intel-
opencl_19.07.12410_amd64.deb
wget https://github.com/intel/compute-runtime/releases/download/19.07.12410/intel-
ocloc_19.07.12410_amd64.deb
```

3. Install Intel OpenCL Runtime

```
sudo dpkg -i *.deb
```

If dependency errors occur:

```
sudo apt --fix-broken install
```

4. Install OpenCL ICD Loader

```
sudo apt update
sudo apt install intel-opencl-icd
```

If errors occur:

```
sudo apt --fix-broken install
sudo apt install intel-opencl-icd
```

5. Verify OpenCL Installation

```
sudo clinfo
```

You should see information about your OpenCL platform and devices. Look for:

- Number of platforms: 1 (or more)
- Platform name: Intel(R) OpenCL (or similar)
- Device type: GPU or CPU

6. Reboot

```
sudo reboot
```

AMD GPU OpenCL

For AMD GPUs, install ROCm:

```
# Add ROCm repository
wget -q -O - https://repo.radeon.com/rocm/rocm.gpg.key | sudo apt-key add -
echo 'deb [arch=amd64] https://repo.radeon.com/rocm/apt/debian/ ubuntu main' | sudo tee /etc/apt/
sources.list.d/rocm.list

# Install ROCm
sudo apt update
sudo apt install rocm-opencl rocm-clinfo

# Add user to video group
sudo usermod -a -G video $USER

# Reboot
sudo reboot

# Verify
clinfo
```

NVIDIA GPU OpenCL

For NVIDIA GPUs, install CUDA:

```
# Install NVIDIA drivers
sudo apt install nvidia-driver-525

# Install CUDA toolkit
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64/cuda-
keyring_1.0-1_all.deb
sudo dpkg -i cuda-keyring_1.0-1_all.deb
sudo apt update
sudo apt install cuda

# Reboot
sudo reboot

# Verify
nvidia-smi
clinfo
```

Installing PhantomSDR-Plus

Clone the Repository and Run the Installer

```
cd ~
git clone https://github.com/sv1btl/PhantomSDR-Plus
cd PhantomSDR-Plus
bash install.sh
```

The installer first asks a few questions about your station — the receiver, what to receive, your callsign and locator, your address on the internet, and which extras you want — and then installs everything on its own, without asking anything more. At the end it starts the receiver and shows the address listeners use and the one port to open on your router (9000, unless you chose another). The questions are described in the next section but one, *The station questions*.

Type `bash install.sh` exactly like that: there is no `chmod` step. A copy downloaded as a ZIP or through the GitHub web page has no executable bits, and the installer sets them itself.

> **Already running PhantomSDR-Plus?** Do not install it again — update it. Fetch the updater once and run it; it shows what would change and asks before it writes anything. Your configuration, markers, admin password, frequency list and chat history are never touched:
>
> `bash cd ~/PhantomSDR-Plus curl -fLO https://raw.githubusercontent.com/sv1btl/PhantomSDR-Plus/main/update.sh bash update.sh`
>
> The *Updating PhantomSDR-Plus* chapter has the details. Re-running the installer over a working site is only needed when a rebuild fails because system packages are missing.

Use the script that matches your system. They do the same work and ask the same questions; only the package manager differs:

System	Script
Ubuntu 22.04 / 24.04 / 26.04, Debian 12 / 13	`./install.sh`
Fedora	`./install_fedora.sh`
Arch	`./install_arch.sh`
openSUSE Tumbleweed	`./install_opensuse.sh`

> **One installer for every Debian and Ubuntu release.** `install.sh` reads `/etc/os-release` and the installed Boost version and adapts itself; the old `install_ubuntu22.sh`, `install-Deb12.sh` and `install_ubuntu26.sh` are gone. It sets `DEBIAN_FRONTEND=noninteractive`, so the `tzdata` that arrives with `python3-matplotlib` cannot stop the run to ask for your time zone and then swallow the answer meant for the next question. On Jammy it calls `install_rade_ubuntu22.sh` directly, because Jammy's `python3-websockets` is 10.1 and RADE needs 11.0 or newer. It knows where `intel-opencl-icd` lives on each release: in the repositories on 22.04 and 26.04, in `non-free` on Debian 12, and in Intel's own graphics repository on 24.04 and Debian 13. And where Boost is 1.87 or newer, the websocketpp header patch stops being optional — the installer verifies it landed and refuses to build without it. No newer compiler is ever needed: the stock GCC accepts `-std=c++23` on all five releases, so do not install `gcc-12` for this.

The installation runs in 22 numbered steps. With the station questions answered at the start, none of the steps stops to ask anything; only the sudo password is asked, once, right after the questions, and it is kept alive for the whole run. Setting `PHANTOM_CLASSIC=1` brings back the old way — each question asked at its own step, fenced by a ⌨**YOUR INPUT IS NEEDED** banner. Every answer can also be given in the environment for a fully unattended run (`PHANTOM_NONINTERACTIVE=1`); see *Installing unattended* below.

Every run writes `install.txt`. When the installer finishes — or dies halfway — it writes a report to `install.txt` in the PhantomSDR-Plus directory: the result, each of the 22 steps as OK / SKIPPED / PARTIAL / FAILED, what it detected (distribution, Boost, compiler, Node.js), which components were installed, and every warning that came up. A run that fails leaves a report ending at the step that failed, with the reason and a note that the installer is safe to re-run. It is the first file to read when something did not work, and the first to attach to a bug report. Each run overwrites it, so keep a copy if you want to compare two installations.

> **Ubuntu 26.04, Arch and openSUSE Tumbleweed build, but have not been run on air.** All three ship a Boost newer than 1.87, which removed the `io_service` API that the vendored websocketpp 0.8.2 was written against. The patched headers the installer copies in bridge that gap (`io_context`, `executor_work_guard`, `boost::asio::post`, the modern resolver), and where Boost is 1.87 or newer the installer treats that patch as mandatory rather than optional — it verifies the copies landed and refuses to build without them. A complete install, receiver driver and all optional components included, has been verified end to end in containers on Boost 1.90 (Ubuntu 26.04), 1.91 (openSUSE Tumbleweed) and 1.92 (Arch). That proves the server compiles and starts — not that it serves a receiver for hours. Treat all three as untested in production until someone reports back.
>
> **openSUSE means Tumbleweed.** That is where `install_opensuse.sh` is verified. Leap 15.6 does not work: its repositories carry no `liquid-dsp-devel` at all, which the backend needs, and Boost is only available under versioned package names. Supporting Leap would mean adding third-party OBS repositories, so it is out of scope for now.

The station questions

`configure-station.sh` — run by the installer at the start, and runnable on its own at any time — asks five groups of questions. ENTER takes the value shown in brackets; when it is run again, the previous answers are the defaults.

#	Asked	Notes
1	Which receiver	The one plugged in is recognised and suggested (RX888, RTL-SDR, RSP1A, Airspy HF+, HackRF, Fobos). For an RTL-SDR it also asks whether it is a Blog V4.
2	What to receive	Ready choices for each receiver (for example *HF 0–30 MHz* for the RX888, *2 m* for an RTL-SDR), or your own centre frequency and sample rate.
3	Your station	Callsign, name, e-mail, QTH locator, city and country, antenna, computer, ITU region (suggested from the locator).
4	Internet	Your public address (IP or DNS name), and whether to list the receiver on sdr-list.xyz and on the websdr.org map. Also whether to serve the receiver over https:// as well (needs a DNS name). Type a name or an IP, not a port — the port comes in step 5.
5	Ports and extras	Admin panel, FreeDV RADE, DRM, statistics, WebSDR relay, start at boot — and the public port.

The answers are saved in **`station.conf`**, and from it the wizard writes everything that has to agree: the receiver's `config-<receiver>.toml` (sample rate, frequency, port, directory listings, websdr.org), `frontend/site_information.json` and the relay's settings. The start scripts read the frequency and sample rate from `station.conf` themselves, so they are never edited. Only the settings the wizard owns are changed, and each file is copied to `<name>.bak-<date>` before it is touched.

Only one port has to be open on your router. Everything listeners use goes through it:

Port	What	Open on the router
9000	the receiver page — `proxy.py`, which also carries `/admin`, `/rade`, `/drm`, `/stats` and `/relay`	**yes, only this one**
9001	spectrumserver	no
9010	admin panel	no
9011	statistics server	no
9012	FreeDV RADE decoder	no
9013	WebSDR diversity relay	no
9015	DRM decoder	no

A port that is already in use on the computer is skipped for the next free one. To change anything later:

```
bash configure-station.sh          # ask again (ENTER keeps each answer)
bash configure-station.sh --show   # print the current answers
```

On an installed station the wizard then carries the change everywhere it is needed — new ports into `admin_config.json` and the statistics service, a new receiver into start at boot — and asks once whether to rebuild the page and restart what runs. Changing a port is therefore this one command, plus the forwarding on your router if the public port moved. The public port must be 1024 or more, because the proxy does not run as root: to be reached on port 80, keep 9000 here and forward the router's outside port 80 to it.

A station installed before the wizard existed keeps its own ports: run there, the wizard offers the ports that station already uses.

What the installer does

Nothing has to be prepared by hand first — no dependency list to paste, no Node.js to fetch, no OpenCL packages to hunt down. Allow anywhere from about twenty minutes to well over an hour, depending on the machine and on whether RADE is installed (its download is the long part).

#	Step
1	Lists the PhantomSDR-Plus services running right now and offers to stop them before anything is touched (only on a computer that already has them)
2	Detects the distribution — then come the station questions and the sudo password, once
3–6	Installs every build dependency, with Node.js 22 through nvm if the system Node is missing or too old
7	Builds the backend with meson
8	Builds the driver for your receiver, with its udev rule, so the server never needs `sudo` for the device
9	Writes the receiver's `.toml` and `site_information.json` from your answers
10–11	Installs the frontend dependencies and builds the desktop and `/mobile` pages
12	Installs OpenCL for the hardware it finds
13	Installs the admin panel — or, without it, only the proxy on the public port
14–16	Installs the WebSDR relay, FreeDV RADE, DRM and the statistics server, if you chose them
17	Re-applies the patched websocketpp headers and verifies them
18	Installs the KiwiSDR client emulation, so Kiwi clients such as AetherSDR can connect
19	Rebuilds the backend from the patched sources
20	Chooses the FFT accelerator — OpenCL only when a device actually answers, otherwise the CPU — and installs start at boot if you chose it, and sets up https:// if you chose it
21	Starts the receiver
22	Prints the summary: the address listeners use, the port to forward, and every step with its verdict

Installing unattended

Every question has an environment-variable override, and the installer also switches to defaults on its own when stdin is not a terminal (a pipe, a container, a CI job). This works the same way in all four installers — `install.sh`, `install_arch.sh`, `install_fedora.sh` and `install_opensuse.sh`. Set `PHANTOM_NONINTERACTIVE=1` and the whole run completes without asking anything:

Variable	Effect
`PHANTOM_NONINTERACTIVE=1`	answer every question with its default
`PHANTOM_SDR=1…7`	RX888 · RTL-SDR · SDRplay RSP1A · skip · Fobos · Airspy HF+ · HackRF (default 4)
`PHANTOM_RTLSDR_V4=y|n`	RTL-SDR Blog V4 driver (default n)
`PHANTOM_SITE_EDIT=y|n`	open `site_information.json` in an editor
`PHANTOM_OPENCL=y|n`	install OpenCL (default y)
`PHANTOM_OPENCL_PROVIDER=1|2|3`	Intel · Mesa/Rusticl · POCL (default: from the detected hardware)
`PHANTOM_ADMIN=y|n`	admin panel (y interactive, n unattended)
`PHANTOM_RADE=y|n`	RADE / FreeDV (y interactive, n unattended)
`PHANTOM_DRM=y|n`	DRM decoder (default y)
`PHANTOM_STATS=y|n`	statistics server (y interactive, n unattended)
`PHANTOM_KIWI=y|n`	KiwiSDR client emulation (default y)
`PHANTOM_RECOMPILE=y|n`	final rebuild (default y)
`PHANTOM_CURLPP=y|n`	continue without curlpp — Arch and openSUSE only (default y)
`PHANTOM_FIX_CLOCK_SKEW=y|n`	reset source file timestamps dated in the future, so meson can build (default y)
`PHANTOM_START_RECEIVER=y|n`	start the receiver at the end (default y)
`PHANTOM_CLASSIC=1`	no station questions: everything is asked on the way, as before
`STATION_…=…`	any answer to the station questions, e.g. `STATION_RECEIVER=rtl STATION_CALLSIGN=SV1XYZ` — see the header of `configure-station.sh`

With the station questions, the extras you chose take their answers from `station.conf` and run without a terminal, so an unattended run includes them without naming them here.

The three sub-installers marked *n unattended* are interactive scripts of their own, so an unattended run skips them by default rather than hang on their prompts. Name them explicitly to include them:

```
# a complete unattended install for an RX888 MkII
PHANTOM_NONINTERACTIVE=1 PHANTOM_SDR=1 ./install.sh

# ... and with the admin panel, RADE and the statistics server too
PHANTOM_NONINTERACTIVE=1 PHANTOM_SDR=1 \
  PHANTOM_ADMIN=y PHANTOM_RADE=y PHANTOM_STATS=y ./install.sh
```

After a reboot the RX888 udev rules and any OpenCL driver are fully in effect; the installer tells you when a reboot or a re-login is needed.

When it sets up the admin panel, the installer offers to install the two systemd units as well, and **strongly recommends** accepting: the CPU over-temperature guard runs inside the panel, so without them a reboot or a crash leaves the machine unprotected. The offer is skipped automatically where systemd is not running (containers, WSL1, OpenRC) — see the Admin Panel guide.

Each extra is a normal script you can also run on its own at any time — `./setup_admin.sh` (manual), `./install_rade.sh` (manual), `./setup-drm.sh` (manual), `./install-stats-server.sh` (manual), `./setup-rx888-udev.sh` and the receiver scripts `./setup-rsp1a.sh`, `./setup-fobos.sh`, `./setup-airspyhf.sh` and `./setup-hackrf.sh` — they are exactly what the installer calls.

Building by hand (reference)

> **IMPORTANT**
>
> **You do not need this section for a normal installation.** `./install.sh` — or the installer for your distribution — does all of it for you; see What the installer does. What follows is a reference for a manual setup, for a distribution none of the installers covers, or for repairing one piece by hand.

Should you ever have to build without the installer — a distribution none of the scripts covers, or repairing a half-finished build:

Build the Backend

```
# Clean any previous builds, Configure build with optimization, Compile
rm -rf build
meson setup build --buildtype=release --optimization=3
meson compile -C build

# Verify binary was created
ls -lh build/spectrumserver
```

Build the Frontend

```
cd frontend

# Install dependencies, Fix any vulnerabilities, Build the frontend
npm install
npm audit fix
npm run build

# Return to project root
cd ..
```

Verify Installation

```
# Check if binary exists
ls -lh build/spectrumserver

# Check if frontend was built
ls -lh frontend/dist/

# List important files
ls -lh *.sh *.toml
```

Installing Opus Audio Codec

> **IMPORTANT**
>
> **You do not need this section for a normal installation.** `./install.sh` — or the installer for your distribution — does all of it for you; see What the installer does. What follows is a reference for a manual setup, for a distribution none of the installers covers, or for repairing one piece by hand.

Opus provides better audio quality and lower latency compared to FLAC.

1. Install libopus System Library

```
sudo apt-get update
sudo apt-get install -y libopus0 libopus-dev
```

2. Install Opus Decoder for Frontend

```
cd PhantomSDR-Plus/frontend

# Install npm dependencies if not already done, Install Opus WASM decoder, Fix any vulnerabilities,
Rebuild frontend
npm install
npm install @wasm-audio-decoders/opus-ml
npm audit fix
npm run build

# Return to project root
cd ..
```

3. Verify Opus Installation

```
# Check if Opus system library is installed
pkg-config --modversion opus

# Check if Opus npm package is installed
cd frontend
npm list @wasm-audio-decoders/opus-ml
cd ..
```

Autorun Spot Reporter (FT8/FT4/WSPR)

PhantomSDR-Plus ships an optional **autorun spot reporter** (in the `autorun/` directory). It decodes FT8/FT4/WSPR server-side straight off the receiver and uploads spots to the reporting networks:

- **FT8 / FT4 → PSK Reporter** (IPFIX/UDP)
- **WSPR → wsprnet.org** (HTTP)

It is controlled entirely from the **admin panel → "Spot Reporting"** tab, and **reporting is OFF by default** — nothing is transmitted or uploaded until you enable it there. The decoder runs as a separate Node.js daemon and taps the receiver's audio locally, so it adds no extra RF hardware.

Requirements

Requirement	Notes
Node.js 22+	Same runtime the frontend build already needs — installed in the Node.js step.
`ws` + `cbor-x` npm packages	Resolved through `autorun/node_modules`, a symlink to the frontend's `node_modules` (both packages are declared in `frontend/package.json`).
`util-linux` (`taskset`)	The admin "Start" button pins the daemon to the E-cores with `taskset`. Present on virtually every distro; the installers add it explicitly.
Callsign + grid	Read from `frontend/site_information.json` (`siteSysop` / `siteGridSquare`) unless overridden in the admin tab. Spots are uploaded under this callsign. PSK Reporter and WSPRnet get a 4- or 6-character locator only — a longer one (KM17VX69JA) is cut to its square (KM17VX).

> ⚠**Report only what you actually receive.** Spots are uploaded to public networks under your callsign — only enable bands/modes your receiver genuinely hears, and use your correct grid square.

Automatic installation

`install.sh` (and the per-distro variants: `install_arch.sh`, `install_fedora.sh`, `install_opensuse.sh`) handle everything for you: they install Node.js 22 and `util-linux`, run the frontend `npm install`, and then create the `autorun/node_modules` symlink automatically. No extra steps are required — the feature is ready as soon as `install.sh` finishes.

Manual installation

> **NOTE**
>
> The installer already does this for you: it creates the symlink and installs `util-linux`. Use the steps below only to repair an installation by hand.

If you installed manually (Option B), create the symlink yourself after the frontend `npm install` so the daemon can resolve its dependencies:

```
cd ~/PhantomSDR-Plus

# 1. Frontend deps must be installed first (ws + cbor-x live there)
cd frontend && npm install && cd ..

# 2. Link the autorun daemon to the frontend's node_modules
ln -sfn ../frontend/node_modules autorun/node_modules

# 3. Ensure taskset is available (Debian/Ubuntu shown; use your package manager)
sudo apt install -y util-linux
```

> **Note:** `autorun/node_modules` is git-ignored, so a fresh `git clone` never contains it — the symlink must be (re)created after every clean checkout. The installers do this for you; the command above is only for manual setups.

Verifying

```
# The symlink should point at ../frontend/node_modules
ls -l autorun/node_modules

# ws + cbor-x must resolve
ls -d frontend/node_modules/ws frontend/node_modules/cbor-x

# taskset must be on PATH
command -v taskset
```

Then open the admin panel, go to the **Spot Reporting** tab, set your identity, tick the bands/modes to decode, enable the destination(s), and press **Start**. A "REPORTING" badge appears on the main waterfall whenever reporting is active. (PSK Reporter uploads batch every 5 minutes and wsprnet every 2 minutes, so a freshly started daemon shows "0 sent" for the first few minutes — that is normal.)

Two different counters are then shown, and it is easy to mistake one for the other. The **SPOTS UPLOADED PER DECODER** tiles count only the current run, so Stop/Start resets them to zero; the number beside each band/mode checkbox is that slot's **all-time** total, kept in `autorun-totals.json` so it

survives restarts. See the Admin Panel guide for both, and for the **Graphs** page that plots CPU frequency, load, temperature and users online over the last 15 minutes to 24 hours.

> **If your machine runs hot**, the panel also carries a Thermal Guard that stops the server when the CPU reaches a dangerous temperature and restarts it once cool. It derives its thresholds from your own CPU's critical trip point, so there is nothing to calculate, and it works whatever start/stop method you use. It is installed with the panel but starts in log-only mode: it records what it *would* have done and changes nothing until you enable it in Settings. Continuous decoding keeps a CPU busy around the clock, so it is worth reading the CRASH tab after a week of autorun to see how close your machine actually gets.
>
> **Server port is auto-detected.** The daemon taps spectrumserver's own `[server] port` directly (loopback + token, bypassing the proxy). It reads that port from the config file the running spectrumserver was launched with, so it works on any port with no configuration — the `[autorun] tap backend: …` line in `autorun.log` shows what it resolved. If your server wasn't running when the daemon started, or you use an unusual setup, pin it in `autorun.json`: `json "server": { "host": "127.0.0.1", "port": 9002 }` A wrong port shows up as `504` / `tap closed 1006` in `autorun.log` with `decodes` stuck at 0.

KiwiSDR Client Emulation (optional)

Since v4.1.0 PhantomSDR-Plus can also answer the **KiwiSDR protocol**, so software written for a KiwiSDR — **AetherSDR**, `kiwiclient` and the rest — connects to your receiver directly, on the same host and port you already publish. It is off until `[kiwi_emulation] enabled = true` is added to the config your receiver runs with.

The installer offers it as step 17, or `./kiwi_install.sh` applies it to a tree that is already installed.

> **Full documentation:** **KiwiSDR Client Emulation** — what the bridge does, how to install it, every `[kiwi_emulation]` key, connecting a client, audio level, the S-meter, the waterfall rate, and a symptom table.

Configuration

> **Installed with the station questions?** Then everything in this chapter is already done from your answers — the `.toml`, `site_information.json`, and the frequency and sample rate the start script uses. Change them with `bash configure-station.sh`. The steps below are for what the wizard does not cover, or for a station set up by hand.

1. Choose Your Configuration File

Select the appropriate config file for your SDR:

- `config-rtl.toml` - RTL-SDR dongles
- `config-rsp1a.toml` - SDRplay RSP1A
- `config-airspyhf.toml` - Airspy HF+ Discovery
- `config-fobos-hf.toml` - RigExpert Fobos SDR, HF1/HF2 direct sampling (0-25 MHz)
- `config-fobos.toml` - RigExpert Fobos SDR, RF path (25-6000 MHz)
- `config-rx888mk2.toml` - RX888 MK2

- `config-hackrf.toml` - HackRF One
- `config.example.hackrf.toml` - HackRF One

For this example, we'll use RTL-SDR.

2. Edit Configuration File

```
nano config-rtl.toml
```

Key Settings to Configure

```
[server]
port = 9002                      # Web interface port (or everything else)
html_root = "frontend/dist/"     # Frontend location
otherusers = 1                   # Show other users (1=yes, 0=no)
iq = true                        # Offer the IQ mode to listeners (true/false)
threads = 2                      # Number of server threads

[websdr]
register_online = true           # Register on sdr-list.xyz (true/false)
name = "Your WebSDR Name"        # Display name
antenna = "Your Antenna Type"    # e.g., "Vertical", "Loop", "Dipole"
grid_locator = "AB12cd"          # Your Maidenhead grid square
hostname = "your.domain.com"     # Your domain or IP address

[input]
sps = 2048000                    # Sample rate (adjust for your coverage)
fft_size = 131072                # FFT size (higher = better resolution)
brightness_offset = -10          # Waterfall brightness adjustment
frequency = 145000000            # Base frequency in Hz (145 MHz for 2m)
signal = "iq"                    # "iq" for complex, "real" for real sampling
fft_threads = 2                  # FFT processing threads
accelerator = "opencl"           # "none", "cuda", or "opencl"
audio_sps = 12000                # Audio sample rate (keep at 12000)
audio_compression = "opus"       # "flac" or "opus"
smeter_offset = -2               # S-meter calibration
waterfall_size = 1024            # Waterfall FFT size
waterfall_compression = "zstd"   # Waterfall compression

[input.driver]
name = "stdin"                   # Input driver
format = "u8"                    # Sample format for RTL-SDR

[input.defaults]
frequency = 145500000            # Default tuning frequency
modulation = "FM"                # Default modulation mode
volume = 50                      # Start volume of a first-time listener, 0-100 (optional)
```

Sample Rate Guidelines

Coverage	Sample Rate	FFT Size
2 MHz	2048000	131072
3.2 MHz	3200000	131072
10 MHz	10000000	1048576
30 MHz	30000000	2097152
60 MHz	60000000	4194304

3. Configure Site Information

```
nano frontend/site_information.json
```

Edit the following fields:

```
{
  "siteSysop": "YourCallsign",
  "siteSysopEmailAddress": "your@email.com",
  "siteGridSquare": "AB12cd",
  "siteCity": "Your City, Country",
  "siteInformation": "https://github.com/sv1btl/PhantomSDR-Plus",
  "siteHardware": "Computer specifications",
  "siteSoftware": "PhantomSDR-Plus v5.5.0",
  "siteReceiver": "Your SDR model",
  "siteAntenna": "Antenna description",
  "siteNote": "Additional information",
  "siteIP": "http://your.domain.com:9002",
  "siteSDRBaseFrequency": 0,
  "siteSDRBandwidth": 2048000,
  "siteRegion": 1,
  "siteChatEnabled": true
}
```

IARU Regions:

- **1**: Europe, Africa, Middle East, Northern Asia
- **2**: Americas (North, Central, South), Caribbean
- **3**: Asia-Pacific, Oceania

4. Customize Frequency Markers (Optional)

```
nano markers.json
```

Add your favorite frequencies, repeaters, and broadcast stations. A marker whose mode is a digital mode, such as `FT8` or `APRS`, starts that decoder when clicked — see Decoders.

5. Edit Start Script

```
nano start-rtl.sh
```

> On a station set up with the station questions, the frequency and sample rate come from `STATION_FREQ` and `STATION_SPS` in `station.conf`: `bash configure-station.sh` changes them, and the block above stays as shipped.

The start script is a self-contained launcher + watchdog. Edit only the **RECEIVER CONFIGURATION** block near the top so the receiver arguments match your setup:

```
# ═══ RECEIVER CONFIGURATION (the only receiver-specific part) ═══
RX_LABEL="RTL-SDR"
RX_COMM="rtl_sdr"                       # process name to monitor/kill
CONFIG="$PHANTOMDIR/config-rtl.toml"    # your .toml
FIFO="$PHANTOMDIR/rtl.fifo"
RX_ARGS="${RX_ARGS:--f 145000000 -s 2048000 -}"   # receiver args (edit these)
```

Parameters inside `RX_ARGS`:

- `-f 145000000`: Center frequency (145 MHz)
- `-s 2048000`: Sample rate (2.048 MSPS)

`CONFIG` points at your `.toml`. You do **not** need to edit anything else in the script — the start/restart/watchdog/logging logic is generic. You can also override the receiver args at launch without editing the file: `RX_ARGS="-f 145000000 -s 2048000 -" ./start-rtl.sh` (`RX888_ARGS` for `start-rx888mk2.sh`).

SDR Device-Specific Setup

RTL-SDR

Install RTL-SDR Tools

```
sudo apt install -y rtl-sdr
```

Test RTL-SDR

```
rtl_test
```

Press Ctrl+C to stop. You should see sample rate information.

Edit Configuration

```
nano config-rtl.toml
```

Common settings for RTL-SDR:

```
sps = 2048000
frequency = 145000000
signal = "iq"
format = "u8"
```

Receivers on SoapySDR (RSP1A, Fobos, Airspy HF+)

The SDRplay RSP1A, the RigExpert Fobos SDR and the Airspy HF+ all reach the server the same way: a SoapySDR driver for the device, and `rx_sdr` (from rx_tools) to stream its samples into `spectrumserver`. One script per receiver installs the whole chain:

Receiver	Script	Installer option	Launcher	Config
SDRplay RSP1A	`./setup-rsp1a.sh`	3	`start-rsp1a.sh`	`config-rsp1a.toml`
RigExpert Fobos SDR	`./setup-fobos.sh`	5	`start-fobos-hf.sh` (HF) / `start-fobos.sh` (RF)	`config-fobos-hf.toml` / `config-fobos.toml`
Airspy HF+	`./setup-airspyhf.sh`	6	`start-airspyhf.sh`	`config-airspyhf.toml`

Each script:

- installs git, cmake, a compiler, libusb and **SoapySDR** from your distribution's own packages — it recognises apt, dnf, pacman and zypper, so it works on Debian/Ubuntu, Fedora, Arch and openSUSE alike;
- builds the receiver's driver from source into `sdr_drivers/` (not tracked by git);
- builds **rx_tools** for `rx_sdr`, unless `rx_sdr` is already installed — one copy serves every receiver;
- installs a **udev rule** and adds you to the `plugdev` group, so the start script never needs `sudo` for the device. **Log out and back in once** for the group to take effect;
- finishes by checking that SoapySDR really lists the driver, and stops with an error if it does not.

Run them as your normal user — they call `sudo` themselves. They are idempotent, so re-running one updates the driver sources with `git pull` and rebuilds. The installer calls exactly these scripts; on a station that is already installed, run the one for your receiver on its own. The shared parts live in `setup-sdr-common.sh`, which is not meant to be run by itself. Two optional overrides apply to all three: `SDR_USER=<name>` puts another account in `plugdev`, and `SDR_SKIP_DEPS=1` skips the package installation.

Check that the receiver is seen, with it plugged in:

```
SoapySDRUtil --find="driver=soapyMiri"   # RSP1A
SoapySDRUtil --find="driver=fobos"       # Fobos
SoapySDRUtil --find="driver=airspyhf"    # Airspy HF+
```

SDRplay RSP1A

`setup-rsp1a.sh` installs the **open-source driver**: libmirisdr-5, the reverse-engineered driver for the Mirics MSi2500/MSi001 chips inside the RSP1 and RSP1A, with its SoapySDR module SoapyMiri (`driver=soapyMiri`). It needs no background service and no root. The script also blacklists the kernel's `msi2500` and `msi001` drivers in `/etc/modprobe.d/blacklist-msi2500.conf`, because they claim the RSP1A as a V4L2 radio before `rx_sdr` can open it. If the device was plugged in while they were loaded, replug it once (or reboot).

SDRplay's own closed API (`driver=sdrplay`, with SoapySDRPlay and the `sdrplay` service) still works — the included file `instructions-for-rsp1a` describes that installation. `start-rsp1a.sh` chooses between the two by itself: if SoapySDR has the `sdrplay` driver it uses the API and restarts the `sdrplay` service first, otherwise it uses libmirisdr-5. A station set up on the API therefore keeps working unchanged. To force one, start it with `RX_DRIVER=miri` or `RX_DRIVER=sdrplay`. The chosen driver is written to `logwebsdr.txt` as `RSP1A driver: miri` or `RSP1A driver: sdrplay`.

RigExpert Fobos SDR

The Fobos streams **one path at a time**, so it has two launchers that share the watchdog lock like every `start-*.sh` — starting one stops the other:

- **`start-fobos-hf.sh`** — the HF1/HF2 inputs in direct sampling, 0-25 MHz, on port 9003. There is no local oscillator in this mode: the ADC digitises 0-25 MHz all the time (`-f 0` is intentional, and rx_sdr's *Failed to set center freq* warning is expected). `rx_sdr` delivers CF32 at 50 Msps; the small converter `cf32_to_real` (built by `setup-fobos.sh` from `cf32_to_real.c`) keeps the I channel and scales it to s16, so `config-fobos-hf.toml` uses `signal="real"` and `format="s16"`.
- **`start-fobos.sh`** — the RF path, 25-6000 MHz, IQ, on port 9002. Choose the window before the first start: `-f` (centre) and `-s` (sample rate) in the `RX_ARGS` line must match `frequency=` and `sps=` in `config-fobos.toml`. The shipped values (97 MHz, 20 Msps) are only a starting point to check that it runs.

> **IMPORTANT**
>
> `config-fobos-hf.toml` ships with `accelerator="opencl"`: a 50 Msps stream through a 1M-bin FFT is as heavy as the RX888's 60 Msps, and on the CPU it drops samples and heats the machine. On a machine with no OpenCL device `spectrumserver` will not start with that setting — set `accelerator="none"` there.

SoapyFobosSDR needs the headers of both libfobos and libfobos-sdr-agile to build, so `setup-fobos.sh` builds both, even for the stock firmware. The Fobos support comes from a script another station wrote after bringing a Fobos up on PhantomSDR-Plus; the HF values in `config-fobos-hf.toml` are the ones that worked there.

Airspy HF+

`setup-airspyhf.sh` installs libairspyhf from the distribution where it is packaged (Debian and Ubuntu, Fedora, openSUSE) and builds it from source where it is not (Arch has it only in the AUR), then builds SoapyAirspyHF (`driver=airspyhf`). `start-airspyhf.sh` streams 912 ksps IQ in CS16, which `config-airspyhf.toml` reads as `format="s16"`. The included file `instructions-for-airspy` describes the manual route.

RX888 MK2

Install RX888 Tools

```
# Install rx_tools
git clone https://github.com/rxseger/rx_tools.git
cd rx_tools
mkdir build && cd build
cmake ..
make -j4
sudo make install
sudo ldconfig

# Install RX888 firmware and support
# Follow manufacturer instructions
```

Run rx888_stream Without sudo (udev rules)

By default the RX-888's Cypress FX3 USB device is only accessible to root, so `rx888_stream` would need `sudo`. Run the bundled helper once to install udev rules and add yourself to the `plugdev` group:

```
./setup-rx888-udev.sh          # re-runs itself with sudo automatically
```

It writes `/etc/udev/rules.d/99-rx888.rules` for all three Cypress FX3 product ids (`04b4:00f1/00f3` bootloader and `04b4:8613` firmware-loaded), reloads udev, and creates a stable `/dev/rx888` symlink. **Log out and back in** (for the group change) and **replug the device** once, then `rx888_stream` — and the `start-rx888mk2.sh` launcher — run without `sudo`. It is idempotent, so it's safe to re-run. To set up a different account: `RX888_USER=<name> ./setup-rx888-udev.sh`.

Edit Configuration

```
nano config-rx888mk2.toml
```

Common settings for RX888:

```
sps = 60000000
frequency = 0
signal = "real"
format = "s16"
fft_size = 4194304
```

HackRF One

The HackRF needs nothing built: `hackrf_transfer`, from your distribution's `hackrf` package, streams the samples straight into `spectrumserver`. `./setup-hackrf.sh` (installer option 7) installs that package with apt, dnf, pacman or zypper and adds a udev rule plus the `plugdev` group, so `start-hackrf.sh` runs without `sudo` — log out and back in once afterwards.

```
./setup-hackrf.sh    # installs the hackrf package and the udev rule
hackrf_info          # with the HackRF plugged in: must list the board
```

`start-hackrf.sh` streams one window at a time. Choose it before the first start: `-f` (centre) and `-s` (sample rate) in its `RX_ARGS` line must match `frequency=` and `sps=` in `config-hackrf.toml`. `-l` and `-g` are the LNA (0-40 dB, 8 dB steps) and VGA (0-62 dB, 2 dB steps) gains, and `-a 1` switches the RF amplifier on. The shipped values (98 MHz, 20 Msps) are only a starting point to check that it runs.

`hackrf_transfer` writes **signed** 8-bit IQ, so the config needs `format="s8"`. `config.example.hackrf.toml` said `u8` until September 2026; with `u8` every sample is off by 128 and the waterfall shows only noise — check an older copy of it.

Several receivers on one computer

One computer can run more than one of the receivers above at the same time — for example an RX-888 for HF and an RTL-SDR for 2 m — each as its own server, with a receiver picker on the page. Each extra receiver is started with `INSTANCE=<name> ./start-<radio>.sh` and set up in `instances/<name>/`; `receivers.toml` lists them for the proxy and for `start-all.sh`. Everything is in Several Receivers. The quickest way to add a receiver is `./add-receiver.sh`, which the installers offer at the end: it installs the driver (`setup-rtlsdr.sh` for an RTL-SDR, the matching `setup-*.sh` for the others) and writes the instance folder and `receivers.toml` for you.

Testing and Verification

1. Test Run

```
# For RTL-SDR
./start-rtl.sh
```

This starts the server **in the background** (it detaches and returns immediately) and begins logging to `logwebsdr.txt`.

2. Check for Errors

The script logs its progress to `logwebsdr.txt` — watch it live:

```
tail -f logwebsdr.txt
```

Look for:

- `Starting the initialization script of the PhantomSDR Server`

- `<receiver> running (PID …)`
- `Server started normally`
- `Server … FAILED …` or `ERROR: receiver binary '…' not found` (fix the receiver args/`.toml`, or install the receiver tool, then re-run the start script)

3. Access Web Interface

Open your browser to:

```
http://localhost:9002
```

(Replace port number with your configured port)

4. Verify Functionality

- Waterfall should be displaying
- Audio should play when clicking on signals
- S-meter should respond to signals
- User count should show "1"

5. Test from Another Device

From another computer on your network:

```
http://YOUR_SERVER_IP:9002
```

6. Check Resource Usage

```
# Monitor CPU usage
htop

# Monitor GPU usage (if OpenCL/CUDA enabled)
nvidia-smi  # For NVIDIA
radeontop   # For AMD

# Monitor network usage
iftop
```

7. Stop the Server

The server runs in the background under a watchdog, so **Ctrl+C won't stop it** (and the watchdog would just restart it). Use the shared stop script, which works for any receiver — it stops the watchdog first, then the receiver and `spectrumserver`:

```
./stop-websdr.sh
```

Setting Up Autostart

The simple way — and what the installer does when you answer yes to *start at boot*:

```
bash setup-autostart.sh               # the start script from station.conf
bash setup-autostart.sh start-rtl.sh  # or name it
bash setup-autostart.sh --remove      # stop starting at boot
```

It installs `phantomsdr-receiver.service`, which runs your start script at boot and `stop-websdr.sh` at shutdown, with the `plugdev` group, so the USB receiver opens without anyone logged in. The rest of this chapter shows the same by hand.

Using systemd (Recommended)

1. Create Service File

```
sudo nano /etc/systemd/system/phantomsdr.service
```

Add the following content (adjust paths and user):

```
[Unit]
Description=PhantomSDR-Plus WebSDR Server
After=network.target

[Service]
Type=simple
User=youruser
WorkingDirectory=/home/youruser/PhantomSDR-Plus
# Run the watchdog in the FOREGROUND (note the --watchdog flag) so systemd can
# track it. Do NOT use the plain "./start-rtl.sh" here — that form detaches into
# the background and exits, which systemd would treat as the service stopping.
ExecStart=/home/youruser/PhantomSDR-Plus/start-rtl.sh --watchdog
ExecStop=/home/youruser/PhantomSDR-Plus/stop-websdr.sh
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

> The `--watchdog` process already auto-restarts the receiver/`spectrumserver` on its own; `Restart=always` is just a backstop for the rare case the watchdog itself exits. Because systemd supervises the server here, you can use `systemctl start/stop/restart` and `journalctl -u phantomsdr -f` in place of running the scripts by hand.

2. Enable and Start Service

```
# Reload systemd
sudo systemctl daemon-reload

# Enable service to start on boot
sudo systemctl enable phantomsdr.service

# Start service now
sudo systemctl start phantomsdr.service

# Check status
sudo systemctl status phantomsdr.service
```

3. Manage Service

```
# Start
sudo systemctl start phantomsdr

# Stop
sudo systemctl stop phantomsdr

# Restart
sudo systemctl restart phantomsdr

# View logs
sudo journalctl -u phantomsdr -f
```

Using Screen (Alternative)

> Usually unnecessary: `./start-rtl.sh` already detaches into the background (via `setsid`) and keeps running after you log out, with its own watchdog. Screen is only handy if you specifically want an interactive session to run the foreground `./start-rtl.sh --watchdog` form.

1. Install Screen

```
sudo apt install -y screen
```

2. Start in Screen Session

```
screen -S phantomsdr
./start-rtl.sh
```

Press Ctrl+A, then D to detach.

3. Reattach to Session

```
screen -r phantomsdr
```

CPU Thermal Protection

> **Full manual: THERMAL_GUARD.md** — the four modes and what a sysop must do for each, the standalone systemd service, enabling the throttle stage without root, testing and troubleshooting.

PhantomSDR-Plus ships a guard that stops the server if the CPU reaches a dangerous temperature and starts it again once it has cooled. It derives its thresholds from the critical trip point your own CPU publishes, so there is nothing to calculate, and it works whatever start/stop method you use.

If you run the admin panel, you already have it — it runs inside the panel and is configured on its Settings page. See the Admin Panel guide. The rest of this section is for installations **without** the panel.

1. Check what the guard sees on your machine

```
cd ~/PhantomSDR-Plus
python3 thermal_guard.py --once
```

```
sensor     : hwmon:coretemp
temperature: 58.0 C
crit       : 100.0 C
mode       : log
thresholds : warn 88.0  throttle 92.0  stop 95.0  resume 75.0 (C)
cpufreq    : not writable — throttle stage disabled
```

Only real CPU-die sensors are used (`coretemp`, `k10temp`, `zenpower`, `cpu_thermal`); `acpitz` and unlabelled thermal zones are ignored on purpose, because they often report a case or board temperature tens of degrees below the CPU. If this prints `sensor : NONE`, that machine cannot be protected — common on a VPS or inside a container — and the guard will stay inactive rather than pretend.

Nothing needs to be installed for this: `thermal_guard.py` uses only the Python standard library.

2. Configure it

The guard reads `admin_config.json` next to `thermal_guard.py`. Create it if you do not have one — with no panel installed you will not. The minimum useful file:

```
{
  "sdr_process_name": "spectrumserver",
  "stop_script":  "stop-websdr.sh",
  "start_script": "start-rx888mk2.sh",
  "thermal_mode": "stop+restart"
}
```

- `thermal_mode` — `log` (record only, the default), `throttle`, `stop`, or `stop+restart`. **This is the one you must set**, because the guard ships inert: left alone it only writes what it *would* have done. You can also pass it on the command line as `--mode stop+restart`, which overrides the file — so if you are happy with every other default you need no config file at all.
- `stop_script` / `start_script` — how the guard stops and starts your server. Without a stop script it falls back to `SIGTERM`, then `SIGKILL` after 10 seconds, on `sdr_process_name`. Without a start script it stops but never restarts.
- Thresholds and timings can be set here too, with the same key names the panel uses — the full table is in the Admin Panel guide.

> **If systemd supervises your SDR server**, point `stop_script` at a small wrapper that runs `systemctl stop your-unit` rather than letting the guard signal the process directly. The guard will win either way — while the CPU is too hot it re-issues the stop every 2 seconds, so anything that revives the server is undone — but a clean stop beats a fight every two seconds.

3. Run it as a service

The repository includes a ready-made unit, `thermal-guard.service`. Edit `User=` and the two paths in it, then:

```
sudo cp thermal-guard.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now thermal-guard
systemctl status thermal-guard
```

`User=` must be an account allowed to run your start/stop scripts — normally the same user that runs the SDR server. **A guard running as a user that cannot stop the server leaves you unprotected while looking protected.**

To arm the guard from the unit instead of the config file, add `--mode` to `ExecStart`:

```
ExecStart=/usr/bin/python3 -u /home/sysop/PhantomSDR-Plus/thermal_guard.py --mode stop+restart
```

To run it without systemd:

```
cd ~/PhantomSDR-Plus
nohup python3 -u thermal_guard.py --mode stop+restart >> thermal.log 2>&1 &
```

`python3 thermal_guard.py --help` lists every option, including `--config` for a config file somewhere other than beside the script.

4. Watch it, then trust it

Everything the guard does is written to `crash.log` in the PhantomSDR-Plus directory, one event per line:

```
[THERMAL] warn: 89.0C (warn=88.0 crit=100.0) sustained 31s
[THERMAL] STOPPED server: 96.0C sustained 62s (stop=95.0 crit=100.0) — Started stop-websdr.sh
[THERMAL] lockout: process reappeared at 94.0C — re-stopped [3 time(s) so far]
[THERMAL] recovered: 74.0C held below 75.0 — lockout cleared
```

```
grep THERMAL crash.log
```

Leave `thermal_mode` at `"log"` for a week first and read that file after your heavy moments — a full rebuild, a hot afternoon. If nothing appears, your machine never came close. Then set the mode to `stop`.

Before relying on it, prove the whole path once by adding a fake reading above your stop threshold:

```
"thermal_test_temp": 97
```

The guard treats it as real, so warn → stop → lockout → recover → restart all run on demand. Remove the line (or set it to `null`) to return to the real sensor. With the mode at `stop` this really does stop the server and disconnect your listeners, so do it when nobody is on — or run the test at `log`, where it shows what *would* have happened without touching anything.

Troubleshooting

Build Errors

Dependency Issues

```
# Ubuntu: Install missing dependencies
sudo apt install --fix-broken

# Fedora: Install missing dependencies
sudo dnf install <package-name>
```

Meson Configuration Fails

```
# Clean and reconfigure
rm -rf build
meson setup build --buildtype=release
```

`meson setup` dies with `ModuleNotFoundError: No module named 'mesonbuild'`

```
Traceback (most recent call last):
  File "/home/<user>/.local/bin/meson", line 3, in <module>
    from mesonbuild.mesonmain import main
ModuleNotFoundError: No module named 'mesonbuild'
```

Nothing is wrong with the source tree. A leftover `pip install --user meson` left a launcher script in `~/.local/bin`, which comes before `/usr/bin` on `PATH` and therefore hides the working copy the package manager installed. A distribution upgrade (Ubuntu 24.04 → 26.04, for example) moves Python to a new version, the old `site-packages` that held `mesonbuild` is no longer on the import path, and the script dies before it does any work. The same thing can happen to `ninja`.

The installer detects this and works around it for the duration of the run, warning you in the process, but repair the system:

```
rm -f ~/.local/bin/meson
hash -r
meson --version        # must print a version
```

Keep a pip-installed meson instead if you prefer, by reinstalling it for the Python this system has now:

```
python3 -m pip install --user --force-reinstall --break-system-packages meson
```

`meson setup` stops with `Clock skew detected`

```
ERROR: Clock skew detected. File /home/pi/PhantomSDR-Plus/build/../meson.build has a time stamp
10392.2144s in the future.
```

Nothing is wrong with the source tree: the system clock is behind the files. meson and ninja refuse to build when an input is newer than the current time, because they cannot tell which outputs are out of date. It happens on a Raspberry Pi with no cell in its RTC holder — every boot starts from the last time it knew, so a build launched before NTP catches up sees the whole tree dated in the future — and on any tree unpacked or copied from a machine whose clock is ahead.

Correct the clock first:

```
timedatectl                      # is the time right? is NTP synchronised?
sudo timedatectl set-ntp true
```

Wait a few seconds for it to settle, then run the installer again. It checks for this before calling meson and offers to reset the offending timestamps (`PHANTOM_FIX_CLOCK_SKEW=y|n`). By hand, from the source tree, that is:

```
find . -path ./build -prune -o -path ./.git -prune -o -newermt now -print0 | xargs -0r touch
```

The backend built but there is no web page

The symptom is an installation that looks like it mostly worked: `build/spectrumserver` exists, the server starts, and the browser gets nothing — because `frontend/dist/` was never produced. Look back through the installer output for:

```
[vite-plugin-top-level-await] missing field `type`
    at Compiler.printSync (node_modules/@swc/core/index.js:257:29)
error during build:
      Failed: Default
```

`vite-plugin-top-level-await` asks for `@swc/core` `^1.12.14`, so a fresh `npm install` resolves 1.16.0, whose `printSync()` rejects the syntax tree the plugin gives it. It has nothing to do with your distribution — every installer now pins the working version, and so does `frontend/package.json`. A machine installed before that pin was added keeps working until its `node_modules` is deleted, which is why this appears out of nowhere on a reinstall.

If you are repairing an older checkout by hand:

```
cd frontend
npm pkg set 'overrides.@swc/core=1.15.33'
npm install
./build-all.sh
```

Runtime Errors

Port Already in Use

```
# Find process using port
sudo lsof -i :9002

# Kill process
sudo kill -9 <PID>
```

Permission Denied for SDR

```
# Add user to plugdev group
sudo usermod -a -G plugdev $USER

# Reload groups (or log out and back in)
newgrp plugdev
```

For an **RX-888 MkII** this isn't enough on its own — the Cypress FX3 device also needs udev rules. Run the bundled helper (installs the rules *and* adds you to `plugdev`), then log out/in and replug the device:

```
./setup-rx888-udev.sh
```

Audio Issues

```
# Try different codec
# Edit config file and change:
audio_compression = "flac"  # or "opus"

# Rebuild
cd frontend && npm run build && cd ..
```

OpenCL Issues

clinfo Shows No Devices

```
# Verify drivers are loaded
sudo clinfo

# Check OpenCL ICD files
ls /etc/OpenCL/vendors/

# Reinstall drivers (Intel example)
sudo apt remove --purge intel-opencl-icd
sudo apt install intel-opencl-icd
```

Performance Not Improved

```
# Verify OpenCL is enabled in config
accelerator = "opencl"

# Try different device
# Add to config file:
[input.opencl]
device_id = 0  # Try 0, 1, 2, etc.
```

Network Issues

Can't Access from Other Devices

```
# Check firewall
sudo ufw status
sudo ufw allow 9000/tcp

# Or disable firewall temporarily for testing
sudo ufw disable

# Fedora / openSUSE (firewalld)
sudo firewall-cmd --permanent --add-port=9000/tcp
sudo firewall-cmd --add-port=9000/tcp
```

On Fedora and openSUSE the firewall is firewalld, not ufw, and it is on by default. `setup_admin.sh` opens the public proxy port there by itself (and in ufw, if you switched it on); on a station installed before, run `./setup_admin.sh --firewall` once, or use the `firewall-cmd` lines above. If the admin panel or the proxy does not come back after a reboot on these systems, SELinux is the usual cause: run `./setup_admin.sh --selinux` (see the troubleshooting table in ADMIN_PANEL_SETUP.md).

High Latency

```
# Reduce waterfall size in config
waterfall_size = 512

# Increase buffer size
buffer_size = 16384

# Use Opus instead of FLAC
audio_compression = "opus"
```

SDR Device Issues

RTL-SDR Not Found

```
# Check if device is recognized
lsusb | grep Realtek

# Test with rtl_test
rtl_test

# Try blacklisting DVB-T drivers
echo "blacklist dvb_usb_rtl28xxu" | sudo tee /etc/modprobe.d/blacklist-rtl.conf
sudo rmmod dvb_usb_rtl28xxu
```

SDRplay Not Found

`grep "RSP1A driver" logwebsdr.txt` shows which driver `start-rsp1a.sh` picked.

With the open driver (libmirisdr-5, `RSP1A driver: miri`):

```
# The kernel's msi2500 driver must not hold the device
lsmod | grep msi2500 && sudo modprobe -r msi2500 msi001

# Test with SoapySDR
SoapySDRUtil --find="driver=soapyMiri"
```

With SDRplay's API (`RSP1A driver: sdrplay`):

```
# Check API installation
systemctl status sdrplay

# Restart API
sudo systemctl restart sdrplay

# Test with SoapySDR
SoapySDRUtil --find="driver=sdrplay"
```

Fobos or Airspy HF+ Not Found

```
SoapySDRUtil --info | grep -i factories   # must list fobos / airspyhf
SoapySDRUtil --find="driver=fobos"         # or driver=airspyhf
id | grep plugdev                          # log out and back in if missing
```

If the driver is missing from the list, re-run `./setup-fobos.sh` or `./setup-airspyhf.sh`: it stops with the reason if the build or the SoapySDR check fails.

Performance Optimization

CPU Optimization

```
# Adjust thread counts based on CPU cores
[server]
threads = 4  # Set to number of cores

[input]
fft_threads = 4  # Set to number of cores
```

Memory Optimization

```
# Reduce memory usage
[input]
waterfall_size = 512  # Lower value = less memory
fft_size = 65536      # Lower value = less memory
```

Network Optimization

```
# Optimize for limited bandwidth
[input]
audio_compression = "opus"  # More efficient than FLAC
waterfall_compression = "zstd"  # Efficient compression
```

Updating PhantomSDR-Plus

Since version 4.1.0 the repository ships **`update.sh`**, an updater that brings an installed receiver up to date with the published tree **without touching the files that make it your site**. It replaces the hand-written `git pull` script that earlier editions of this guide asked you to create, and it does not need git at all: the published tree is downloaded as a tarball and compared with yours file by file, so it works the same whether you cloned the repository, unpacked an `update.zip`, or copied the tree off a USB stick.

If your installation does not have update.sh yet

An older tree will not contain the script. Fetch it once — it is the only step you ever do by hand:

```
cd ~/PhantomSDR-Plus
curl -fLO https://raw.githubusercontent.com/sv1btl/PhantomSDR-Plus/main/update.sh
```

From then on everything — sources, frontend, documentation, installers, and `update.sh` itself — is brought in by the tool.

Running it

```
cd ~/PhantomSDR-Plus
bash update.sh
```

It downloads the published tree, compares it with yours and prints a report. Then it asks **Update these files now?** — ENTER means *no*, and then nothing at all has been written. Answer `y` and it:

1. stops the receiver, the admin panel, the proxy and the relay **of this installation** (another installation on the same computer keeps running),
2. saves every file it is about to replace in a dated archive in `update-backups/`,
3. writes the new files,
4. rebuilds only what changed — the server, the web page, or both — without asking anything,
5. starts again exactly what it stopped.

Three kinds of file are treated differently:

Files	What happens
`station.conf`, `config.toml`, `config-<receiver>.toml`, `markers.json`, `admin_config.json`, `autorun.json`, `frontend/site_information.json`, `frontend/variant.json`, `frequencylist/`, `chat_history.txt`, the logs, `build/`, `frontend/dist/`	**Never touched**, never even shown in a prompt. These are what make the machine *your* receiver.
`start-*.sh`, `stop-websdr.sh`, the `*.service` units, `install*.sh`, `recompile.sh`, `setup_admin.sh`, `proxy.py`, `admin_server.py`, `frontend/src/bands-config.js` and the other files a sysop has reason to edit	**Updated if you never edited them, asked about if you did.**
Everything else	Updated.

Each release carries `update-known.txt`, the fingerprint of every version of every file ever published. A file of yours that matches one of them is simply older, and it is updated without a question — so even a first update asks only about the files you really changed. For each of those you choose:

```
    start-rx888mk2.sh  [K]eep mine / [u]pstream / [b]oth  (ENTER = Keep mine)
```

- **Keep mine** — your file stays exactly as it is.
- **upstream** — the new version is installed; yours is in the backup.

- **both** — the new version is written beside yours as `start-rx888mk2.sh.new`, to merge in your own time.

Nothing is ever deleted: files that have gone from the repository are reported, and removed only if you ask with `--prune`. The exit status of a run that writes nothing is `0` when you are up to date and `10` when an update is waiting, so a cron job running `bash update.sh --check` can tell you when there is something to do.

Undoing an update

```
./update.sh --restore LAST
```

Every file an update overwrites is saved first, in one dated archive per run in the plain, visible folder `update-backups/`:

```
update-backups/phantomsdr-backup-20260923-164530.zip
```

The archive holds only the files that run replaced, each at the path it has in your tree, so it can be copied to another machine as a single file, opened anywhere, or unpacked straight back over the installation. Where `zip` is not installed it is written as `.tar.gz` instead. The three most recent are kept. Either of these puts back what an update replaced:

```
./update.sh --restore LAST
./update.sh --restore phantomsdr-backup-20260923-164530.zip
```

Each archive also carries its own `restore.sh`, for the case where `update.sh` itself is the problem: unpack the archive and give it your installation directory, for example `./restore.sh /home/pi/PhantomSDR-Plus`. While restoring, `update.sh` runs from a copy outside the tree, because it is usually one of the files being put back.

Backups made before 23 September 2026 sit in the hidden folder `.update-backups/<timestamp>/`; they can still be restored the same way and count towards the three that are kept.

Other options

```
./update.sh --apply --yes     # never asks; every file you edited is KEPT
./update.sh --check           # report only, never ask — for cron (10 = update waiting)
./update.sh --from FILE|DIR   # the new version from a .zip/.tar.gz or a folder — no network
./update.sh --ref v5.5.0      # a tag, branch or commit instead of the current tree
./update.sh --list-excludes   # print the never-touch rules as they resolve here
./update.sh --verbose         # list every file, not only the first 40
```

You can add your own never-touch rules by putting one glob per line in `update-exclude.txt` in the root folder of the installation.

If the rebuild fails on a very old installation

`update.sh` updates files, not system packages. If your tree is old enough that the build now needs libraries you do not have, `recompile.sh` will stop with a compiler or meson error. That is not a broken update — you need the dependencies:

```
./install.sh
```

The installer is itself brought up to date by the same run, and your configuration survives it too.

Without an internet connection on the receiver

Download the whole repository on another computer (GitHub → *Code* → *Download ZIP*), carry it over, and:

```
bash update.sh --from ~/PhantomSDR-Plus-main.zip
```

Everything else is exactly as above — report, question, backup, rebuild, restart.

Updating by hand

If you would rather apply a file drop yourself:

```
cd ~
unzip -o update.zip
cd PhantomSDR-Plus
./stop-websdr.sh
cp -a ~/update/. .
chmod +x *.sh
./recompile.sh
./start-rx888mk2.sh          # or the start script for your receiver
```

Back up your configuration first — `config-*.toml`, `frontend/site_information.json`, `admin_config.json` and `markers.json` — because a file drop cannot tell your edits from the release's. That is exactly the problem `update.sh` exists to solve.

Backup and Restore

Files to Backup

- Configuration files: `*.toml`
- Site information: `frontend/site_information.json`
- Markers: `markers.json`
- Custom scripts: `start-*.sh`, `stop-*.sh`
- Chat history: `chat_history.txt`
- Background image: `frontend/src/assets/background.jpg`

Backup Command

```
cd ~/PhantomSDR-Plus
tar -czf phantomsdr-backup-$(date +%Y%m%d).tar.gz \
  *.toml \
  *.sh \
  markers.json \
  chat_history.txt \
  frontend/site_information.json \
  frontend/src/assets/background.jpg
```

Restore Command

```
tar -xzf phantomsdr-backup-YYYYMMDD.tar.gz
```

Security Considerations

Firewall Configuration

```
# Allow only necessary ports
sudo ufw allow 9000/tcp
sudo ufw enable

# Fedora / openSUSE (firewalld)
sudo firewall-cmd --permanent --add-port=9000/tcp
sudo firewall-cmd --add-port=9000/tcp
```

Reverse Proxy (Optional)

Consider using nginx or Apache as a reverse proxy for:

- SSL/TLS encryption
- Domain name mapping
- Load balancing
- Access control

User Limits

Edit `config.toml`:

```
[server]
max_users = 100  # Limit concurrent users
```

Getting Help

Resources

- **Documentation**: This guide, README.md, USER_GUIDE.md
- **GitHub Issues**: https://github.com/sv1btl/PhantomSDR-Plus/issues
- **Live Demo**: http://phantomsdr.no-ip.org:8900/

Reporting Issues

When reporting issues, include:

1. Operating system and version
2. SDR device model
3. Configuration file contents
4. Error messages
5. System resource usage (CPU, RAM, GPU)

Community Support

- Check existing GitHub issues before creating new ones
- Provide detailed information about your setup
- Include logs and error messages
- Be patient and respectful

Appendix A: Complete Dependency List

Ubuntu 24.04 Package List

```
build-essential
cmake
pkg-config
meson
libfftw3-dev
libwebsocketpp-dev
libflac++-dev
zlib1g-dev
libzstd-dev
libboost-all-dev
libopus-dev
libliquid-dev
git
util-linux (taskset — for the autorun spot reporter)
psmisc
wget
curl
rtl-sdr (for RTL-SDR)
airspy (for Airspy)
hackrf (for HackRF)
libclfft-dev (for OpenCL)
ocl-icd-opencl-dev (for OpenCL)
clinfo (for OpenCL)
```

Appendix B: Configuration Examples

Example 1: RTL-SDR for VHF/UHF

```
[input]
sps = 2048000
frequency = 145000000
signal = "iq"

[input.driver]
format = "u8"

[input.defaults]
frequency = 145500000
modulation = "FM"
```

Example 2: RX-888 mk2 for HF (0-30 MHz)

The receiver most sysops run. This is a complete `[input]` section rather than a fragment, with the values from the `config-rx888mk2.toml` that ships with the repository.

```
[input]
sps = 60000000            # 0-30 MHz by direct sampling
fft_size = 4194304        # see the note below
fft_threads = 8
brightness_offset = -9    # more negative if the waterfall shows black patches
frequency = 0             # baseband: the RX-888 samples from DC
signal = "real"           # not "iq" - direct sampling gives a real stream
accelerator = "opencl"    # "none" if there is no OpenCL runtime
audio_sps = 12000
audio_compression = "flac"
waterfall_size = 1024
waterfall_compression = "zstd"
smeter_offset = 5
analog_smeter_offset = 5

[input.driver]
name = "stdin"
format = "s16"

[input.defaults]
frequency = 7120000
modulation = "LSB"
```

Feed it from `rx888_stream`, which `start-rx888mk2.sh` does for you:

```
rx888_stream -f /path/to/SDDC_FX3.img -s 60000000 -g 50 -a 0 -m low --pga -d -r -o - \
  | build/spectrumserver --config config-rx888mk2.toml
```

On `fft_size`: 4194304 is the size to use at 60 MSPS. 8388608 doubles the waterfall resolution and doubles the memory and CPU cost of every transform with it, which on most machines buys sharper bins at the price of dropped frames. Start at 4194304 and only go higher if the server is comfortably idle.

Example 3: HackRF for Wideband FM

```
[input]
sps = 10000000
frequency = 100900000
signal = "iq"

[input.driver]
format = "s8"

[input.defaults]
frequency = 100900000
modulation = "WBFM"
```

Installation complete! You should now have a fully functional PhantomSDR-Plus server.

73 de SV1BTL, 9A7AOF, F1NSK & SV2AMK

CHAPTER 3

Admin Panel Setup

ADMIN_PANEL_SETUP.md

PhantomSDR-Plus Admin Panel — Setup Guide

> ⚠**Security notice:** Keep this admin panel on your home network or behind a VPN. Public exposure of the admin port is not recommended without proper authentication hardening.

Overview

The admin panel consists of two Python services:

Service	File	Role
Admin panel	`admin_server.py`	Flask web app, binds to `127.0.0.1` (internal only)
Reverse proxy	`proxy.py`	Exposes both the SDR and the admin panel on a single public port

Both services read their configuration from `admin_config.json`, which is written by `setup_admin.sh`. **Do not edit port constants directly in the Python files** — all ports and the SDR host IP are stored in `admin_config.json`.

What the admin panel gives you

- **Dashboard** — live server status, CPU/RAM, top processes, recent log output, terminal
- **Config Editor** — view, edit and save any `.toml`, `.sh`, `.json`, `.h`, `.cpp` file in your installation
- **Log Viewer** — tail any log file in real time (SDR server, admin panel, RADE, crash, autorun, proxy)
- **Markers** — view and edit frequency markers
- **Chat History** — view the WebSDR chat log, clear it entirely, or delete individual messages without restarting the server
- **Waterfall Message** — broadcast a persistent banner message to all connected users, visible on the waterfall display in real time
- **Users** — live list of connected listeners with their name or callsign, location and IP address, tuned frequency, mode and duration, a Kick button and a Ban ▾ menu (8, 16 or 24 hours), and a map of where they are
- **Chat message deletion** — Delete button that removes that single message from the log immediately
- **Waterfall broadcast messages** — lets you push a persistent text banner to every connected user's waterfall
- **Spot Reporting** — start/stop the autorun FT8/FT4/JS8/WSPR decoder, pick bands/modes, and report spots to PSK Reporter / wsprnet (OFF by default), with two counters: per-decoder tiles for the current run and an all-time total beside each band/mode checkbox
- **Graphs** — CPU frequency, CPU load, CPU temperature and connected users plotted over the last 15 minutes / 1 hour / 4 hours / 12 hours / 24 hours

- **Thermal Guard** — stops the server if the CPU overheats and restarts it once cool, with thresholds derived from your own CPU; works with any start/stop method, and ships in log-only mode so it does nothing until you enable it
- **Settings** — change admin password, SDR base directory, process name and the spectrumserver port (with the listener port shown under it); run the default start or stop script with ▶ START / ■ STOP; an **HTTPS** card that checks https, or guides you through turning it on
- **Receivers** — every receiver of the station with the range it covers, its bandwidth, its gain and whether it is on air; change a receiver's settings and gain, stop or start, rename or delete one receiver, and prepare a new receiver for `add-receiver.sh`
- **System Health** (Dashboard) — every receiver on air or off with its uptime, free disk space and the size of each log folder, the last `crash.log` entry and the recent alerts
- **DRM** (Dashboard) — the DRM decoder: whether its helper runs, how many decoders are in use, and who is decoding which frequency (station, codec, WMER); switch DRM on or off and set the maximum number of DRM listeners (1–20), applied within 5 s without a restart. The Users page shows those listeners as DRM
- **Receive Diversity** (Dashboard) — every listener combining your receiver with another one by receive diversity: name and city, frequency and mode, the partner (PhantomSDR+, KiwiSDR, UberSDR or WebSDR) and its address. Only the admin panel sees it, never the public Users list
- **Visitors** — private visitor statistics for 1 to 31 days: visitors, sessions, listening hours, most at the same time, per day, busiest hours, countries, and the addresses with the most sessions at once, with Ban ▾
- **Alerts** — an email and/or a Telegram message when a receiver goes off the air, comes back or is restarted, the thermal guard changes stage, or `crash.log` gets a new entry
- **Backup & restore** — one zip with the station's own settings, and a restore that copies every file it replaces to `admin-backups/` first
- **Themes** — Phosphor green, Amber display, Metal panel, Modern dark, Light, and Auto (light by day, dark at night), chosen with **THEME** in the top bar of every page

Requirements

- PhantomSDR-Plus already installed and running
- Python 3.8+

> **NOTE**
>
> **The installer normally does this for you.** `./install.sh` — and each of the four distro installers, both RADE installers and the stats-server installer — sets the admin panel up by default, installing `pip` first with the system's own package manager if it is missing. This page is what happens during that step, and what to do if you skipped it or want to change something afterwards.

The panel's Python libraries (`flask`, `psutil`, `aiohttp`, `tomli-w`) are installed by `setup_admin.sh`; it prints the exact command to run by hand if that ever fails.

Step 1 — Files

All four files ship with the repository, so there is nothing to copy:

```
admin_server.py
manage_admin.sh
setup_admin.sh
proxy.py
```

The installer sets the executable bits itself. On a tree copied by hand, run the script as `bash setup_admin.sh`.

Step 2 — Run the setup script

```
./setup_admin.sh
```

> **Without the admin panel:** `bash setup_admin.sh --proxy-only` installs only `proxy.py` on the public port, with the ports from `station.conf`. It is what the installer does when you decline the panel, because the proxy is what listeners — and `/rade`, `/stats`, `/relay` — reach.

The script will:

1. Check that Python 3 is installed
2. Verify `admin_server.py` and `manage_admin.sh` are present
3. Record `127.0.0.1` as `sdr_host` in config (see The `sdr_host` setting)
4. Ask for three port numbers. Each offers a default in brackets that a bare Enter accepts, so a normal setup is three keystrokes:
5. **Spectrumserver port** — the port your SDR server listens on (default `8900`, which is what every `config-*.toml` in the repository ships with)
6. **Admin panel internal port** — where `admin_server.py` binds locally (default `3000`)
7. **Proxy public port** — the single external port that combines SDR + admin (default `8902`) On a station set up with the station questions (`configure-station.sh`), the three defaults are that station's own ports from `station.conf` — 9001, 9010 and 9000 — so Enter is right, and the start script is already preselected.

Invalid answers are rejected and re-asked, but only five times — after that the default is used. A run whose input is not a terminal (piped, cron, unattended) takes the defaults straight away instead of waiting for input that never comes.

1. Install `flask`, `psutil`, `aiohttp` and `tomli-w` via pip
2. Grant `ss` the `cap_net_admin` capability (fallback path of Kick Users, used only when the panel runs without the proxy)
3. Ask which script starts and which stops the receiver — the panel drives the server through these, and so does the thermal guard
4. Ask how far the CPU over-temperature guard may act on its own — `log`, `throttle`, `stop` or `stop+restart` — and offer to run `setup-cpufreq-perms.sh` so the throttle stage works without root (see THERMAL_GUARD.md)
5. Write `admin_config.json` with all settings
6. Offer to install two systemd units (panel + proxy) — auto-start on boot and restart after a crash. **Recommended, and the default** (a bare Enter accepts it): the CPU over-temperature guard only runs while the panel runs, so without the units a reboot leaves the machine unprotected. The step is skipped automatically where systemd is not PID 1 (containers, WSL1, OpenRC), or where you have no sudo.

After setup, open your browser via the proxy:

```
http://YOUR_SERVER_IP:<proxy_port>/admin
```

Default password: **`admin`**

> ⚠**Change the password immediately** — on first login a one-step wizard asks only for a new password. The folder and the ports are already in `admin_config.json` (written by `setup_admin.sh`, which the installer runs); the wizard shows them for reference and leaves them untouched. Change them later in Settings, or with `./configure-station.sh`.

Step 3 — Start / Stop / Restart

There are two ways to run the panel and the proxy. **Pick one and stay with it** — the warning at the end of this section explains what happens if you mix them.

> **Method B (systemd) is strongly recommended** wherever systemd is available. The CPU over-temperature guard runs inside the panel, so a panel that dies at 03:00 or a machine that reboots leaves the CPU unprotected until someone starts it again by hand. `setup_admin.sh` now offers the units by default. Use Method A when there is no systemd — containers, WSL1, OpenRC/sysvinit — or when you deliberately want to supervise things yourself.

Method A — `manage_admin.sh` (no root, nothing installed)

```
./manage_admin.sh start      # start admin panel and proxy
./manage_admin.sh stop       # stop both
./manage_admin.sh status     # show running status and PIDs
```

There is deliberately **no `restart`**. It used to kill the panel and immediately start its own copy, which collides with systemd on any machine using Method B. To restart under Method A, run `stop` then `start` — see Restarting the panel below.

One script controls **both** processes. It checks that `aiohttp` is installed before starting the proxy and prints a clear error with the fix command if it is missing.

Nothing starts by itself: after a reboot, or if the panel crashes, you start it again by hand. That is fine for a receiver you watch, but note that the CPU over-temperature guard runs inside the panel — while the panel is down, nothing is protecting the machine (see THERMAL_GUARD.md).

Method B — systemd units (starts at boot, restarts after a crash)

The repository ships two ready units, `phantomsdr-admin.service` and `phantomsdr-proxy.service`. Edit `User=` and the paths in both, then:

```
sudo cp phantomsdr-admin.service phantomsdr-proxy.service /etc/systemd/system/
sudo systemctl daemon-reload
./manage_admin.sh stop        # free the ports first
sudo systemctl enable --now phantomsdr-admin
sudo systemctl enable --now phantomsdr-proxy
```

Day-to-day commands then become:

```
sudo systemctl start   phantomsdr-admin
sudo systemctl stop    phantomsdr-admin
sudo systemctl restart phantomsdr-admin
systemctl status       phantomsdr-admin   # no sudo needed
sudo journalctl -u phantomsdr-admin -f    # live logs
```

The same four with `phantomsdr-proxy`, or both at once: `sudo systemctl restart phantomsdr-admin phantomsdr-proxy`.

`admin.log` and `proxy.log` keep working exactly as before — the units append to those same files.

One thing to do once: install the tmpfiles rule at `tmpfiles/phantomsdr-logs.conf` (edit its paths and user first). systemd creates those two logs as `root` before it drops to `User=`, and the panel's **Clear Logs** button then fails on them with `Permission denied` — see “Clear Logs” says Permission denied below. `setup_admin.sh` installs the rule for you.

The admin unit carries `SupplementaryGroups=cpufreq`, which is what lets the thermal guard's throttle stage lower the CPU clock without the panel running as root. Create that group first with `./setup-cpufreq-perms.sh`, or systemd will refuse to start the unit; delete the line if you do not use the throttle stage.

To go back to Method A:

```
sudo systemctl disable --now phantomsdr-admin
sudo systemctl disable --now phantomsdr-proxy
```

⚠**Do not mix the two.** With the units enabled, `./manage_admin.sh stop` is undone by systemd five seconds later, and `./manage_admin.sh start` gives you a second, unmanaged copy fighting the systemd one over the same port. (`restart` was removed from the script for exactly this reason; it now prints an error telling you to use `systemctl`.) `./manage_admin.sh status` only reads, so it stays useful either way.

Restarting the panel

The rest of this document says "restart the panel" in a number of places. It means whichever of these matches your method:

```
# Method B — systemd units (the normal setup)
sudo systemctl restart phantomsdr-admin phantomsdr-proxy

# Method A — no units installed
./manage_admin.sh stop && ./manage_admin.sh start
```

The unit ships with `KillMode=process`, and that line is load-bearing. If you start the SDR from the panel, the receiver is a child of the admin unit and inherits its cgroup. Under systemd's default `KillMode=control-group`, restarting the panel would take spectrumserver, its watchdog and the autorun daemon down with it, and stall for the full 90-second stop timeout first. With `KillMode=process` systemd signals only the panel, so `sudo systemctl restart phantomsdr-admin` leaves a busy receiver on air.

Upgrading a unit installed before v4.1.0. Older unit files have no such line, and restarting the panel will not add one: `systemctl restart` re-runs the program, it does not change the program's configuration. Edit the *installed* copy — the one in the repository is only a template systemd never reads:

```
sudo nano /etc/systemd/system/phantomsdr-admin.service
```

Add `KillMode=process` in the `[Service]` section, next to `Restart=always`. Then make systemd re-read the file and confirm:

```
sudo systemctl daemon-reload
systemctl show phantomsdr-admin -p KillMode
```

The second command must print `KillMode=process`. If it still prints `control-group`, systemd is running its cached copy — `systemctl show phantomsdr-admin -p NeedDaemonReload` prints `yes` while a reload is pending. Leading whitespace in unit files is ignored, so indentation does not matter.

Step 4 — Open firewall port

Open only the **proxy public port** in your firewall. There is no need to expose the admin panel internal port externally:

```
sudo ufw allow <proxy_port>
```

Step 5 — Kick Users feature

The Users page shows every currently connected listener. Each row shows the name or callsign the listener typed (`--` when none), the location with the IP address under it, the tuned frequency, mode and how long they have been connected. To disconnect one user, click the **Kick** button on their row — only that listener's connections are closed. All other listeners stay connected and hear nothing.

Next to the mode the Users page shows the decoder a listener is running in their page, for example `USB · FT8` (DRM listeners read `DRM`). Pages opened before the server was updated report it only after they are reloaded.

The IP addresses are on this page only. Since 5.3.0 the public users list (`/users`) and the User Statistics carry none, as on a WebSDR; the panel reads its list from `/~~users`, which spectrumserver answers only on its own computer and never through the proxy.

The kick is instant and surgical: the server does not restart, no audio is interrupted for anyone else, and the kicked user can reconnect immediately (the feature is for removing misbehaving or stuck connections; to keep someone out, use the ban below).

Ban ▾ opens a small menu — **8**, **16** or **24 hours** — and the chosen length disconnects the listener from every receiver of the station and refuses that address for that long (24 hours is the most the receiver accepts) (close code 4001, which the page does not retry). The ban is kept in `admin_bans.json` next to the panel and sent to every receiver again once a minute, so a receiver restart does not lift it. While bans are in force, a **Banned** list under the table shows each one with its name, address and end time, and an **Unban** button lifts it at once. Everyone behind the same internet connection shares the address and is banned with it.

Under the hood the kick uses `ss -K dst <IP> dport <port>` to tear down the specific TCP socket. This requires a Linux capability that the setup script grants automatically. If you skipped setup or the button reports an error, apply it manually:

```
sudo setcap cap_net_admin+ep $(which ss)
# Verify:
getcap $(which ss)    # should show: cap_net_admin=ep
```

Configuration — admin_config.json

All runtime configuration lives in `admin_config.json` in your PhantomSDR-Plus directory. `setup_admin.sh` writes the initial file; subsequent changes can be made via the Settings page or by editing the file directly.

Key fields written by `setup_admin.sh`:

Key	Description
`port`	Admin panel internal port (`admin_server.py` binds here)
`public_port`	Spectrumserver port of the main receiver (8901 behind proxy.py) — **not** the port listeners use, that is `proxy_port`. proxy.py forwards to it; Kick, the Users page and the user count query it
`proxy_port`	Public port the proxy listens on
`sdr_host`	Host the proxy uses to reach spectrumserver — `127.0.0.1` (see below)
`password_hash`	SHA-256 hash of the admin password
`sdr_base_dir`	Path to your PhantomSDR-Plus installation
`sdr_process_name`	Process name to monitor (default: `spectrumserver`)
`start_script` / `stop_script`	Scripts the panel runs to start and stop the SDR server — also what the thermal guard acts through

> **Several receivers on one computer?** Set `start_script` to `start-all.sh`. The stop script already stops every receiver, and Restart and the thermal guard use the start script, so with `start-all.sh` they bring back every receiver in `receivers.toml` instead of only the main one. The Users page, the user count and the kick then cover every receiver as well. See Several Receivers.

Thermal guard keys (all optional — the panel writes them when you save the Thermal Guard settings, and any that are missing fall back to these defaults):

Key	Default	Description
`thermal_enabled`	`true`	Master switch for the guard
`thermal_mode`	`"log"`	`log` (record only, never act) · `throttle` · `stop` · `stop+restart`
`thermal_warn`	`null`	Warn temperature °C; `null` = derive from your CPU's critical trip point
`thermal_throttle`	`null`	Temperature at which the CPU clock ceiling is lowered; `null` = auto
`thermal_stop`	`null`	Temperature at which the server is stopped; `null` = auto
`thermal_resume`	`null`	Temperature the CPU must fall below to recover; `null` = auto
`thermal_sustain_s`	`60`	Seconds the stop temperature must be held before acting
`thermal_warn_sustain_s`	`30`	Seconds the warn/throttle temperatures must be held
`thermal_resume_s`	`300`	Seconds below the resume temperature before the lockout is released
`thermal_max_stops_hour`	`2`	Thermal stops per hour before the automatic restart is disabled
`thermal_test_temp`	`null`	Inject a fake temperature to test the guard; `null` = use the real sensor

Changes to these take effect on the next sample — within a couple of seconds, with no restart.

To change ports after initial setup, edit `admin_config.json` and restart:

```
sudo systemctl restart phantomsdr-admin phantomsdr-proxy   # Method B
./manage_admin.sh stop && ./manage_admin.sh start        # Method A
```

The `sdr_host` setting

`proxy.py` connects to `spectrumserver` over loopback — `sdr_host` is `127.0.0.1` in `admin_config.json`, written there by `setup_admin.sh`. You should not need to change it.

Set it to a real address only if `spectrumserver` runs on a **different machine** than the proxy. Then restart the panel.

> **Changed:** earlier versions required `sdr_host` to be the machine's LAN IP, because `spectrumserver` closed WebSockets arriving from the loopback address. That filter has been removed, so local connections work normally. Two consequences: browsing `http://localhost:<port>` on the server machine itself now shows the full GUI (it used to load the page but stay blank), and the proxy no longer breaks when DHCP reassigns the machine's IP. If you are upgrading and your `admin_config.json` still holds a LAN IP, change it to `127.0.0.1` or re-run `setup_admin.sh`.

proxy.py — what it is and when you need it

`proxy.py` puts both the SDR server and the admin panel on a **single public port**, routing by path prefix:

Path	Routed to
`/admin*`	Admin panel (`localhost:<port>`)
Everything else	Spectrumserver (`<sdr_host>:<public_port>`)

Without the proxy you must expose two ports separately. With the proxy you expose only one.

The proxy also removes the 4 MB WebSocket message-size cap (important for large FFT frames from the RX-888 at 60 MSPS), forwards real client IPs via `X-Forwarded-*` headers, and keeps long-lived connections alive through NAT with a 30-second heartbeat.

Changing ports after setup

Edit `admin_config.json` directly:

```
{
  "port":        3000,
  "public_port": 9001,
  "proxy_port":  9002,
  "sdr_host":    "127.0.0.1"
}
```

Then restart both services:

```
sudo systemctl restart phantomsdr-admin phantomsdr-proxy   # Method B
./manage_admin.sh stop && ./manage_admin.sh start        # Method A
```

If you also need `admin_server.py` to bind on a different port at launch (e.g. the systemd service), you can override with the environment variable:

```
ADMIN_PORT=3000 python3 admin_server.py
```

By default `admin_server.py` binds to `127.0.0.1` only. To run it standalone without the proxy (development/testing), bind to all interfaces:

```
ADMIN_BIND=0.0.0.0 python3 admin_server.py
```

Useful manual commands

```
# Start admin panel manually (foreground)
python3 ~/PhantomSDR-Plus/admin_server.py

# Kill the admin panel
pkill -f admin_server.py

# Kill the proxy
pkill -f proxy.py

# Free a port that is stuck in use
sudo fuser -k 3000/tcp

# Check what is listening on a port
ss -tlnp | grep 3000
```

Log files

File	Contents
`admin.log`	Admin panel output
`proxy.log`	Proxy startup banner + one access-log line per proxied request
`logwebsdr.txt`	Launcher and watchdog log — start/stop, receiver driver, websdr.org registration
`spectrumserver.log`	The receiver process's own output — FFT and OpenCL setup, per-client connections, errors. The biggest of these files; it is rotated, so `spectrumserver.log.1` may sit beside it
`rade.log`	RADE/FreeDV sidecar log
`autorun.log`	Decoder daemon log — spots, SNR, drift
`crash.log`	Crash reports and thermal events: `[CRASH]` / `[TERMINATE]` backtraces, `[EXIT]` notes and `[THERMAL]` lines from the guard. **Normally empty** — anything in it is worth reading

All of these are readable from the panel's **Log Viewer** tabs (LOGWEBSDR, SPECTRUMSERVER, ADMIN, RADE, CRASH, AUTORUN, PROXY). The viewer shows the last 150 lines of whichever tab is selected and refreshes every 3 seconds when auto-refresh is on; it reads only the tail, so the size of the file does not matter.

Rotating proxy.log and admin.log

`proxy.log` records every proxied request and `admin.log` every request that reaches the panel itself. Neither is rotated by default. The dashboard's polling used to dominate both — `/admin/api/status` fires every ~3 seconds for as long as a tab is open — but those polls are now filtered out of both files (see below), so rotation is there to bound slow growth rather than a flood.

The repo ships a logrotate config at `logrotate/phantomsdr` (daily, or sooner if a log passes 10 MB; keeps 7 gzipped archives in `logproxy/`, so they do not clutter the project root). **It is site-specific — open it and change the three log paths, the `olddir` path and the `su` user to match your machine before installing**, then:

```
sudo cp logrotate/phantomsdr /etc/logrotate.d/phantomsdr
sudo logrotate -d /etc/logrotate.d/phantomsdr     # dry run, should list all three logs
```

`logrotate/phantomsdr` in the repo and `/etc/logrotate.d/phantomsdr` are two **independent copies** — `cp` does not link them. Editing the repo file changes nothing on a running system until you re-run the `sudo cp` above. Only the `/etc` copy is what logrotate actually reads.

The config uses `copytruncate`, which is required: `manage_admin.sh` starts both processes with `>> <log>` and neither reopens its stdout, so a rename-based rotation would leave them writing into the rotated file while the new log stays empty forever.

Archives land in `logproxy/` (created automatically by `createolddir`) and are named `proxy.log.1.gz` … `proxy.log.7.gz`, newest first. Read one with `zcat logproxy/proxy.log.1.gz | less`, or search all of them at once with `zgrep "pattern" logproxy/*.gz`.

The panel's **Clear Logs** button truncates **every** file in the table above, `autorun.log` and `proxy.log` included (`CLEAR_EXCLUDE` in `admin_server.py` is empty). Note what that costs on `autorun.log`: its decode history is the only record of what was heard and when, and nothing can reconstruct it — what survives a clear is whatever logrotate has already rotated into `logproxy/`, so a clear discards everything decoded since the last rotation. Clearing it under a running decoder is safe: the daemon holds it open `O_APPEND` and goes on appending to the same inode. The toast names what was cleared, and if a file could not be written — a `proxy.log` owned by a different user, say, on a split-user install — it names that file and the error instead of reporting success, and leaves that log's pane on screen rather than blanking it. The files are truncated in place, never deleted: both `manage_admin.sh` and the systemd units open them `O_APPEND`, so an unlinked file would go on filling an invisible inode until the service restarts.

Successful polls are filtered out of **both** access logs, by the `QuietPollFilter` in `proxy.py` and the one in `admin_server.py`. The dashboard hits `/admin/api/status` every 3 seconds, and `/admin/api/logs`, `/admin/api/autorun/status`, `/admin/api/users`, `/admin/api/graph-stats` and `/admin/api/chat` every 5 seconds; unfiltered they are ~95% of both files. Only plain `200`s on those paths are dropped — any other status, path or method is logged as before, so a failing poll stays visible and state-changing actions (`kick`, `autorun/start`, …) are separate paths and always recorded. The one deliberate exception is `/admin/api/logs/clear`, filtered in both files: its own access line is written *after* the truncate, so without the filter every successful clear left one fresh line behind in the log it had just emptied, and the button looked broken.

Because `proxy.log` is an access log, it contains visitor IP addresses and user-agent strings — keep that in mind before sharing it when asking for help.

"Clear Logs" says Permission denied

On a machine where the systemd units were installed on a clean tree, the button can come back with:

```
✗ Could not clear admin.log ([Errno 13] Permission denied: /home/you/PhantomSDR-Plus/admin.log),
  proxy.log ([Errno 13] Permission denied: /home/you/PhantomSDR-Plus/proxy.log)
```

It is always those two files and never the other five, because they are the only ones systemd creates itself. `StandardOutput=append:` is opened by PID 1 **before** it drops to `User=`, so a log that does not exist yet is created `root:root 0644`. The panel runs as your user and cannot truncate it. The other five logs are created by the launcher scripts, which already run as your user, so they clear normally. Where the two logs predate the units — `manage_admin.sh` created them with `>>` — they are already owned correctly and the button works, which is why the problem only shows up on a fresh systemd-first install.

Confirm with `ls -l admin.log proxy.log`: the failing ones say `root root`. Repair them:

```
sudo chown "$(id -un):$(id -gn)" admin.log proxy.log
sudo chmod 664 admin.log proxy.log
```

No restart is needed — systemd keeps writing through the file descriptor it already holds, and appending does not change ownership. Press **Clear Logs** again and both are emptied.

Do not delete the files instead. Both are held open `O_APPEND`; unlinking one leaves systemd filling an invisible inode until the service restarts, and the file that reappears is root-owned again.

To stop it coming back, install the tmpfiles rule the repo ships at `tmpfiles/phantomsdr-logs.conf`. It re-creates both logs with the right owner at every boot, before the units start, so a removed log — manual cleanup, a reinstall, a moved installation directory — never returns root-owned. **It is site-specific: open it and change the two paths and the user:group to match your machine first**, then:

```
sudo cp tmpfiles/phantomsdr-logs.conf /etc/tmpfiles.d/99-phantomsdr-logs.conf
sudo systemd-tmpfiles --create /etc/tmpfiles.d/99-phantomsdr-logs.conf
```

As with `logrotate/phantomsdr`, the repo file and the `/etc` copy are independent — editing the repo copy changes nothing until you re-run the `cp`. On systemd 254 and newer the rule also repairs the ownership and mode of a log that already exists, so it fixes the current failure as well as future ones; on older versions the `chown` above is what unblocks the button.

`setup_admin.sh` does all of this for you when it installs the units, so a setup made with the script never hits this. The steps above are for a hand-installed pair, or for an installation that predates the script's log-ownership step.

Logrotate does not reintroduce the problem: the shipped config uses `copytruncate`, which keeps the same inode and therefore the same owner.

Access summary

Setup	SDR	Admin panel
Without proxy	`http://YOUR_IP:<public_port>`	`http://YOUR_IP:<port>/admin`
With proxy	`http://YOUR_IP:<proxy_port>`	`http://YOUR_IP:<proxy_port>/admin`

Chat message deletion

The Chat History page lists every message in the current chat log. Each entry has a **Delete** button that removes that single message from the log immediately — no server restart required.

How it works:

- The admin panel rewrites the chat log file in place, stripping only the selected message line.

- The change takes effect for any user who reloads the chat; already-loaded chat history in open browser tabs is not retroactively updated.
- Clearing the entire log (the **Clear All** button) truncates the file, which also takes effect without a restart.
- A **chat reply** is stored as an ordinary message that begins with a marker naming the message it answers, for example `[RE:2026-09-25 12:00:01|SV1BTL]`, so in this list it shows with that marker in front. Deleting a reply removes only that reply. Deleting the message it answers leaves its replies in place, showing only the name and time they answer.

The delete button is rendered consistently across all five Svelte App variants. If a deletion appears not to take effect, confirm the admin panel has write permission to the chat log file:

```
ls -l ~/PhantomSDR-Plus/chat.jsonl   # path depends on your config
```

Waterfall broadcast messages

The **Waterfall Message** panel lets you push a persistent text banner to every connected user's waterfall display without touching the server process.

Sending a message

1. Open the admin panel and go to **Waterfall Message**.
2. Type your message text and choose a colour (hex, e.g. `#ffdd00`).
3. Click **Send** — the message appears on all active waterfall views immediately.

Clearing the message

Click **Clear** to remove the banner from all waterfalls. The message state is held in memory by the admin panel; it is cleared automatically if the admin panel process is restarted.

How it works

The admin panel exposes two internal endpoints that the proxy forwards:

Endpoint	Method	Purpose
`/admin/api/waterfall-message`	`POST`	Set or clear the current banner text and colour
`/admin/api/waterfall-message`	`GET`	Return the current message state as JSON

The frontend polls the message state and renders it as an overlay on the waterfall canvas. No WebSocket reconnection or page reload is needed on the client side.

Typical uses

- Announce scheduled maintenance: `"Server restart in 10 minutes"`
- Flag band conditions: `"Solar flux 180 — 10m wide open"`
- Welcome message: `"Welcome to SV1BTL WebSDR — Athens, KM17"`

System Graphs

The **Graphs** page plots four measures against one shared time axis:

- **CPU frequency** — average clock across all cores, in GHz
- **CPU load** — total utilisation, in percent
- **CPU temperature** — in °C, with dashed guides at 70 °C (warm) and 80 °C (critical)
- **Users online** — listeners connected to the WebSDR at that moment

Pick a range with **15 MIN / 1 HOUR / 4 HOURS / 12 HOURS / 24 HOURS**. Hovering the plot draws a crosshair and reads out all four values at that instant. The four cards above the plot always show the newest sample.

How sampling works

A background thread in `admin_server.py` takes one sample every **2 seconds**. Samples are held in two tiers: the last **1 hour** at full 2-second resolution (1800 points), and **24 hours** of 30-second averages (2880 points) for the longer ranges. The page picks the tier that covers the range you chose, and says `30s averages` in the header when you are looking at the aggregated one.

Storing a full day at 2-second resolution would mean ~43000 points — tens of MB of memory, a multi-megabyte first load, and about 40 points per pixel of canvas, which no screen can render. Averaging to 30 seconds keeps a day in 2880 points.

Sampling starts together with the admin panel, not when you first open the page, so the page opens onto history that already exists.

The buffer is **memory-only**: nothing is written to disk, and the history is lost when the admin panel restarts. That is deliberate — it keeps the feature out of log rotation and off your disk.

2 seconds gives a near-live view and is short enough not to alias CPU frequency, which on a modern CPU swings between its idle and turbo clock from one sample to the next. A sample costs about a millisecond, so the sampler is negligible next to the SDR server. The page polls at the tier's own cadence — every 2 seconds on the live ranges, every 30 seconds on the aggregated ones, because asking more often than points are produced only returns empty replies.

To change the cadence or the retention, edit these constants near the top of the sampler block in `admin_server.py` and restart:

```
GRAPH_INTERVAL_S   = 2          # seconds between fine samples
GRAPH_FINE_S       = 3600       # how far back the fine tier reaches (1h)
GRAPH_COARSE_EVERY = 15         # fine samples per coarse point (15 x 2s = 30s)
GRAPH_COARSE_S     = 24 * 3600  # how far back the coarse tier reaches (24h)
```

Where the numbers come from

Measure	Source
CPU frequency	`psutil.cpu_freq()`, falling back to `/sys/devices/system/cpu/cpu*/cpufreq/`. This is the same kernel data `cpufreq-info` formats, so **cpufrequtils is not required**.
CPU load	`psutil.cpu_percent()`
CPU temperature	`psutil.sensors_temperatures()`, falling back to `/sys/class/thermal`, then the `sensors` command
Users online	spectrumserver's own `/users` endpoint — the authoritative session list. Counting sockets with `ss` cannot see real client IPs behind proxy.py.

If a measure is unavailable on your machine, that panel says so rather than drawing a flat zero line. A reading that fails mid-run breaks the line, so a gap in the data never looks like a measured value.

Why four panels and not one graph

GHz, percent, degrees and a head-count share no common scale. Drawing them on one plot needs several y-axes, and the crossings that produces are artefacts of the scaling rather than facts about the server. Four stacked panels on one time axis keep the comparison honest. Every panel is anchored at zero, so curve height stays proportional to the value.

Colour is reserved for temperature, where amber and red mark the thresholds above. The other panels share one colour because each holds a single series that its own title already names.

Endpoint

`GET /admin/api/graph-stats?since=<epoch>` (login required) returns the samples newer than `since`, so the page polls incrementally instead of re-downloading the whole buffer every tick.

Thermal Guard

> **Full manual:** **THERMAL_GUARD.md** — the four modes and what a sysop must do for each, running it without the panel, enabling the throttle stage without root, testing and troubleshooting. This section covers the panel side.

Stops the SDR server when the CPU gets dangerously hot, and starts it again once it has cooled. It is part of the panel — nothing extra to install — and appears as a **THERMAL GUARD** card on the Dashboard with its settings under **Settings**.

It ships **inert**: the mode starts at `log`, so out of the box it only records what it *would* have done. Nothing touches your server until you change the mode.

It works with any start/stop method

The guard never tries to work out what supervises your server. It acts through whatever scripts you set as **Default start script** and **Default stop script** in Settings, and while the CPU is over-temp it re-issues the stop on **every check** (every 2 s). Anything that brings the server back — the watchdog inside `start-*.sh`, a systemd unit, a cron job, a `tmux` session — is undone within a couple of seconds until the machine has cooled. That is the *lockout*.

If no stop script is configured, it falls back to `SIGTERM` to the process named in `sdr_process_name`, then `SIGKILL` after a 10-second grace period.

Thresholds come from your own CPU

Rather than a fixed number that is wrong on most hardware, the guard reads the critical trip point the kernel publishes for your CPU and works backwards from it:

Your CPU reports	warn	throttle	stop	resume
Intel, limit 100 °C	88	92	**95**	75
AMD Ryzen, limit 95 °C	83	87	**90**	70
Raspberry Pi, limit 85 °C	73	77	**80**	60

This matters: a fixed 95 °C would be near-normal on an Intel, but on a Ryzen — whose Tctl sits at 95 °C by design under boost — it would stop the server on a healthy machine, and on a Pi, which throttles at

80 °C, it would never fire at all. Any threshold can still be set by hand in Settings; leave a field blank for *auto*.

To see what your machine looks like to the guard:

```
python3 thermal_guard.py --once
```

```
sensor     : hwmon:coretemp
temperature: 58.0 C
crit       : 100.0 C
mode       : log
thresholds : warn 88.0  throttle 92.0  stop 95.0  resume 75.0 (C)
cpufreq    : not writable — throttle stage disabled
```

The four stages

Stopping the server is the last resort, not the only tool:

1. **warn** — logged, nothing else.
2. **throttle** — lowers the CPU clock ceiling by 20 %. Needs a cpufreq driver *and* write access to `scaling_max_freq`, which normally means running as root; where that is unavailable (the usual case for an unprivileged panel, and on most VPS) the stage **disables itself** and the guard falls back to warn → stop. `./setup-cpufreq-perms.sh` grants that write access to a `cpufreq` group, so the stage works without running the panel as root — see THERMAL_GUARD.md.
3. **stop** — runs your stop script and holds the server down (see lockout above).
4. **resume** — after the CPU has stayed below the resume temperature for `thermal_resume_s` (5 minutes by default), the clock ceiling is restored and, in `stop+restart` mode, your start script runs again.

Each stage must be **held** for its sustain time — 30 s for warn and throttle, 60 s for stop — so a brief spike during a compile never trips it.

Repeated trips are capped by `thermal_max_stops_hour` (2 by default). Once that is reached the *automatic restart* is disabled while the protection keeps working, so an overheating machine cannot flap between running and stopped.

Which sensors are used

Only real CPU-die sensors: `coretemp` (Intel), `k10temp` / `zenpower` (AMD), `cpu_thermal` / `soc_thermal` (ARM, Raspberry Pi). The hottest reading on the chip is used, not the average.

`acpitz` and unlabelled thermal zones are **deliberately ignored** — they often report a case or board temperature tens of degrees below the actual CPU, so a guard trusting them would never fire when it mattered. If your machine has no usable sensor (common on a VPS or inside a container) the card says `no trusted CPU sensor` and the guard stays inactive rather than pretending to protect you.

What it writes

Everything goes to `crash.log`, i.e. the **CRASH** tab of the Log Viewer, one event per line:

```
[THERMAL] warn: 89.0C (warn=88.0 crit=100.0) sustained 31s
[THERMAL] throttle: 93.0C sustained 30s — cpufreq max lowered 20%
[THERMAL] STOPPED server: 96.0C sustained 62s (stop=95.0 crit=100.0) — Started stop-websdr.sh
[THERMAL] lockout: process reappeared at 94.0C — re-stopped [3 time(s) so far]
[THERMAL] recovered: 74.0C held below 75.0 — lockout cleared
[THERMAL] auto-restart: Started start-rx888mk2.sh
```

The lockout line is rate-limited to one per minute, so a supervisor that keeps fighting the guard cannot flood the log.

Testing it before you trust it

Set **TEST TEMPERATURE** in Settings to a value above your stop threshold. The guard treats it as a real reading, so the whole path — warn, throttle, stop, lockout, recover, restart — runs on demand. Clear the field to return to the real sensor.

> Running this test with the mode set to `stop` really does stop the server and disconnect your listeners. Do it when nobody is on, or leave the mode at `log`, where the test shows what *would* have happened without touching anything.

Suggested way to adopt it

1. Leave the mode at **`log`** for a week and carry on as normal.
2. Read the CRASH tab after your heavy moments — a full rebuild, a hot afternoon. If nothing appears, your machine never came close.
3. If the logged temperatures look right, set the mode to **`stop`** (or **`stop+restart`** to have it come back by itself).
4. Prove it once with the test-temperature field, then leave it alone.

Recovering from a lockout

CLEAR LOCKOUT on the Dashboard card clears the lockout and the rate limit. It does not disarm the guard: if the CPU is still too hot, the next check simply stops the server again. Use it after you have fixed the cooling.

Endpoint

`GET /admin/api/thermal` (login required) returns the guard's full state — sensor, thresholds, stage, lockout, last event. `POST` with `{"action":"reset"}` does the same as the CLEAR LOCKOUT button.

Without the admin panel

`thermal_guard.py` is stdlib-only and runs on its own, reading the same `admin_config.json`:

```
python3 thermal_guard.py --once              # print sensor, trip point and thresholds
python3 thermal_guard.py                     # run as a service (log-only until armed)
python3 thermal_guard.py --mode stop+restart # arm it without writing any config file
python3 thermal_guard.py --config /path/to/other.json
python3 thermal_guard.py --help
```

`--mode` overrides `thermal_mode` from the config file, so a machine with no panel can be armed straight from its systemd unit with no JSON at all.

Spot Reporting (Autorun FT8/FT4/JS8/WSPR)

The **Spot Reporting** tab controls the autorun daemon (`autorun/index.js`) — a Node.js process that decodes FT8/FT4/JS8/WSPR server-side straight off the receiver and uploads spots to the reporting networks:

- **FT8 / FT4 / JS8 → PSK Reporter** (IPFIX/UDP)
- **WSPR → wsprnet.org** (HTTP)

> **JS8 is spotted at Normal speed only** — the 15 s calling cycle, where heartbeats and CQs are. Spots come from heartbeats, compound frames and directed messages; group destinations such as `@ALLCALL` and callsigns the decoder could not resolve (`<....>`) are never reported. JS8 shares the PSK Reporter queue with FT8 and FT4 and is counted separately, exactly as they are.

Reporting is OFF by default. Nothing is uploaded until you enable a destination and press **Start**. Decoding and uploading are independent: the daemon can decode and log with reporting off, so you can verify activity before anything goes public.

> ⚠Spots are uploaded to public networks under **your callsign**. Only enable bands/modes your receiver genuinely hears, and set the correct grid square.

Prerequisites

The autorun daemon needs Node.js 22+, the `ws` + `cbor-x` npm packages (resolved via the `autorun/node_modules` symlink → `frontend/node_modules`), and `util-linux` (`taskset`). The installers set all of this up automatically — see the Autorun Spot Reporter section of INSTALLATION.md. If **Start** fails, that symlink or `taskset` is the usual cause (see Troubleshooting below).

Using the tab

1. **Identity** — callsign and grid. Pre-filled from `frontend/site_information.json` (`siteSysop` / `siteGridSquare`, truncated to 6 chars); override here if needed. The reporters take a 4- or 6-character locator only (KM17 or KM17VX): a longer one is cut to its 6-character square, and anything that is not a locator is refused when you save.
2. **Band × mode matrix** — tick the combinations to decode. Rows are bands, columns are FT8 / FT4 / JS8 / WSPR; unsupported cells are disabled. WSPR additionally covers the LF/MF bands (2200m, 630m) and an extra 80m EU channel.
3. **Destinations** — enable **PSK Reporter** and/or **wsprnet** (both off by default).
4. **Max slots** — a safety cap on how many band/mode slots can run at once.
5. **Start / Stop** — spawns/kills the daemon (pinned to the top CPU cores with `taskset`, chosen automatically for the machine — see *Resource requirements & limitations* below). **Save** / **Reload** persist and re-read the config; **Free All** clears every slot.

The status card polls every 5 s and shows running state, decode/upload counts and the last upload time. A **REPORTING** badge appears on the main waterfall while reporting is active.

> **"0 sent" for the first few minutes is normal.** PSK Reporter uploads batch every **5 minutes** and wsprnet every **2 minutes** — spots queue until the next flush. The status card shows the pending count and cadence.

Changing bands or modes while it is running

Changes do not take effect until the daemon is restarted. `autorun/index.js` reads `autorun.json` once, at startup. There is no file watcher and no reload signal — the only signals it handles are SIGINT and SIGTERM, both meaning shut down. A running daemon therefore keeps decoding the slots it was launched with, whatever you save afterwards.

To apply a change:

1. Tick / untick the bands and modes you want
2. **SAVE CONFIG**
3. **STOP**

4. **START**

Or simply **STOP** → change the boxes → **START**, since START saves the config before launching.

> ⚠**Pressing START without stopping first does not apply the change.** START saves the config, then the start request answers `already running` and nothing relaunches. You get a "saved" toast beside an "already running" error while the daemon carries on with the **old** bands — easy to read as success. Always STOP first.

What a restart does to the counters:

- **The all-time counts beside each checkbox survive** — they live in `autorun-totals.json`, which is not deleted on shutdown.
- **The per-decoder tiles reset to zero**, being per-run by design.
- **Nothing queued is lost**: shutdown flushes pending spots before exiting.
- A **newly enabled** band starts at `0`; a band you had used **before** resumes its previous total.
- A band you **deselect keeps its number** next to the now-unchecked box; the history is not deleted.

Decoding stops only for the few seconds between STOP and START.

Spot counters

Two different counts are shown, and they answer different questions.

SPOTS UPLOADED PER DECODER — a tile per decoder (FT8 / FT4 / JS8 / WSPR) with the spots uploaded **since the daemon last started**, under it the decode count and how many are still queued. Stop/Start resets these to zero. A decoder whose destination is switched off shows its decodes and `reporting off` rather than a bare `0`, because zero uploads is a setting there, not a fault.

The number beside each checkbox in BANDS & MODES is that band+mode's **all-time** uploaded spots. These are kept in `autorun-totals.json` and survive restarts. **Amber with a dot in front** (`·123`) means that slot has decoded but not uploaded yet. That is normal between flushes — PSK Reporter uploads every 5 minutes and wsprnet every 2 — so FT8/FT4 counts sit amber for the first few minutes after a start. It also stays amber if that decoder's destination is switched off. A plain `0` with no dot means that slot is enabled but has decoded nothing yet — WSPR sits at zero for several minutes after a start, because it runs on a 2-minute cycle while FT8 is already counting in the hundreds. A slot the daemon has never run shows nothing at all. Hover any number for the uploads, the decodes and the time of the last upload.

FT8 and FT4 share one PSK Reporter upload queue, so the daemon counts each spot against its own mode and band as it goes out; the split is not estimated from the totals afterwards.

Clearing the counters. FREE ALL SLOTS unticks every band/mode, turns both destinations off, stops the daemon and **erases the all-time counters** — after it, no number is shown beside any checkbox. This is the only way to reset them, and it cannot be undone. The button stops the daemon and waits for it to exit before deleting `autorun-totals.json`, because the daemon rewrites that file on shutdown; deleting it under a live daemon would simply bring the numbers back.

Resource requirements & limitations

The daemon is deliberately lightweight and runs on low-resource hardware (a 4-core i5 is fine), but there are real limits to be aware of.

Autorun CPU pinning is config-aware and automatic. The **autorun daemon** is always launched by the admin panel (`admin_server.py`), so its pin is set on every install without any configuration: it derives a `taskset` core range from the CPU count, reserving a few of the top cores for decoding, or runs unpinned on ≤ 4 cores.

spectrumserver pinning is up to your own start method. How you launch spectrumserver varies between installs (SDR type, tooling, personal scripts), so this document does not assume any particular launcher. The admin panel does not start or pin spectrumserver — that is entirely down to whatever command or service you use to run it. If you want spectrumserver kept off the cores the autorun daemon uses, pin it yourself with `taskset` in your own launch command (see the override section below). If you don't, it simply runs unpinned and the OS scheduler balances it — correct and safe, you just lose the deliberate core separation.

For reference, the autorun daemon reserves these top cores (so if you pin spectrumserver, keep it to the lower cores to avoid overlap):

Logical CPUs	autorun daemon uses	leave for spectrumserver
≤ 4	*unpinned*	*unpinned*
6	`5`	`0-4`
8	`6-7`	`0-5`
12 (e.g. 8P+4E hybrid)	`8-11`	`0-7`
16	`12-15`	`0-11`

On a **4-core (or fewer)** machine there is nothing to segregate, so the autorun daemon also runs *unpinned* and shares all cores — correct and safe, but the SDR FFT and the decode bursts compete for the same cores.

Known limitations:

1. **The real bottleneck is spectrumserver + SDR bandwidth, not the daemon.** An RX888 @ 30 MHz needs OpenCL/GPU; a low-core CPU without a capable GPU cannot sustain that FFT. Pair low-resource hardware with a narrower SDR (RSP1A ≈ 10 MHz, RTL-SDR ≈ 2.4 MHz).
2. **WSPR is the CPU long pole.** Its Fano decoder is a JS port (~30 s per band, single-threaded). The 4-worker pool runs 4 decodes in parallel; enabling **more than ~4 WSPR bands** at once can queue past the 120 s slot, especially while the SDR is competing for cores. FT8/FT4 decodes are cheap by comparison.
3. **Slot-count scaling.** The ~2–2.5 cores / 600–700 MB figure is for the full ~28 slots on an 8-core box. On 4 shared cores, keep it to a handful of bands (guideline: ≤ 6 FT8/FT4 + ≤ 3 WSPR) and watch the pool's `queued` stat.
4. **Pinning assumes Intel hybrid topology** (lower cores = faster P-cores). On AMD or CPUs with interleaved SMT numbering the auto split is still valid (no overlap, no crash) but "lower cores are faster" may not literally hold, and on a hyperthreaded 4C/8T part the top cores are SMT siblings — confined, not fully isolated. Where the auto range isn't ideal, use the **manual override** below.
5. **Band coverage is limited by the receiver.** The RX888 config (`sps=60000000` → 30 MHz Nyquist) cannot reach 6 m and above; the band table is 160 m–10 m. Other SDRs cover whatever their tuned window allows.
6. **Reporting cadence.** PSK Reporter flushes every 5 min, wsprnet every 2 min — "0 sent" in the first few minutes is normal, not a fault.

Manual CPU-pin override (advanced)

The auto-derived core split is right for most machines, but you can force a specific pinning where it isn't (AMD CCX/CCD, ARM big.LITTLE, interleaved SMT). Two independent overrides, each accepting a `taskset -c` core list (`2-3`, `0,2,4`, `0-2,5`) or `none`/`off`/`unpinned` to disable pinning. An invalid value is ignored and the auto-derive runs — a typo can never stop a launch.

Autorun daemon — `AUTORUN_CORES` env var, or a `"cores"` field in `autorun.json` (env wins). The `"cores"` field is preserved across admin-panel **Save**, so a hand-edit sticks. Two ways to set it:

Option A — `autorun.json` (persistent, recommended). Add a `"cores"` line to the config at the repo root, then **Stop → Start** the daemon on the Spot Reporting tab:

```
// autorun.json
{ "identity": { "callsign": "SV1BTL", "grid": "KM17VX" },
  "reporting": { "pskreporter": true, "wsprnet": false },
  "slots": [ /* … */ ],
  "cores": "2-3" }          // or "none" to run unpinned
```

Option B — environment variable (one-off). The daemon is spawned by the admin panel, so the var must be in the **admin panel's** environment — set it and restart the panel (env wins over the `autorun.json` value):

```
# Method A — the var must be set on the process that starts:
./manage_admin.sh stop && AUTORUN_CORES=2-3 ./manage_admin.sh start

# Method B — systemd ignores a var set on the systemctl command line,
# so put it in the unit instead:
sudo systemctl set-environment AUTORUN_CORES=2-3
sudo systemctl restart phantomsdr-admin
```

spectrumserver — pin it yourself by prepending `taskset -c <cores>` to whatever command you use to start the server. This works with any start method (a direct launch, an SDR pipeline, a service unit, etc.):

```
# pin the server to cores 0-3, direct launch:
taskset -c 0-3 ./build/spectrumserver --config <your-config>.toml &lt; <your-input>
# or in an SDR pipeline:
<your-sdr-source> | taskset -c 0-3 ./build/spectrumserver --config <your-config>.toml
```

⚠**Make it survive restarts.** An inline `taskset` only applies to that one launch. If something auto-restarts the server (a watchdog, a systemd service, a cron/`@reboot` entry, …), add the `taskset` prefix inside the script or unit that actually starts it — otherwise the restart drops the pin.

Verify the pin took — print the running processes' actual CPU affinity:

```
taskset -cp "$(pgrep -f 'autorun/index.js')"   # autorun daemon
taskset -cp "$(pgrep -x spectrumserver)"       # spectrumserver
```

Accepted values (both overrides): a `taskset -c` list (`2-3`, `0,2,4`, `0-2,5`), or `none`/`off`/`unpinned` for no pinning. Anything malformed is ignored and the auto-derive runs, so a typo can never block startup.

On a Raspberry Pi / any ≤4-core box you normally need neither — the auto path already runs both processes unpinned, which is the correct choice on a small, homogeneous CPU. The override is for larger non-Intel-hybrid machines.

Files and endpoints

Item	Purpose
`autorun.json`	Saved config (identity, slots, destinations, max slots, optional `cores` pin override). Written by the tab; git-ignored.
`autorun-status.json`	Live status the status card reads (pid, counts, last upload). Written every 15 s; removed on stop.
`frontend/dist/autorun-active.json`	Public badge data served at `/autorun-active.json`. Empty when reporting is off.
`autorun.log`	Daemon stdout/stderr.
`GET/POST /admin/api/autorun/config`	Read / write `autorun.json` (validates callsign, grid, band/mode combos, slot cap).
`GET /admin/api/autorun/status`	Running state (`pgrep`) + `autorun-status.json`.
`POST /admin/api/autorun/start` / `stop`	Spawn (`taskset -c <auto> node autorun/index.js`) / `SIGTERM`. The core range is derived from the CPU (see below).

> **Note:** after upgrading `admin_server.py` you must restart the panel to load new autorun routes or an updated band/mode matrix. The daemon runs as a separate process, so it survives admin-panel restarts.

System Health

A card on the **Dashboard**, refreshed every 30 seconds (or with **↻ refresh**):

- **Receivers** — every receiver of the station (the one on `public_port` and every receiver in `receivers.toml`): a green dot and *on air, up 15h 5m* when it answers, a red **OFF THE AIR** when it does not.
- **Disk & log folders** — free space on the disk that holds the installation (amber from 80 % used, red from 90 %), and the size of `logs/`, `logproxy/` and each `instances/<id>/logs/`.
- **Recent alerts** — what the alert monitor has seen since the panel started (see Alerts), whether or not a message was sent.
- **Last crash.log entry** — the newest non-empty `crash.log` of all receivers, with its date and its last 12 lines.

Visitor statistics

The **Visitors** page in the sidebar (on a phone: ☰ **MENU → Visitors**, or the **VISITOR STATISTICS** button on the Users page) counts the station's listeners from the private logs `logs/users_YYYY-MM-DD.jsonl` of every receiver — the copies with the IP addresses, which the public pages no longer show. Choose **Today**, **7**, **14** or **31 days**.

- **Visitors** (distinct addresses), **sessions**, **listening hours** and the **most listening at the same time**.
- **Per day** — the same four figures for each day.
- **Busiest hours** — when sessions started, by UTC hour.
- **Countries** — visitors per country.

- **Decoders** — how many sessions ran each decoder (counted from the log lines that carry the decoder, i.e. since the server was updated).
- **Addresses** — the 20 addresses with the most sessions *at the same time* first, then the most sessions; a flood like the one of 16 September 2026 stands out at the top (**AT ONCE** turns red above 3). Each row has **BAN ▾** (8, 16 or 24 hours).

Days and hours are UTC. The station's own decoders (local connections) are left out; a session still open counts at most 12 hours. Names are not in these logs, so the page shows addresses, not names. The figures are recomputed at most once a minute.

Alerts (email / Telegram)

Settings → Alerts — Email / Telegram. A monitor inside the admin panel looks every 30 seconds at:

Event	When
off the air	a receiver has not answered for about a minute (two checks in a row)
back on air	it answers again
restarted	it answers as a new process — restarted by the watchdog, a start script or the thermal guard
thermal guard	the guard changes stage (normal / warn / throttle / stopped)
crash.log	a `crash.log` of any receiver gets a new entry; the message carries its last lines

Each event has its own tick box. The same alert is sent at most once per 5 minutes. The first check after the panel starts only learns the current state, so restarting the panel sends nothing.

Telegram (the simplest):

1. In Telegram open **@BotFather**, send `/newbot` and follow the steps; it gives a **bot token** (`123456789:AA…`).
2. Open your new bot and press **Start** — a bot cannot write to you before that.
3. Open **@userinfobot**; the **Id** it answers is your **chat ID**.
4. Tick **TELEGRAM**, paste both, press ✉ **SEND TEST**.

Email: tick **EMAIL** and fill in your provider's SMTP server, port and security (SSL on 465, STARTTLS on 587), user, password and **TO**. **FROM** may be a full address, `Name <address>`, or only a name — a name alone is shown as the sender's name over the **USER** address, so the user must then be a full address.

✉ **SEND TEST** saves the form first, then sends; the line under the buttons shows the answer of each channel, or what is still missing. The settings are kept in `admin_config.json` under `"alerts"` — the panel's private file, never part of a backup or an update. A saved password or token is shown as `********` and stays as it is unless you type a new one.

Receivers

Receivers in the left menu (on a phone: **RX** in the bottom bar) lists every receiver of the station — from `receivers.toml`, or only the main one on a single-receiver station — with its name, its `?rx=` name, the hardware and its gain, the range it covers and its centre, the bandwidth, the frequency and mode a new listener starts on, and whether it is on the air. Each receiver has its own buttons:

Button	What it does
✎ **CHANGE SETTINGS**	the form described below
■ **STOP** / ▶ **START**	stops only this receiver (it asks first; the others keep running), or starts it again. Starting takes about 20 seconds; the button says *STARTING …* until the receiver is on the air
RENAME BUTTON	only on a receiver without the change form: the name on its picker button, saved in `receivers.toml`; nothing restarts
DELETE	only on an extra receiver that is not the default one — see below

✎ **CHANGE SETTINGS on an extra receiver that tunes** (one in `instances/<name>/`): give the centre frequency in MHz, pick the bandwidth from the sample rates that hardware can do, and set where a new listener starts, in which mode, its gain and the name on its button; the form shows the new range before you apply it. A frequency the hardware cannot tune, a sample rate it cannot do, or a start frequency outside the new range is refused. ✔ **APPLY AND RESTART THIS RECEIVER** then:

- sets `-f`, `-s` and the gain in `instance.env`, and `frequency`, `sps`, `fft_size` and the start frequency and mode in `config.toml` — a crystal correction (a `frequency` a few Hz away from `-f`) is kept as the same error in ppm, so it stays right on the new band, and `fft_size` is scaled so the waterfall keeps its resolution
- sets `siteSDRBaseFrequency` and `siteSDRBandwidth` in `www/site_information.json` — the receiver's page reads it when it opens, so nothing is rebuilt
- sets the button name in `receivers.toml`; the receiver's directory-listing name and hardware name stay as they are
- keeps the old files in `admin-backups/retune-<name>-<date>/` and restarts **only that receiver** — its listeners are cut off for a few seconds, the others notice nothing

✎ **CHANGE SETTINGS on the main receiver and on a direct-sampling one** (RX888, Fobos HF — it covers 0 Hz to half its sample rate and has nothing to tune): the form has the name on its button, the start frequency, the mode and the gain. The start frequency and mode go into its `.toml`, the name into `receivers.toml`, and the old files into `admin-backups/settings-<name>-<date>/`. ✔ **APPLY** restarts that receiver only when the start frequency, the mode or the gain changed; restarting the main receiver restarts RADE and DRM with it. The main receiver's band itself is changed with `bash configure-station.sh`: its page carries its band from the build, so it needs a page rebuild, and the wizard restarts the services with sudo. On a single-receiver station without `receivers.toml` there is no picker button, so there is no name to set.

AT VOLUME (0 – 100 %), in both forms, is the volume slider position a first-time listener starts at; it goes into the receiver's `.toml` as `volume` under `[input.defaults]`. Left empty, the line is removed and the page starts at 50 %. A listener who has moved the slider once keeps their own setting on later visits. A changed volume restarts that receiver, like a changed start frequency.

IQ FOR LISTENERS, **WIDE IQ** and **WIDE IQ LISTENERS AT ONCE**, in both forms, are the receiver's IQ settings: `iq`, `iq_wide` and `iq_wide_listeners` under `[server]` in its `.toml` (see the wider IQ section of the User Guide). *off* hides the IQ button from listeners. Wide IQ offers 24 or 48 kHz besides the 12 kHz everyone gets, to as many listeners at once as the last field says (`0` = no limit). Only a setting you change is written, and an example line such as `# iq_wide=48` becomes the real line. The server reads these settings only when it starts, so a changed IQ setting restarts that receiver.

On every receiver, changing only the name restarts nothing.

Gain. For an RX888 the form has the VGA gain (0–127), the gain mode (low/high), the attenuation (0–63) and **PGA on**; for an RTL-SDR the RF gain from the steps the tuner supports, or *auto* (the stick's own AGC). They are written into the receiver's argument line — `RX888_ARGS` or `RX_ARGS` — in `instances/`

`<name>/instance.env` for an extra receiver, and in `main.env` for the main one. `main.env` is the main receiver's `instance.env`: every start script reads it last, it is not in git and `update.sh` never replaces it. The gain cannot live in `station.conf`, because `configure-station.sh` rewrites that file. Without `main.env` the start script keeps its built-in gain (`-g 50 -a 0 -m low --pga` for the RX888). Other receiver types keep their gain in the same argument line; change it there by hand.

DELETE removes an extra receiver for good. Type its short name to confirm; it then stops that receiver (watchdog first) and goes no further while any of its processes is still running, takes its `[[receiver]]` block out of `receivers.toml` (the old file goes to `admin-backups/deleted-<name>-<date>/`), and moves its folder `instances/<name>/` there too — or deletes the folder if you untick *keep a copy*. Its button disappears from the pages; the driver and its log file `logwebsdr-<name>.txt` stay.

The proxy re-reads `receivers.toml` whenever it changes, so a rename, a deletion or an added receiver reaches the listeners without restarting the proxy. Open pages re-read the receiver list every 15 seconds, so a stopped receiver's button goes away and comes back on its own.

ADD NEW RECEIVER opens a form with the questions of `add-receiver.sh`: the receiver type (with *RTL-SDR Blog V4* for an RTL stick), centre frequency, bandwidth, a short name (suggested from the band, and checked to be free), the name on its button, its antenna and the station's public address, and whether to install the driver, show the receiver picker on the main page, set this panel's start script to `start-all.sh`, start the receiver and restart the proxy. Above it, the page lists the USB receivers this computer sees, so you know the stick is plugged in. ▶ **PREPARE** checks the answers, saves them in `instances/.add-rx.env`, and gives two lines to run in a terminal, each with **COPY**:

```
cd ~/PhantomSDR-Plus
./add-receiver.sh --answers instances/.add-rx.env
```

The script takes every answer from that file and asks only for your sudo password (the driver and the proxy restart need it — the admin panel never runs anything as root), then removes the file. Reload the page afterwards and the new receiver is in the list. `--answers FILE` also works on its own: one `ADD_RX_…=value` per line; the names are listed at the top of `add-receiver.sh`.

Settings — ports, start/stop and HTTPS

PORTS, under **UPDATE PASSWORD**, lists every port this station uses — the public port, https (443/80) when it is set up, each receiver's spectrumserver, the admin panel, statistics, RADE, DRM and the relay — read from `admin_config.json`, `station.conf` and `receivers.toml`, the same files `proxy.py` reads. A green ● means something listens on the port now, ○ that nothing does. Only the ports marked *open on the router* need forwarding; the others stay on this computer and are reached through the public port. Services that are not installed are left out.

SPECTRUMSERVER PORT (MAIN RECEIVER) is `public_port`: the port the main receiver's spectrumserver listens on — 8901 behind `proxy.py`, or 9001 on a station set up with `configure-station.sh`. It is **not** the port listeners use; that one is shown read-only under the box as **Listener port (proxy)** (`proxy_port`), or as *none* on a station without a proxy, where the two are the same. **SAVE SETTINGS** checks the number: the proxy's own port is refused, and a port that differs from the `[server] port` of the running receiver asks for confirmation first — with a wrong number `proxy.py` forwards listeners to a port where nothing answers, from its next restart on. To really move spectrumserver, set `[server] port` in the receiver's `.toml`, its line in `receivers.toml` and this box to the same number, then restart the receiver and the proxy.

▶ **START** and ■ **STOP**, next to **DEFAULT START SCRIPT** and **DEFAULT STOP SCRIPT**, run the **saved** script at once, after a confirmation — both drop every listener. If you changed a selection, save first: the

button refuses an unsaved one. The first seconds of the script's output appear under the buttons; the whole output of these scripts, and of the thermal guard's restarts, goes to `logs/admin-scripts.log`.

HTTPS shows whether the station is served over https.

- On an https station, **CHECK** runs `setup-https.sh --status`: is Caddy running, and does the certificate answer?
- On an http-only station, **SET UP HTTPS** checks what it can — `station.conf`, that your DNS name resolves (an IP address cannot get a certificate), whether ports 443 and 80 are free or already Caddy's — reminds you to forward TCP 443 and 80 on the router, and gives the two lines to run in a terminal, each with **COPY**. `setup-https.sh` asks for your sudo password there: the admin panel never runs anything as root. Reload the page when it has finished, and the button becomes **CHECK**. See Secure Access — https://.

Backup & restore of the station settings

Settings → Backup & restore of the station settings.

DOWNLOAD BACKUP gives one zip, `phantomsdr-settings-<call>-<date>.zip`, with this station's own settings:

- every `*.toml` of the installation (receiver configs and `receivers.toml`) and `station.conf`
- `frontend/site_information.json`, `frontend/src/bands-config.js`, `markers.json`
- `autorun.json` (spot reporting), `websdr_relay.json`, `drm.json`, `admin_bans.json`
- for every extra receiver: `instances/<id>/config.toml`, `instance.env`, `markers.json`

and a `BACKUP-INFO.json` with the date, the computer and the file list. **Not** in it: `admin_config.json` (admin password, alert secrets), the logs, and the program itself.

RESTORE takes a backup zip and writes back **only** files of that same list — anything else in the zip is skipped and named. Before a file is replaced, the current one is copied to `admin-backups/restore-<date-time>/` (git-ignored, never in an update), so a wrong restore can be undone by hand. The answer says what to do next: restart the receivers after a `.toml`, `instance.env` or `station.conf`; rebuild the page (`./recompile.sh --frontend`) after site information or band settings; restart `phantomsdr-websdr-relay` after the relay settings.

On a phone or tablet

Below 768 pixels wide the left menu becomes a drawer. The bottom bar has **DASH**, **RX** (Receivers), **USERS**, **GRAPHS**, **LOGS** and ☰ **MENU**, which slides in the full menu with every page; it closes when you pick a page or tap beside it, and **MENU** stays lit while you are on a page that is not in the bar. On a phone the tables turn into one card per row (the Receivers list already on a tablet, up to 1100 pixels), forms drop to one column, and a card title's grey note moves under the title. Top Processes, the Markers editor and the Spot Reporting band/mode grid stay tables and scroll sideways.

Themes

The **THEME** selector sits in the top bar of every page, between **ONLINE** and the clock, and again in **Settings → Appearance**; the two always show the same choice. A theme applies at once, no reload.

Theme	Look
Phosphor green	the classic green terminal (default)
Amber display	the same in warm amber, like an old rig's display
Metal panel	the receiver page's grey anodised finish, lit from the upper right
Modern dark	charcoal with a blue accent, no glow
Light	white and grey, black text — for daylight
Auto (day/night)	**Light** from sunrise to sunset, **Modern dark** at night

All themes use the same fonts and sizes; only colours and finish change. *On air* stays green and warnings amber and red in every theme, and the **Graphs** follow the theme too. **Auto** computes sunrise and sunset at the station's locator (`siteGridSquare` in Site Info) and switches by itself while the page is open; without a locator it takes 07:00–19:00 local time as day. The choice is kept per browser, so a PC and a phone can use different themes. The login page keeps the green look.

Endpoints (new pages)

Endpoint	Purpose
`GET /admin/api/health`	System Health card: receivers, disk, log folders, last crash.log entry, recent alerts
`GET /admin/api/visitor-stats?days=N`	Visitor statistics for 1–31 days (cached one minute)
`GET` / `POST /admin/api/alerts`	Read / save the alert settings (secrets come back as `********`)
`POST /admin/api/alerts/test`	Save-then-send test; answers per channel
`GET /admin/api/backup`	Download the settings zip
`POST /admin/api/restore`	Restore a settings zip (multipart, field `file`)
`POST /admin/api/run-default-script`	Run the saved default start or stop script (`{"which": "start"}` or `"stop"`)
`GET /admin/api/https`	The https state, from `station.conf`
`POST /admin/api/https/check`	Run `setup-https.sh --status`
`POST /admin/api/https/precheck`	The checks before turning https on, and the commands to run
`GET /admin/api/receivers-info`	Every receiver with its range, bandwidth and state
`POST /admin/api/receivers-info/retune`	Change a receiver's settings (frequency, bandwidth, start frequency, mode, start volume, gain, button name) and restart it when needed
`POST /admin/api/receivers-info/rename`	The name on a receiver's button (`{"id", "name"}`); nothing restarts
`POST /admin/api/receivers-info/power`	Stop or start one receiver (`{"id", "action": "stop"}` or `"start"`)
`POST /admin/api/receivers-info/delete`	Remove an extra receiver (`{"id", "confirm": <id>, "keep_copy"}`)
`GET /admin/api/receivers-info/add-defaults`	Starting values for the Add form, and the USB receivers seen
`POST /admin/api/receivers-info/add-plan`	Check the Add form and write `instances/.add-rx.env`
`GET /admin/api/station-grid`	The Site Info locator, for the Auto theme

Troubleshooting

Problem	Fix
Admin panel not starting	Check `admin.log` — usually a missing Python package
Proxy not starting	Run `python3 -c "import aiohttp"` — if it fails: `pip3 install aiohttp --break-system-packages`
Fedora / openSUSE: panel or proxy does not start at boot (`status=209/STDOUT`, `avc: denied { open } … admin.log` or `proxy.log` in `sudo ausearch -m avc -ts boot`)	SELinux forbids systemd to open log files in your home folder. Run `./setup_admin.sh --selinux` (labels both logs `var_log_t`), then `sudo systemctl restart phantomsdr-admin phantomsdr-proxy`. New installs do this automatically
Fedora / openSUSE: everything runs, but nobody outside this machine can connect	firewalld drops the public port. Run `./setup_admin.sh --firewall` (opens the proxy port, permanent). New installs do this automatically; `setup_websdr_relay.sh` offers it for the relay port and `setup-https.sh` opens 443/80
`systemctl is-active` stays `activating`, log says `address already in use`	A copy started by hand with `manage_admin.sh start` still holds the port. `./manage_admin.sh stop`, then `sudo systemctl restart phantomsdr-admin phantomsdr-proxy`. With the units installed, `manage_admin.sh start` now hands over to systemctl instead of starting a second copy
`proxy.log` is empty	First check the proxy is actually running (`./manage_admin.sh status`). If it is, you are on an old version: the launcher must use `python3 -u` (unbuffered — otherwise the startup banner never leaves the 8 KB buffer), and `proxy.py`'s `main()` must call `logging.basicConfig()` (otherwise aiohttp's access logger has no handler and every request line is discarded, because `web.AppRunner` — unlike `web.run_app` — does not configure logging).
Proxy starts but waterfall is blank	Check `sdr_host` in `admin_config.json` — normally `127.0.0.1`. If it holds an old LAN IP from a previous version, change it and restart the panel
Kick button says "no connections"	Run `getcap $(which ss)` — if no `cap_net_admin`, run `sudo setcap cap_net_admin+ep $(which ss)`
Status always shows OFFLINE	Go to Settings → SDR Process Name — set it to the exact name shown by `ps -eo comm,args | grep -v grep`
Users page shows no data	Verify the endpoint works: `curl http://127.0.0.1:<public_port>/~~users`
External browser gets NetworkError	Check proxy is running: `./manage_admin.sh status`
Proxy broke after a network change	Only relevant if `sdr_host` still holds a LAN IP — set it to `127.0.0.1` and restart the panel
Chat delete button has no effect	Check write permission on the chat log file: `ls -l ~/PhantomSDR-Plus/chat.jsonl`
Waterfall message not appearing	Confirm the proxy is running (`./manage_admin.sh status`) and the frontend is on a recent build that includes the overlay renderer
Waterfall message lost after restart	Expected — message state is in-memory only; re-send it after restarting the admin panel
Spot Reporting "Start" fails / daemon exits immediately	Check `autorun.log`. Usually the missing `autorun/node_modules` symlink (`ln -sfn ../frontend/node_modules autorun/node_modules`) or `taskset` not installed (`sudo apt install -y util-linux`).

Problem	Fix
Spot Reporting: daemon runs but `decodes` stays 0 and `autorun.log` shows `504` / `tap closed 1006`	The audio tap can't reach spectrumserver. The daemon auto-detects the port from the **running** server's config; the `[autorun] tap backend: HOST:PORT` log line must match your `[server] port`. If it's wrong (or the server wasn't running at start), pin it in `autorun.json`: `"server": { "host": "127.0.0.1", "port": 9002 }`, then Stop→Start.
Spot Reporting shows "0 sent"	Normal for the first few minutes — PSK Reporter flushes every 5 min, wsprnet every 2 min. Confirm a destination is enabled and the daemon is running.
New bands/modes not showing in the matrix	Restart the panel — the matrix is loaded at admin start.
`autorun.log` shows `MODULE_TYPELESS_PACKAGE_JSON` / "Reparsing as ES module … performance overhead"	Cosmetic warning (decoding still works). `frontend/src/modules/package.json` is missing `"type": "module"`; add that line near the top, then Stop→Start. No frontend rebuild needed — the daemon imports the source file directly.
Spot Reporting "Start" fails with `taskset: … Invalid argument` on an old/small PC	Should not happen with the config-aware launcher — the core range is derived from the CPU count and goes unpinned on ≤ 4 cores. If seen, `admin_server.py` predates that change; update it (`_autorun_taskset_prefix`) and restart the panel.
REPORTING badge not on the waterfall	Reporting must be ON with at least one slot; the badge reads `/autorun-active.json`. Rebuild the frontend if `dist/index.html` predates the badge.
✉ **SEND TEST** says "Nothing to send to: …"	The message names what is missing: tick **TELEGRAM** or **EMAIL**; Telegram needs both the bot token and the chat ID; email needs the SMTP server and **TO**.
Email test: `5.5.2 … Sender address rejected: need fully-qualified address`	The sender is not a full address. Put your address in **FROM**, or keep only a name there and make **USER** your full address; restart the panel if it predates that rule.
Telegram test fails with `Forbidden` / no message arrives	Open your bot in Telegram and press **Start** first; check the chat ID with @userinfobot (your own Id, not the bot's).
No alert when a receiver stopped	It takes about a minute (two checks); the same alert comes at most once per 5 minutes; a panel restart sends nothing; check the event's tick box.
Visitors page shows "no visitors"	The page reads `logs/users_*.jsonl` (and `instances/<id>/logs/`); those are written by the receivers' statistics logging — check they exist for the chosen days.
THEME selector missing, or a new theme does not apply	Restart the panel, then reload the page with **Ctrl+F5** so the browser drops the old copy.

CHAPTER 4

Thermal Guard

THERMAL_GUARD.md

Thermal Guard — Sysop Manual

CPU over-temperature protection for PhantomSDR-Plus.

A WebSDR is one of the few things a computer does that keeps every core busy, all day, forever. If a fan clogs, a pump stalls or the room gets hot in August, nothing tells you — the server just keeps decoding until the CPU is cooking itself. The thermal guard is the piece that notices and pulls the plug for you.

It ships **inert**: out of the box it only writes to a log. Nothing is protected until you choose a mode. This manual is about making that choice and living with it.

> **In a hurry?** Run `python3 thermal_guard.py --once`, read the thresholds it prints, then set `thermal_mode` to `stop+restart` in the admin panel. That is the whole job. Everything below is the *why*.

Contents

1. What it does
2. Quick start
3. How it decides
4. The four modes — what each one does and what you must do
5. Choosing a mode
6. Configuration reference
7. Running it without the admin panel
8. Enabling the throttle stage without root
9. Unattended operation — when nobody is watching
10. Testing it before you trust it
11. Reading the log
12. Lockout and the rate limit
13. Troubleshooting
14. Turning it off or removing it

---

1. What it does

Every 2 seconds the guard reads your CPU's die temperature and compares it against four thresholds. When the temperature stays above one of them for long enough, it moves up a ladder:

```
normal  ──►  warn  ──►  throttle  ──►  stop  ──►  (cool down)  ──►  restart
          log a line   lower the      run your       hold the         run your
                       CPU clock      stop script    server down      start script
```

Two design decisions make it work on *any* sysop's machine, which is worth understanding before you rely on it:

It never asks how you run your server. systemd, a watchdog loop, cron, tmux, a bare shell — the guard does not know and does not care. Instead, while the machine is over-temperature it holds a **lockout**: it re-issues the stop on *every* tick. If your watchdog restarts the server, the guard stops it again within two seconds. Whatever is trying to keep the server alive loses, until the CPU has cooled. This is the part that makes it universal, and it is tested against a real watchdog.

It never hard-codes a temperature. "Stop at 95 °C" is a bad rule: on an Intel with Tjmax 100 that is normal-ish load, on a Ryzen it is *below* where Tctl sits under boost by design, and on a Raspberry Pi it is unreachable because the SoC throttles itself at 80. So the guard reads the critical trip point your own kernel publishes (`tempN_crit`) and works backwards from it. Same code, sensible numbers everywhere.

---

2. Quick start

1. See what your machine looks like to the guard. Do this first, always:

```
cd ~/PhantomSDR-Plus
python3 thermal_guard.py --once
```

```
sensor     : hwmon:coretemp
temperature: 65.0 C
crit       : 100.0 C
mode       : log
thresholds : warn 88.0  throttle 92.0  stop 95.0  resume 75.0 (C)
cpufreq    : not writable — throttle stage disabled
```

If `sensor` says `NONE`, stop here and read Troubleshooting — the guard cannot protect a machine it cannot measure, and it will tell you so rather than pretend.

2. Leave it in `log` mode for a few days. It is already running inside the admin panel. Let it watch a normal week — summer afternoons, a big build, a busy Saturday contest — and see whether it would ever have fired.

3. Arm it. Admin panel → **Settings** → **THERMAL GUARD** → **MODE** → `stop+restart` → **SAVE THERMAL SETTINGS**. It takes effect within about 2 seconds; no restart needed.

---

3. How it decides

The sensor

The guard only trusts real CPU-die sensors:

Source	Typical hardware
`coretemp`	Intel
`k10temp`, `zenpower`, `zenpower3`	AMD
`cpu_thermal`, `soc_thermal`	Raspberry Pi, ARM SBCs
`x86_pkg_temp` thermal zone	fallback on x86

`acpitz` and unlabelled thermal zones are **deliberately excluded**. On a typical desktop `acpitz` reads about 28 °C while the CPU package is at 59 °C — a guard trusting it would simply never fire. Reporting "disabled, no trusted sensor" is more honest than pretending to protect.

`psutil` is used if it happens to be installed, but is never required. The guard is pure standard library.

The thresholds

Each one is an offset below the critical trip point the machine itself publishes:

Threshold	Offset below crit	What it means
warn	−12 °C	Something is wrong, log it
throttle	−8 °C	Try to cool it down by slowing the CPU
stop	−5 °C	Too close to the edge — stop the server
resume	−25 °C	Genuinely cool again, safe to come back

Which lands like this in practice:

Your CPU reports	warn	throttle	stop	resume
Intel, crit 100 °C	88	92	**95**	75
AMD Ryzen, crit 95 °C	83	87	**90**	70
Raspberry Pi, crit 85 °C	73	77	**80**	60

If the machine publishes no critical point at all, the guard falls back to a conservative fixed set and says so. You can override any threshold with an absolute number (see Configuration reference), but you should not need to.

Sustain times — why a spike never acts

Nothing happens on a single hot reading. A threshold must be held **continuously**:

- **warn** and **throttle**: 30 s (`thermal_warn_sustain_s`)
- **stop**: 60 s (`thermal_sustain_s`)
- **resume**: 300 s below the resume threshold before a restart (`thermal_resume_s`)

A compile spike, a burst of decoding, a fan ramping up — all far too short to matter. One dip below the threshold resets the clock. On top of that the guard requires 3 consecutive valid readings before it will act at all after starting, and it discards any reading outside 20–125 °C as a sensor glitch rather than a temperature.

---

4. The four modes

`thermal_mode` is the single switch that decides how far up the ladder the guard is allowed to go. Each mode **includes everything the modes before it do**.

Mode	Warns	Throttles	Stops	Restarts	Server can go down?
`log`	✓	—	—	—	No — touches nothing
`throttle`	✓	✓	—	—	No
`stop`	✓	✓	✓	—	Yes, stays down until you act
`stop+restart`	✓	✓	✓	✓	Yes, comes back on its own

`log` — watch and report (the shipping default)

What it does: everything the other modes do, except acting. It follows the whole ladder and writes what it *would* have done to `crash.log`. It never runs your stop script, never touches the CPU frequency, never interrupts a listener.

What you must do: nothing — it is already running. After a week, open the **CRASH** tab in the admin panel, or run:

```
grep THERMAL crash.log
```

- **Nothing there?** Your cooling is fine and you can arm the guard with confidence.
- **`STOP level reached ... NOT stopping`?** The guard would have shut your server down. That is a real thermal problem you have been running with — fix the cooling *and* arm the guard.

Use it when: you have just installed the guard and do not yet know how your machine behaves. This is a shakedown mode, not a destination — a guard in `log` mode protects nothing.

`throttle` — slow the CPU down, never interrupt service

What it does: at the throttle threshold it lowers the maximum CPU frequency by 20 %, then restores it once the temperature is back below the warn line. Your WebSDR stays up throughout. Listeners may notice slightly worse performance under heavy load; most will notice nothing.

If the temperature keeps climbing past the stop threshold anyway, this mode **logs it and does nothing** — it will not stop the server.

What you must do:

1. Give the guard write access to the CPU frequency limit — it needs it, and by default only root has it. Run `./setup-cpufreq-perms.sh` once; see section 8.
2. Confirm with `python3 thermal_guard.py --once` that it prints `cpufreq : writable, throttle stage available`. If it says *not writable*, this mode does nothing at all beyond what `log` does, and will say so in the log every time.

Use it when: you are chasing a marginal cooling problem and want the machine defended without ever dropping listeners. **Be aware this is not full protection** — if throttling is not enough, nothing else happens. Good as a first step; poor as a final answer.

`stop` — shut the server down and leave it down

What it does: at the stop threshold it runs your stop script, then holds the **lockout** — re-stopping the server every 2 seconds for as long as the machine is over-temperature, so nothing can revive it behind your back. When the CPU has been below the resume threshold for 5 minutes the lockout clears. **The server does not come back on its own.** You restart it when you are satisfied the cause is fixed.

What you must do:

1. Make sure the guard knows how to stop your server. In the admin panel that is the configured stop script (`./stop-websdr.sh` for the shipped launchers). If no stop script is set, the guard falls back to `SIGTERM` on the process name, then `SIGKILL` after a 10-second grace — which works, but a proper stop script is cleaner.
2. Accept that **your WebSDR will be offline until you notice**. Have some way of finding out: an uptime monitor, the CRASH tab, or simply checking daily.

Use it when: the machine matters more than the service, you are usually around, or a thermal event is serious enough that you want to inspect the box before it carries on. Also the right choice if you do not trust an automated restart on your hardware.

`stop+restart` — shut down, cool off, come back by itself

What it does: everything `stop` does, plus: once the CPU has held below the resume threshold for 5 minutes, it runs your start script and the WebSDR comes back on its own.

Flapping is prevented by `thermal_max_stops_hour` (default 2). After two thermal stops within an hour the guard **disables the automatic restart** and leaves the server down for you to look at. Note the important detail: the rate limit turns off the *restart*, never the *protection* — the lockout keeps stopping the server while it is hot, no matter how many times it has already tripped.

What you must do:

1. Make sure **both** scripts are configured and actually work standalone — the start script as well as the stop script. Test them by hand first: `./stop-websdr.sh` then `./start-rx888mk2.sh`.
2. Nothing else. This is the fire-and-forget mode.

Use it when: the WebSDR is public, unattended, or remote — which is most of them. This is the recommended mode for a normal installation.

---

5. Choosing a mode

Your situation	Mode
Just installed the guard, do not know the machine yet	`log` for a week
Public WebSDR, unattended, you want it to look after itself	**`stop+restart`**
Remote site you cannot easily reach	**`stop+restart`**
You want to inspect the machine after any thermal event	`stop`
Hardware you do not trust to survive an automatic restart	`stop`
Chasing marginal cooling, must not drop listeners	`throttle`, then move on to `stop+restart`
Machine has no trusted CPU sensor	none of them will work — fix the sensor first

For most sysops the answer is `stop+restart`, and the honest summary of the others is: `log` protects nothing, and `throttle` protects a little.

---

6. Configuration reference

All keys live in `admin_config.json` next to `thermal_guard.py`, and every one of them is also editable in **Settings → THERMAL GUARD**. The guard **re-reads the file on every tick**, so changes take effect within ~2 seconds — no restart.

Key	Default	Meaning
`thermal_enabled`	`true`	Master switch. `false` disables the guard entirely.
`thermal_mode`	`"log"`	`log` | `throttle` | `stop` | `stop+restart` — see section 4.
`thermal_warn`	`null`	Absolute warn threshold in °C. `null` = derive from the CPU's crit point.
`thermal_throttle`	`null`	Absolute throttle threshold. `null` = derived.
`thermal_stop`	`null`	Absolute stop threshold. `null` = derived.
`thermal_resume`	`null`	Absolute resume threshold. `null` = derived.
`thermal_sustain_s`	`60`	Seconds the stop threshold must be held before stopping. Minimum 5.
`thermal_warn_sustain_s`	`30`	Seconds for the warn and throttle stages. Minimum 5.
`thermal_resume_s`	`300`	Seconds below the resume threshold before the lockout clears. Minimum 30.
`thermal_max_stops_hour`	`2`	Thermal stops per hour after which the automatic restart is disabled. Protection continues.
`thermal_test_temp`	`null`	Pretend the CPU is this temperature. For testing only — see section 10.

The standalone guard additionally uses `start_script`, `stop_script`, `sdr_process_name` and `sdr_base_dir` from the same file.

Overriding a threshold. Only do this if you have a reason. A common one: your machine legitimately runs hot during builds and you would rather not have that count. Raising `thermal_stop` buys you headroom at the cost of margin against the critical point — never set it above your crit value, or the CPU will hit its own emergency shutdown first and the guard becomes decoration.

---

7. Running it without the admin panel

Plenty of sysops never install the panel. The guard runs perfectly well on its own — same file, same behaviour.

Try it in the foreground first:

```
cd ~/PhantomSDR-Plus
python3 thermal_guard.py --once           # what would happen here?
python3 thermal_guard.py --mode log       # watch it live, Ctrl-C to quit
```

`--mode` overrides `thermal_mode` from the command line, so you never have to write JSON by hand:

```
python3 thermal_guard.py --mode stop+restart
python3 thermal_guard.py --config /etc/phantomsdr/thermal.json
```

Then make it permanent with the sample unit shipped in the repository:

```
sudo cp thermal-guard.service /etc/systemd/system/
sudo nano /etc/systemd/system/thermal-guard.service   # set User= and paths
sudo systemctl daemon-reload
sudo systemctl enable --now thermal-guard
systemctl status thermal-guard
```

Read the comments at the top of `thermal-guard.service` before enabling it. Two things matter most:

- **`User=` must be an account that can actually run your start and stop scripts.** Running the guard as root is usually the wrong answer: it would run those scripts as root too.
- Either point it at an `admin_config.json` containing `start_script` and `stop_script`, or pass `--mode` on the `ExecStart` line. A minimal config is enough:

```
{
  "sdr_base_dir": "/home/youruser/PhantomSDR-Plus",
  "sdr_process_name": "spectrumserver",
  "start_script": "start-rx888mk2.sh",
  "stop_script": "stop-websdr.sh",
  "thermal_mode": "stop+restart"
}
```

Everything the guard does still goes to `crash.log` next to that config file, plus the systemd journal (`journalctl -u thermal-guard`).

---

8. Enabling the throttle stage without root

By default `python3 thermal_guard.py --once` reports:

```
cpufreq    : not writable — throttle stage disabled
```

This is normal and it is not a bug. The kernel creates

```
/sys/devices/system/cpu/cpuN/cpufreq/scaling_max_freq
```

owned by `root:root`, mode `0644`. An admin panel running as an ordinary user cannot write it, so the guard skips the throttle stage and says so in the log rather than failing silently.

The wrong fix is running the guard as root. It also runs your start and stop scripts, and those should stay unprivileged.

The right fix is to hand write access on those specific files to a dedicated group. One script does it:

```
cd ~/PhantomSDR-Plus
./setup-cpufreq-perms.sh
```

It asks for your password once — and only once, at setup — then:

1. creates a system group `cpufreq` and adds you to it;
2. installs `/etc/tmpfiles.d/99-phantomsdr-cpufreq.conf` so the group ownership and `0664` mode are re-applied **on every boot** — sysfs permissions do not survive a reboot on their own;
3. applies the change immediately so you can test without rebooting.

Then, in order:

```
# 1. log out and back in (or reboot) — a new group only reaches your
#    processes through a fresh login
id | grep cpufreq

# 2. restart the panel; the guard tests for write access once, at startup
sudo systemctl restart phantomsdr-admin
#    (no logout needed this way — systemd rebuilds the group list at every
#     start. Only if the units are NOT installed: ./manage_admin.sh stop && ./manage_admin.sh start
#     — never both, they fight over port 3000)

# 3. confirm
python3 thermal_guard.py --once
#    cpufreq    : writable, throttle stage available
```

To undo it completely: `sudo ./setup-cpufreq-perms.sh --revoke`.

Notes

- The throttle stage still only *acts* if `thermal_mode` is `throttle` or higher. Granting permission changes nothing on its own.
- The guard lowers whatever limit is currently in force by 20 % and restores exactly that value afterwards. If you already cap your CPU (for example `MAX_SPEED` in `/etc/init.d/cpufrequtils`), your cap is what gets restored — the guard will not quietly hand you back a higher clock than you asked for.
- Some machines have no cpufreq driver exposed at all — many VPS, most containers. The script detects this and refuses to change anything. Use `stop` or `stop+restart` there; they do not depend on cpufreq.

---

9. Unattended operation

The most common question about the guard: *what happens when nobody is watching?* The short answer is that nobody watching is the case it was built for.

It does not need a browser open. The guard is a background thread inside `admin_server.py`, ticked from the graph sampler every 2 seconds. It runs whether or not anyone is logged into the panel, whether or not a browser is open, whether or not you are awake. In standalone mode it is a systemd service, which is even more independent. The dashboard card is a *window* onto the guard, not the guard itself.

It acts without asking. There is no confirmation dialog and no notification to wait for. At the stop threshold it stops the server, full stop. That is the entire point.

But only if you gave it permission. In `log` mode it will faithfully record a machine cooking itself and do nothing about it. If the reason you are reading this section is "*I am not around to watch it*", the mode you want is `stop+restart` — it is the only one that both protects the machine and brings the service back without you.

It only helps while the panel is alive. The guard is part of `admin_server.py`, so a panel that died at 02:00 takes the protection with it, and a reboot leaves the machine unguarded until you log in and start it again. If nobody is watching, let systemd watch: the repository ships `phantomsdr-admin.service` and `phantomsdr-proxy.service`, which start at boot and restart the panel within five seconds of a crash. For an unattended receiver this is **strongly recommended** — arming `stop+restart` without it protects you only until the next reboot. See ADMIN_PANEL_SETUP.md.

How you find out afterwards. In rough order of usefulness:

Where	What you get
`crash.log` / the **CRASH** tab	Every action, timestamped. The permanent record.
Dashboard **THERMAL GUARD** card	Live state: current temperature, stage, lockout, stops in the last hour.
Graphs page (CPU temperature, 24 h)	The shape of the event — how fast it climbed, how long it took to cool.
`journalctl -u thermal-guard`	Standalone mode only.

A sensible routine for an unattended site: `stop+restart` armed, and `grep THERMAL crash.log` whenever you happen to log in. If it is empty, there is nothing to know.

A worked example. Fan fails at 03:00 on a machine in `stop+restart`:

```
03:14  temperature crosses 88 °C, holds 30 s   →  warn logged
03:16  crosses 92 °C, holds 30 s               →  throttle applied (if permitted)
03:19  crosses 95 °C, holds 60 s               →  STOP — stop script runs, lockout on
03:19  watchdog restarts the server            →  guard re-stops it, logs the lockout
03:26  CPU has been under 75 °C for 5 minutes  →  lockout clears, start script runs
03:41  it overheats and stops a second time    →  rate limit reached, no more restarts
                                                  server stays down, protection stays on
```

You wake up to a WebSDR that is down, a `crash.log` that says exactly why, and — most importantly — a CPU that never got near its critical point.

---

10. Testing it before you trust it

Do not wait for a real heatwave to find out whether your stop script works.

`thermal_test_temp` makes the guard believe the CPU is at a temperature you choose. Everything else behaves completely normally — the sustain timers, the ladder, the lockout, your actual scripts.

1. Admin panel → **Settings → THERMAL GUARD → TEST TEMPERATURE**.
2. Enter a value above your stop threshold (for example `97` on an Intel).
3. Save, and watch the dashboard card and the CRASH tab.
4. **Clear the field when you are done.** A test temperature left in place means the guard is no longer reading the real sensor.

In `log` mode this shows you the whole ladder with nothing at risk — the safe first test, and the one to run before arming anything.

In `stop+restart` mode this is a **live test**: your server really will stop, and really will come back about five minutes after you clear the field. Do it when nobody is listening. It is worth doing once, because it is the only way to know that your stop and start scripts work when called by something other than you.

Standalone equivalents:

```
python3 thermal_guard.py --once                # thresholds only, no action
python3 thermal_guard.py --mode log            # watch the ladder live
```

---

11. Reading the log

Everything is appended to `crash.log` next to the config file, one event per line, each prefixed `[THERMAL]`:

```
grep THERMAL crash.log
```

Lines are deliberately single-line and grep-friendly. What you will see:

Line	Meaning
`warn: 88.4C (warn=88.0 crit=100) sustained 30s`	First rung. Nothing has been done.
`throttle: 92.1C sustained 30s — cpufreq max lowered 20%`	CPU clock reduced.
`throttle level reached: ... (no cpufreq write access — stage skipped)`	See section 8.
`STOP level reached: ... mode is 'log', NOT stopping`	It would have stopped the server. Arm it.
`STOPPED server: 95.2C sustained 60s (stop=95.0 crit=100) — ran stop-websdr.sh`	The real thing.
`lockout: process reappeared at 96.0C — re-stopped (...) [12 time(s) so far]`	Something is restarting your server; the guard is winning. Rate-limited to one line per minute.
`rate limit: 2 thermal stops within the hour — automatic restart is now DISABLED`	Stays down for you to investigate. Protection continues.
`recovered: 54.0C held below 75.0 — lockout cleared`	Cool again.
`auto-restart: ran start-rx888mk2.sh`	Back online.
`back to normal: 59.0C (warn=88.0)`	Dropped off the ladder without ever stopping.

12. Lockout and the rate limit

These are the two behaviours that surprise people, so they are worth stating plainly.

The lockout is what makes the guard work against *your* setup specifically. When it stops the server it does not simply issue one stop and hope. It marks itself locked, and on every 2-second tick, while the temperature is still above the resume threshold, it checks whether the server process is back — and if it is, stops it again. Your watchdog, systemd's `Restart=always`, a cron job, an impatient sysop: all of them lose that argument until the CPU cools. It clears itself automatically once the temperature has held below the resume threshold for `thermal_resume_s` (default 5 minutes).

The CLEAR LOCKOUT button on the dashboard card clears the lockout *and* the stops-per-hour counter, for when you have fixed the cooling and do not want to wait. It does **not** disarm the guard: if the machine is still hot, the very next check stops the server again. That is intended.

The rate limit (`thermal_max_stops_hour`, default 2) stops a broken machine from cycling up and down all night. After the limit is reached the guard **disables the automatic restart** and leaves the server down. Read that carefully: it disables the restart, not the protection. The lockout keeps re-stopping the server while it is over-temperature no matter how many times it has already tripped. A guard that went quiet exactly when the machine was at its worst would be worse than no guard at all.

13. Troubleshooting

`sensor : NONE — no trusted CPU sensor on this machine` The guard found no allow-listed sensor and refuses to guess. Try `sensors` (from `lm-sensors`; `sudo apt install lm-sensors && sudo sensors-detect`). Inside a VM or container there is often genuinely no CPU die sensor exposed — the guard cannot protect that machine and correctly reports itself disabled rather than pretending.

`cpufreq : not writable — throttle stage disabled` Expected unless you have run `./setup-cpufreq-perms.sh` — see section 8. It only affects the throttle stage; `stop` and `stop+restart` are unaffected.

Ran `setup-cpufreq-perms.sh`, still says not writable Two likely causes: you have not logged out and back in since (a new group only reaches processes through a fresh login — check with `id`), or the panel has not been restarted since (the guard tests for write access once, at startup). Do both, in that order.

Thresholds look wrong for my CPU Check what the machine publishes: `cat /sys/class/hwmon/hwmon*/temp*_crit`. If your crit is unusual, or absent, set `thermal_stop` and friends to absolute values.

It stopped my server and I do not think it was hot Check whether `thermal_test_temp` is still set from a test. That is by far the most common cause.

It never fires even though the machine gets hot Confirm the mode is not `log`, confirm `thermal_enabled` is `true`, and compare the temperature you are seeing against the thresholds from `--once`. Remember the threshold must be held *continuously* for the sustain time.

The server keeps restarting during a thermal event That is the lockout working, and the `lockout:` lines are it reporting success — your supervisor keeps trying, the guard keeps undoing it. Nothing to fix.

The panel is not running, so is the guard running? No — in panel mode the guard lives inside `admin_server.py`. If you want protection independent of the panel, use the standalone service in section 7.

---

14. Turning it off or removing it

- **Pause it:** set `thermal_mode` to `log`. It keeps watching and reporting but never acts.
- **Disable it completely:** set `thermal_enabled` to `false`.
- **Standalone:** `sudo systemctl disable --now thermal-guard`.
- **Undo the cpufreq permissions:** `sudo ./setup-cpufreq-perms.sh --revoke`.

Deleting `thermal_guard.py` is also safe — `admin_server.py` imports it inside a `try`, and simply reports the guard as unavailable if it is missing.

See also

- Admin Panel Setup — the Thermal Guard tab in context
- Installation — thermal protection during a fresh install
- Project Structure — where `thermal_guard.py` sits

CHAPTER 5

Connection Limits

CONNECTION_LIMITS.md

Connection Limits — Sysop Manual

Protecting a public receiver from connection floods.

A WebSDR is a public service that hands anyone who asks a continuous stream of audio and spectrum. That is the point of it, and it is also the problem: nothing about a normal listener looks different, to the server, from someone opening sessions in a loop. On 16 September 2026 this receiver was hit by exactly that — one address opened 134 listener sessions inside a single minute and sat on 181 of them at once, while the station's genuine audience was around a dozen.

This manual is about the three layers that stop it, what each one can and cannot do, and how to tell whether yours are set sensibly.

> **In a hurry?** The example `config.toml` already ships with sensible values, so a fresh installation is protected without you doing anything. If your own `.toml` predates this and has no `[limits]` keys beyond `audio`/`waterfall`/`events`, the per-IP limits are simply **off** — copy the block from the example to switch them on. The one command worth knowing afterwards is `grep Refused spectrumserver.log`, which tells you whether anyone is being turned away.

Contents

1. What you are actually defending against
2. The three layers
3. Per-IP limits — the listener policy
4. Idle connections — the silent attack
5. The kernel guard — `setup-firewall.sh`
6. Configuration reference
7. What a refused listener sees
8. Reading the logs and counters
9. Choosing your numbers
10. Traps worth knowing about
11. Turning it all off

---

1. What you are actually defending against

It is worth being precise, because the word "DDoS" covers two very different things and only one of them can be solved on your own machine.

Single-source flooding. One host, or a small handful, opening connections as fast as it can. This is what happened here, and it is entirely fixable: the connections arrive from an address you can count, and counting is enough.

A genuine distributed attack. Thousands of hosts, often with forged source addresses, filling your uplink. By the time those packets reach your network card the bandwidth has already been spent. **Nothing described in this manual helps**, and nothing you install on the server can — the only answer is a service upstream of you, which is a different conversation involving a real domain and a provider such as Cloudflare in front of the receiver.

Everything below addresses the first case. That is not a limitation worth apologising for: the first case is what actually happens to amateur stations.

---

2. The three layers



They are deliberately not redundant. The kernel layer is fast but stupid — it counts packets from an address and knows nothing about listeners. The server layer is precise but only sees a connection once it has been accepted and parsed, which costs something. The browser layer exists so that a refused listener is told what happened instead of staring at a page that never loads.

---

3. Per-IP limits

What counts as one listener

A listener is **one `/audio` connection** — or, where `[kiwi_emulation]` is enabled, **one Kiwi sound connection**, which counts exactly the same. That is the same unit the user count, `users.json` and the labels on the waterfall already use, so a Kiwi client consumes a slot and appears in the user list just as a browser does.

This matters because a single desktop browser tab opens **four** sockets to your server:

Socket	Purpose
`/audio`	the demodulated audio stream — **this is the listener**
`/waterfall`	the spectrum display
`/events`	user list, other listeners' frequencies
`/chat`	the chat panel

A phone on `/mobile` opens two (`/audio` and `/chat`), because it has no waterfall. So a limit that counted raw sockets would refuse everybody: three tabs is twelve sockets. With the audio analyser open, a desktop tab opens a fifth, a hidden `/waterfall` that feeds its measurements; three such tabs (fifteen sockets) still fit the derived limit of sixteen.

The three keys

```
[limits]
per_ip=3            # simultaneous listeners from one address
per_ip_sockets=0    # simultaneous sockets of any path; 0 derives per_ip * 4 + 4
per_ip_rate=40      # new sockets per minute from one address
per_ip_ban_s=600    # how long an address that breaks the rate is refused
```

`per_ip` is the headline number and the one you will think about. Three is a good starting point.

`per_ip_sockets` exists because `/audio` is not the only thing worth flooding. `/waterfall` is the heaviest stream your server produces, and a limit that counted only listeners could be sidestepped simply by opening waterfall sockets instead. Left at `0` it becomes `per_ip * 4 + 4`, which fits `per_ip` full desktop tabs with a little slack for sockets the kernel has not finished reaping after a page reload.

`per_ip_rate` is what actually ends a flood. A cap alone does not: refusing a connection is cheap but not free, and an attacker can keep reconnecting as fast as the refusals come back, burning your CPU on setup and teardown. The rate limit gives them a ban instead of an answer. Remember one tab costs four, so 40/minute is about ten page loads a minute from one address.

> **A deliberate ordering.** The rate is charged *before* the simultaneous cap is tested. This looks backwards until you consider an address already sitting at its cap and reconnecting in a loop: if the cheap "you are at the cap" refusal answered it every time, the rate violations would never accumulate and the ban that ends the loop would never trigger.

Who is exempt

Loopback is never limited. The autorun spot-reporting tap, the admin panel and a browser opened on the server machine itself all arrive from `127.0.0.1`, and letting them consume slots would mean your own station locking out your listeners.

This also covers `proxy.py`: connections through the proxy reach `spectrumserver` from loopback, and the real client address arrives in the `X-Forwarded-For` header, which is what the limits are keyed on.

Since v5.0.0 that header is believed **only from a loopback peer** — a proxy on the same machine. Sent by anyone else it is ignored and the TCP address counts, because otherwise a direct visitor could write `X-Forwarded-For: 127.0.0.1` and pass as loopback, past every limit on this page. `proxy.py` also drops any `X-Forwarded-For` or `X-Real-IP` a client sends before adding its own, and answers the sysop kick (`/~~kick`) only for clients on the same machine. A station running several receivers behind the proxy is described in Several Receivers; the limits there apply per receiver.

---

4. Idle connections

There is an attack that none of the above can see.

Open a TCP connection. Send nothing. Hold it.

Such a peer never completes a request, so it never reaches the code where the per-IP limits live — no listener exists, no client object exists, nothing has a name to count. Meanwhile it costs the attacker a few bytes and costs you a file descriptor and a slot for as long as it is held.

```
[limits]
idle_per_ip=8       # idle (pre-request) connections per address
idle_total=512      # ceiling across all addresses
```

The limit is on **how many**, not **how long**, and that distinction is the whole design. **There is no deadline on an idle connection, and deliberately so.** A deadline seems the obvious answer and is a trap (see §10): there are legitimate peers that connect and then sit silent for a long time before speaking. What separates them from an attack is quantity — a legitimate peer holds one such connection, an attacker holds thousands.

An idle connection therefore lives until it makes a request or dies, and its slot is released either way.

`idle_total` bounds a flood spread across many source addresses. When the ceiling is reached, **new** idle connections are refused rather than established ones evicted, so a legitimate long-lived connection is never what gets dropped to make room.

Both are enforced at the moment the socket is accepted, using the peer address — there is no request yet, so there is no `X-Forwarded-For` to consult. That is fine: loopback is uncounted, which covers everything arriving through `proxy.py`, and a direct flood carries its own address.

---

5. The kernel guard

`setup-firewall.sh` installs an nftables table that sheds volume before `spectrumserver` ever reads a byte. It is deliberately blunt, and its numbers sit well above the ones in `config.toml`, so a normal listener can never be caught by both.

```
./setup-firewall.sh --show        # print the rules, change nothing
sudo ./setup-firewall.sh --check  # validate against your kernel
sudo ./setup-firewall.sh --apply  # load, with a 60-second auto-rollback
sudo ./setup-firewall.sh --persist # reload at boot
sudo ./setup-firewall.sh --status  # the per-rule packet counters
sudo ./setup-firewall.sh --remove  # undo all of it
```

It covers four things: a concurrent-connection ceiling per source address on your proxy and receiver ports, a new-connection rate per source, an SSH brute-force brake, and Windows file sharing closed to everything outside private address ranges — `smbd` listens on `0.0.0.0` on many machines, and whether that is reachable depends on a router the script cannot see.

Ports are read from `admin_config.json`, the same file `proxy.py` uses, so it follows your setup.

Why it cannot lock you out

Two properties, both deliberate:

- The table's policy is **accept**, and it only ever drops specific named patterns. A rule that fails to match lets the packet through to whatever else you have configured. It cannot black-hole the machine.
- **Established connections are accepted in the very first rule.** The SSH session you are typing in is never affected by anything that follows.

On top of that, `--apply` arms an automatic rollback: load the rules, then confirm within 60 seconds or they are removed again. Say nothing, close the terminal, lose your connection — the ruleset disappears by itself.

Use that window properly. Check **from another device, off your own network** that the receiver still loads and that a *new* SSH session still opens. Testing with the session you already have proves nothing, because it was accepted by rule one.

6. Configuration reference

Every key below lives under `[limits]` in your `.toml` file. **All of them default to off or to generous values**, so a configuration that does not mention them behaves exactly as it always did.

Key	Default	Meaning
`per_ip`	`0` (off)	Simultaneous listeners per address
`per_ip_sockets`	`0` → `per_ip * 4 + 4`	Simultaneous sockets of any path per address
`per_ip_rate`	`0` (off)	New sockets per minute per address
`per_ip_ban_s`	`600`	Seconds an address is refused after breaking the rate
`idle_per_ip`	`8`	Idle, pre-request connections per address
`idle_total`	`512`	Idle connections across all addresses

Setting any key to `0` disables that particular check.

The three that are not limits

`[limits]` also holds `audio`, `waterfall` and `events`, inherited from upstream PhantomSDR, and **none of the three is enforced.** `waterfall` and `events` are read by no code at all. `audio` is read in exactly one place: it is posted as `max_users` in the registration JSON sent to the directories in `[websdr]` `register_urls`, so it is the figure sdr-list.xyz and the others display, and it does nothing whatsoever when `register_online=false`. If you want a cap on listeners, `per_ip` above is the one that acts.

One more, under `[server]`

```
[server]
min_client_version=0
```

Not a limit but a refusal, and it shares the same close code, so it belongs here. Pages announce the build they were loaded from as `/audio?v=N`, and anything below `min_client_version` is turned away with "*this page is out of date — please reload it*".

It exists because the server cannot reach JavaScript that is already running in somebody's browser. A tab keeps the code it loaded until someone reloads it, so a change to the page only reaches listeners who happen to reload — and a station that has just changed how sessions behave may need the old pages gone now, not eventually.

Only `/audio` is checked, because that is the socket a session lives on, and loopback and the Kiwi paths are exempt so the autorun tap and Kiwi clients are untouched. `0` disables it and is the default. Leaving it on once the old pages have drained is fine: a current diversity client that is refused on a bare `/audio` retries with the marker, so what is still turned away is a diversity client on an older PhantomSDR-Plus build and any third-party tool that opens `/audio` without `?v=`. Set it back to `0` if another station reports that it can no longer use you as a diversity partner.

The configs that ship with PhantomSDR-Plus set `min_client_version=2`, the current build (`CLIENT_VERSION` in `frontend/src/clientVersion.js`), with a comment that explains it; without the line it is `0`. If a future update raises `CLIENT_VERSION`, raise this value to match.

7. What a refused listener sees

Nothing ever reconnects

Start here, because it governs everything else: **a dropped `/audio` socket ends the session.** There is no automatic reconnection, by design.

A reconnect is the right instinct for a chat application and the wrong one for a receiver. `/waterfall` and `/events` never came back with it, so a retried session was a live audio socket bolted to a frozen waterfall; and stopping the server no longer cleared it, because every listener was back a few seconds later, filling the user list with sessions nobody was really in. The page goes quiet and the listener loads it again — which is what this receiver did before a reconnect was ever added.

For the limits in this manual that also removes a trap: against a rate limit, an automatic retry would extend the very ban it was trying to get around.

The two close codes

Code	Meaning	Who sends it
4003	refused — over a per-IP limit, or the page is out of date	`spectrumserver`
4001	kicked or banned by the sysop	`spectrumserver`, when the admin panel calls `/~~kick`

Both are in the 4000–4999 private range, so neither can be confused with a protocol status, and both are treated as final by every page. The reason text travels in the close frame rather than as a message, because the first frame on `/audio` is reserved for the receiver's settings and anything sent ahead of it would break every normal connection.

On the page

Refused as the page loads — the listener never gets going, so both pages explain themselves. The desktop shows a panel: "*This receiver refused the connection*", the reason, and the note that everyone sharing an internet connection counts as one address. `/mobile` shows the same in its notice area.

Refused or kicked mid-session — the desktop page simply stops, deliberately without a message: the waterfall's draw loop halts and that is all. `/mobile` shows one line telling the listener to reload, except after a kick, where it too stays silent.

That silence after a kick is intentional. **Kicking is the sysop ending a session, not a punishment** — nothing is announced, and whoever wants back in loads the page again and is an ordinary listener one second later. The only thing the 4001 code buys is that the browser knows this was not a dropped connection, so the kick actually takes instead of undoing itself.

A **ban** — the admin panel's Ban ▾ — is the one exception: it is the same kick with an 8-, 16- or 24-hour refusal attached, so the reload is turned away with 4001 until the ban runs out or the sysop lifts it.

The kick is done by `spectrumserver` itself, not by `proxy.py`: listeners connect to the receiver's own port directly, so the proxy owns none of their sockets and its version of a kick reaches nobody. The panel calls `/~~kick?ip=&secs=`, which is loopback-only and checks the real TCP peer rather than `X-Forwarded-For`, that being client-supplied. `secs` defaults to zero — no ban — but it accepts a length if you do want the address kept out for a while, and that reuses the same ban machinery as `per_ip_rate` so the reconnect is refused at the door. The panel's Ban 24h sends `secs=86400`, the most it accepts, and `/~~unban?ip=` lifts a ban early.

8. Reading the logs and counters

The server

```
grep Refused spectrumserver.log
```

```
Refused /audio from 203.0.113.5: too many simultaneous connections
Refused /audio from 198.51.100.7: too many connection attempts
```

Nothing there means nobody is being turned away. Logging is throttled to at most one line per address every five seconds — without that, an attack would turn into unbounded disk writes, which is simply a slower way of taking the station down.

The kernel

```
sudo ./setup-firewall.sh --status
```

Read the `counter packets N` on each rule; that is the number actually dropped. All zeros means nothing has been blocked and your thresholds are comfortably loose.

Do not be alarmed by the `elements = { ... }` lists in the sets above the rules. Those are your ordinary listeners being *tracked* against the limits — an entry appears on an address's first connection and ages out a couple of minutes later. Tracking is not blocking.

---

9. Choosing your numbers

Guessing is unnecessary; your own logs will tell you. This is the measurement that set `per_ip=3` here, run over a week of `logs/users_*.jsonl`:

Concurrent sessions from one address	Number of addresses
1	682
2	54
3	13
more than 3	**8**

758 distinct addresses over seven days, and a limit of three would have touched eight of them. The flood sat at 181.

To repeat it on your own station, pair the session ids per address in `logs/users_*.jsonl` and take the maximum overlap. Note that the log records `tune` and `disconnect` events but has no `connect` event, so a session's start has to be inferred from its first `tune`.

Remember NAT. A club, a school, an office or a mobile carrier appears as a single address, and everyone behind it shares one allowance. This is inherent to any limit keyed on an address; the only mitigation is choosing a number you are comfortable with. If a listener ever reports being unable to connect, this is the first thing to suspect — check `grep Refused` before anything else.

---

10. Traps worth knowing about

Never put a deadline on an idle connection

There is no `handshake_s` knob any more, and this is why. A 30-second deadline on idle connections looks like the obvious defence against a slowloris. It was built, switched on, and produced this within a minute:

```
Idle deadline: closed 1 connection(s) idle past 30s: 192.87.173.88
```

`192.87.173.88` is `etgd-websdr.ewi.utwente.nl` — **websdr.org's own callback host**. It connects back to your server and then sits idle, for well over 30 seconds, before sending its `GET /~~orgstatus`. The deadline cut it every time and registration stopped. The setting was removed rather than left as an option, because a footgun that breaks your directory listing is not worth the sixty lines that implement it.

The same reasoning applies to websocketpp's `open_handshake_timeout`, which is still there. The ten-minute value in `spectrumserver.cpp` looks absurd and is load-bearing: it is what allows the websdr.org callback to sit idle at all. There is a second reason too — the `/~~orgstatus` handler answers from a thread holding a `dup()` of the socket, and websocketpp ends a timed-out connection with `shutdown()`, a socket-level call that reaches through the duplicate and would break the callback even if the idle time were not an issue.

If you need to bound silent connections, bound their number, not their age. That is what `idle_per_ip` and `idle_total` are.

An nftables file must be idempotent

`nft -f` **appends** to a table that already exists rather than replacing it. A ruleset file without a `delete table` prologue therefore doubles the entire chain every time it is reloaded — which is what a naive systemd unit does on every service restart. `setup-firewall.sh` generates the prologue for you; if you write your own rules, do the same.

```
sudo ./setup-firewall.sh --status | grep -c dport   # should be 8, not 16
```

---

11. Turning it all off

The server limits: set the keys to `0`, or delete them from your `.toml`, and restart the receiver. There is no other state; nothing persists across a restart.

The firewall:

```
sudo ./setup-firewall.sh --remove
```

That deletes the table, removes the saved ruleset and disables the boot service. Your machine returns to exactly the state it was in before — which, on most installations, means no firewall at all. That is worth thinking about before you remove it.

CHAPTER 6

Several Receivers

MULTI_RECEIVER.md

Several Receivers — Sysop Manual

> **Port numbers.** The examples below use the ports of a station set up before the station questions (8900, 8901, …). On a station set up with `configure-station.sh`, `proxy.py` already owns the public port 9000 and the main receiver is on 9001, so give further receivers 9002, 9003 and so on. The station questions keep `receivers.toml` in step: when a port or the address changes, `configure-station.sh` moves the main receiver to its new port and writes every link as `<address>/?rx=<id>` — a link without `?rx=` would keep a visitor on the receiver the `rx` cookie remembers. Never give a receiver the public port itself: the proxy would forward to itself, so it now refuses that.

Running two or more receivers on one computer, with a receiver picker on the page.

Since v5.0.0 one PhantomSDR-Plus computer can run several receivers at the same time — for example an RX-888 for HF and an RTL-SDR for 2 m — and listeners switch between them with buttons in the page header, the way OpenWebRX offers its profiles. Every receiver keeps its own waterfall, chat, markers, listener list and station details; nobody on one receiver is ever disturbed by someone on another.

> **In a hurry?** A station with one receiver needs none of this and sees no change. For a second receiver: create `instances/<name>/` with its own `config.toml`, start it with `INSTANCE=<name> ./start-<radio>.sh`, list both receivers in `receivers.toml`, and restart the proxy. The worked example in section 3 goes through it line by line.
>
> **The easiest way:** run `./add-receiver.sh` (the installers offer it at the end). It asks which receiver and what it should cover, installs its driver, writes everything described in section 3 and section 4, and offers to start the receiver and restart the proxy. The rest of this manual explains what it does, for when you want to change something by hand.
>
> **From the admin panel:** the **Receivers** page lists every receiver with what it covers. ✎ **CHANGE SETTINGS** changes an extra receiver's frequency and bandwidth (and restarts only that one), and on every receiver the start frequency, mode, gain and button name — an extra receiver keeps its gain in `instances/<id>/instance.env`, the main one in `main.env`, which every start-*.sh reads last.* ■ *STOP* / ▶ *START stops or starts one receiver while the others keep running, and DELETE** removes an extra receiver (stops it, takes it out of `receivers.toml`, moves its `instances/<id>/` folder to `admin-backups/`). It also prepares a new receiver: it saves your answers and gives `./add-receiver.sh --answers instances/.add-rx.env` to run in a terminal. See Admin Panel.

Contents

1. How it works
2. Two ways to publish the receivers
3. Adding a second receiver
4. receivers.toml
5. Starting and stopping
6. What listeners see

7. The S-meter above 30 MHz
8. Security
9. How many receivers fit
10. Traps worth knowing about

1. How it works

Each receiver is a complete spectrumserver of its own, with its own SDR front end, its own configuration and its own internal port. They never share samples, settings or listeners. What ties them together is `proxy.py`, the reverse proxy that already serves the admin panel: it reads `receivers.toml` and sends every request to one receiver.

```
                     ┌──► :8900  spectrumserver  RX-888   (HF, the main receiver)
listeners ──► proxy.py ┤
                     └──► 127.0.0.1:9002  spectrumserver  RTL-SDR  (2 m, instance "vhf")
```

The proxy decides by these, in order:

1. `?rx=<id>` in the address — `http://your.host:8899/?rx=vhf`. The page adds it to every socket, poll and link it opens, so a tab stays on its receiver.
2. The `rx` cookie, which the proxy sets whenever a request carries a valid `?rx=`.
3. The Host header, if a receiver lists that name under `hostnames`.
4. Otherwise the default receiver.

The **main receiver** is the one started the usual way (`./start-rx888mk2.sh`). Every other receiver is a **named instance**: started with `INSTANCE=<name>`, configured in `instances/<name>/`, and given its own log (`logwebsdr-<name>.txt`), server log, watchdog lock and FIFO. Processes are told apart by a `PHANTOMSDR_INSTANCE` tag in their environment, not by their name, so two receivers may run the very same program — two `rtl_sdr`, or `rx_sdr` for both an RSP1A and an Airspy — and a restart of one never touches the other.

2. Two ways to publish the receivers

A — the main receiver keeps its own port. The main receiver stays exactly where it was (say `:8900`), listeners there notice nothing, and the other receivers are reached through the proxy's port (`proxy_port` in `admin_config.json`, say `:8899`) with `?rx=`. Nothing has to be moved and nothing goes off the air; it does mean two public ports.

```
HF:  http://your.host:8900/
2 m: http://your.host:8899/?rx=vhf
```

B — one public port for everything. The proxy takes over the public port and the main receiver moves to an internal one. Set `[front] port` in `receivers.toml`, move the main receiver's `[server] port` and set `host = "127.0.0.1"` on it, set `public_port` in `admin_config.json` to the new internal port, and add `public_port = <the public port>` under `[websdr]` so the directory listing keeps announcing the right one. Everything then lives behind one port; the cost is a short outage while you move it.

```
HF:  http://your.host:8900/
2 m: http://your.host:8900/?rx=vhf
```

Either way the receiver picker works the same. Layout A is the safer start; B can follow later without changing anything for the second receiver.

---

3. Adding a second receiver

`add-receiver.sh` does all of the steps below — run it and answer its questions; it also installs the driver (`setup-rtlsdr.sh` for an RTL-SDR, the matching `setup-*.sh` for the others), picks a free internal port and names the receiver after what it covers. The steps are written out here so that you can check its work or change it later.

The example adds an RTL-SDR Blog V4 for 2 m as instance `vhf`, next to an RX-888 that keeps port 8900 (layout A).

1. The driver. The RTL-SDR Blog V4 needs the RTL-SDR Blog driver rather than the distribution's `rtl-sdr` package, and the kernel's DVB-T driver must let go of the stick. `install.sh` (receiver option 2, "RTL-SDR Blog V4: yes") does both. Check with `rtl_test -t`, which must say `RTL-SDR Blog V4 Detected`.

2. The instance folder. Everything that belongs to the receiver lives in `instances/vhf/`, which is never committed and never overwritten by an update:

```
instances/vhf/
├── config.toml          its spectrumserver configuration (required)
├── instance.env         optional: receiver arguments and CPU pinning
├── markers.json         its own markers
└── www/                 its own copies of the page's per-station files
    ├── site_information.json
    └── wf-message.json
```

spectrumserver runs with `instances/vhf/` as its working directory, so its chat history, markers, FFTW wisdom and `logs/` are its own as well.

3. `config.toml`. Start from `config-rtl.toml` and change:

```
[server]
port=9002                              # its own internal port
host="127.0.0.1"                       # reachable only through the proxy
html_root="www/"                       # its own few files...
html_fallback_root="../../frontend/dist/"   # ...and the shared page for the rest

[input]
sps=2400000                            # must match -s in instance.env
frequency=145000000                    # must match -f in instance.env

[input.defaults]
frequency=145500000
modulation="FM"

[kiwi_emulation]
enabled = false                        # see the traps below
```

`html_fallback_root` lets the instance keep only its own `site_information.json` and `wf-message.json` while everything else — the built page itself — comes from `frontend/dist`, so one frontend build reaches every receiver. A symlinked copy of `frontend/dist` does not work: the server refuses files that resolve outside its own folder.

4. `instance.env`. Read by the launcher after its own settings:

```
RX_ARGS="-f 145000000 -s 2400000 -g 29.7 -"   # -f and -s match config.toml
SPECTRUM_CORES=8-11                           # keep it off the main receiver's cores
```

5. Its page details. Copy `frontend/site_information.json` to `instances/vhf/www/` and edit it: `siteReceiver`, `siteAntenna`, `siteSDRBaseFrequency` and `siteSDRBandwidth` (these choose which band buttons appear), and the keys described in section 6.

6. Start it.

```
INSTANCE=vhf ./start-rtl.sh
```

7. List it in `receivers.toml` and restart the proxy (`sudo systemctl restart phantomsdr-proxy`). The second receiver is now at `http://your.host:8899/?rx=vhf`, and both pages show the picker.

---

4. receivers.toml

Copy `receivers.toml.example` to `receivers.toml`. Without that file the proxy serves a single receiver exactly as it always has.

```
# [front]
# port = 8900        # layout B only: the proxy also listens on this public port

[[receiver]]
id       = "hf"
name     = "HF 0-30 MHz (RX-888 MkII)"
port     = 8900
default  = true
launcher = "start-rx888mk2.sh"
url      = "http://your.host:8900/"

[[receiver]]
id       = "vhf"
name     = "2 m (RTL-SDR Blog V4)"
port     = 9002
launcher = "start-rtl.sh"
instance = "vhf"
url      = "http://your.host:8899/?rx=vhf"
```

Key	Meaning
`id`	Short name used in `?rx=`; letters, digits, `-` and `_`
`name`	The text on the receiver's button
`port`	Its spectrumserver's `[server] port`
`host`	Optional; where that spectrumserver runs (default `127.0.0.1`)
`default`	The receiver for requests that name none; at most one
`launcher`	Its `start-*.sh`, for `start-all.sh`; leave it out for a receiver this computer does not start
`instance`	Its instance name; leave it out for the main receiver
`url`	Where its button sends a listener; without it, `/?rx=<id>` on the proxy
`hostnames`	Optional DNS names routed straight to this receiver

The proxy also serves the list as `/receivers.json` (ids, names and links — never the internal ports), which is what the picker reads.

5. Starting and stopping

Command	Does
`./start-rx888mk2.sh`	Starts or restarts the main receiver
`INSTANCE=vhf ./start-rtl.sh`	Starts or restarts the `vhf` receiver
`./start-all.sh`	Starts every receiver in `receivers.toml` that has a `launcher`
`./stop-websdr.sh vhf`	Stops only `vhf`
`./stop-websdr.sh main`	Stops only the main receiver
`./stop-websdr.sh`	Stops every receiver, as it always did

While a receiver is stopped, its button disappears from the picker — the proxy lists only the receivers that answer, and open pages re-read the list every 15 seconds — and comes back when the receiver starts again; with only one receiver left running, the *Receivers:* line disappears altogether. A stopped receiver does come back by itself whenever every receiver is started: `./start-all.sh`, the admin panel's **Restart** and a thermal-guard restart all start everything listed in `receivers.toml`. To keep one off for good, delete it on the admin panel's **Receivers** page, or remove (or comment out) its `[[receiver]]` block by hand — the proxy re-reads `receivers.toml` when it changes, so a rename, a removal or a new block takes effect without a proxy restart (only the first `receivers.toml` on a station, or a change of `[front] port`, still needs one).

Set the admin panel's **Default start script** to `start-all.sh`. Its Stop already stops every receiver, and its Restart and the thermal guard use the start script, so with `start-all.sh` they bring back every receiver instead of only the main one.

The RADE sidecar belongs to the main receiver only. A named instance never starts or stops it.

6. What listeners see

The receiver picker. On a station with more than one receiver the page header gets a line of its own, *Receivers:*, with one button per receiver; the current one is shown in yellow. The /mobile page has the same buttons as a second row in its top bar, and they open the other receiver's /mobile page.

Show All strips. A *Show All* checkbox after the receiver buttons puts each other receiver under the listener's waterfall as a live strip. A click there opens that receiver tuned to the frequency clicked, and a page the listener left reopens on the view they left it with. Each strip is a waterfall-only socket to the other receiver (`/waterfall?rx=<id>`): it is not a listener there — the user count and the per-IP listener limit do not see it, `per_ip_sockets` does — and it costs about 10 KB/s of upload for each listener who ticks the box. It is off by default and needs nothing on the server. When the other receiver covers more than one band, the strip's header offers that receiver's bands in a dropdown (the same ones its band buttons show); a band frames the strip as its band button does, and a click then opens the receiver framed on that band. The choice is kept per receiver in the listener's browser.

Switching by frequency. A frequency outside this receiver's coverage that another receiver covers — typed, from a link, a bookmark or a spot — opens the receiver that covers it. The page learns each receiver's range by opening its `/waterfall` socket for a moment, and remembers it for a day.

Users and statistics per receiver. The *Connected Users* and *User Statistics* windows show the receiver whose page they are on, even with several receivers open side by side in one browser. The statistics come from that receiver's own logs — `logs/` for the main receiver, `instances/<id>/logs/` for the others.

Each receiver's own details. A page opened on a second receiver loads that receiver's `site_information.json` before it starts, so the band buttons, the starting frequency, the users list, the station details and the Receiver and Antenna links are all its own. A plain visit to the main receiver makes no extra request.

Keys in `site_information.json` for this:

Key	Meaning
`siteReceiverId`	The receiver this page belongs to (its `id`); leave it out on the main receiver
`siteReceiversList`	Where the picker reads the list, e.g. `http://your.host:8899/receivers.json`; needed on a receiver whose page is not served through the proxy
`siteReceiverURL`	Where the *Receiver* name in *Open Additional Info* links to
`siteAntennaURL`	Where the *Antenna* name links to; `""` shows the name without a link

When the picker stays hidden. The picker only shows a list that names the host the page was opened on. Opened by LAN address (`http://192.168.1.10:8900/`) it stays hidden; opened as `http://your.host:8900/` it appears. That is deliberate: a station that copied someone else's `site_information.json` without editing it never shows that station's receivers as its own.

---

7. The S-meter above 30 MHz

The IARU Region 1 standard puts S9 at −73 dBm below 30 MHz and at **−93 dBm above**, six dB per S-unit on both. The S-meters (the analog needle, the digital bar and the /mobile bar) follow the tuned frequency: above 30 MHz they read VHF S-units, below it they read as before. The dBm and dBµV figures are never changed. If you prefer the meters to rest at 0 on an empty channel, like a VHF/UHF transceiver's, add `"siteSMeterGateDb": 6` to the receiver's `site_information.json` (no rebuild needed): above **60 MHz** the needle and the bar then come up only when a signal stands that many dB above the noise in its own neighbourhood (the passband's strongest point against the spectrum ±100 kHz around it). It is **off by default** — a calibrated meter showing the real band noise is the more honest reading. The /mobile page, which has no waterfall, estimates the noise from the received level itself.

Calibrate a VHF receiver's dBm with `analog_smeter_offset` (needle and figures) and `smeter_offset` (digital bar) under `[input]` in its `config.toml`, then restart that receiver. Set the receiver's gain first: an RTL-SDR reports levels relative to its own full scale, so every gain change moves the readings. Without a signal generator, a 50 Ω terminator in place of the antenna, USB with a 2.7 kHz filter, should read about −136 dBm (thermal noise in 2.7 kHz is −139.7 dBm, plus the stick's noise figure).

For an RTL-SDR, `add-receiver.sh` writes a starting calibration: a fixed gain of `-g 29.7` in `instance.env` (the tuner's automatic gain cannot be calibrated) and `analog_smeter_offset=-55`, `smeter_offset=-38`, measured on an RTL-SDR Blog V4 at that gain with a −71 dBm (63 µV) signal. Another stick of the same model reads within a few dB of that; check it with a known signal, and recalibrate if you change the gain.

---

8. Security

Putting the proxy in front of listeners closed two holes, both fixed in v5.0.0:

- **The sysop kick through the proxy.** spectrumserver allows `/~~kick` only from its own computer, and to it every request relayed by the proxy comes from its own computer. Anyone who could reach the proxy port could therefore disconnect and ban any listener. The proxy now answers `/~~kick` only for clients on the same computer. The same holds for `/~~users` (the users list with IP addresses) and `/~~unban`, added in 5.3.0; spectrumserver also refuses those two when the request carries `X-Forwarded-For`, as everything relayed by the proxy does.
- **Faked client addresses.** spectrumserver believed an `X-Forwarded-For` header from anyone, so a visitor could claim to be `127.0.0.1` and walk past the per-IP limits. The header is now believed only from a local proxy, and the proxy drops any copy a client sends before adding its own.

Bind every receiver that is reached through the proxy to `127.0.0.1` (`[server] host`), so its port cannot be used around the proxy. The per-IP limits in Connection Limits apply per receiver.

---

9. How many receivers fit

The software sets no limit; the hardware does. Narrowband receivers are cheap: an RTL-SDR at 2.4 Msps uses about 5–10 % of one core, a few dozen MB of memory, and about 38 Mbit/s of USB.

- **USB 2.0 is the usual ceiling.** Every USB 2 device shares one 480 Mbit/s bus, even in a blue USB 3 socket. Three to four RTL-SDRs per bus is a safe figure; add them one at a time and watch every waterfall for gaps.
- **Power:** use a powered hub beyond two sticks.
- **A second wideband receiver** (another RX-888, or a HackRF at 20 Msps) is a different matter: it competes with the first for USB 3, CPU and GPU.
- **Listeners cost more than receivers.** Each listener costs CPU on the receiver they are on and upload bandwidth, so the total number of listeners matters more than the number of receivers.

---

10. Traps worth knowing about

- **Two sticks of the same model have the same serial.** Every RTL-SDR Blog V4 reports `00000001`. Give each its own with `rtl_eeprom -s <serial>` (one stick plugged in at a time) and name it in `instance.env` (`RX_ARGS="-d <serial> …"`), or after a reboot the sticks may swap receivers.
- **Kiwi clients cannot choose a receiver.** A KiwiSDR client dials a bare address and port; through the proxy it always lands on the default receiver. Keep `[kiwi_emulation]` off on the others.
- **Cookies are per host, not per port.** A visitor who used the second receiver on `:8899` carries `rx=vhf` to `:8900` too. That is why a page decides which receiver it is from `siteReceiverId`, never from the cookie.
- **Directory listings.** Give each receiver its own name in `[websdr]`, and on a receiver behind the proxy set `[websdr] public_port` to the port listeners use, or the listing advertises the internal one.
- **A frontend rebuild briefly shows "Not Found".** `frontend/dist` is rebuilt in place, so for a few seconds during a build the page is missing for every receiver.
- **Nothing starts the receivers at boot** unless you arrange it. After a reboot, `./start-all.sh` brings them all back.

CHAPTER 7

Secure Access (HTTPS)

HTTPS.md

Secure Access — https://

A PhantomSDR-Plus station can be reached over **https://** as well as http://. The browser then shows the padlock, the connection is encrypted, and browsers allow the page their better audio path (the AudioWorklet), which they keep from plain http pages. The http address keeps working exactly as before, side by side.

It is done by **Caddy**, a small web server placed in front of the receiver on port 443. Caddy obtains a free certificate from **Let's Encrypt** for your station's DNS name and renews it by itself, every couple of months, with nothing for you to do.

What you need

A DNS name	Let's Encrypt certifies names, not bare IP addresses. A free dynamic-DNS name (no-ip, DuckDNS, dynu and the like) pointing at your public address is fine, and it may change address — the name follows.
Two more ports on the router	Forward TCP **443** (https) and TCP **80** to the receiver computer, as well as your public port (9000). Port 80 is only used to prove to Let's Encrypt that the name is yours when the certificate is issued and renewed.
A station set up with the station questions	`station.conf` must exist. On a station installed before the station questions, run `bash configure-station.sh` once first — it offers the ports your station already uses, so nothing moves.

Turning it on

At installation

Among the station questions, under **Internet**, answer yes to:

```
  Also serve the receiver over https://? [y/N]: y
```

The installer installs Caddy, obtains the certificate and, at the end, shows both addresses.

On a station that already runs

```
cd ~/PhantomSDR-Plus
bash setup-https.sh
```

It reads your DNS name from `station.conf`, installs Caddy (from your distribution; on Ubuntu 22.04 from Caddy's own repository), points it at the receiver, rebuilds the page so that it links its https address, restarts the proxy, and waits for the certificate. Answering yes to the https question in `bash configure-station.sh` does exactly the same.

When it finishes:

```
  ✔ https://myname.ddns.net/ is live

  https://myname.ddns.net/   — and http://myname.ddns.net:9000/ as before
```

Checking and turning it off

```
bash setup-https.sh --status    # is it on, and does the certificate answer?
bash setup-https.sh --remove    # turn https off; http stays exactly as it is
```

The admin panel shows the same under **Settings → HTTPS**: **CHECK** on an https station, and on an http-only one **SET UP HTTPS**, which checks what it can and gives the commands above to copy into a terminal. See Admin Panel.

Changing the DNS name later with `bash configure-station.sh` moves Caddy to the new name by itself.

How it works

```
  visitor ──https──► Caddy :443 ──► proxy.py :9014 (this computer only) ──► spectrumserver, panel,
RADE …
  visitor ──http─────────────────► proxy.py :9000 ─────────────────────────► the same
```

Port	What	Open on the router
443	Caddy — https	yes
80	Caddy — the certificate check, and a redirect to https	yes
9000	proxy.py — http, as before	yes
9014	proxy.py — where Caddy hands the visitors over	no — inside the computer

Caddy reaches the proxy on a port of its own because, seen from the proxy, every visitor coming through Caddy would otherwise come from the computer itself — and a request from the computer itself is trusted: it may disconnect listeners, and it is not counted against the per-IP limits. On port 9014 the proxy takes the visitor's real address from Caddy and never treats a visitor as local, so the listener list, the per-IP limits and the sysop's kick all work exactly as over http, and nobody can kick a listener through https.

What changes for listeners

- **Both addresses work.** Directories and the websdr.org map keep listing the http address; websdr.org's callback and Kiwi clients use http.
- **Better audio over https.** Browsers give a secure page the AudioWorklet, a steadier audio path than the one plain http pages have to use.
- **Receive Diversity with http-only partners.** A browser does not let an https page open plain `ws://` connections, and most PhantomSDR+ stations and every KiwiSDR offer nothing else. From the https page they are therefore reached through the station's diversity relay — see *Receive Diversity over https* below. Partners that have https themselves, and UberSDRs, are reached directly.
- **Rig control (TCI-CAT).** Connections to `127.0.0.1` on the listener's own computer are allowed from an https page in current Chrome, Edge and Firefox. The page connects only once the listener opens

TCI-CAT or turns **CAT Sync** on; Chrome and Edge then ask once for permission to reach apps and devices on the listener's own computer or network. Refusing it only disables TCI-CAT.

Receive Diversity over https

An https page reaches http-only diversity partners — most PhantomSDR+ stations, every KiwiSDR — through the station's own relay, `websdr_relay.py`, the same one that serves WebSDR partners. The browser connects securely to `wss://myname.ddns.net/relay/…`, and the relay connects onward over `ws://`. The panel does this by itself. The relay carries diversity audio only (PhantomSDR+ `/audio`, KiwiSDR `/kiwi/…/SND`), connects only to public addresses, and keeps its usual limits. Without it running, an https page reaches only partners that have https themselves.

So on an https station, keep the relay on:

- **Turning https on with the station questions:** once https is on, the relay question that follows defaults to yes — keep it, and the relay is installed and started as well. `bash setup-https.sh` on its own does not install it.
- **A station that already runs https:** check with `systemctl is-active phantomsdr-websdr-relay`. If it is not active, run `bash setup_websdr_relay.sh`, or `bash configure-station.sh` and answer yes to the relay question. Behind the proxy it needs no router port and no rebuild.
- **Check:** `https://myname.ddns.net/relay/status` answers with a few lines beginning `"service": "PhantomSDR-Plus WebSDR relay"`.

Through the relay the partner sees your station's address, not the listener's, so a partner with a per-IP limit counts all your relayed listeners together. A station served over http only needs none of this: an http page opens `ws://` and `wss://` connections directly.

The local network only — `--lan`

Without a DNS name, https can still be used inside the house:

```
bash setup-https.sh --lan
```

Caddy then issues its own certificate for the computer's local address and name. Browsers do not know that certificate and warn the first time; accept it, or make the computers you use trust Caddy's root certificate:

- on the receiver computer itself: `sudo caddy trust`
- on another computer: copy `/var/lib/caddy/.local/share/caddy/pki/authorities/local/root.crt` from the receiver (the exact path is printed by `sudo caddy environ`, under `caddy.AppDataDir`) and import it into the browser or the system as a trusted authority.

A web server is already running on this computer

If nginx, Apache or another Caddy setup already uses port 443 or 80, `setup-https.sh` does not take it over: it stops and prints what to add to your server instead. For nginx, inside the `server { … }` block of your https site:

```
location / {
    proxy_pass http://127.0.0.1:9014;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header X-Forwarded-For $remote_addr;
    proxy_read_timeout 1d;
}
```

For Apache (with `mod_proxy`, `mod_proxy_http` and `mod_proxy_wstunnel`):

```
ProxyPass        / http://127.0.0.1:9014/ upgrade=websocket
ProxyPassReverse / http://127.0.0.1:9014/
```

Then mark the station as https in `station.conf` so that the proxy opens port 9014 and the page links its https address — the wizard will say once more that the port is taken, which is expected:

```
STATION_HTTPS=y bash configure-station.sh
```

Troubleshooting

Symptom	What to check
`no certificate yet`	The DNS name must point at your public address (`ping myname.ddns.net` from outside your network), and the router must forward 443 and 80 to this computer. Caddy keeps trying by itself; its log: `sudo journalctl -u caddy -n 50`.
The provider blocks port 80	Caddy also proves the name over port 443 alone, so 443 is enough in most cases; leave 80 forwarded if you can.
`too many failed authorizations` in Caddy's log	Let's Encrypt allows a few failures per hour. Fix the cause, wait an hour, then `sudo systemctl restart caddy`.
Caddy's log shows `Invalid response … /.well-known/acme-challenge/…` (a 502 or the receiver's page), or `Timeout during connect`	The router sends 80 or 443 somewhere else — often to the receiver's public port. The rules must be TCP 80 → this computer, port **80**, and TCP 443 → this computer, port **443**; switch off the router's own remote management on those ports. Then `sudo systemctl restart caddy` to try again at once.
https works at home but not from outside	The router forwards 443 to another computer, or not at all.
The listener list or the users panel is empty over https	Run `bash configure-station.sh` once and say yes to the rebuild: the page must know its https address.
`Port 443 is already used by another web server`	See *A web server is already running on this computer* above.

CHAPTER 8

Statistics Server

sdr-stats/README.md

System Stats Installation Guide

A comprehensive guide to add real-time server monitoring to your PhantomSDR application using the automated installation script.

Overview

This feature adds a **Stats** button to your PhantomSDR interface that displays real-time system information:

- CPU usage, cores, frequency, temperature, and top processes
- Memory usage (used/total/percentage)

Stats automatically update every 5 seconds while the modal is open.

> **Not the same as the admin panel's Graphs page.** This modal is a live snapshot that refreshes while it is open and keeps no history. The admin panel has a separate **Graphs** page that samples CPU frequency, load, temperature and users online every 2 seconds and plots them over the last 15 minutes to 24 hours — see the Admin Panel guide.

Prerequisites

- **Linux server** (Ubuntu/Debian recommended)
- **Internet connection** (for downloading Node.js and packages)
- **Sudo access** (for systemd service setup)
- **PhantomSDR-Plus** already installed and running
- **lm-sensors** (recommended for Intel/AMD CPU temperature - automatically installed by script)

Installation Methods

> **NOTE**
>
> **The main installer already offers this.** `./install.sh` — and the four distro installers — run `install-stats-server.sh` by default as part of the normal PhantomSDR-Plus installation, so on a fresh machine the stats server is usually already in place. Use this page to install it separately, or to change its port, address or service afterwards.

Option 1: Automated Installation (Recommended)

The automated script handles everything for you.

Step 1: Run the script

```
cd ~/PhantomSDR-Plus
chmod +x install-stats-server.sh   # only if it is not already executable
./install-stats-server.sh
```

IMPORTANT: Do NOT run as root or with sudo. Run as your regular user.

Step 2: Follow the interactive prompts

The script will guide you through:

1. Node.js Installation (if needed)

```
⚠ Node.js / npm is not installed
Would you like to install Node.js 22 via nvm now? (y/n): y
```

The script installs Node.js 22 through nvm, on any Linux distribution — no root and no apt source needed. It asks the same question when Node.js is present but older than 22.

2. Installation Directory

```
Installation directory [/home/user/sdr-stats-server]:
```

Press Enter for default, or specify a custom path.

3. Port Configuration (Enhanced!)

```
Port Configuration:
  Default port: 3001
  Common alternatives: 8080, 5000, 8888
Enter port number [3001]:
```

The script validates your port:

- Checks if it's a valid number
- Warns if port < 1024 (requires root privileges)
- Validates range (1-65535)
- Checks if port is already in use
- Shows which process is using it if occupied
- Allows you to choose a different port

Example scenarios:

Using default port:

```
Enter port number [3001]: ↵
✓ Port 3001 selected
```

Choosing custom port:

```
Enter port number [3001]: 8080
✓ Port 8080 selected
```

Port already in use:

```
Enter port number [3001]: 3001
⚠ Port 3001 is currently in use
Process using port 3001:
node    12345 user   20u  IPv6 123456  TCP *:3001 (LISTEN)
Choose a different port? (y/n): y
Enter port number [3001]: 3002
✓ Port 3002 selected
```

Invalid port:

```
Enter port number [3001]: abc
✗ Port must be a number
Enter port number [3001]: 99999
✗ Port must be between 1 and 65535
Enter port number [3001]: 3001
✓ Port 3001 selected
```

4. Server Address

```
What is your server's public address?
Examples: Your_site_IP, 192.168.1.100, localhost
Server address: Your_site_IP
```

Enter your server's public domain or IP address.

5. Confirmation

```
Please confirm your settings:
  Installation directory: /home/user/sdr-stats-server
  Port: 3001
  Server address: Your_site_IP
  Stats URL will be: http://Your_site_IP:3001
Continue with these settings? (y/n): y
```

6. Systemd Service Setup (Optional)

```
Would you like to set up the server as a system service (auto-start on boot)? (y/n): y
```

Recommended: Choose **y** to have the server start automatically on boot.

Step 3: Installation Complete! ✓

The script will display a summary:

```
╔═══════════════════════════════════════════╗
║          Installation Complete! ✓         ║
║          Open the port in the router!     ║
╚═══════════════════════════════════════════╝

Configuration Summary:
  Installation: /home/user/sdr-stats-server
  Port: 3001
  Server: Your_site_IP
  API URL: http://Your_site_IP:3001/api/system-stats
```

Configure Your PhantomSDR Application

After the script completes, you need to update your Svelte application.

Step 1: Update `site_information.json`

Since `site_information.json` was already edited during the initial setup, you only need to add the new `siteStats` line with your port:

```
{
    "siteSysop": "your name or callsign",
    "siteSysopEmailAddress": "mail@mail.net",
    "siteGridSquare": "QTH locator",
    "siteCity": "City Country",
    "siteInformation": "https://github.com/sv1btl/PhantomSDR-Plus",
    "siteHardware": "Hardware you are using, ",
    "siteSoftware": "Software you are using",
    "siteReceiver": "Receiver model",
    "siteAntenna": "Receiving Antenna.",
    "siteNote": "This is a bright new open-source WebSDR project, under active development.",
    "siteIP": "http://Your_site_IP:port",
  "siteStats": "http://Your_site_IP:3001",  ← ADD THIS LINE (use your port)
  "siteSDRBaseFrequency": 0,
  "siteSDRBandwidth": 30000000,
  "siteRegion": 1,
  "siteChatEnabled": true
}
```

Note: Use the port you selected during installation!

Step 2: Update `App.svelte` (the latest GitHub version already includes the updated file)

Make these **4 changes** to your App.svelte file:

Change 1: Ensure that `siteStats` is declared (around line 50)

```
import {
  siteSysop,
  siteSysopEmailAddress,
  siteInformation,
  siteGridSquare,
  siteCity,
  siteHardware,
  siteSoftware,
  siteReceiver,
  siteAntenna,
  siteNote,
  siteIP,
  siteStats,  // ← ADD THIS LINE
  siteSDRBaseFrequency,
  siteSDRBandwidth,
  siteRegion,
  siteChatEnabled,
} from "../site_information.json";
```

Change 2: Update systemStats object (around line 530)

```
let systemStats = {
  cpu: { usage: 0, cores: 0, temperature: null, frequency: null, topProcesses: [] },  // ← ADD
topProcesses: []
  memory: { used: 0, total: 0, percent: 0 }
};
```

Change 3: Update fetch URL (around line 541)

```
async function fetchSystemStats() {
  try {
    const response = await fetch(`${siteStats}/api/system-stats`);  // ← CHANGE THIS LINE
    if (response.ok) {
      systemStats = await response.json();
    } else {
      console.error('Failed to fetch system stats:', response.statusText);
    }
  } catch (error) {
    console.error('Error fetching system stats:', error);
  }
}
```

Change 4: Add top processes display (in CPU section, around line 3838)

Add this code after the temperature section:

```
{#if systemStats.cpu.frequency}
<div style="display: flex; justify-content: space-between; margin-bottom: 0.25rem;">
  <span>Frequency:</span>
  <span style="color: #4ade80;">{systemStats.cpu.frequency.current} GHz{#if
systemStats.cpu.frequency.max} <span style="opacity: 0.7;">(max {systemStats.cpu.frequency.max})</
span>{/if}</span>
</div>
{/if}

{#if systemStats.cpu.temperature !== null}
<div style="display: flex; justify-content: space-between; margin-bottom: 0.25rem;">
  <span>Temperature:</span>
  <span style="color: {systemStats.cpu.temperature > 70 ? '#fbbf24' :
'#4ade80'};">{systemStats.cpu.temperature}°C</span>
</div>
{/if}

<!-- ADD THIS ENTIRE SECTION: -->
{#if systemStats.cpu.topProcesses && systemStats.cpu.topProcesses.length > 0}
<div style="margin-top: 0.75rem; padding-top: 0.75rem; border-top: 1px solid
rgba(255,255,255,0.1);">
  <h4 style="margin: 0 0 0.5rem 0; font-size: 0.85rem; color: rgba(0, 225, 255, 0.8);">Top
Processes:</h4>
  {#each systemStats.cpu.topProcesses as process}
  <div style="display: flex; justify-content: space-between; margin-bottom: 0.25rem; font-size:
0.85rem;">
    <span style="overflow: hidden; text-overflow: ellipsis; white-space: nowrap; max-width:
70%;">{process.name}</span>
    <span style="color: {process.cpu > 50 ? '#ef4444' : process.cpu > 25 ? '#fbbf24' :
'#4ade80'};">{process.cpu}%</span>
  </div>
  {/each}
</div>
{/if}
```

Step 3: Rebuild Your Application

```
cd /path/to/your/phantomsdr-app
npm ./recompile.sh
```

Testing the Installation

Test 1: Check API Endpoint

```
curl http://localhost:3001/api/system-stats
```

Expected output (JSON):

```
{
  "cpu": {
    "usage": 45.2,
    "cores": 8,
    "coresUsed": 1.3,
    "temperature": 62.5,
    "frequency": {
      "current": 3.0,
      "max": 3.9,
      "limit": 4.4
    },
    "topProcesses": [
      {"name": "node", "cpu": 12.5},
      {"name": "phantomsdr", "cpu": 8.3}
    ]
  },
  "memory": {
    "used": 8.5,
    "total": 16,
    "percent": 53
  }
}
```

Test 2: Check Service Status

If you installed as a service:

```
sudo systemctl status sdr-stats.service
```

Should show: `Active: active (running)`

Test 3: Test in Browser

1. Open your PhantomSDR web interface
2. Scroll to **Additional Info** section
3. Click **Open Additional Info** button
4. Look for the **Stats** button next to PC info
5. Click it to open the stats modal
6. Verify stats are displaying and updating

Service Management

If you installed as a systemd service, use these commands:

```
# Start the service
sudo systemctl start sdr-stats.service

# Stop the service
sudo systemctl stop sdr-stats.service

# Restart the service (after updating files)
sudo systemctl restart sdr-stats.service

# Check status
sudo systemctl status sdr-stats.service

# View real-time logs
sudo journalctl -u sdr-stats.service -f

# View last 50 log entries
sudo journalctl -u sdr-stats.service -n 50

# Enable auto-start on boot
sudo systemctl enable sdr-stats.service

# Disable auto-start
sudo systemctl disable sdr-stats.service

# Edit service
sudo nano /etc/systemd/system/sdr-stats.service

# Delete service
sudo rm /etc/systemd/system/sdr-stats.service
```

Manual Installation (Option 2)

> **IMPORTANT**
>
> **You do not need this section for a normal installation.** `install-stats-server.sh` does all of it for you and is run by the main installer by default. Follow it only for a manual setup or to repair one piece by hand.

⚠**Warning:** Manual installation may use outdated files. The **automated script is strongly recommended** as it:

- Installs the latest version with improved temperature detection
- Auto-installs and configures lm-sensors
- Handles all dependencies automatically

If you still prefer manual installation:

1. Install Node.js

```
# Any distribution - nvm needs no root and no apt source.
# NOT NodeSource: deb.nodesource.com now answers HTTP 403 on every path.
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.4/install.sh | bash
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
nvm install 22 && nvm use 22 && nvm alias default 22
```

2. Install lm-sensors (Required for Intel/AMD temperature)

```
sudo apt-get update
sudo apt-get install -y lm-sensors
sudo sensors-detect --auto
```

3. Create Directory

```
mkdir ~/sdr-stats-server
cd ~/sdr-stats-server
```

4. Download Latest Files

Important: the server files are no longer kept as copies in the repository - the installer generates them.

Instead, run the automated script once to generate the latest files, then copy them:

```
# Run the automated script from the repository root
cd ~/PhantomSDR-Plus
./install-stats-server.sh
# When prompted, use a temporary port like 9999
# After installation completes, copy the generated files
cp ~/sdr-stats-server/system-stats-server.js ~/your-manual-install-dir/
cp ~/sdr-stats-server/package.json ~/your-manual-install-dir/
```

OR manually create the files using the latest code from the installation script.

5. Edit Port (if needed)

```
nano ~/sdr-stats-server/system-stats-server.js
```

Change line 6:

```
const PORT = process.env.PORT || 3001;  // Change this number
```

6. Install Dependencies

```
npm install
```

7. Test Run

```
npm start
```

8. Set Up Service (Optional)

Follow the systemd service setup from the automated installation section.

Changing the Port After Installation

If you need to change the port after installation:

Step 1: Edit the server file

```
nano ~/sdr-stats-server/system-stats-server.js
```

Change line 6:

```
const PORT = process.env.PORT || 3001;  // Change this number
```

Step 2: Update site_information.json

```
"siteStats": "http://Your_site_IP:NEW_PORT"
```

Step 3: Restart the service

```
sudo systemctl restart sdr-stats.service
```

Step 4: Rebuild your Svelte app

```
cd /path/to/phantomsdr-app
npm run build
```

Troubleshooting

Issue: Port already in use

Error:

```
Error: listen EADDRINUSE: address already in use :::3001
```

Solution:

1. Find what's using the port:

```
sudo lsof -i :3001
```

1. Stop that process:

```
kill <PID>
# or
sudo systemctl stop sdr-stats.service
```

1. Start the service again:

```
sudo systemctl start sdr-stats.service
```

Alternative: Run the installation script again and choose a different port when prompted.

Issue: Service won't start

Error:

```
status=217/USER
```

Solution:

1. Check the service file:

```
cat /etc/systemd/system/sdr-stats.service
```

1. Verify the `User=` line matches your username:

```
whoami
```

1. Edit if needed:

```
sudo nano /etc/systemd/system/sdr-stats.service
```

1. Reload and restart:

```
sudo systemctl daemon-reload
sudo systemctl restart sdr-stats.service
```

Issue: Stats not showing in browser

Symptoms: Modal opens but no data, or shows 0 values

Solutions:

1. Check browser console (F12) for errors
2. Verify API is accessible:

```
curl http://localhost:3001/api/system-stats
```

1. Check service is running:

```
sudo systemctl status sdr-stats.service
```

1. Verify `siteStats` in `site_information.json` matches your server address and port
2. Check CORS errors - ensure server allows your domain

Issue: Temperature shows null

Cause: Temperature reading not available on your system

Solutions:

1. **For most Linux systems:** Check if thermal zone exists:

```
cat /sys/class/thermal/thermal_zone0/temp
```

1. **Install lm-sensors:**

```
sudo apt-get install lm-sensors
sudo sensors-detect
sensors
```

1. **Note:** Temperature may not be available on:
2. Virtual machines
3. Some VPS providers
4. Windows Subsystem for Linux (WSL)
5. Non-Linux systems

This is normal - the rest of the stats will still work!

Issue: Stats values don't match `top`

Cause: CPU sampling period was too short in older versions

Solution: The current version uses 1-second sampling for accurate readings. If you have an old installation:

1. Update the server file:

```
nano ~/sdr-stats-server/system-stats-server.js
```

1. Find line 18 and change:

```
}, 100);  // Old value
```

to:

```
}, 1000);  // New value (1 second)
```

1. Restart:

```
sudo systemctl restart sdr-stats.service
```

Or simply re-run the installation script to get the latest version.

Issue: Node.js not found

Error:

```
node: command not found
```

Solution:

1. Install Node.js:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.4/install.sh | bash
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
nvm install 22 && nvm use 22 && nvm alias default 22
```

1. Verify:

```
node --version
npm --version
```

What You'll See

Once installed and configured, the stats modal displays:

CPU Section

- **Usage**: Overall CPU utilization percentage
- **Cores**: How much of the CPU is actually in use, expressed as whole cores — e.g. `1.3 / 12 in use` means the machine is doing 1.3 cores' worth of work. This is exact and needs no threshold. It is deliberately not a count of busy cores: the scheduler spreads one core's work across many, so any such count would only ever approximate this figure.
- **Frequency**: Average current CPU clock across all cores in GHz, with the fastest core in parentheses (if available)
- **Temperature**: CPU temperature in Celsius (if available), colour-coded: green up to 69 °C, orange 70-79 °C, red at 80 °C and above
- **Top Processes**: Top 5 CPU-consuming processes with color-coded percentages:
- Green: up to 50% CPU
- Orange: 51-80% CPU
- Red: above 80% CPU

Memory Section

- **Used**: Memory in use (GB)
- **Total**: Total system memory (GB)
- **Usage**: Memory usage percentage (color-coded)

♻Auto-Update

- Stats refresh every **5 seconds** while modal is open
- Stops updating when you close the modal (saves resources)

Temperature Detection

The stats server uses advanced multi-method temperature detection:

Supported Systems:

- **Intel Processors**: Reads from coretemp sensors via lm-sensors
- **AMD Processors**: Reads from k10temp sensors (Tdie/Tctl)
- **ARM Processors**: Reads from thermal zones (Raspberry Pi, etc.)

Requirements:

- **Intel/AMD**: Requires `lm-sensors` package (auto-installed by script)
- **ARM/Raspberry Pi**: Works out of the box, no additional packages needed

Installation Priority:

1. Tries `sensors` command for Package/Core temperature (Intel/AMD)
2. Falls back to direct coretemp reading (Intel)
3. Falls back to thermal_zone0 (ARM/Raspberry Pi)

If temperature shows `null`, install lm-sensors:

```
sudo apt-get install lm-sensors
sudo sensors-detect --auto
sudo systemctl restart sdr-stats.service
```

Features Summary

Real-time Monitoring - Live system stats from your server **CPU Metrics** - Usage, cores, temperature, top processes **Memory Tracking** - Used, total, and percentage **Auto-Updates** - Refreshes every 5 seconds **Color-Coded** - Visual indicators for warnings **Lightweight** - Minimal resource usage **Easy Installation** - Automated script handles everything **Port Validation** - Smart port selection with conflict detection **Service Management** - Auto-start on boot

Security Notes

- The stats server accepts connections from any origin (CORS: *)
- For production, consider restricting CORS to your domain only
- Port selection validates input and checks for conflicts
- CPU temperature reading is read-only and safe
- No sensitive system information is exposed

Support & Resources

Files Included:

In this folder (the installer itself lives in the repository root, as `PhantomSDR-Plus/install-stats-server.sh` - there is only ever one copy):

- `package.json` - Dependencies, identical to what the installer writes
- This document, also available in de, el, es, fr, hr and ru

Created by the installer in `~/sdr-stats-server/`:

- `system-stats-server.js` - Backend server, generated with the port you chose
- `package.json` - Dependencies
- `node_modules/` - Installed by `npm install`

Getting Help:

- Check the troubleshooting section above
- Review service logs: `sudo journalctl -u sdr-stats.service -n 50`
- Test API manually: `curl http://localhost:3001/api/system-stats`
- Verify configuration files

Unistall stats-server:

- Delete the folder /sdr-stats-server
- If you've created a service, just delete it using:

```
sudo rm /etc/systemd/system/sdr-stats.service"`
```

- Modify `App.svelte`: Find the part of the code:

```
                   <!-- In case you don't want the Stats Button to appear, please comment this
button section (12 lines)-->
                    <!-- System Stats Button -->
                    <button
                      type="button"
                      class="glass-button text-white py-1 px-2 ml-2 rounded text-xs"
                      on:click={openSystemStats}
                      title="System Resources"
                      aria-haspopup="dialog"
                      aria-expanded={showSystemStats}
                      aria-controls="system-stats-dialog"
                      style="color:rgba(0, 225, 255, 0.993); font-size: 0.75rem;"
                    >
                      Stats
                    </button>
  ```
  and replace with:
```

```
                <!-- In case you don't want the Stats Button to appear, please comment this button
section (12 lines)-->
                 <!-- System Stats Button -->
                 <!--
                 <button
                   type="button"
                   class="glass-button text-white py-1 px-2 ml-2 rounded text-xs"
                   on:click={openSystemStats}
                   title="System Resources"
                   aria-haspopup="dialog"
                   aria-expanded={showSystemStats}
                   aria-controls="system-stats-dialog"
                   style="color:rgba(0, 225, 255, 0.993); font-size: 0.75rem;"
                 >
                   Stats
                 </button>
                 -->
```

```

- Recompile using the command:

```
  cd PhantomSDR-Plus
 ./recompile.sh
 ```

- Restart the PC and start the server

Author: Created for PhantomSDR-Plus
Version: 1.1 (Improved CPU temperature detection for Intel/AMD) **Updated:** January 2026
License: MIT

Quick Command Reference

```bash
# Installation
cd ~/PhantomSDR-Plus
chmod +x install-stats-server.sh
./install-stats-server.sh

# Service Management
sudo systemctl start sdr-stats.service      # Start
sudo systemctl stop sdr-stats.service       # Stop
sudo systemctl restart sdr-stats.service    # Restart
sudo systemctl status sdr-stats.service     # Status
sudo journalctl -u sdr-stats.service -f     # Logs

# Testing
curl http://localhost:3001/api/system-stats # Test API
curl http://localhost:3001/api/health       # Health check

# Troubleshooting
sudo lsof -i :3001                          # Check port
whoami                                      # Check username
node --version                              # Check Node.js
```

Happy monitoring!

CHAPTER 9

Decoders

DECODERS.md

PhantomSDR-Plus — Decoder User Manual

This guide covers every built-in decoder available in PhantomSDR-Plus. All decoders share the same activation workflow, described below, followed by per-decoder setup instructions.

Table of Contents

1. How to Start a Decoder
2. FT8
3. FT4-FT2
4. JS8
5. CW — Morse Code
6. QRSS Grabber
7. WSPR
8. HF FAX / WEFAX
9. NAVTEX
10. FLDIGI — ALE 2G, RTTY, PSK31, Olivia, FST4W, MFSK, MSK144, DominoEX, THOR, THROB, Hellschreiber, MT63, Packet and APRS
11. SSTV
12. SERVICES
13. DRM
14. General Tips

1. How to Start a Decoder

All decoders are accessed from the **Decoder Options** section, located below the Audio Spectrogram controls on the main panel.

Steps:

1. Click the **Decoder: OFF** button to switch it to **Decoder: ON** (it turns blue when active).
2. Open the dropdown menu that appears to the right of the button and select the decoder you want.
3. The chosen decoder's panel will appear below the controls — follow the decoder-specific instructions in the relevant section of this guide.
4. To stop decoding, select **— Select decoder —** from the dropdown, or click the **Decoder: ON** button to turn it back OFF.

One-touch decoder buttons

The dropdown is not the only way in. The main panel carries a **Decoders** button row — directly below **Wheel Tuning Steps** — with one button per decoder:

Button	Decoder	Button	Decoder
FT8	FT8	**SSTV**	SSTV
FT4	FT4	**FLDIGI**	ALE 2G HAM, RTTY, PSK31·63·125, QPSK, Olivia, Contestia, FST4W, MFSK, MSK144, DominoEX, THOR, THROB, Hellschreiber, MT63, Packet, APRS
FT2	FT2	**NAVTEX**	NAVTEX
CW	CW	**FAX**	HF FAX / WEFAX
JS8	JS8	**SERVICES**	HFDL (aircraft), Maritime FSK / SITOR, Weather RTTY, DSC (ships), Time Signals, HF Selcall, ALE 2G
WSPR	WSPR	**DRM**	DRM — digital broadcasting

Pressing a button does the whole sequence in one go: it selects the decoder, switches the decoder ON, and scrolls its panel into view. The button turns blue while its decoder is running. **Press the same button again to switch the decoder off** — its window closes with it.

RADE-v2U, **RADE-v1L** and **RADE-v1U** are deliberately *not* in this row. They are digital-voice modes rather than text decoders, so they have their own buttons next to the **Modes** heading, and inside the **Modes** and **Bands** popup windows. Those behave in exactly the same way — press to start, press again to stop. See the RADE manual. **RADE-v2U** starts RADE V2 (beta), always on USB; **RADE-v1L** and **RADE-v1U** start RADE v1.

The buttons, the dropdown and the ON/OFF button all drive the same state, so whichever you use, the others follow.

Where a decoder has its own list of frequencies — the QRSS **Band** list, the HF FAX and NAVTEX stations, the FLDIGI **Known frequency** list, and the SERVICES and DRM station lists — its Tune button also frames the band as **To Band** does: the waterfall shows the whole band the station is in, with that band's brightness, contrast and tuning step. The band is chosen by the station's listed frequency, not by the dial, so 10100.800 kHz (DWD weather RTTY) opens on 30 m even though the dial sits a little below 10100 kHz.

Starting a decoder from a frequency marker

A frequency marker (the yellow labels above the waterfall, from `markers.json`) normally tunes the receiver and sets the mode written in it. When that mode is the name of a digital mode, clicking the marker starts its decoder instead, exactly as if you had pressed the decoder's button:

Marker mode	Starts	The marker's frequency is
`FT8`, `FT4`, `FT2`, `JS8`, `WSPR`, `SSTV`	that decoder	the dial frequency
`NAVTEX`	NAVTEX	the signal frequency (the dial goes 500 Hz below)
`FAX`, `WEFAX`	HF FAX	the published centre (the dial goes 1900 Hz below)
`RTTY`, `RTTY-WX`, `SITOR`	FLDIGI as Amateur RTTY (`RTTY`); SERVICES on Weather RTTY or Maritime FSK / SITOR	the signal frequency (the dial goes the panel's audio centre below)
`PSK31`, `OLIVIA`, `FST4W`, `MFSK`, `MSK144`, `DOMINOEX`, `THOR`, `THROB`, `HELL`, `MT63`	FLDIGI in that variant	as above
`MFSK16`, `MFSK32`, `MFSK64`, `FELDHELL`	FLDIGI in that variant and sub-mode	as above
`PSK63`, `PSK125`, `BPSK31/63/125`, `QPSK31/63/125`, `CONTESTIA`	FLDIGI in that variant and sub-mode	as above
`PACKET`, `APRS`	FLDIGI in that variant	the channel; 300 Bd (USB) below 30 MHz, 1200 Bd (FM) above
`RADEU`, `RADE`, `FREEDV` / `RADEL`	FreeDV RADE V1 on upper / lower sideband (`RADE` and `FREEDV` mean upper)	the dial frequency
`RADE2`	FreeDV RADE V2 (beta), always upper sideband	the dial frequency
`HFDL`	SERVICES · HFDL	the dial frequency (USB; the HFDL signal sits 1440 Hz above it)
`DSC`, `SELCALL`	SERVICES on DSC or HF Selcall	the DSC frequency or the Selcall channel (the dial goes 1.7 kHz below)
`TIME`, `DCF77`, `MSF`, `RWM`, `WWV`, `WWVH`	SERVICES on Time Signals, with that station	the carrier (the dial goes 1 kHz below)
`ALE`	FLDIGI on ALE 2G HAM on an HFLINK channel, otherwise SERVICES on ALE 2G	the dial frequency (LSB for the networks that use it)
`DRM`	DRM	the centre of the DRM channel
`MORSE`	the CW decoder, with CW and a ±250 Hz filter	the signal frequency

A decoder that is already running stays on. `CW` remains an ordinary receiver mode, because beacons and stations use it — it does not start the CW decoder. Markers are shown only while the waterfall spans 3.5 MHz or less. The decoder sets its own mode and filter, as it does when started from its button. To start the CW decoder from a marker, give it the mode `MORSE`. Decoder markers for every decoder come with PhantomSDR-Plus in `frequencylist/decodermarkers.json`; a marker of your own in `mymarkers.json` on the same frequency always wins.

> Only one decoder can be active at a time. Switching to a different decoder automatically stops the previous one.
>
> The **QRSS grabber** is not part of this dropdown — it has its own **QRSS** section and can run at the same time as a decoder.

Decoding runs in the background. SSTV, HF FAX, NAVTEX, SERVICES, FLDIGI and CW each run in a separate Web Worker thread, so the decoding work never competes with audio playback or the waterfall. Starting or stopping a decoder does not interrupt the audio, and the interface stays responsive while a picture or a page is being received. All decoders are fed raw audio taken *before* noise reduction and mute — so muting the receiver, or changing those settings to suit your ears, does not affect decoding. The AGC is different: the receiver applies it before the audio reaches your browser, so the decoders share it with you. While any decoder is open it is therefore held on **Slow**, whichever AGC button is selected, and a small *Slow (decoder)* note next to the AGC heading says so; your own choice returns when the last decoder closes. DRM is the exception: it is decoded on the station's server (chapter 13).

A running decoder owns the mode and the passband. Normally the mode follows the band plan in `bands-config.js`: move the dial into a segment marked LSB or AM and the receiver switches to it. While a decoder is running that no longer happens. The decoder holds the mode it needs (USB for most, its own for RADE) and the passband it needs — PSK31 about ±100 Hz, Olivia, MFSK, DominoEX, THOR, THROB, Hellschreiber and MT63 their full bandwidth, RTTY its shift — and both survive retuning, including a jump to another band. Without this, tuning FT8 on 40 m would flip the receiver to LSB the moment the dial moved, and the narrow decoder passbands would widen back to the full SSB filter.

The band's own mode returns the moment you switch the decoder off. You can still override the mode by hand at any time: the mode buttons are a deliberate choice and always win. The CW decoder holds **CW** in the same way (CW-L stays CW-L), and the link works both ways: starting the CW decoder puts the receiver in CW, and clicking **CW** in the Mode selector starts the CW decoder. Only the CW Browser uses USB, 200–3000 Hz.

Decoder ID — recognising the mode automatically

Above the decoder buttons sits a **Decoder ID** frame. It answers the question "*what am I listening to?*" when you come across a digital signal you cannot place, and then offers the matching decoder in one click.

It is **off by default** and costs nothing while it is off. Press **On** and leave the signal tuned in. For about half a minute it shows *Listening…* with a progress bar — the timing measurements have to see two complete FT8 cycles before they mean anything — and then it names the mode:

```
Decoder ID [On]   NAVTEX / SITOR-B  99% 170 Hz shift · 100.00 Bd · 43 dB S/N   [ Use NAVTEX ]
```

- The **mode** and a **confidence** figure. Green above 75%, amber above 50%, orange below — treat an orange answer as a hint, not a verdict.
- **What was actually measured**: occupied bandwidth, tone spacing, symbol rate, burst length, the UTC grid it fits, signal-to-noise. This is the evidence behind the verdict, so you can judge it for yourself.
- **Use** starts the right decoder, exactly as if you had pressed its button — and sets the receiver up to work that mode. For the modes that live in the FSK panel it also selects the correct variant — RTTY, PSK31 or Olivia in FLDIGI, Weather RTTY in SERVICES.
- Runners-up are listed after **or**. Click one to try it instead, so a wrong first answer costs you nothing.
- **** means the frequency you are tuned to is a known calling frequency for that mode, and that this counted towards the ranking.

It never switches your decoder by itself. It only ever suggests. And once you start a decoder — from the button row, from the dropdown, or from Decoder ID's own **Use** button — Decoder ID switches itself off: its job was to choose one, and the decoder now wants the same audio.

It sets the receiver up for the mode. Choosing a mode is a decision to work it, so the receiver follows: the tuned frequency moves to the middle of the waterfall and the view becomes about 100 kHz wide around it, the sideband becomes the one that mode is worked on, and the passband narrows to the window that mode lives in — the whole 3 kHz sub-band for FT8, FT4, FT2 and JS8, 1350–1650 Hz for WSPR, 250–750 Hz for NAVTEX, 800–2700 Hz for HF FAX, 900–2600 Hz for SSTV, and ±250 Hz either side of the dial for CW, which also switches the receiver to CW. A narrow passband keeps the neighbouring signals out of the decoder, which is what you would have done by hand. Two exceptions: SSTV keeps the sideband it chose for itself — LSB on 80 and 40 m by convention, where forcing USB would invert its tone mapping — and the FSK modes keep the window their own panel derives from the variant's shift or bandwidth, which is tighter than any fixed figure. Pressing **Use** for the decoder that is already running still switches it off, and then the receiver is left exactly as it was.

The slow modes take longer. WSPR sits on a two-minute grid and SSTV pictures run for a minute or two, so those two are named later than the rest — give them three or four minutes on frequency. HF FAX and SSTV are also recognised only when they are tuned the usual way — the picture band sitting between about 1500 and 2300 Hz in the audio, which is where their decoders expect it too. Retuning starts the measurement again from scratch, because everything gathered so far describes the frequency you have just left.

What it recognises: FT8, FT4, JS8, WSPR, CW, NAVTEX/SITOR-B, RTTY at 45.45 Bd, Weather RTTY (DWD), PSK31, Olivia, HF FAX, SSTV and DSC. **Use DSC** and the Weather RTTY suggestion start SERVICES on that service (chapter 12).

Recentre & retry. Where the signal sits in the audio passband makes no difference between about 400 Hz and 2700 Hz. Outside that range the measurement degrades, and an amber **Recentre & retry** button appears. Pressing it moves the dial so that the signal lands in the middle of the passband, and starts a fresh measurement. It is a single calculated move, not a search.

Three honest limits:

- **FT8, JS8 and FT2 are the same signal.** The same 8-FSK modulation, the same tone spacing, and JS8 at Normal speed uses FT8's 15-second cadence. Nothing in the signal itself separates them. Frequency does: on an FT8 calling frequency it reports FT8, on a JS8 one it reports JS8, and anywhere else it honestly reports the family — *FT8 / JS8 / FT2*. Start either decoder and see which one produces text. FT2 has a faster rhythm of its own (3.75 s) that is not measured, so an FT2 signal is usually not named at all.
- **It goes quiet rather than guessing.** Below roughly 10 dB signal-to-noise it reports *Nothing above the noise* or *Not recognised* instead of naming a mode it cannot really see. This is deliberate: a confident wrong answer is worse than no answer at all. The 10 dB is measured across the signal's own width, so narrow modes go much deeper: in a normal 2.5 kHz receiver bandwidth FT8 is named down to about −5 dB, FT4 to about 0 dB, and WSPR and CW to about −10 dB.
- **A fading signal is read a different way.** WSPR transmits for 110 seconds without a break, which is longer than an ordinary QSB fade, so a deep fade cuts the transmission into pieces and its two-minute rhythm can no longer be read from them. It is then recovered by another route — matching the whole listening history against the two-minute grid instead of against single transmissions — and the evidence line says *120 s grid (through fading)*. That is weaker evidence than a transmission heard whole, so the confidence shown is deliberately lower. A fade deep enough to bury the signal in the noise cannot be recovered at all, and Olivia is stricter than the rest: it will not be named until its symbol rate has been measured, so it falls silent earlier than the other modes.

2. FT8

What it is: FT8 is a popular weak-signal digital mode used by amateur radio operators worldwide. Transmissions last exactly 15 seconds, and the protocol can copy signals up to 20–25 dB below the noise floor. It is the most commonly used mode for long-distance (DX) contacts.

Recommended frequencies (USB)

Band	Frequency
160m	1.840 MHz
80m	3.573 MHz
40m	7.074 MHz
30m	10.136 MHz
20m	14.074 MHz
17m	18.100 MHz
15m	21.074 MHz
12m	24.915 MHz
10m	28.074 MHz

Setup

1. Tune to an FT8 frequency listed above and set mode to **USB**.
2. Enable the decoder and select **FT8** from the dropdown. Or simply press the **FT8** button.
3. The **FT8 Messages** panel appears automatically below.

Reading the output

The message list shows decoded transmissions as they arrive. Each 15-second cycle produces a new batch of messages. The **Farthest** field (top-right of the panel) shows the greatest distance decoded in the current session in kilometres. A long message ends in “…”: click its row to read the whole message, wrapped, and click again to close it. While a row is open the list stays where it is; otherwise it follows the newest messages. FT4 and FT2 use the same list.

Typical message format: `CQ DX AA1BB FN31` — a CQ call from callsign AA1BB located in grid square FN31.

The **SNR** shown with each decode is a real signal-to-noise ratio, measured on the decoded signal against the noise around it and referred to a 2500 Hz bandwidth, the way WSJT-X reports it. Weak stations read negative, down to about −20 dB, and strong ones positive; on the same recordings the two programs agree on average, typically within about 3 dB for a single decode. FT4 and FT2 use the same method.

Busy bands and weak stations. Each decoded station is subtracted from the audio and the rest is decoded again, up to three times, as WSJT-X does, so a station hidden under a stronger one next to it still comes out; on recordings from real bands the decoder finds 86–88 % of what WSJT-X finds. When the normal error correction gives up on a weak FT8 signal, a deeper one (ordered-statistics decoding) tries again; it is held to strict checks, so noise does not turn into made-up messages. FT4 and FT2 use the subtraction too.

> **After an update:** the FT8/FT4/FT2 decoder is a separate file that browsers keep for a long time. If every SNR still sits between about 0 and +5 dB, reload the page with a hard refresh (**Ctrl+Shift+R**, or **Cmd+Shift+R** on a Mac).
>
> **Tip:** FT8 is tightly time-synchronised. Your browser uses your computer clock; if your system clock drifts by more than a couple of seconds, decoding will fail. Keep your system time synchronised to NTP.

---

3. FT4-FT2

What it is: FT4 is a faster variant of FT8 designed for contest-style operation. Each transmission cycle is 7.5 seconds (half of FT8), making it twice as fast but requiring a slightly stronger signal. FT2 is an even faster variant of FT8. It is an ultra-fast 77-bit mode with TR periods of 3.75 seconds (= 1/2 of FT4) — still experimental.

Recommended frequencies (USB)

Band	Frequency FT4
80m	3.575 MHz
40m	7.047 MHz
30m	10.140 MHz
20m	14.080 MHz
15m	21.140 MHz
10m	28.180 MHz

Band	Frequency FT2
160m	1.843 to 1.846
80m	3.578 to 3.581
60m	5.360 (Check local regional regulations)
40m	7.052 to 7.062
30m	10.144
20m	14.084
17m	18.108
15m	21.144
12m	24.923
10m	28.184

Setup

1. Tune to an FT4 frequency and set mode to **USB**.
2. Enable the decoder and select **FT4** or **FT2** from the dropdown. Or simply press the **FT4** / **FT2** button.
3. The **FT4 Messages** or **FT2 Messages** panel appears below, identical in layout to the FT8 panel.

> **Note:** FT4 and FT8 use different spectral formats and are not interchangeable. Make sure you are on an FT4 frequency when using this decoder.
>
> **Clock:** FT4 and FT2 also decode when your computer clock or the audio path is up to about a second late (FT2 about 0.9 s); the decoder's own timing then corrects itself from the decodes.

---

4. JS8

What it is: JS8 (the mode used by JS8Call) takes FT8's weak-signal engine and turns it into a keyboard-to-keyboard conversation mode. Where FT8 sends fixed 13-character exchanges, JS8 sends free text a few characters at a time and stitches consecutive transmissions back into whole sentences — so anything longer than about twelve characters arrives over several 15-second cycles. It decodes about as deep as FT8, which makes it usable when voice and CW are not.

Recommended frequencies (USB)

Band	Frequency
160m	1.842 MHz
80m	3.578 MHz
40m	7.078 MHz
30m	10.130 MHz
20m	14.078 MHz
17m	18.104 MHz
15m	21.078 MHz
12m	24.922 MHz
10m	28.078 MHz

Speeds

JS8 has five speeds. Every station in a conversation must use the same one. **Normal** is the calling speed and is where almost all activity is — start there.

Speed	Cycle	Bandwidth	When it is used
Slow	30 s	25 Hz	Very weak paths; the most sensitive
Normal	15 s	50 Hz	The standard calling speed
Fast	10 s	80 Hz	Quicker exchanges, needs a stronger signal
Turbo	6 s	160 Hz	Strong local signals
Ultra	4 s	250 Hz	Experimental, rarely seen on the air

Setup

1. Tune to a JS8 frequency and set mode to **USB**.
2. Enable the decoder and select **JS8** from the dropdown, or simply press the **JS8** button.
3. The **JS8 Decoder** panel appears below. Leave **Speed** on *Normal* unless you know the station you are after is using another.

Reading the output

The panel has two lists.

Messages still arriving appear at the top in green, with a blinking cursor. A JS8 message can take four cycles — a full minute on Normal — so this is where you watch a sentence build up. Nothing has gone wrong if it sits there for a while.

Completed messages fill the main list, one row each:

Column	Meaning
Mode	Always `JS8`
Hz	Audio frequency of the signal inside the passband
dB	Signal-to-noise ratio in a 2500 Hz reference bandwidth
Message	The decoded text; **callsigns are shown in green**

Typical messages:

- `SV1BTL KM17: HB` — a heartbeat: the station is announcing it is on the air, with its grid square.
- `KN4CRD: K0OG SNR -05` — a directed message: KN4CRD tells K0OG it is hearing them at −5 dB.
- `MP 100W 8M/BALUN JN58KH AUGSBURG, MARTIN` — free text, in this case a station description that arrived over four cycles.

A message shown **dimmed and in italics** timed out before its last frame arrived. The text is real, but it may be cut short.

Sync offset

The **Sync offset** slider sets how long after the UTC cycle boundary the receiver starts capturing, which compensates for delay through the audio pipeline. Leave **Auto** ticked: the decoder measures the timing of the signals it hears and adjusts on its own. Only touch the slider if you have a reason to.

Spectrum marks

The small spectrum strip in the panel draws a yellow vertical line at the frequency of every signal decoded in the last cycle, so the Hz column can be read straight off the display.

Seeing your own spots

If the receiver is uploading JS8 spots to PSK Reporter (see the admin panel's Spot Reporting tab), the **JS8 map** button in the **Note:** panel opens PSK Reporter's map filtered to this receiver's callsign and mode JS8 — the same thing the **FT8 map** and **FT4 map** buttons do for those modes.

> **Tip:** JS8 is much quieter than FT8. On a mid-band afternoon you may see one transmission every couple of minutes, and long stretches of nothing at all are normal. Give it five or ten minutes before concluding something is wrong. 20m (14.078) and 40m (7.078) are the usual places to look.
>
> **Tip:** Like FT8, JS8 is tightly time-synchronised and uses your computer's clock. If your system time drifts by more than a second or two, nothing will decode. Keep it synchronised to NTP.

---

5. CW — Morse Code

What it is: The CW decoder listens for Morse code (continuous wave) signals and converts them to text in real time. It automatically tracks the signal frequency and adapts to the operator's sending speed.

Recommended frequencies

CW is active across amateur bands, typically at the lower portion of each band. Common spots:

Band	Segment
40m	7.000–7.040 MHz
20m	14.000–14.070 MHz
15m	21.000–21.080 MHz

Setup

1. Tune to a CW signal. Starting the decoder switches the receiver to **CW** (CW-L stays CW-L); clicking **CW** in the Mode selector starts the decoder as well.
2. Enable the decoder and select **CW** from the dropdown. Or simply press the **CW** button.
3. The **CW Decoder** panel appears below.

Reading the output

- The panel header shows the detected signal frequency in Hz (e.g. `≈ 700 Hz`) and the estimated sending speed in words per minute (e.g. `· 22 WPM`).
- If no signal is detected the header shows **scanning…**
- On an empty channel nothing prints, and the speed and tone are not shown. The decoder only starts printing when there is a clear difference between key-down and key-up, which plain noise does not produce, while a signal still readable at about −10 dB has plenty.
- Decoded text scrolls in amber monospaced text. The blinking cursor (▉) marks where text is currently being written.
- The **Pitch** slider before **Clear** sets the audio tone of the CW signal while the decoder runs: 500 to 800 Hz in 50 Hz steps, 600 Hz until you change it. Your browser remembers the choice. Plain CW listening is at 600 Hz too.
- **Save** writes the decoded text to a file; **Clear** erases it.

> **Tips:** - Centre your passband on the CW tone. The decoder works best when the CW signal sits between roughly 400–900 Hz in the audio spectrum. - Speeds from about 8 to 40 WPM are copied, including Farnsworth sending (characters sent fast with longer gaps between them): the speed lock holds and letters are not split into separate words. Heavily hand-keyed (irregular) Morse may still reduce accuracy. - The decoder performs best on a single clean signal. Strong QRM from nearby signals on the same band may confuse it. - Fading (QSB) is followed: a signal that fades and comes back keeps being read, and the first word of a transmission is read too — also at the start of a reply after a pause in the QSO. - Each letter is decided on the signal itself rather than on the dots and dashes counted first, which roughly halves the wrong letters on weak and hand-keyed signals.

CW Browser — every signal at once

Switch on the **Browser — all signals** knob in the CW window to decode every CW signal between 200 and 3000 Hz at once, one line each, like the PSK and Olivia browsers. While it runs the receiver is held on **USB** with a 200–3000 Hz filter; your own mode and filter come back when you switch it off.

- Each line shows the signal's frequency (the carrier, in kHz), its speed, the station's callsign — green with **CQ** when it calls CQ — and its text. Click a line to switch to **CW** on that station with the normal decoder.

- A signal gets its own decoder once it has been on for about two seconds; the last few seconds are decoded too, so its first words are not lost. When a station stops, its line stays for a minute, then goes.
- Stations 40 Hz apart are kept apart, and signals down to about −6 dB (in 3 kHz) read well. Lines that do not look like Morse text (a data signal, noise) are not shown, and neither are the keying sidebands of a strong station, which can stand out of the noise up to about 160 Hz either side of it.
- The callsign is the one sent twice or more, otherwise the one after **DE** or **CQ**; letters that came out spaced apart ("Y O 8 B N C") are joined back up.
- Each line keeps its last 4000 characters in a box about six lines tall: once it holds more than six lines it gets its own scroll bar (the mouse wheel works too), and the box holds still while you read. **Save** writes every line — with time, frequency, speed and callsign — including the lines already dropped.
- About two dozen stations at once cost roughly 10 % of one processor core in the listener's browser; the receiver is not loaded at all.

---

6. QRSS Grabber

What it is: QRSS is CW sent so slowly that a single dot lasts seconds instead of milliseconds. At that speed the signal occupies only a fraction of a hertz, so an analysis narrow enough to match it can pull the trace out from 20–30 dB *below* the noise floor. There is nothing to read by ear — QRSS is **looked at**, not listened to. The grabber runs its own very long FFT over the receiver audio and paints the classic grabber display: frequency on the vertical axis, time scrolling from left to right, the newest column always at the right-hand edge.

Most of what you will see comes from MEPT beacons (Manned Experimental Propagation Transmitter) — low-power transmitters, often a few hundred milliwatts into a wire, identifying continuously in very slow Morse or in patterned FSK.

Where to look

QRSS beacons live in narrow 100–200 Hz windows near the bottom of each band. You do not have to remember them: the grabber has a **Band** list of the windows below, and picking one tunes there and sets the receiver up for it (see *Opening the grabber*).

Band	Window centre	Note
30m	10,140.00 kHz	The main QRSS window — busiest by far, day and night
40m	7,039.90 kHz	
40m	7,000.85 kHz	Knights window, 7,000.8–7,000.9 kHz
20m	14,096.90 kHz	
80m	3,569.90 kHz	
80m	3,568.60 kHz	Older window
80m	3,500.85 kHz	Knights window, 3,500.8–3,500.9 kHz
160m	1,837.90 kHz	
160m	1,843.30 kHz	Older window
630m	476.10 kHz	
2200m	137.70 kHz	LF QRSS / DFCW
60m	5,288.55 kHz	
17m	18,105.90 kHz	
15m	21,095.90 kHz	
12m	24,925.90 kHz	
10m	28,125.70 kHz	
10m	28,000.85 kHz	Older window
10m	28,322.00 kHz	Alternative
6m	50,294.30 kHz	

Several bands appear twice because two conventions are genuinely in use. The modern windows sit 200 Hz below that band's WSPR frequency; the older *Knights* windows are somewhere else entirely, and on 40 and 80 m much lower down. If a band is quiet on one, try the other.

These are conventions, not regulations, and some are regional — the 10 m figures in particular vary. Treat the list as a starting point and follow local practice.

Opening the grabber

The QRSS grabber is **not** in the Decoder dropdown. It has its own **QRSS** section, alongside the Spectrogram and Decoder controls.

1. Click **Show**. The grabber canvas appears and its controls unfold next to the button.
2. Pick a window from the **Band** list and press **Tune**. This does everything the mode needs in one step: it tunes to the window, switches the receiver to **CW**, and sets a passband just wide enough for the slice you are watching. The dial then reads the true QRSS frequency, and the traces land on the centre line of the display.
3. Set **Speed** to match the beacon's dot length.
4. **Centre** and **Span** work as before; changing either re-shapes the passband to match, so the display and the receiver stay in step.
5. Click **Hide** to stop. The band's normal mode comes back.

You can still do it by hand instead — tune in **USB** about 1 kHz below the window so the traces land near 800 Hz of audio. The **Band** list only saves you the arithmetic.

While a window from the list is tuned, the grabber holds the receiver the way a decoder does: CW and its passband survive retuning, and the window follows the dial, so the trace stays on the centre line as you hunt along the band. It hands the receiver back when you hide the grabber, click a mode button, or start a decoder.

Speeds

Setting	Dot length	Resolution	Analysis window	New column every
QRSS 3	3 s	0.73 Hz	1.4 s	0.7 s
QRSS 6	6 s	0.37 Hz	2.7 s	1.4 s
QRSS 10	10 s	0.18 Hz	5.5 s	2.7 s
QRSS 30	30 s	0.09 Hz	10.9 s	5.5 s
QRSS 60	60 s	0.05 Hz	21.8 s	10.9 s

A setting faster than the beacon wastes sensitivity — the trace comes out thin and noisy. A setting slower than the beacon smears consecutive dots and dashes into one bar. The panel opens at **QRSS 6**, which is a reasonable place to start on an unknown signal: it is more sensitive than QRSS 3, and quicker to update and more forgiving of drift than QRSS 10. Once you can see the shape of the keying, move to the speed the beacon is actually sending — every step up in speed setting costs 3 dB of sensitivity.

Controls

Control	What it does
Band	The QRSS windows listed above. **Tune** goes to the selected one in CW with a matching passband.
Speed	Sets the transform length — the trade between frequency resolution and time resolution (table above).
Centre	Audio frequency at the middle of the display, 100–3000 Hz.
Span	Height of the displayed slice: 20, 50, 100 or 200 Hz, opening at 100 Hz — the width of a QRSS sub-band. A narrower span spreads each trace over more pixels. Wider spans fit more bins into the panel than it has pixel rows; each row then shows the strongest bin it covers, so nothing can hide between rows.
Gain	−10 to +40 dB. Shifts the colour map relative to the measured noise floor; raise it to brighten faint traces.
Color	Rainbow, Green or Grayscale.
Clear	Wipes the canvas and resets the noise reference.

Reading the display

Frequency ticks run down the left edge, highest frequency at the top. Under the canvas a status line shows the selected speed, the resolution actually in use (e.g. `0.183 Hz/bin · 5.5 s window`) and the column rate (e.g. `2.7 s/column`).

The colour map is referenced to the **current** noise, measured continuously from the empty parts of each column, not to an absolute level. Volume, AGC, antenna and band changes therefore do not require re-adjusting Gain — the display keeps a constant contrast against whatever the band noise happens to be. As with the decoders, the audio is taken before noise reduction and mute, so you can mute the receiver and keep watching.

What the shapes mean:

- A **steady carrier** draws a straight horizontal line.
- **Slow Morse** draws that line as a chain of short and long segments — dots and dashes read left to right.

- **Curved or drifting traces** are an unstabilised beacon oscillator warming up or cooling down. This is normal, and the drift signature is often how a regular grabber-watcher recognises a station.
- **Vertical streaks** across the whole slice are static crashes or local noise bursts, not signal.

> **Tips:** - Be patient. At QRSS 30 a single column takes 5.5 seconds, so a complete callsign can take ten minutes or more to cross the screen. Leave it running. - Tuning from the **Band** list already narrows the passband around the window. If you tuned by hand, do it yourself: it does not change what the transform resolves, but it keeps strong neighbours from working the receiver's AGC. - The grabber is independent of the decoders — you can leave it running while a decoder works on something else. - A wide span at a slow speed is the heaviest combination; the whole transform runs in the browser, so on a modest machine prefer a 50 Hz span.

---

7. WSPR

What it is: WSPR (Weak Signal Propagation Reporter, pronounced "whisper") is an ultra-weak-signal beacon mode that maps HF propagation paths worldwide. Each transmission takes approximately 110 seconds and fits inside a 200 Hz-wide slot. The decoder waits for a complete 2-minute UTC-aligned slot before decoding.

Recommended frequencies (USB, dial)

Band	Dial frequency
160m	1.836.600 MHz
80m	3.568.600 MHz
40m	7.038.600 MHz
30m	10.138.700 MHz
20m	14.095.600 MHz
17m	18.104.600 MHz
15m	21.094.600 MHz

Setup

1. Tune to a WSPR dial frequency above and set mode to **USB**.
2. The WSPR signal occupies the 1400–1600 Hz audio range. Selecting **WSPR** does the rest for you: the receiver switches to USB and the passband narrows to 1350–1650 Hz, which covers the whole range the decoder searches. You do not need to adjust it further.
3. Enable the decoder and select **WSPR** from the dropdown. Or simply press the **WSPR** button.
4. The **WSPR-2 Decoder** panel appears below.

Reading the output

The panel shows a progress bar for the current 2-minute slot:

- **Cyan bar filling** — collecting signal data (0–116 s into the slot).
- **Amber bar pulsing** — decoding in progress (last ~4 s of the slot).
- **Empty bar** — waiting for the next even UTC minute.

Each successfully decoded spot is shown in a table with the following columns:

Column	Meaning
UTC	Time of the spot (even minute)
Callsign	The station that transmitted
Grid	Maidenhead locator of the transmitter
Power	Transmitted power in dBm
Freq	Exact audio frequency (Hz) within the WSPR passband
SNR	Signal-to-noise ratio in dB

Click **Clear** to erase the spot list.

On a busy band the decoder works the way WSJT-X does: it takes up to 40 candidates per slot, subtracts every station it has decoded and searches again, up to three passes, so the weaker stations between strong ones are found as well.

> **Tip:** WSPR decoding requires very accurate system time (within ±1 second of UTC). The first slot after enabling the decoder will begin at the next even UTC minute — a short wait is normal.

---

8. HF FAX / WEFAX

What it is: HF Radiofax (also known as WEFAX) is used by coast guard and meteorological services worldwide to broadcast weather maps, sea-state charts, and surface analyses over shortwave. The decoder reconstructs the image line by line as it is received.

Setup

1. Enable the decoder and select **HF FAX / WEFAX** from the dropdown. Or simply press the **FAX** button.
2. The **HF FAX / WEFAX Receiver** panel appears below.
3. **Select a station** from the Station dropdown. Over 20 stations are available covering Europe, Asia, Oceania, and the Americas (e.g. DDH3/DDK3 Germany, SVJ4/GR Greece, JMH Japan, NMG USA New Orleans, CBM Punta Arenas Chile). The timetables follow the NOAA worldwide radiofax schedule (edition of 7 March 2025); Charleville (VMC, Australia) includes its extra broadcasts on 11030 kHz only.
4. If the station broadcasts on more than one frequency, select the desired frequency from the **Frequency** sub-dropdown.
5. Click ▶ **Tune** to automatically tune the waterfall to that station.
6. The mode is forced to **USB** automatically.

Broadcast schedule

When a station is selected, a **Next Transmissions** countdown table appears showing the next 4 scheduled broadcasts in UTC, with a live countdown:

- Normal (grey/green) — transmission is coming up.
- **Amber** — transmission starts within 3 minutes; arm the decoder now.
- **Red** ● — transmission starts within 30 seconds; reception is imminent.

Parameters

Most stations use the standard defaults (marked ★). Change only if you know the station uses non-standard settings.

Parameter	Default	Description
LPM	120 ★	Lines per minute — the drum rotation speed
IOC	576 ★	Index of cooperation — determines pixels per line
Shift	800 Hz ★	Frequency shift between black and white tones

Controls

- ⇔ **Auto-align** — enabled by default. Automatically synchronises to the phasing signal at the start of each image. Disable only if you are experiencing alignment problems on a known-good signal.
- ⇅ **Invert** — swaps black and white. Use if the image appears as a negative (white areas where black should be).
- ↺ **Refresh** — clears the canvas and resets the decoder. Use this between transmissions or if the image tears or drifts.
- ⭳ **Save PNG** — saves the current canvas as a PNG file to your computer.

Status indicators

At the bottom of the image, two tone indicators show:

- **300 Hz phasing** — lights cyan when the start-of-image phasing tone is detected.
- **450 Hz stop** — lights red when the end-of-image stop tone is detected.

Both signals are recognised by their rhythm across the whole width of a line, which the real start and stop signals always fill and chart content practically never does, so hatched areas in a chart are not taken for a start signal. They are recognised down to about 0 dB SNR, and on very weak signals the image is lined up from the average of several phasing lines, so the chart still starts at the left edge.

> **Note:** The image scrolls upward — the newest received line always appears at the bottom of the canvas. If you see **[PHASING]** in the header, the decoder has locked onto a new image start.

---

9. NAVTEX

What it is: NAVTEX is the international maritime broadcast system for coastal safety information — navigational warnings, weather forecasts, and search-and-rescue notices. It uses 100-baud FSK (SITOR-B with FEC) and is received on dedicated channels worldwide.

Available channels

Channel	Frequency	Usage
International	518 kHz	English-language, international
Domestic	490 kHz	National language broadcasts
HF (×5)	4209.5 / 6314 / 8416.5 / 12579 / 16806.5 kHz	Long-range HF NAVTEX

Setup

1. Enable the decoder and select **NAVTEX** from the dropdown. Or simply press the **NAVTEX** button. The receiver switches to **USB** automatically.
2. The **NAVTEX Receiver** panel appears.
3. Select the desired channel from the **Station** dropdown (e.g. `International — 518 kHz`).
4. Click ⇒ **Tune & Set IF** to automatically tune the waterfall and narrow the passband to the correct audio window. Mode is set to **USB** automatically. The dial is placed 500 Hz below the channel centre, so the NAVTEX signal appears at 500 Hz audio. The frequency display still reads **518.000**: while HF FAX, NAVTEX or the FLDIGI window is on, it shows the signal's own frequency (the dial plus the decoder's audio centre — 1900 Hz for FAX, 500 Hz for NAVTEX, the panel's centre for FLDIGI), and a frequency typed then is taken as the signal's.

Broadcast schedule

The schedule table lists all known stations on the selected channel with:

- Their ITU identifier letter and country flag
- Next broadcast time in UTC
- Live countdown to the next transmission
- **Amber** ● within 2 minutes / **Red** ● within 30 seconds — arm the decoder now

Reading the output

Decoded text appears in teal monospace. Message boundaries are clearly marked:

```
── ZCZC MA12 ──
... message content ...
── NNNN ──
```

`ZCZC` marks the start of a message. The four characters after it identify the station (`M`), subject (`A` = navigational warnings), and sequential number (`12`). The banner waits for the whole header, so it reads `ZCZC MA12`, not `ZCZC ????`. `NNNN` marks the end.

If you tune in after a message has started, or its header is lost to noise, the text still appears, under a **── (header missed) ──** banner. That banner only opens once several characters in a row arrive with both of their NAVTEX transmissions agreeing, which noise practically never produces. It closes at `NNNN` as usual, with **── (signal lost) ──** if the signal disappears first, or when a new message begins. A weak signal keeps its sync through corrupted characters: every NAVTEX character is sent twice, so the decoder holds on through errors and lets go only when the signal is really gone, and if it locked one bit away from the true character boundary it moves to the right one without closing the message. At −6 dB in 2.5 kHz this roughly doubles the text that gets through.

Click **Clear** to erase the message buffer.

> **Note:** NAVTEX operates on MF and LF frequencies (518/490 kHz). Reception range is typically 200–400 nautical miles from the transmitter. The HF channels (4–17 MHz) provide much greater range.

---

10. FLDIGI — ALE 2G, RTTY, PSK31, Olivia, FST4W, MFSK, MSK144, DominoEX, THOR, THROB, Hellschreiber, MT63, Packet and APRS

What it is: One window for the narrow-band keyboard modes that the program fldigi made popular, with fourteen variants selected from one dropdown. One is true FSK (Frequency-Shift Keying): Amateur RTTY; its utility relatives, Maritime FSK (SITOR) and Weather RTTY, use the same decoder in SERVICES (chapter 12). The others share the same window: **PSK31** (also **PSK63** and **PSK125**, each in BPSK or QPSK), which is phase-shift keying; **Olivia** (with its sibling **Contestia**) and **MFSK16/32/64**, multi-tone modes with forward error correction; **DominoEX** and **THOR**, incremental frequency keying (THOR with error correction); **THROB**, two tones at once, one character per pulse; **Hellschreiber**, which paints text as a picture instead of decoding it; **MT63**, 64 carriers at once; and **Packet (AX.25)** and **APRS**, which decode AFSK packet radio — the 1200 baud packet of VHF/UHF and the 300 baud packet of HF. Each variant comes with a preset tuned to its standard parameters. The button and the dropdown entry are both called **FLDIGI**; the decoding is done here, in the browser, and fldigi itself is not needed.

Three of the variants come from outside fldigi. **ALE 2G HAM** is the automatic link establishment (MIL-STD-188-141) of the amateur HFLINK network, and lists its calls and soundings; the government, military and utility networks are in SERVICES → ALE 2G (chapter 12). **FST4W**, the very slow beacon mode of the 2200 m, 630 m and 160 m bands, and **MSK144**, for meteor scatter on 6 m and 2 m, come from WSJT-X. None of the three prints running text: each lists what it decodes one line at a time, with the time.

Variants and presets

Variant	Center	Shift	Baud	Framing	Encoding
Amateur RTTY	1000 Hz	170 Hz	45.45	5N1.5	ITA2
PSK31·63·125 (BPSK/QPSK)	1000 Hz	—	31.25 / 62.5 / 125	—	Varicode
Olivia / Contestia	1000 Hz	—	see mode	—	7-bit + FEC
FST4W	1500 Hz	—	1.46 / 0.56 / 0.18 / 0.09	T/R 120–1800 s	LDPC + CRC-24
MFSK16·32·64	1500 Hz	—	see mode	—	Varicode + FEC
MSK144	1500 Hz	—	2000 bit/s	T/R 15 / 30 s	LDPC + CRC-13
DominoEX	1500 Hz	—	see mode	—	nibble Varicode
THOR	1500 Hz	—	see mode	—	Varicode + FEC
THROB / THROBX	1500 Hz	—	1, 2 or 4	—	tone pairs
Hellschreiber	1500 Hz	—	see mode	—	picture
MT63	1000 Hz (lowest carrier at 500 Hz)	—	5, 10 or 20	—	Walsh + interleave
ALE 2G	1625 Hz (tones 750–2500 Hz)	—	125	24-bit words, ×3	Golay (24,12)
Packet (AX.25)	1200 / 2200 Hz tones, or 1700 Hz at 300 Bd	1000 Hz, or 200 Hz at 300 Bd	1200 or 300	HDLC	AX.25
APRS	as Packet	as Packet	1200 or 300	HDLC	AX.25 + APRS

The panel adapts to the variant you choose. **Shift**, **Baud**, **Framing**, **Encoding**, **Invert mark / space** and **Auto shift detect** are hidden for every variant except the FSK ones (Amateur RTTY, and Maritime FSK and Weather RTTY in SERVICES), because none of the other modes has a mark/space tone pair or UART-style framing. In their place, **Center audio** becomes a free-entry number field (the signal can sit anywhere in the passband), and Olivia, MFSK, DominoEX, THOR, THROB, Hellschreiber and MT63 each gain a **Mode** selector and — all except Hellschreiber — a **Squelch** slider with its own scale. Packet and APRS hide the same FSK controls and show a **Speed** selector instead; APRS also has a **Show raw packet** checkbox. The **Known frequency** list appears only for the variants that have one. ALE 2G HAM replaces all of these with its own **network** and **channel** lists (see its notes). FST4W and MSK144 have no **Mode** or **Squelch**; they show **T/R period** and **F Tol** instead (MSK144 also **Decode**), and **⟳ Auto-tune Center** is hidden for both, because they search the whole **F Tol** range themselves.

Setup

1. Enable the decoder and select **FLDIGI** from the dropdown. Or simply press the **FLDIGI** button.
2. The decoder panel appears. Its title follows the variant — *FSK / RTTY Decoder*, *BPSK31 Decoder* (or whichever PSK mode is chosen), *Olivia Decoder*, *Contestia Decoder*, *MFSK Decoder*, *DominoEX Decoder*, *THOR Decoder*, *THROB Decoder*, *Hellschreiber Decoder*, *MT63 Decoder*, *Packet (AX.25) Decoder* or *APRS Decoder*. ALE 2G HAM shows *ALE 2G HAM Decoder*; FST4W names its period, e.g. *FST4W-120 Decoder*; MSK144 shows *MSK144 Decoder*.
3. Select the **Variant**. The parameters update automatically.

4. For PSK31, Olivia, MFSK, DominoEX, THOR, THROB, Hellschreiber and MT63, set **Mode** to match the transmission — see each mode's notes below. For Packet and APRS, set **Speed** — see the Packet and APRS notes below. For ALE 2G, pick a network and a channel; for FST4W and MSK144, set **T/R period** and **F Tol** — see their notes below.
5. Where the variant has one, use the **Known frequency** dropdown to select a common frequency, then click **Tune** to jump to it. The other modes have no fixed calling frequencies: find the signal on the waterfall and use ⟳ **Auto-tune Center**.
6. Fine-tune frequency until decoded text becomes stable and readable.

Known frequencies by variant

Amateur RTTY

- 3590 kHz (80m), 7043 kHz (40m), 10143 kHz (30m), 14083 kHz (20m), 21083 kHz (15m), 28083 kHz (10m)

PSK31

- 3580.15 kHz (80m), 7040.15 kHz (40m), 10142.15 kHz (30m), 14070.15 kHz (20m), 18100.15 kHz (17m), 21080.15 kHz (15m), 24920.15 kHz (12m), 28120.15 kHz (10m)

Olivia

- 3583 kHz (80m), 7040 kHz (40m), 7072.5 kHz (40m), 10143 kHz (30m), 14072.5 kHz (20m), 14108.5 kHz (20m, 32/1000), 18099 kHz (17m), 21072.5 kHz (15m), 24922.5 kHz (12m), 28122.5 kHz (10m)

FST4W (the signal's centre; the USB dial goes 1.5 kHz below it)

- 137.5 kHz (2200 m, dial 136.000), 475.7 kHz (630 m, dial 474.200), 1838.1 kHz (160 m, WSPR sub-band, dial 1836.600)

MSK144 (the same: the dial goes 1.5 kHz below)

- dial 144.360 MHz (2 m, IARU Region 1), 144.150 MHz (2 m, North America), 50.260 and 50.280 MHz (6 m), 70.230 MHz (4 m), 28.145 MHz (10 m)

MFSK, DominoEX, THOR, THROB, Hellschreiber, MT63

- No fixed calling frequencies; the list is not shown. Find the signal on the waterfall.

Packet and APRS (the same list for both; each entry also sets the right speed)

- 144.800 MHz — APRS in Europe (IARU Region 1)
- 144.390 MHz — APRS in North America
- 145.175 MHz — APRS in Australia
- 144.640 MHz — APRS in China
- 144.660 MHz — APRS in Japan
- 145.825 MHz — the ISS packet digipeater
- 10147.6 kHz — HF APRS on 30 m, USB dial (300 Bd)

Parameters

Parameter	Applies to	Description
Center audio (Hz)	all	The audio frequency of the mark/space midpoint; for PSK31 the carrier, for the multi-tone modes the centre of the tone block or carrier group. A dropdown for the FSK variants, a free-entry field for all the others. For Packet and APRS a free-entry field at 300 Bd only; at 1200 Bd the tones are fixed at 1200 / 2200 Hz and the field just shows them
Shift (Hz)	FSK only	Frequency difference between mark and space tones
Baud	FSK only	Symbol rate
Framing	FSK only	Data bits, parity, stop bits (e.g. 7N1 = 7 data, no parity, 1 stop)
Encoding	FSK only	Character set (CCIR-476, ITA2/Baudot, or ASCII)
Invert mark / space	FSK only	Swaps mark and space tones
Auto shift detect	FSK only	Attempts automatic shift measurement from the incoming signal
Mode	PSK31 only	BPSK31/63/125, QPSK31/63/125 — must match the transmission
Mode (tones / Hz)	Olivia, Contestia only	Number of tones and bandwidth — must match the transmission exactly
Squelch (FEC S/N)	Olivia, Contestia only	How strong the error-correction match must be before text is printed
Browser — all signals	PSK31, Olivia, Contestia	A knob: decodes every signal of the chosen **Mode** (PSK31: every BPSK31 signal) between 300 and 2700 Hz at once, one line each — see the PSK Browser and Olivia notes
Mode	MFSK, DominoEX, THOR, THROB, Hellschreiber, MT63	The sub-mode (speed, bandwidth, interleave) — must match the transmission
Squelch (FEC metric)	MFSK	The 0–100 decoding quality the signal must reach before text is printed; default 22
T/R period	FST4W, MSK144	FST4W: 120, 300, 900 or 1800 s — must match the transmission. MSK144: 15 or 30 s; a message repeated within one period is listed once
F Tol	FST4W, MSK144	How far either side of **Center audio** the decoder searches: ±50, 100 or 150 Hz for FST4W, ±50 to ±300 Hz for MSK144
Decode	MSK144	**Fast**, **Normal** or **Deep** — how thoroughly each block is searched; Deep finds the most and uses the most CPU
Squelch (tone / noise)	DominoEX, THOR	0–100: the current tone against the floor of the other tone bins; default 25
Squelch (S/N)	THROB	In dB; default 4
Squelch (FEC S/N)	MT63	The Walsh-code match quality; default 4
Reverse	FSK Hell 245 / 105, Hell 80	Paints the other tone as ink
Speed	Packet and APRS	**1200 Bd · VHF/UHF (FM)** or **300 Bd · HF (USB)**. It also chooses the receiver mode
Show raw packet	APRS only	Also prints the packet as received (TNC2 format) above each decoded line

Parameter	Applies to	Description
Squelch (SNR)	Amateur and Weather RTTY	Stops noise printing as random letters. Starts at −5 dB for Amateur RTTY and −7 dB for Weather RTTY; the far left (**off**) disables it. The SNR figure turns grey while the squelch is closed

Signal metrics

The status bar shows live measurements, and the fields change with the variant:

Variant	Fields shown
FSK variants	**Mark / Space** (measured tone frequencies), **SNR**, **Lock**, **Timing**
PSK31	**Carrier** (Hz, after automatic frequency correction), **IMD** (dB), **SNR**, **Lock**, **Timing**
Olivia	**Centre** (Hz), **Mode**, **S/N**, **FEC** (%), **Sync**
PSK Browser, Olivia Browser	**Signals** (how many lines are on screen)
MFSK16 / 32 / 64	**Centre** (Hz, after automatic frequency correction), **Mode**, **S/N**, **FEC** (%), **Sync**
ALE 2G	**Dial** (warns when it is not a listed channel), **In sync**, **Events**
FST4W	**Period** (how much of the current period has been recorded), **Search** (Hz), **Decodes**
MSK144	**Search** (Hz), **Decodes**, **CPU** (the share of real time the decoder needs; above 100 % it falls behind)
DominoEX, THOR, THROB, MT63	**Centre** (Hz; THROB and MT63 after automatic frequency correction), **Mode**, **S/N**, **Quality** (%), **Sync**
Hellschreiber	**Centre**, **Mode**, **Audio** (input level, dBFS)
Packet and APRS	**Tones** (Hz), **DCD** (lit while a packet is on the air), **Frames** (packets decoded), **Stations** (different callsigns heard), **Audio** (input level, dBFS)

`Timing/Sync` reads `LOCKED/SYNCED` once the decoder is tracking, and `SEARCH` while it is still hunting.

SNR for the FSK variants and PSK31 is measured on the signal itself and given in a 3 kHz bandwidth, so the two read on the same scale: within about 1 dB of the true value from +20 dB down to −5 dB. An empty frequency reads about −12 to −15 dB, and for PSK31 0 dB is already a strong signal. Olivia's **S/N** is different: it is the error-correction quality figure. For MFSK, **S/N** is fldigi's tone-to-noise figure and **FEC** its 0–100 decoding quality. For DominoEX and THOR, **S/N** is the received tone against the other tone bins and **Quality** the squelch scale (THOR: its error-correction confidence); for THROB the two-tone S/N; for MT63 the error-correction match. None of these is a 3 kHz SNR: they are the figures the squelch acts on.

Amateur RTTY (45.45 baud, 170 Hz) copies without errors down to about 0 dB, with about 1.5% of characters wrong at −3 dB and 20% at −6 dB.

Additional controls

- ⇒ **Set IF Band-Pass** — narrows the receiver passband to tightly bracket the signal. The width follows the variant: 300 Hz for Amateur RTTY (narrow enough to remove a stronger station 250 Hz away, while still copying a signal tuned 40 Hz off), 594 Hz for Weather RTTY, mark/space plus margin for Maritime FSK, about ±100 Hz for PSK31 (±200 Hz PSK63, ±400 Hz PSK125), the full tone-block width plus margin for Olivia, about ±400 Hz for 300 Bd packet, and the ordinary FM channel for 1200 Bd packet. The passband follows the decoder when **Auto-tune** or **Auto shift detect** moves the tones, and when you change the **Baud** setting. MFSK, DominoEX, THOR, THROB, Hellschreiber and MT63 get their full width plus a margin (at least 60 Hz either side, 15 % of the width for the wide

modes). ALE 2G gets 500–2750 Hz (mirrored below the dial on an LSB network), FST4W its **F Tol** range plus 150 Hz either side, MSK144 ±1.1 kHz plus **F Tol**.

- ⟳ **Auto-tune Center** — automatic search for the signal. For the FSK variants it looks for a balanced pair of tones; for PSK31 it finds the carrier; for Olivia it finds the strongest block of the selected bandwidth; for 300 Bd packet it finds the strongest pair of tones 200 Hz apart. The button is hidden for 1200 Bd packet, where the tones are fixed. **Auto shift detect** only changes the shift when a real two-tone signal is present, so it no longer jumps on noise. MFSK, DominoEX, THOR, THROB, Hellschreiber and MT63 are searched the same way as Olivia, over their own width. The button is hidden for ALE 2G, FST4W and MSK144.

ALE 2G notes

ALE (MIL-STD-188-141, second generation) is how government, military, aid-agency and amateur HF networks find a working frequency: the stations scan a set of channels, send short *soundings* to say they are there, and call each other with a few seconds of an 8-tone signal (125 Bd, 750–2500 Hz).

1. Choose **ALE 2G HAM**, the entry just before Packet in **Variant**. In place of the usual settings the panel shows a **network** list and a **channel** list.
2. Pick a network and one of its channels, then click ⇒ **Tune & Set IF**. The listed frequency is the dial; the receiver goes to USB, or LSB for the few networks marked so, with a 500–2750 Hz filter.
3. Wait. Soundings come every half hour or hour, and calls only when a link is needed, so a channel can stay quiet for a long time.
4. **Ham (HFLINK)** lists all 31 amateur ALE channels of the HFLINK channel plan, from 475.5 kHz to 144.1625 MHz; 14109 and 7102 kHz are among the main HFLINK sounding and texting channels. It is the only network in FLDIGI. The other 51 networks (Government, Military — MARS among them — and Utility) come from KiwiSDR's ALE list and are in SERVICES → ALE 2G (MIL-STD-188-141) (chapter 12), which works the same way. They are public lists and may be out of date.
5. Each event shows the time, the frequency, its kind (sounding, call, call acknowledged, link established, message), the **from** and **to** addresses, and any commands or text (AMD and DTM messages). Identical repeats are merged into one line with ×N, and **Save** writes the events to a text file.
6. **In sync** counts how many of the 17 decoders, each on its own frequency offset, are reading ALE words at that moment. A signal up to about ±1 kHz off is still found.

PSK31 notes

PSK31 is the most common keyboard-to-keyboard mode on HF. It is only 62 Hz wide, so several QSOs sit side by side within a few hundred hertz of the watering-hole frequency, and tuning is a matter of picking one trace out of the group on the waterfall.

- The decoder corrects its own tuning error automatically over roughly **±25 Hz** (±50 Hz PSK63, ±100 Hz PSK125), so you only need to get close. **Carrier** in the metrics row shows where it actually settled.
- **IMD** measures transmitter quality, not reception: a clean signal reads about −20 dB or better. A poor reading means the other station is overdriving, not that you are mistuned.
- Text appears character by character with no error correction, so a weak signal degrades into occasional wrong letters rather than stopping.
- **PSK63 and PSK125** are the same mode two and four times faster, and as many times wider; **QPSK31/63/125** add error correction (a Viterbi decoder), which repairs short bursts of noise at about the same signal level. Pick the sub-mode in **Mode** — the wrong one prints nothing. Most PSK activity is still BPSK31; PSK63 is common in contests.

- **PSK Browser (BPSK31).** The **Browser — all signals** knob under **Mode** decodes every BPSK31 signal between 300 and 2700 Hz at once, as fldigi's signal browser does: one line per signal (up to 16), with its frequency, its lock quality and the latest text. Signals are found by themselves within a few seconds, and one that stays silent for 30 s is dropped. Click a line to decode only that signal, as ordinary BPSK31 on its carrier. While it runs, **Mode** shows BPSK31 and is locked; switched off, the window goes back to the Mode you had. The passband opens to 200–2800 Hz while browsing. Each line keeps its last 4000 characters in a box about six lines tall, with its own scroll bar once it holds more than six lines (the mouse wheel works too); the box holds still while you read. **Save Text** writes every line, also the ones already dropped. While a browser runs, the settings of the single decoder (Center audio, ⟳ Auto-tune Center and its readouts) are hidden.

Olivia notes

Olivia trades speed for robustness. It is far slower than PSK31 but decodes signals that are inaudible by ear, which makes it popular for weak-signal and long-distance contacts.

- **The Mode setting must match the transmission exactly.** A wrong tones/bandwidth combination decodes nothing at all — not garbled text, but silence. The panel opens on **8 / 250**, the narrow configuration left running on the calling frequencies; **16 / 500** and **32 / 1000** are the other two in common use, and **16 / 1000** is also offered.
- **Contestia** is Olivia with a smaller error-correction block: about twice as fast, upper case only, and a little less robust. Its configurations are in the same **Mode** list, under *Contestia* (8 / 250 is the usual one, 4 / 125 and 4 / 250 for weak paths); the panel title changes to *Contestia Decoder*. An Olivia setting never decodes Contestia, nor the other way round.
- **Olivia Browser.** Switch on the **Browser — all signals** knob under **Mode** to decode every Olivia (or Contestia) signal of that configuration between 300 and 2700 Hz at once, one line each, in the same way as PSK Browser; click a line to decode only that signal. It works one configuration at a time: on **8 / 250** it does not see 16 / 500 stations. Because Olivia is used below the noise, it does not look for signals by their strength: the passband is covered by about a dozen decoders side by side (about 5 % of one processor core), so a station appears once its first characters are decoded, a few seconds after it starts. Within one bandwidth only the strongest copy is shown, and a line appears only once it holds a few characters of real text.
- Olivia sends no preamble, so the decoder has to search for synchronisation. **Allow a few seconds** after tuning before text appears. The **Sync** field shows `SEARCH` until it locks.
- The error correction works on blocks, so text arrives in **bursts rather than a steady stream**, and there is a delay of several blocks between transmission and display.
- **Squelch (FEC S/N)** sets how confident the error correction must be before printing. The default of 4.0 keeps noise out; 3.0 is the floor, below which random noise starts printing occasional characters. A good signal reads 8–9 on the **FEC** meter, so there is plenty of room to raise the squelch on a busy band.
- When a transmission ends, the text simply stops. Each block is also checked on its own, so the few blocks the squelch still lets through while it settles no longer print a burst of random characters.

FST4W notes

FST4W is the WSJT-X beacon mode for the low bands, where it follows WSPR: four tones carrying a callsign, a locator and a power, sent once per T/R period of 2, 5, 15 or 30 minutes. The longer the period, the weaker the signal it decodes: about −31 dB SNR (in 2.5 kHz) for FST4W-120, and real 30-minute signals near −44 dB.

- **It decodes once per period, after the period ends.** Periods start on the UTC clock — FST4W-120 at every even minute, FST4W-1800 on the hour and the half hour — and the **Period** bar shows how much of the current one has been recorded. A period joined part-way through is skipped, because

half a message cannot be decoded; the next one is recorded whole. Decoding a 30-minute period takes a few seconds.

- **Pick the right T/R period.** The decoder looks only at the one you choose. Most LF/MF beacons use FST4W-120; FST4W-300, -900 and -1800 are for the weakest paths.
- **F Tol** sets how far either side of **Center audio** (1500 Hz) it searches. WSJT-X's default of ±100 Hz covers the beacon sub-bands; ±150 Hz is the most the passband allows.
- Each line shows UTC, SNR (dB), DT (the time offset, s), the frequency (MHz) and the message, as in WSJT-X, and **Save Text** writes them in the same style.
- The computer's clock must be right to about a second, as for WSPR.

MFSK16 / MFSK32 / MFSK64 notes

MFSK16, MFSK32 and MFSK64 are the IZ8BLY modes as sent by fldigi: 16 tones, one at a time, spaced one baud apart, protected by convolutional error correction and interleaving. They are faster than Olivia and a little less robust. MFSK16 (15.625 baud, 234 Hz wide) prints cleanly down to about −12 dB SNR in 2.5 kHz, MFSK32 (31.25 baud, 469 Hz) to about −10 dB, and MFSK64 (62.5 baud, 938 Hz) to about −7 dB.

- **Pick the MFSK16·32·64 variant**, then the speed in its **Mode** list. The three modes differ only in speed, and the setting must match: the wrong one prints nothing readable.
- **Tuning:** put the tone block on the **Center audio** frequency, or press ⟳ **Auto-tune Center**. The decoder corrects the last quarter of a tone by itself — about ±4 Hz for MFSK16, ±8 Hz for MFSK32 and ±16 Hz for MFSK64 — and **Centre** shows where it settled. A signal further off is found by the decoder itself, up to about ±75 Hz from the set centre: allow a few seconds at the start of a transmission.
- Text lags the signal by a couple of seconds, the time the interleaver and the error correction need. One or two stray characters at the very start of a transmission are normal; fldigi shows them too.
- **Squelch (FEC metric)** holds printing back until the decoding quality reaches the setting. The default of 22 keeps noise out (noise alone reaches about 20); a good signal reads 90 % or more on the **FEC** meter, and 0 turns the squelch off. When a transmission ends with fldigi's end-of-transmission mark, printing stops at once instead of letting a few seconds of noise through.
- MFSK picture transfers are not decoded.

MSK144 notes

MSK144 is the WSJT-X mode for meteor scatter: signals reflected for a fraction of a second by the ionised trails of meteors, mostly on 6 m and 2 m. Each station repeats its message without a break all through its 15- or 30-second period, 72 ms per message, so a ping of a tenth of a second can carry it.

- **Expect long gaps.** On a quiet band nothing appears for minutes. During the big showers (Quadrantids, Perseids, Geminids) and in the early morning there are many more pings.
- Each line shows UTC (to the second), SNR (dB), T (when the ping came in its period, s), the frequency (MHz) and the message. A message heard again in the same T/R period is listed once, unless the new copy is stronger. Weak copies of the same message are averaged, as in WSJT-X.
- **F Tol** sets how far either side of 1500 Hz the decoder searches; stations are seldom exactly on frequency, so ±100 Hz is a good start. **Decode** sets how hard each block is searched, and **Deep** finds the most. **CPU** in the metrics row shows the load: above 100 % the decoder falls behind, so choose **Normal** or **Fast** on a slow computer.
- The 40-bit short messages (Sh) are not decoded: they need both stations' callsigns, which a listener does not know.

DominoEX and THOR notes

DominoEX and THOR send 18 tones, one at a time, but a character is not a tone: it is the *step* from the previous tone (incremental frequency keying). The same tone never repeats, and a slow drift of the whole signal cancels out, so both modes forgive a few hertz of mistuning and a drifting transmitter. **DominoEX** sends its characters straight, without error correction: a bad tone costs one character. **THOR** adds the same convolutional error correction and interleaving as MFSK, so it copies through static crashes and fading at the cost of a couple of seconds' delay.

- **Mode** must match the speed being sent. DominoEX offers Micro, 4, 5, 8, 11, 16, 22, 44 and 88; THOR offers Micro, 4, 5, 8, 11, 16, 22 and the fast 25 x4, 50 x1, 50 x2 and 100 (these four use a stronger code and a one-second interleaver). DominoEX 11 and THOR 16 are the most heard.
- **Tuning:** put the tone block on **Center audio**, or press ⟳ **Auto-tune Center**. A tuning error of a quarter of the signal's width still copies.
- **Squelch (tone / noise)** compares the received tone with the floor of all the other tone bins, on a 0–100 scale. Noise alone stays at or under 20; readable copy reads 35 and up; the default is 25. This is not fldigi's own figure: fldigi's reads 26 on some perfectly clean signals, which would have made any useful default cut them off. When a transmission ends, printing stops within a few symbols instead of letting noise through.
- The receiver follows fldigi's default settings: three sub-bin paths, hard decisions for DominoEX, soft decisions and preamble detection for THOR. DominoEX's optional MultiPSK error correction (off by default in fldigi) is not decoded, nor is the secondary text channel (the station's idle beacon text) or THOR's picture mode.

THROB notes

THROB sends each character as one pulse of two tones at once (or one tone for a few characters), from 9 tones (THROB) or 11 (THROBX), at 1, 2 or 4 pulses a second. It is slow but very robust in fading. THROB has only capitals, figures and a few marks; THROBX has a larger set.

- **Tuning is critical.** The tones of THROB 1 and 2 are only 8 Hz apart, so the decoder captures ±3 Hz of mistuning (±6 Hz for THROB 4) and corrects the rest with its own frequency tracking. Press ⟳ **Auto-tune Center** first: it finds the signal to within a hertz or two.
- **Mode** must match, THROB or THROBX and the speed.
- **Squelch (S/N)** — noise alone stays under 1 dB, a copyable signal reads 10 dB and up; the default is 4 dB.

Hellschreiber notes

Hellschreiber is not decoded at all: the transmitter scans each character of a 7 × 14 font column by column, and the receiver *paints* what it hears. The panel therefore shows a picture instead of text, and your eye reads the letters. That is why Hell stays readable when no text mode is: a damaged letter is a smudged letter, not a wrong one.

- **Mode:** Feld Hell (the common one, 245 Hz wide), Slow Hell, Hell X5 and X9 (faster and wider), and the two-tone modes FSK Hell 245, FSK Hell 105 and Hell 80. The speed must match, or the letters come out stretched or squashed.
- **Each line of text is painted twice**, one copy above the other, as fldigi paints it: a small speed difference between the stations slants the text, and one of the two copies is always whole.
- There is no squelch: noise shows as speckle, which the eye ignores. For the two-tone modes, **Reverse** swaps which tone is ink, for a station with the tone order the other way round.
- **Save Picture** saves the picture as a PNG; **Clear** starts a clean page.

MT63 notes

MT63 sends 64 carriers at once, each carrying one bit per symbol, and spreads every character over all of them and over 32 (short) or 64 (long) symbols in time. A character therefore survives the loss of a good part of its carriers and a burst of noise, which is why MT63 is used for emergency traffic: it copies through interference that wipes out narrower modes.

- **Mode:** MT63-500, -1000 or -2000 (the bandwidth), each with **short** or **long** interleave. Both must match. MT63-1000 long is the most used.
- **Tuning:** fldigi sends the lowest carrier at 500 Hz, so the centre is 500 Hz plus half the bandwidth: 750, 1000 or 1500 Hz, set automatically when you pick the mode. The decoder finds the last 8 carriers of error by itself (±125 Hz for MT63-1000).
- **Text lags the signal** by the interleaver: from about 1.6 s (MT63-2000 short) to 12.8 s (MT63-500 long). The last characters of a transmission arrive after the signal has stopped.
- **Squelch (FEC S/N)** — noise alone reads about 3, a locked signal 4.5 and up; the default is 4. The squelch hides the moment the synchroniser needs to lock at the start.

Packet and APRS notes

Packet radio sends data in short bursts called frames. Each frame carries the sender's callsign, the destination, the digipeaters it went through, and a checksum. **APRS** (Automatic Packet Reporting System) uses these frames to report positions, weather, messages and status, mostly on one shared frequency per region.

- **Packet** prints every frame, the way a TNC monitor does: time (UTC), `SOURCE>DESTINATION,PATH`, the frame type in angle brackets, then the text. Connected-mode traffic (SABM, UA, I-frames, RR, DISC) shows up as well as broadcasts.
- **APRS** prints only the broadcast frames APRS uses, and turns each into plain text: position with a Maidenhead locator, symbol (car, house, weather station, …), speed and course, altitude, comment, weather readings in metric units, messages and acknowledgements, objects, items, status and telemetry. Compressed and Mic-E positions are decoded too. With **Show raw packet** on, the received packet is printed above its decoded line.
- **APRS map.** Under the decoded text, the APRS variant shows a map of every station heard with a position, the same width and height as the text pane: a labelled marker per station (objects and items under their own name), a track line for moving stations, and an amber marker for this receiver, placed from its grid square. Click a station for its decoded line, when it was last heard, and its distance and bearing from the receiver; click a decoded line in the pane to zoom the map to that station. **Auto fit** (on by default) re-frames the map as stations arrive and move; panning or zooming by hand turns it off, and ticking it turns it back on. **Fit all** frames everything once, **Clear** empties the map, and **–** minimises it to its title bar (□ brings it back) — stations keep collecting meanwhile. The map images come from OpenStreetMap, so the listener's browser needs internet access to draw them; the decoding itself stays local.
- **Speed 1200 Bd** is the VHF/UHF standard (Bell 202 tones, 1200 / 2200 Hz). The decoder switches the receiver to **FM** and tunes it on the channel itself: pick **144.800 MHz** in Europe, or tune any packet channel and press ⇒ **Set IF Band-Pass**. The 12.5 kHz **Wheel Tuning Step** moves along the VHF channel grid.
- **Speed 300 Bd** is HF packet, two tones 200 Hz apart. The receiver goes to **USB** and the dial is the published frequency; the tones sit at 1600 / 1800 Hz. Use ⟳ **Auto-tune Center** if the signal is a little off.
- **Only frames with a correct checksum are printed**, so noise never prints as garbage — a quiet pane means a quiet channel. If **DCD** lights but **Frames** does not count up, the signals are there but too weak or distorted to pass the checksum.

- The decoder works with FM receivers that use de-emphasis and those that do not, and with transmitters that boost one tone more than the other.

Mode note: The decoder takes control of the demodulation mode and IF passband while active. Both are restored automatically when you disable the decoder. All variants use **USB**, except Packet and APRS at 1200 Bd, which use **FM**.

Polarity note (FSK variants only): For Weather RTTY you usually need to check **Invert mark / space**. For Maritime FSK (SITOR/NAVTEX-style) and Amateur RTTY, leave it unchecked — amateur RTTY sends mark as the higher radio frequency, and USB keeps it as the higher audio tone, which is the unchecked case. If decoded text is garbled, toggling this checkbox is the first thing to try. The checkbox is hidden for every variant except the three FSK ones: the others have no mark/space pair, and packet decoding does not depend on which tone is which.

Letters/figures note (FSK variants only): Baudot carries letters and figures in two separate states, and noise can flip the decoder into the wrong one — which garbles every following character, not just the damaged one. The decoder therefore returns to letters on every space, standard practice that repairs a corrupted shift within a word or two instead of a whole line. The cost is that groups of digits separated by spaces need the sender to repeat the figures shift after each space, as transmitters normally do.

11. SSTV

What it is: Slow-Scan Television transmits still pictures over a normal SSB voice channel, one line at a time, as a frequency-modulated tone between 1500 Hz (black) and 2300 Hz (white). A full picture takes between 36 seconds and 4½ minutes depending on the mode.

Recommended frequencies (USB)

Band	Frequency	Mode	Notes
20m	14.230 MHz	USB	The main international SSTV calling frequency — by far the most active
20m	14.233 MHz	USB	Secondary, used when 14.230 is busy
15m	21.340 MHz	USB	
10m	28.680 MHz	USB	Active during band openings
40m	7.171 MHz	LSB	
80m	3.845 MHz	LSB	Regional, evenings

Sideband is chosen for you. Starting the decoder selects **LSB below 10 MHz** and **USB above it**, following normal amateur practice. On the wrong sideband the tone mapping is inverted and the picture will not decode. If you meet a station that ignores the convention, simply change the sideband by hand — the decoder stays running, clears the frame and starts fresh on the new setting.

Setup

1. Enable the decoder and select **SSTV** from the dropdown. Or simply press the **SSTV** button. The receiver switches sideband automatically — USB above 10 MHz, LSB below.
2. Tune so the picture tones fall in the middle of the passband. A correctly tuned signal has its sync pulses at 1200 Hz and the picture content between 1500 and 2300 Hz.
3. Leave **Mode** on **Auto** unless you already know what is being sent. The image builds line by line as it is received.

Modes

Setting	Picture	Duration
Auto	Detected automatically	—
Martin M1 / M2	320×256 colour	114 s / 58 s
Scottie S1 / S2	320×256 colour	110 s / 71 s
Scottie DX	320×256 colour	269 s
Robot 36 / 72	320×240 colour	36 s / 72 s

Auto works two ways: it reads the **VIS header** — the digital mode code sent in the first 300 ms of a transmission — and, if the header was missed (you tuned in late, or it was lost in QSB), it identifies the mode from the sync-pulse timing instead. Selecting a specific mode forces that mode, but the decoder still verifies the sync before it draws anything, so a wrong choice produces no picture rather than noise. Robot 36 is recognised correctly even when you tune in in the middle of a picture (its line timing also fits Robot 72 and Scottie DX, and used to be mistaken for them).

Weak pictures keep drawing to the end: even when no single sync pulse can be told apart from the noise, the decoder sees from many lines together that the picture is still there. When the transmission really ends, it stops within a few seconds.

Reading the output

The status line under the controls reports what the decoder is doing:

Status	Meaning
`Waiting for VIS / AUTO lock`	Listening; nothing identified yet
`Martin M1? verifying sync…`	A candidate was found and is being confirmed against the next lines
`Martin M1 lock (VIS)`	Locked from the VIS header
`Martin M1 lock (AUTO)`	Locked from sync-pulse timing
`Martin M1 lock (MANUAL)`	Started by the ● **Force** button
`Candidate rejected (no sync)`	The candidate failed confirmation — normal on noise
`— sync lost, resetting`	The signal disappeared mid-picture
`Frame complete`	The full picture was received

The detected mode also appears in green next to the **SSTV** title, and the line counter shows progress.

Controls

Button	Action
▶ **Start**	Arms the decoder. It does not draw a picture by itself — a lock still has to come from the VIS header or from sync detection.
■ **Stop**	Stops decoding.
↺ **Reset**	Clears the canvas and re-arms in place. Use it between pictures, or after retuning.
● **Force**	Starts drawing **immediately** in the mode selected in the dropdown, skipping VIS and sync detection entirely. Only available when Mode is not **Auto**.
Save	Saves the current image as a PNG.

When to use Force. If you can see a picture in the waterfall but the decoder will not lock onto it — an unusual mode, a signal too weak or too distorted for the detectors, or a header you missed — select the mode by hand and press ● **Force**. The frame is anchored at the moment you click, and the mode badge shows `MANUAL` so you can tell it from a VIS or AUTO lock. The decoder still snaps to a real sync pulse if it finds one, so a click that is slightly early or late is corrected.

Because Force skips every safety check, it will happily paint noise if you press it on an empty channel, and it will not stop on its own — press ■ **Stop** or ↺ **Reset**.

Notes

- **Noise will not start the decoder.** Static crashes, sparks and mains hash used to be enough to trigger a decode. Every detection stage now tests that the tone is a genuine tone and confirms the candidate against the following lines before a single pixel is drawn. Expect the decoder to sit quietly on an empty band.
- **The picture drifts diagonally if you are off frequency.** SSTV is unforgiving of tuning error. If lines slant, adjust the dial in small steps and let the next picture start.
- Colour balance and sharpness are fixed; there are no adjustments to make.

---

12. SERVICES

SERVICES (the button used to be called **HFDL**) is one window for the utility services. The **Service** list at the top left of the window chooses what it decodes: **HFDL** (aircraft), **Maritime FSK / SITOR**, **Weather RTTY**, **DSC** (ships), **Time Signals**, **HF Selcall** and **ALE 2G (MIL-STD-188-141)**. Your browser remembers the choice. One service runs at a time; each sets the mode and filter it needs, has its own frequency list with ⇒ **Tune & Set IF**, and decodes in your browser, so it costs the receiver nothing. Every service has **Save** (a text file) and **Clear**, and the Users window shows which service a listener is running. The amateur ALE channels are in the FLDIGI window, as ALE 2G HAM (chapter 10).

HFDL — aircraft messages

HFDL (HF Data Link) carries the messages that aircraft over oceans and remote areas exchange with ground stations on shortwave: position reports, ACARS text, logons and frequency data. The decoder is dumphfdl (Tomasz Lemiech, GPL-3) compiled to WebAssembly. It runs in your browser on the USB audio, so it costs the receiver nothing.

Setup

1. Press **SERVICES** and choose **HFDL** in the **Service** list. The receiver switches to USB with a 150–2750 Hz filter.

2. Pick a **ground station** and one of its frequencies, then click ⇒ **Tune & Set IF**. The listed frequency is the USB dial; the HFDL signal sits 1440 Hz above it.
3. Wait. Ground stations send a squitter every 32 s, and aircraft answer when they have traffic. The decoder finds the exact audio rate (±1 %) and a frequency offset (±240 Hz) by itself and locks on the first frame it decodes.

Frequencies

The station list covers every HFDL ground station. Heard well from Europe in daytime: 17967 kHz (Al Muharraq, Bahrain) and 21955 kHz (Canarias); in the evening and at night the lower frequencies of Shannon, Reykjavik and Canarias take over. HFDL frequency markers start the decoder when you click them.

Reading the output

Each line shows the time (UTC), the frequency, the direction (▼ ground→air or ▲ air→ground), the ground station, the aircraft — registration, flight and 24-bit ICAO address, each a link to FlightAware — the message type, its SNR and bit rate, and a position when the message carries one. ACARS text appears underneath. Hover over a line to see the whole frame. Tick **Squitters** to see the ground stations' announcements as well.

Map shows the aircraft that reported a position, inside the panel; ⤢ **Large map** opens the same map in a window you can move and resize. Each aircraft points along its track and leaves a red trail. **Save** writes the messages to a text file, **Clear** empties the list.

Maritime FSK / SITOR and Weather RTTY

These two utility broadcasts use the FSK decoder and panel of the FLDIGI window (chapter 10), with the **Service** list where FLDIGI has its **Variant** list. **Shift**, **Baud**, **Framing**, **Encoding**, **Invert mark / space**, **Auto shift detect**, ⇒ **Set IF Band-Pass** and ⟳ **Auto-tune Center** work as described there.

Service	Center	Shift	Baud	Framing	Encoding
Maritime FSK / SITOR	500 Hz	170 Hz	100	7N1	CCIR-476
Weather RTTY	1000 Hz	450 Hz	50	5N1.5	ITA2

1. Choose **Maritime FSK / SITOR** or **Weather RTTY** in the **Service** list.
2. Pick a **Known frequency** and click **Tune**. The listed frequency is the signal; the USB dial goes the panel's centre below it.
3. Fine-tune until the text is stable. Weather RTTY needs **Invert mark / space** checked (the preset does it) and has a **Squelch (SNR)** slider, starting at −7 dB; Maritime FSK leaves it unchecked.

Maritime FSK / SITOR: 2174.5 kHz (MF NBDP working channel), 518 and 490 kHz (NAVTEX), 4209.5 kHz (HF NAVTEX), and the HF SITOR maritime safety channels 4210, 6314, 8416.5, 12579, 16806.5, 19680.5, 22376 and 26100.5 kHz. NAVTEX also has its own button, with the station schedules (chapter 9).

Weather RTTY: the DWD (German Weather Service) outlets 4583, 7646 and 10100.8 kHz (Programme 1, English) and 11039, 14467.3 and 147.3 kHz (Programme 2, German; 147.3 kHz uses an 85 Hz shift, which its list entry sets). While a DWD frequency is chosen, **NEXT BROADCASTS** shows the coming programme.

DSC — Digital Selective Calling

DSC is the digital calling system of the GMDSS: ships and coast stations call each other, test their equipment and send distress, urgency and safety alerts on fixed MF/HF channels. A call lasts a few seconds (100 Bd, two tones 170 Hz apart).

1. Choose **DSC** in the **Service** list and pick a frequency. The **distress & safety** channels (2187.5, 4207.5, 6312, 8414.5, 12577 and 16804.5 kHz) are watched by every coast station and are the busiest; the coast→ship and ship→coast calling channels follow in the list.
2. Click ⇒ **Tune & Set IF**. The dial goes 1.7 kHz below the DSC frequency, in USB, with a 1400–2000 Hz filter.
3. Wait. Coast stations answer and test many times an hour; by day 8414.5 and 12577 kHz are usually the busiest, at night 2187.5, 4207.5 and 6312 kHz.

Each call shows the time, the frequency, its category in colour (**DISTRESS** red, **URGENCY** orange, **SAFETY** yellow, **ROUTINE** blue), the format and its fields: who calls whom (an MMSI with its country, or a coast station), the commands, the working frequency, and for a distress alert its nature, position (, a link to a map) and time. ✓ means the error-check character agrees; *✓ (1 rebuilt)* means one character was lost in both copies and rebuilt from it. **Offset** shows the tuning error and **S/N** the signal-to-noise ratio.

Time Signals

Decodes the date and time sent by the long-wave and shortwave time stations, and shows how late the minute mark reached your page.

1. Choose **Time Signals** in the **Service** list, pick the station and its frequency, and click ⇒ **Tune & Set IF**. The dial goes 1 kHz below the carrier, in USB, with a 700–1300 Hz filter.
2. Wait. A whole minute has to come in clean before the time can be shown: one to two minutes on a good signal. The seconds fill a row of 60 cells as they arrive.
3. **DCF77** (Germany, 77.5 kHz) and **MSF** (England, 60 kHz) are read from the carrier's on/off keying: date, time and summer time.
4. **WWV / WWVH** (USA, 2.5 to 25 MHz) are read from the 100 Hz subcarrier: date, time and DUT1.
5. **RWM** (Russia, 4996, 9996 and 14996 kHz) sends no date: only the minute mark and DUT1, so it gives timing only.
6. CHU (Canada) closed on 22 June 2026 and is not offered.

The panel shows UTC and the date; the time as the transmitter sends it (*sent as*, its own civil time); the **Station time**, which is UTC in the receiver's own time zone, worked out from the station's QTH locator, with summer time applied by your browser; DUT1 and any warnings; and **Arrived**, how late the minute mark reached the page against your computer's clock. That figure includes the WebSDR's own delay (a few tenths of a second) and your clock's error. When fading spoils every single minute, the decoder combines several minutes and accepts a time only when it fits them clearly better than any other.

HF Selcall

Selcall (CCIR 493-4) is the selective calling of HF land-mobile and maritime networks — Codan, Barrett and others: a burst of a second or two that rings one radio, or a group, by its number. It uses the same signal and characters as DSC.

1. Choose **HF Selcall** in the **Service** list, pick a channel (the 7 and 8 MHz channels of KiwiSDR's list) and click ⇒ **Tune & Set IF**. The channel is the FSK centre; the dial goes 1.7 kHz below it, in USB.
2. Each call shows the calling and the called station's numbers (4 or 6 digits), the type of call, and any commands and data; a Codan GPS position is shown as its raw digits. Stations differ in which tone they send as mark, so both polarities are read.

ALE 2G (MIL-STD-188-141)

The soundings, calls and messages of government, military and utility HF networks: 51 networks from KiwiSDR's ALE list, grouped as **Government**, **Military** (MARS among them) and **Utility**. It is the same decoder and panel as ALE 2G HAM in the FLDIGI window, which lists only the amateur HFLINK channels — see the ALE 2G notes in chapter 10.

1. Choose **ALE 2G (MIL-STD-188-141)** in the **Service** list, pick a network and one of its channels, and click ⇒ **Tune & Set IF**. The listed frequency is the dial; the receiver goes to USB, or LSB for the networks marked so, with a 500–2750 Hz filter.
2. Wait. Soundings come every half hour or hour, calls only when a link is needed.

These are public lists and may be out of date. The Users window shows *ALE* for a listener running it.

---

13. DRM

DRM (Digital Radio Mondiale) is digital broadcasting on shortwave and medium wave. Unlike the other decoders it is decoded on the station's server: when you press **DRM**, the server runs a decoder for you — ubersdr-drm, the headless Dream decoder that UberSDR uses — on the whole 10 kHz DRM channel, and sends you the decoded audio and its status. It therefore plays in every browser, xHE-AAC included. A station runs at most five DRM decoders at a time, one per listener.

Setup

1. Press **DRM** (or pick it in the dropdown). The mode under the frequency reads **DRM** and the passband covers the whole 10 kHz channel.
2. Pick a broadcast in the selector and click ⇒ **Tune**, or tune the dial to the centre of a DRM channel yourself — the decoder follows the dial.
3. Watch the lamps: once **Audio** is green, sound comes out.

While DRM is on, the only sound is decoded DRM. The receiver's own audio is muted, so with no DRM signal you hear nothing. On a weak or fading signal a damaged frame fades out into a short reverb and silence instead of a burst of noise, and the decoder resynchronises by itself when the signal comes back; each frame also gets a second round of error correction.

The station selector

The selector lists the schedule of the DRM Reception Project (drmrx.org), which the server refreshes every hour: frequency, station, UTC times, the days when it is not daily, the area the broadcast is beamed to, ✓ when listeners confirmed it on air, and HFCC when the broadcaster registered it with the HFCC. The groups are **Beamed at this station (locator) — on air now**, **On air now — beamed elsewhere, nearest first**, **Later (next 24 h)** and **Medium wave**. **Confirmed only**, on by default, hides the entries nobody has confirmed. The ranking follows the station's grid locator (`grid_locator` in the `[websdr]` section of the config): a broadcast counts as beamed at the station when its target area comes within about 1000 km of it; when the target is unknown ("Various"), the transmitter's country is used instead. Without a locator the on-air broadcasts are listed in one group. Broadcasts beamed far away are often weak or absent. **heard here** marks a broadcast this receiver has decoded in the last 30 days — the decoder notes every service it decodes, never who listened — and **Confirmed only** keeps those too; one that no schedule lists, such as a test transmission, gets its own group at the top, **Heard on this receiver — on air now, in no schedule**, over the half-hours it was heard.

Good catches from Europe (winter schedule from 25 October 2026): Radio Romania International on 9720 kHz (17:00–18:00 UTC), 9570 kHz (18:00–19:00), 6110 kHz (19:00–20:00) and 5955 kHz (19:00–

19:30). Broadcast schedules change at the end of March and the end of October, and the selector follows them.

Reading the panel

Item	Meaning
Signal, Time, Frame, FAC, SDC, Audio	The sync lamps: green OK, amber data error, red CRC error, grey not present. Sound comes out when **Audio** is green.
WMER	The figure that predicts decoding: from about 10 dB the audio decodes.
MER, SNR	Modulation error ratio and signal-to-noise ratio, in dB.
Mode, BW, QAM, Prot	Robustness mode (A–E), channel bandwidth, MSC constellation and protection levels.
Codec	AAC, AAC+SBR or xHE-AAC, as the station signals it.

Below them come the station's name, country, language and audio mode, and its text message. "No decoder on the server for …" means this station cannot decode that codec.

For sysops. The decoder is built by `./setup-drm.sh` (the installers run it; `update.sh` offers it) and the helper `autorun/drm-helper.js` starts and stops with the receiver; `DRM_ENABLED=0` in front of the start script switches it off. See the installation manual. After an update, run `./setup-drm.sh` again: it patches and rebuilds the decoder (fewer dropouts, no bursts of noise on xHE-AAC), and the new decoder is used once the helper restarts with the receiver.

---

14. General Tips

Decoder: ON must be engaged first. The dropdown is disabled (greyed out) until you click the Decoder button to turn it on.

One decoder at a time. Selecting a new decoder from the dropdown automatically stops the previously active one and resets any mode or passband changes it made.

Mode is managed for you. HF FAX and NAVTEX switch the receiver to USB as soon as you select them, and SSTV picks the sideband from the band (LSB below 10 MHz, USB above), and FAX also adjusts the passband when you click its Tune button. NAVTEX narrows the passband to 350–650 Hz audio as soon as it starts, and FLDIGI to the window of its variant — from the button, the dropdown or a marker. A click on the waterfall puts a decoder's own filter back; dragging the filter edges or turning the mouse wheel leaves a filter you shaped by hand as it is. FT8, FT4, FT2, JS8 and WSPR do the same as soon as you select them — from the button row or the dropdown — switching the receiver to USB and setting their own passband: the whole 3 kHz sub-band for the FT8 family, 1350–1650 Hz for WSPR. When you stop the decoder, the band's default mode is restored.

Switching a decoder on no longer interrupts the audio. The decoders run on their own threads, so there is no gap, click or dropout when one starts, stops, or is swapped for another.

System clock accuracy matters. FT8, FT4, and WSPR are time-critical. They decode in fixed UTC-aligned windows. If your computer clock is off by more than 1–2 seconds, decode rates will drop significantly. Use an NTP client to keep your clock accurate.

The noise filters do not reach the decoders. NR, NB, NS and AN are listening aids for your ears only. Every decoder — FT8, FT4/FT2, CW, WSPR, FAX, NAVTEX, every FLDIGI mode, SSTV and the QRSS grabber — taps the audio *before* those filters, so set them however sounds best without worrying about decode quality. For the same reason the decoders keep running while the receiver is muted or squelched: you can silence the speaker and leave a decoder or an overnight QRSS grab collecting. The

one thing that does follow what you hear is the Audio Spectrogram, which is meant to show the filtered audio.

Signal quality beats signal strength. Most of these decoders are designed for weak signals. A quieter band with lower noise is often more productive than a loud, interference-filled signal. Use the waterfall and passband controls to identify and avoid QRM before enabling a decoder.

Use the Refresh or Clear buttons liberally. FAX images drift if the dial frequency is slightly off, and text decoders accumulate noise characters. A fresh start after tuning adjustments often produces much cleaner output.

Automatic spot reporting and server graphs

FT8, FT4 and WSPR decodes can also be uploaded automatically by the server itself — FT8/FT4 to PSK Reporter, WSPR to WSPRnet — by an autorun daemon the sysop starts from the admin panel. It is independent of the decoders in your browser: it keeps running whether or not anyone is listening, and nothing you decode in the browser is reported.

The sysop follows it with two counters, which are easy to mistake for each other: the per-decoder tiles count the spots uploaded since the daemon last started, while the number beside each band/mode checkbox is that slot's all-time total and survives restarts. The same panel has a **Graphs** page plotting CPU frequency, CPU load, CPU temperature and users online over the last 15 minutes to 24 hours. Both are described in the Admin Panel guide.

PhantomSDR-Plus — sv1btl fork — phantomsdr.no-ip.org

CHAPTER 10

RADE README

RADE_README.md

RADE v1 Digital Voice for PhantomSDR-Plus

> **On a station set up with the station questions (`configure-station.sh`)** the sidecar listens only inside the computer, on `PORT_RADE` (9012), and the page reaches it as `/rade` on the public port through `proxy.py` — port 8074 below is then neither used nor forwarded. On a station set up before the wizard, everything below applies as written.

RADE (Radio AutoencoDEr) is FreeDV's flagship HF digital voice mode. It uses a Machine Learning / DSP hybrid (the FARGAN neural vocoder) to deliver high-quality speech over HF radio at SNRs as low as −2 dB, in just 1500 Hz of RF bandwidth — narrower than an SSB signal.

This document covers the complete integration of RADE v1 receive support into PhantomSDR-Plus, implemented as a Python sidecar (`rade_helper.py`) that bridges the browser to the `radae_rxe.py` + `lpcnet_demo` decode pipeline.

It also decodes **RADE V2 (beta)**, the new waveform of FreeDV 3.0, with the **RADE-v2U** button — see RADE V2 (beta) — RADE2.

Installing it — the short way

```
cd ~/PhantomSDR-Plus
./install_rade.sh
```

That is the whole installation. The script installs the system packages through whichever package manager the machine has (apt, pacman, dnf or zypper), falls back to pip for anything a distribution does not ship, upgrades `websockets` if the distro version is older than the 11.0 the sidecar needs, clones and builds radae, checks the model weights, rebuilds the frontend and starts the sidecar. On Ubuntu 22.04 it hands over to `install_rade_ubuntu22.sh` for you. `./install.sh` offers to run it as part of the normal PhantomSDR-Plus installation, so on a fresh machine RADE is already there. It also builds FreeDV's rade_c for RADE V2 (beta); if that step fails, RADE V1 is installed anyway.

This document explains how RADE works and how to run it — it is not the install procedure. If you need to do the installation by hand, on a system the script does not cover or to repair one step, follow the manual guide linked below; it is the single place the steps are written down. Everything here — the architecture, the patched files, the port, using RADE in the browser, the environment variables, troubleshooting and concurrency measurement — applies to every installation however it was made.

- **Manual installation on Linux** - the by-hand route, step by step, covering Ubuntu, Debian, Fedora, Arch and Raspberry Pi OS

Table of Contents

1. Installing it — the short way
2. How It Works
3. Architecture Overview

4. RADEL vs RADEU
5. RADE V2 (beta) — RADE2
6. Prerequisites
7. Installing by hand
8. The Patched Files
9. Sidecar control
10. Port 8074
11. Using RADE in the Browser
12. Verification and Debugging
13. Environment Variables
14. Files Changed
15. Signal Flow Summary
16. Troubleshooting
17. Measuring Concurrency on Your Own Hardware
18. Updating RADE v1

How It Works

RADE v1 cannot run in the browser — it requires PyTorch and the FARGAN neural vocoder, which are too large for WASM. The solution is a Python sidecar process (`rade_helper.py`) that runs on the same server as PhantomSDR-Plus.

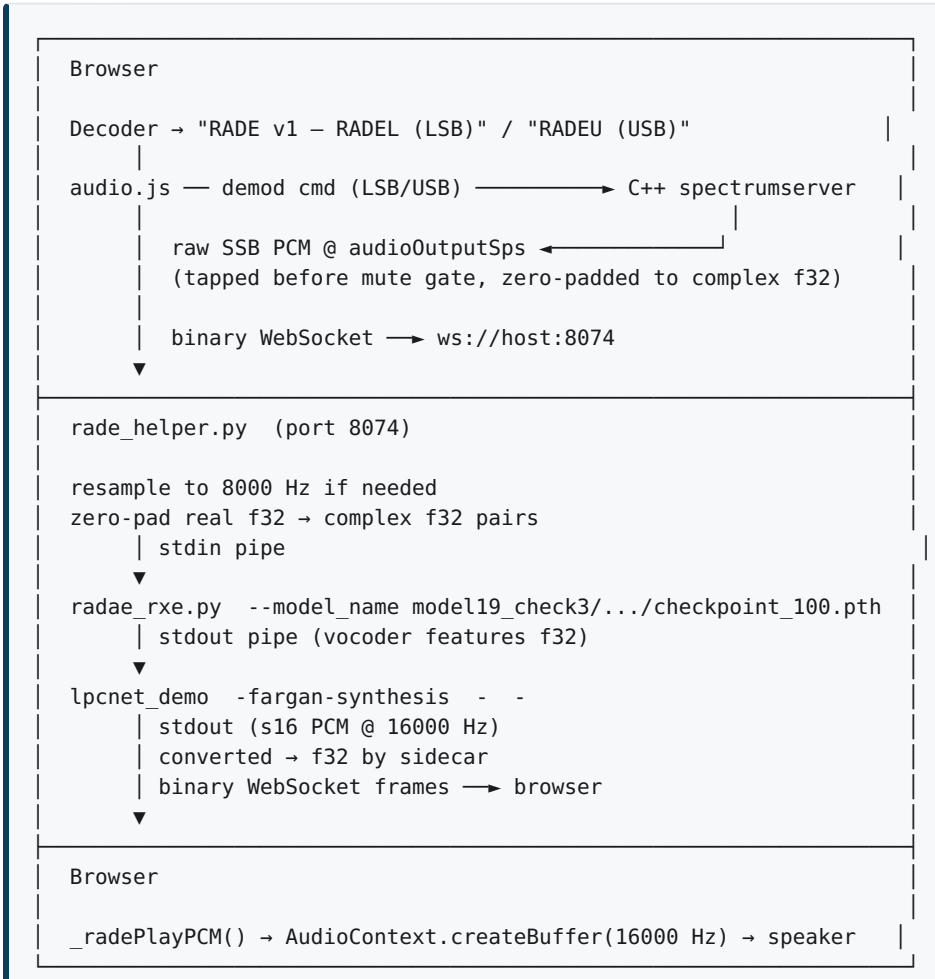
> **Important:** `freedv_rx` from the codec2 repository does **not** support RADEV1. The RADE decode pipeline lives entirely in the separate `radae` repository by David Rowe (VK5DGR). Do not attempt to use codec2's `freedv_rx` for RADE.

When you select RADE in the browser:

1. The frontend sets the underlying demodulation to USB or LSB — the C++ server demodulates the SSB signal as normal.
2. The raw demodulated PCM is tapped in `audio.js` **before** any mute or squelch gating — `radae_rxe.py` needs continuous input to maintain frame sync.
3. Each PCM chunk is zero-padded from real f32 to complex f32 (real + 0.0 imaginary) — this is what `radae_rxe.py` expects on stdin.
4. The sidecar pipes those samples into `radae_rxe.py`, which outputs vocoder features.
5. `lpcnet_demo -fargan-synthesis` converts the features into s16 speech at 16000 Hz.
6. The sidecar converts s16 → f32 and sends it back to the browser as binary WebSocket frames.
7. The browser plays the decoded speech via the Web Audio API at 16000 Hz.

The C++ server (`spectrumserver.cpp`) is not modified at all.

Architecture Overview



RADEL vs RADEU

RADE v1 is always transmitted over SSB. By convention:

Mode	Sideband	Use on bands
RADEL	LSB	160 m, 80 m, 40 m (≤ 10 MHz)
RADEU	USB	20 m, 17 m, 15 m, 12 m, 10 m (> 10 MHz)

RADE V2 (beta) — RADE2

RADE V2 is the new waveform of FreeDV 3.0: about 800 Hz wide instead of 1500 Hz, with no pilot symbols, and about 3 dB more sensitive at low SNR. It is **still a beta at FreeDV** — only stations testing FreeDV 3.0 send it, and the signal may still change before the release. PhantomSDR-Plus decodes it next to RADE V1:

Button	Version	Sideband	Filter
RADE-v2U (RADE2)	RADE V2 (beta)	USB on every band	1040–1930 Hz
RADE-v1L / **RADE-v1U** (RADEL / RADEU)	RADE V1	LSB / USB	700–2200 Hz

V2 is decoded on the server by FreeDV's C port **rade_c** (`radae_rx --v2` and its own `lpcnet_demo`), built in its own folder `~/rade_c`, so the V1 installation in `~/radae` is never touched. `./install_rade.sh` clones and builds it (step "rade_c (RADE V2, beta)"); if that step fails it only warns, and RADE V1 is installed anyway. `rade_helper.py` uses rade_c from the next RADE2 connection — no restart is needed — and without it RADE2 tells the listener "RADE V2 is not installed on this station yet". A V2 decode costs about 1 % of one CPU core per listener (V1: about 70–80 %).

To build rade_c by hand (on Ubuntu 22.04 first `sudo apt-get install autoconf automake libtool`):

```
cd ~
git clone https://github.com/freedv/rade_c.git
cd rade_c
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j4
```

On the air

- Tune within about **±30 Hz** of the station's frequency. Further off, V2 still shows "Synced" but the speech is garbled.
- Each over starts **2–3 s late**: while V2 locks it keeps the signal, then plays the beginning of the over.
- Below about **−2 dB SNR** V2's audio is muted and the panel says "Searching for signal…" — unless the last 2 seconds look like real V2 (the decoder sees its signal most of the time and its frequency estimate stays steady); then it plays down to about **−6 dB**. On busy LSB phone frequencies heard on USB (40 m) V2 can lock onto ordinary voices and play garbled sound; those never look steady, so they stay muted. `RADE_V2_MIN_SNR` and `RADE_V2_WEAK_SNR` set the two levels.
- Between overs the panel goes back to "Searching for signal…".

What the sidecar does for V2. The receiver's audio is resampled to 8 kHz in one continuous stream — in separate pieces V2 kept losing lock; it is brought up to a level the decoder works at — at the receiver's low level V2 decoded only the first seconds of each over; and the decoded speech is levelled towards −16 dBFS (`RADE_V2_TARGET_DB`), because it is only as loud as the transmitting station's microphone. RADE V1 now goes through the same continuous resampling, which lets it decode weaker and fading signals better than before.

When FreeDV 3.0 is released, run `./install_rade.sh` again — or `git pull` in `~/rade_c` and `cmake --build build` — and new RADE2 connections then use the new build.

Prerequisites

Requirement	Version	Notes
PhantomSDR-Plus	any	with Vite/Svelte frontend
Linux	Ubuntu 24.04+ / Debian Bookworm+	tested
Python	3.8+	for `rade_helper.py` and `radae_rxe.py`
radae repo	latest master	provides `radae_rxe.py` and `lpcnet_demo`
cmake	3.10+	to build `lpcnet_demo` from radae
PyTorch	2.0+	required by `radae_rxe.py`
Node.js	16+	for `npm run build`
websockets (Python)	10–16+	`pip3 install websockets` — all versions supported
matplotlib	any	required by `radae_rxe.py` at import time
numpy	1.23+	required; also enables resampling
scipy	any	optional, enables accurate resampling

Installing by hand

`./install_rade.sh` does the whole installation. When you want to do it yourself — a system the script does not cover, or one step to repair — the procedure lives in **RADE v1 — Manual Installation Guide**, and only there. It covers Ubuntu, Debian, Fedora, Arch and Raspberry Pi OS in one pass:

Step 1	System packages, and the Node.js 22+ check
Step 2	Python packages, including the PEP 668 flag and the CPU-only torch wheel
Step 3	Clone and build the radae repository, verify `lpcnet_demo` and the model weights
Step 4	Verify the decode pipeline offline, before wiring it to the browser
Step 5	Build the PhantomSDR-Plus frontend
Step 6	Sidecar control
Step 7	Open port 8074
Steps 8–10	Start the server, verify RADE, use it in the browser

The rest of this document assumes that is done.

The Patched Files

Copy the following files from the patch set into your PhantomSDR-Plus repository.

New file — place in the repository root, alongside the start script you use:

```
rade_helper.py
```

Patched frontend files — place in `frontend/src/`:

```
audio.js
App.svelte
```

What the patches do

`audio.js` — 5 changes:

- Constructor: 5 new RADE state fields (`decodeRADE`, `_radeSideband`, `_radeSocket`, `_radeCallback`, `_radeReady`, `_radeNextTime`)
- Pre-boost PCM save: `pcmArrayPreBoost` saved before the FLAC 300× gain boost so RADE receives original amplitude (the boost would saturate `radae_rxe.py`)
- `playAudio()`: RADE PCM tap using pre-boost audio, before mute/squelch gate
- `playAudio()`: `if (this.decodeRADE) return` guard — suppresses raw SSB playback while RADE decoded speech plays
- New methods: `setRADEDecoding()`, `setRADECallback()`, `_radePlayPCM()` with gapless scheduled playback via `_radeNextTime` clock (plays at **16000 Hz** — the output rate of `lpcnet_demo`)

Each Svelte variant — 6 changes:

- `demodulationDefaults`: RADEL `{type:'LSB', offsets:[2200,-700]}`, RADEU `{type:'USB', offsets:[-700,2200]}` — passband starts 700 Hz from carrier, 1500 Hz wide
- State variables: `radeEnabled`, `radeConnected`, `radeSynced`, `radeSnr`, `_radeDeactivate()`
- `_radeDeactivate()`: stops the decoder and **restores the band default mode** from `bands-config.js` — identical behaviour to FAX, NAVTEX, FSK deactivation
- `_deactivateAll()`: RADE cleanup line
- `activateSelectedDecoder()`: `radel` and `radeu` branches
- Decoder dropdown: two new `<option>` entries
- Status panel: connection dot, sync/SNR status, error banner

Startup scripts

The receiver launchers (`start-rx888mk2.sh`, `start-airspyhf.sh`, `start-rtl.sh`, `start-rsp1a.sh`, `start-fobos-hf.sh`, `start-fobos.sh`, `start-hackrf.sh`) start the sidecar themselves once the server is up, and `stop-websdr.sh` stops it with the server. `rade.sh` remains for running the sidecar standalone — see Sidecar control.

Sidecar control

You normally do not need a separate RADE control script. The receiver launchers start the sidecar once the server is confirmed up, restart it if it exits, and report its state in their closing summary — `✔ RADE sidecar running`, or a `·` note explaining why it was not activated (not installed, `RADE_ENABLED=0`, or no `python3`). `./stop-websdr.sh` stops it along with the server. To run the server without RADE, start it with `RADE_ENABLED=0`.

`rade.sh` ships for standalone use — running the sidecar without the PhantomSDR-Plus launcher, or testing it on its own:

```
chmod +x ~/PhantomSDR-Plus/rade.sh
cd ~/PhantomSDR-Plus
./rade.sh start      # start sidecar with self-restarting watchdog
./rade.sh stop       # stop sidecar + watchdog + all lpcnet_demo children
./rade.sh restart    # stop then start cleanly
./rade.sh status     # show running / stopped + process info
```

Sidecar activity is logged to `~/PhantomSDR-Plus/rade.log`:

```
tail -f ~/PhantomSDR-Plus/rade.log
```

> **Do not mix the two while the server is running.** If a launcher's watchdog is active it will bring the sidecar back within about five seconds, so `./rade.sh stop` looks like it failed. To stop RADE while the server runs, either start the server with `RADE_ENABLED=0`, or stop everything with `./stop-websdr.sh`.
>
> If you do run `rade.sh` standalone, **always use `./rade.sh stop`** — `pkill -f rade_helper.py` alone will not beat its own watchdog, which respawns the process within 3 seconds.

Port 8074

The sidecar listens on TCP port **8074** and the browser connects to it directly, so the port has to be reachable from outside. Opening it — ufw, firewalld, iptables, the router NAT rule, and the Nginx proxy to use when an ISP blocks the port outright — is Step 7 of the manual guide.

Two things worth repeating here, because they cost the most time:

```
# Is the sidecar listening?
ss -tlnp | grep 8074
```

> **NAT hairpin warning:** testing with `curl` from the server to its own public hostname often returns `Connection refused` even when the port is open — many routers do not loop traffic back. Always test from an outside machine, or use **https://portchecker.co**.

Using RADE in the Browser

1. Open your PhantomSDR-Plus web interface
2. Find active stations at **qso.freedv.org**
3. Tune to the station's dial frequency
4. In **Decoder Options**, select:
5. **RADE v2 · RADE2** for RADE V2 (beta), USB on every band
6. **RADE v1 — RADEL (LSB)** for 40 m / 80 m / 160 m
7. **RADE v1 — RADEU (USB)** for 20 m / 17 m / 15 m / 12 m / 10 m
8. Click **Decoder: ON**

The RADE-v1L / RADE-v1U buttons (one-touch)

Steps 4 and 5 can be replaced by a single press. A pair of **RADE-v1L** / **RADE-v1U** buttons is available in three places:

- next to the **Modes** heading on the main panel;
- inside the **Modes** popup window;
- inside the **Bands** popup window.

Pressing one selects the decoder, switches the decoder ON, closes the popup you pressed it in, and scrolls the RADE panel into view. The button turns blue while RADE is running. **Press the same button again to switch RADE off** — the panel closes and mode and passband revert as described below. The buttons, the **Decoder Options** dropdown and the ON/OFF button all share the same state.

A **RADE-v2U** button sits before RADE-v1L and RADE-v1U in the same three places (and a RADE2 button on `/mobile`); it starts RADE V2 (beta) on USB — see RADE V2 (beta) — RADE2.

Panel indicator states

Indicator	Meaning
Red — "Connecting to sidecar…"	Port 8074 unreachable or sidecar not running
Yellow — "Searching for signal…"	Sidecar connected, no RADE frame detected yet
Green — "Synced · SNR x.x dB"	Decoding — speech is playing

When you turn the decoder OFF

Mode and passband automatically revert to the correct default for the current frequency as defined in `bands-config.js` — identical to FAX, NAVTEX, FSK.

Verification and Debugging

Is the sidecar running?

```
ps aux | grep rade_helper | grep -v grep
ss -tlnp | grep 8074
```

Watch live activity

```
tail -f ~/PhantomSDR-Plus/rade.log
```

When a browser connects:

```
[RADE] client connected: x.x.x.x:XXXXX
[RADE] x.x.x.x:XXXXX  sps=12000 sideband=LSB
[RADE] x.x.x.x:XXXXX spawning pipeline (torch_threads=1)
```

Quick WebSocket smoke test

```
python3 - &lt;&lt; 'EOF'
import asyncio, websockets, json

async def test():
    async with websockets.connect('ws://localhost:8074') as ws:
        await ws.send(json.dumps({'type': 'init', 'sps': 8000, 'sideband': 'LSB'}))
        print(await ws.recv())   # expect: {"type": "status", "connected": true}

asyncio.run(test())
EOF
```

Browser console (F12)

```
[RADE] ▶ ENABLED LSB @ 12000 Hz → helper ws://localhost:8074
```

Good — WebSocket opened. The Hz value matches your server's `audioOutputSps` (typically 8000–12000 Hz).

```
[RADE] sidecar socket error
```

Port 8074 is unreachable — check firewall and router NAT rule.

Environment Variables

Variable	Default	Description
`RADE_HELPER_PORT`	`8074`	TCP port the sidecar listens on
`RADE_HELPER_HOST`	`0.0.0.0`	Bind address (`127.0.0.1` if behind a proxy)
`RADAE_DIR`	`~/radae`	Root of the radae repository
`RADE_MODEL`	`RADAE_DIR/model19_check3/checkpoints/checkpoint_epoch_100.pth`	Model weights
`LPCNET_DEMO`	`RADAE_DIR/build/src/lpcnet_demo`	lpcnet_demo binary
`RADE_AUXDATA`	`1`	Set to `0` to pass `--noauxdata` to `radae_rxe.py`
`RADE_TORCH_THREADS`	`1`	PyTorch/OpenBLAS threads per `radae_rxe.py` instance — limits CPU usage
`RADE_PIN_CORES`	`1`	Pin each decode pipeline to its own core set; `0` disables pinning
`RADE_CORES_PER_CLIENT`	`2`	Cores allocated per client when pinning is enabled
`RADE_C_DIR`	`~/rade_c`	Root of rade_c (RADE V2)
`RADE_C_RX`	`RADE_C_DIR/build/src/radae_rx`	rade_c receiver program
`RADE_C_LPCNET`	`RADE_C_DIR/build/src/lpcnet_demo`	rade_c's own lpcnet_demo
`RADE_V2_MIN_SNR`	`-2`	V2 audio is muted below this SNR (dB)
`RADE_V2_WEAK_SNR`	`-6`	Down to this SNR (dB) V2 still plays while the signal looks like real V2; set it to `RADE_V2_MIN_SNR` or higher to switch that off
`RADE_V2_TARGET_DB`	`-16`	Level V2 speech is brought towards, in dBFS (`0` = off)

Files Changed

File	Type	Notes
`rade_helper.py`	**New**	Python sidecar: WebSocket server + two-process decode pipeline
`frontend/src/audio.js`	Modified	4 patches
`frontend/src/App.svelte`	Modified	6 patches
`rade.sh`	**New**	RADE sidecar control script for standalone use (start/stop/restart/status)
`start-rx888mk2.sh`	Modified	starts, watchdogs and reports the sidecar; same for `start-airspyhf.sh`, `start-rtl.sh`, `start-rsp1a.sh`, `start-fobos-hf.sh`, `start-fobos.sh`, `start-hackrf.sh`
`stop-websdr.sh`	Modified	stops the sidecar along with the server
`spectrumserver.cpp`	**Unchanged**	No C++ modifications required

Signal Flow Summary

```
Antenna → RX-888 MK2 → PhantomSDR-Plus C++ server
                              |
                    FFT + DDC + SSB demodulation
                    (USB or LSB — set by RADEL/RADEU)
                              |
                    Opus/FLAC encode → WebSocket → Browser
                              |
                         audio.js decode()
                              |
                         playAudio(pcmArray)
                              |
                    ┌─────────┴──────────────────────────────┐
                    │  rawPcm tap (before mute gate)         │
                    │  real f32 → zero-padded complex f32 pairs  │
                    └─────────┬──────────────────────────────┘
                              | WebSocket binary → ws://host:8074
                              ▼
                        rade_helper.py
                              | resample to 8000 Hz if needed
                              | stdin pipe
                              ▼
          radae_rxe.py  --model_name model19_check3/.../checkpoint_epoch_100.pth
          (PyTorch FARGAN neural vocoder, auxdata ON by default)
                              | stdout pipe (vocoder features f32)
                              ▼
          lpcnet_demo  -fargan-synthesis  -  -
                              | stdout: s16 PCM @ 16000 Hz
                              | sidecar converts s16 → f32
                              | WebSocket binary frames → browser
                              ▼
                    audio.js  _radePlayPCM()
                    AudioContext.createBuffer(16000 Hz)
                              |
                           Speaker
```

Troubleshooting

Red banner — "Sidecar not reachable"

```
# Is sidecar running?
ps aux | grep rade_helper | grep -v grep

# Start it manually for testing
python3 ~/PhantomSDR-Plus/rade_helper.py &

# Check port is open externally — use portchecker.co NOT curl from the server
# (curl from the server uses NAT loopback and gives false "Connection refused")
```

Port 8074 closed on portchecker.co despite iptables rule

The ISP may be filtering the port, or the router NAT rule is missing. Options:

1. Try a different port: `RADE_HELPER_PORT=8080 python3 rade_helper.py`
2. Proxy through your existing public port via Nginx (see Step 7 of the manual guide)

`radae_rxe.py: error: unrecognized arguments: model_path`

The model path must use `--model_name`, not a positional argument:

```
# WRONG — positional argument
python3 radae_rxe.py model19_check3/checkpoints/checkpoint_epoch_100.pth

# CORRECT — named argument
python3 radae_rxe.py --model_name model19_check3/checkpoints/checkpoint_epoch_100.pth
```

`ModuleNotFoundError: No module named 'matplotlib'`

```
pip3 install matplotlib
```

`radae_rxe.py` imports matplotlib unconditionally at the top of the file.

`ModuleNotFoundError: No module named 'torch'`

```
pip3 install torch
```

`lpcnet_demo: No such file or directory`

The radae build did not complete. Rebuild:

```
cd ~/radae/build
cmake .. && make -j$(nproc)
ls src/lpcnet_demo     # should exist now
```

`size mismatch` error in `inference.sh`

`model19_check3` requires `--auxdata` when encoding with `inference.sh`:

```
./inference.sh model19_check3/checkpoints/checkpoint_epoch_100.pth \
    wav/brian_g8sez.wav /dev/null \
    --rate_Fs --pilots --pilot_eq --eq_ls --cp 0.004 \
    --write_rx rx.f32 --auxdata    # ← required for model19_check3
```

Note: when **decoding** with `radae_rxe.py`, `--auxdata` is the default — do not pass it.

Yellow indicator — "Searching for signal" — never syncs

- Confirm the correct sideband: RADEL for ≤ 10 MHz, RADEU for > 10 MHz
- Check qso.freedv.org to confirm a station is currently transmitting
- RADE v1 uses 30 carriers in 1500 Hz BW — it appears as a compact cluster on the waterfall
- Allow up to 1.5 seconds for acquisition

`underrun!!!` from aplay during the offline pipeline test

Expected during offline file testing only. `radae_rxe.py` processes slower than file I/O, causing the audio buffer to starve. This does not occur during live reception because the browser feeds audio at real-time rate.

`ConnectionClosedError: received 1011 (internal error)`

The sidecar accepted the WebSocket connection but crashed internally before responding. This is caused by an incompatible version of `rade_helper.py` — an older version attempted to pass an asyncio `StreamReader` as a subprocess stdin, which fails silently and closes with error 1011.

Fix: Replace `rade_helper.py` with the current version from the patch set. The current version uses `os.pipe()` for the inter-process pipe and is compatible with websockets 10.x through 16.x+.

Multiple simultaneous users

Each browser connection spawns its own independent `radae_rxe.py` + `lpcnet_demo` pair, so each user can tune to a different frequency freely.

By default `radae_rxe.py` uses **all available CPU cores** for PyTorch matrix operations, causing ~900% CPU usage per instance. `rade_helper.py` limits this with two optimisations:

1. `OMP_NUM_THREADS=1` (and MKL/OpenBLAS equivalents) — limits PyTorch to 1 thread
2. Numpy-vectorised audio conversion functions — eliminates Python loop overhead

Result: each instance uses roughly **~8–10%** of one core, sufficient for real-time RADE decode. If you hear audio dropouts, raise to 2 threads:

```
RADE_TORCH_THREADS=2 ./start-rx888mk2.sh     # or your receiver's launcher
# standalone: RADE_TORCH_THREADS=2 ./rade.sh restart
```

Simultaneous RADE users	Approximate CPU
1	~9%
5	~45%
10	~90%
20	~180%

On an i5-12450H (12 threads) with spectrumserver using ~42% and rx888_stream ~11%, you have roughly **1000% headroom** — enough for **20+ simultaneous** RADE users before CPU becomes a concern.

> **The table above assumes CPU cost scales linearly with users. Measurement says it does not.** A `rade_loadtest.py` run on that same i5-12450H showed per-listener CPU well below this table up to ~24 listeners, then rising sharply above it, with the machine stopped by **package temperature at 32 listeners** — not by CPU and not by bandwidth. Treat these figures as a rough guide for small numbers only and measure your own box: see Measuring Concurrency on Your Own Hardware. Note the load test drives the pipeline with noise rather than a real RADE signal, so absolute CPU under genuine traffic may differ; the *shape* of the curve is the reliable part.

Measuring Concurrency on Your Own Hardware

The numbers above are from one specific box. `rade_loadtest.py` (repository root) measures the same thing on **yours** — it ramps synthetic RADE listeners against the sidecar in steps and reports the last listener count your machine sustained before something gave way.

After each step it lets the box settle, then samples package temperature, total CPU, available RAM, swap, and the combined RSS of every `radae_rxe.py` and `lpcnet_demo` process. It stops at the first **knee** — whichever limit is breached first — and prints the last stable count.

Requirements

```
sudo apt install python3-websockets python3-psutil
```

Run it **on the SDR host** (it reads local sensors and process memory), with spectrumserver and the RADE sidecar already running:

```
cd ~/PhantomSDR-Plus
python3 rade_loadtest.py
```

Ctrl-C tears down all connections cleanly at any point.

Before the first run

Open the file and check the two blocks at the top:

- **`CONFIG`** — `WS_URL` defaults to `ws://127.0.0.1:8074`; change it if you moved the sidecar off its default port. `RADE_SPS` (12000) and `RADE_SIDEBAND` (`USB`) should match what your frontend reports.
- **`handshake_messages()`** — the single `init` frame it sends must match the RADE init in `frontend/src/audio.js`. If that handshake has drifted, every listener fails to connect and the test ends immediately with connect failures.

Knee thresholds

Tune these in `CONFIG` to taste — they are deliberately conservative:

Threshold	Default	Stops when
`TEMP_KNEE_C`	`85.0`	Package temperature reaches this ceiling
`MIN_AVAIL_MB`	`800`	Available RAM falls below this
`SWAP_GROWTH_MB`	`50`	Swap grows past the baseline taken at startup
`FAIL_LIMIT`	`3`	This many listeners fail to connect in one step
`MAX_LISTENERS`	`60`	Hard stop even if no knee is ever hit

Ramp shape is controlled by `STEP` (listeners added per step, default 2), `SETTLE_S` (25 s of settling before sampling — this is what lets heat build) and `SAMPLE_S` (5 s CPU averaging window). A full run to the default 60-listener ceiling therefore takes roughly 15 minutes.

Reading the results

Each step prints a line and appends a row to `rade_loadtest.csv`:

Column	Meaning
`listeners`	Synthetic listeners connected at this step
`connect_fails`	Listeners that failed to establish or hold a connection
`pkg_c`	Package temperature, median of 5 samples (spike-resistant)
`cpu_pct`	**Whole-system** CPU percent, not per-listener
`avail_mb`	Available RAM
`swap_mb`	Swap in use
`dec_rss_mb`	Combined RSS of all decoder processes
`dec_procs`	Decoder process count — expect **2 per listener** (`radae_rxe.py` + `lpcnet_demo`)

A measured run

On the i5-12450H referenced above, a full run ended like this:

```
KNEE at n=32: temp 91.0°C ≥ 85.0
Last stable concurrency: 30 RADE listeners
```

Heat was the binding constraint, and it was not close. At the knee there was still 7004 MB of RAM available — 6.2 GB clear of the `MIN_AVAIL_MB` cutoff — and zero connect failures on every step.

Two columns are worth reading carefully:

`dec_rss_mb` overstates real memory cost. It grows at a very steady 295 MB per listener (292, 294, 294 … 295 across all 16 steps), but *available RAM* only falls about **202 MB per listener**. The two processes behind each listener share library pages, and RSS counts those pages once per process. Extrapolating the `avail_mb` slope, the memory knee would arrive near **63 listeners** — past the 60-listener hard stop, so on this box it can never trip. Size memory from `avail_mb`, not from `dec_rss_mb`.

`cpu_pct` is flat and then it is not. It sits at ~5.3% of the whole machine through 24 listeners, then goes superlinear:

Listeners	24	26	28	30	32
`cpu_pct`	6.7%	18.4%	30.1%	49.6%	65.2%

That is a 10× rise in CPU while listener count grows by a third. The inflection reproduces at the same point across runs, so treat it as a property of the box rather than noise — per-listener CPU cost is roughly 3.4% of one core below the inflection and about 20% above it. Do not extrapolate a per-user CPU figure taken at low listener counts.

> **The test has no CPU knee.** The four cutoffs are temperature, available RAM, swap growth and connect failures — CPU is recorded but never ends the run. In the run above CPU reached 65% and would have kept climbing had temperature not tripped first. If you care about a CPU ceiling, watch the column yourself or add a threshold.
>
> **What this does and does not prove.** Each synthetic listener streams low-amplitude random noise, not a real RADE signal. That is enough to make the full decode pipeline run and consume resources, so the resource figures are meaningful — but the test says nothing about decode *quality* or audio continuity under load. Confirm those with real listeners on a real signal.

If cores saturate before the knee

RADE pins one thread hard per instance, so individual cores can reach ~90 °C while total CPU still looks idle. `rade_helper.py` spreads pipelines across cores round-robin to counter this. If a run shows early thermal knees, widen the core allocation per client:

```
RADE_CORES_PER_CLIENT=3 ./rade.sh restart
```

No file edit is needed — this is an environment variable (see Environment Variables); `RADE_PIN_CORES=0` disables pinning entirely.

Updating RADE v1

The PhantomSDR-Plus integration (`rade_helper.py`, frontend patches) is a bridge only — all RADE decode logic lives in the `radae` repository. Updates are therefore almost always a simple `git pull` + rebuild with no changes to PhantomSDR-Plus itself.

Standard update (new code, same model)

```
# 1. Pull latest radae code
cd ~/radae
git pull

# 2. Rebuild lpcnet_demo (in case C code changed)
cd build
cmake ..
make -j$(nproc)

# 3. Restart the sidecar — no server restart needed
cd ~/PhantomSDR-Plus
./rade.sh restart
```

That is all. No frontend rebuild, no server restart, no file edits.

New model weights only

If a new checkpoint is released (e.g. `model20`) without code changes, point the sidecar at the new weights using the env var — no file editing required:

```
# One-off: start with new model
RADE_MODEL=~/radae/model20/checkpoints/checkpoint_epoch_100.pth ./rade.sh start

# Or permanently — add to your shell profile (~/.bashrc):
export RADE_MODEL=~/radae/model20/checkpoints/checkpoint_epoch_100.pth
```

What each type of change requires

What changed in radae	Action required
New model weights (new `.pth` file)	Set `RADE_MODEL` env var, `./rade.sh restart`
Code change in `radae_rxe.py`	`git pull`, `./rade.sh restart`
`lpcnet_demo` C code changed	`git pull`, rebuild, `./rade.sh restart`
`radae_rxe.py` renamed or moved	Update `RADAE_RX` path in `rade_helper.py` (one line)
`--model_name` argument renamed	Update `radae_cmd` in `rade_helper.py` (one line)
New output sample rate (≠ 16000 Hz)	Update `SPS_OUT` in `rade_helper.py` + `createBuffer()` in `audio.js`
RADE v2 uses a different binary	Update `RADAE_RX` in `rade_helper.py` (one line)

Verifying the update worked

After restarting, confirm the new code is running:

```
# Check sidecar picked up new radae_rxe.py
./rade.sh status

# Tail log to see startup lines
tail -20 ~/PhantomSDR-Plus/rade.log
```

The log should show the model path you expect:

```
[RADE] model : /home/sv1btl/radae/model19_check3/checkpoints/checkpoint_epoch_100.pth
```

Staying up to date

Subscribe to the radae repository releases page to be notified of new models and code updates:

```
https://github.com/drowe67/radae/releases
```

Check the FreeDV blog for announcements about new RADE waveforms and models:

```
https://freedv.org/blog/
```

Developed and tested on PhantomSDR-Plus (sv1btl/PhantomSDR-Plus fork) with RX-888 MK2 on Ubuntu 24.04. The C++ server is not modified.

RADE is developed by David Rowe VK5DGR and the FreeDV team. See freedv.org/radio-autoencoder.

Full uninstall and fresh re-install

Here's the full teardown before re-running install_rade.sh:

1. Stop the sidecar cd ~/PhantomSDR-Plus && ./rade.sh stop
2. Remove the radae repo and build rm -rf ~/radae
3. Remove torch (installed in user local) pip3 uninstall -y torch rm -rf ~/.local/lib/python3.11/site-packages/torch*
4. Remove apt Python packages (optional — skip if used by other things) sudo apt-get remove -y python3-numpy python3-scipy python3-matplotlib python3-websockets sudo apt-get autoremove -y
5. Verify everything is gone python3 -c "import torch" 2>&1 # should say ModuleNotFoundError ls ~/radae 2>&1 # should say No such file or directory
6. Fresh install chmod +x ~/PhantomSDR-Plus/install_rade.sh ~/PhantomSDR-Plus/install_rade.sh

CHAPTER 11

RADE General Install (Linux)

RADE_General_INSTALL_MANUAL_LINUX.md

RADE v1 — Manual Installation Guide

> **On a station set up with the station questions (`configure-station.sh`)** the sidecar listens only inside the computer, on `PORT_RADE` (9012), and the page reaches it as `/rade` on the public port through `proxy.py` — port 8074 below is then neither used nor forwarded. On a station set up before the wizard, everything below applies as written.

All Linux — Ubuntu / Debian / Fedora / Arch / Raspberry Pi OS · PhantomSDR-Plus

> **Pre-condition:** The patched files (`rade_helper.py`, `rade.sh`, `audio.js`, `App.svelte`) are already placed in the PhantomSDR-Plus directory tree. This guide builds everything around them.
>
> > **IMPORTANT**
> >
> > **You do not need this guide for a normal installation.** `./install_rade.sh` in the PhantomSDR-Plus folder does every step below for you — system packages, the Python modules, the radae build, the model-weights check and the sidecar start — and it works on apt, pacman, dnf and zypper systems alike. `./install.sh` (and the four distro installers) offer to run it for you as part of the normal installation. Follow this guide only for a manual setup, for a system the script does not cover, or to repair one step by hand.
>
> **Raspberry Pi users:** this is your guide too. Raspberry Pi OS is Debian, so follow the Debian blocks throughout, and read the **Raspberry Pi / Bookworm** notes where they appear — they cover the two things that differ on a Pi: PEP 668 and the CPU-only torch wheel.

Step 1 — System packages

Choose the block that matches your distribution.

Ubuntu / Debian / Raspberry Pi OS

```
sudo apt update
sudo apt install -y \
    build-essential cmake git \
    python3 python3-pip \
    nodejs npm \
    alsa-utils
```

Fedora / RHEL / Rocky

```
sudo dnf install -y \
    gcc gcc-c++ cmake git \
    python3 python3-pip \
    nodejs npm \
    alsa-utils
```

Arch / Manjaro

```
sudo pacman -Sy --needed \
    base-devel cmake git \
    python python-pip \
    nodejs npm \
    alsa-utils
```

Node.js version check (all distros)

RADE's frontend build requires Node.js 22 or later:

```
node --version
```

If below 22.x, install it with `nvm` (any distro, no root needed):

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.4/install.sh | bash
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
nvm install 22 && nvm use 22 && nvm alias default 22
```

Not NodeSource. `deb.nodesource.com` and `rpm.nodesource.com` now answer HTTP 403 on every repository path, so the old `curl -fsSL https://deb.nodesource.com/setup_NN.x | sudo -E bash -` line no longer works — and on Debian/Ubuntu it leaves an apt source behind that breaks every later `apt update`. The install scripts use `nvm` for the same reason.

Step 2 — Python packages

Ubuntu 24.04 and earlier / Fedora / Arch

Plain `pip3 install` works on these systems:

```
pip3 install websockets matplotlib numpy scipy

# torch — CPU-only build (~150–250 MB, avoids the ~3 GB CUDA wheel)
# Use this if the server has no GPU, which is the typical case for a WebSDR
pip3 install torch --index-url https://download.pytorch.org/whl/cpu
```

If your server **does** have an NVIDIA GPU and CUDA installed, you can omit `--index-url` to get the full CUDA build. There is no performance benefit for RADE — `radae_rxe.py` uses PyTorch for CPU matrix ops only.

Ubuntu 23.04+ / Debian Bookworm+ / Raspberry Pi OS (PEP 668 systems)

These distributions block bare `pip3 install`. Add the flag:

```
# All packages except torch
pip3 install --break-system-packages websockets matplotlib numpy scipy

# torch — CPU-only build
pip3 install --break-system-packages torch \
    --index-url https://download.pytorch.org/whl/cpu
```

> **Raspberry Pi / Bookworm — two rules that bite:**
>
> 1. Every `pip3 install` requires `--break-system-packages` (PEP 668 enforcement). Without it the install is blocked entirely.
> 2. Plain `pip3 install torch` downloads the CUDA wheel (~3 GB). On a Pi there is no CUDA — the `--index-url` above is what gets you the lean ARM64 wheel (~150 MB) instead.

Alternatively, use a virtual environment to avoid the flag entirely:

```
python3 -m venv ~/rade-venv
source ~/rade-venv/bin/activate
pip install websockets matplotlib numpy scipy torch \
    --index-url https://download.pytorch.org/whl/cpu
```

> If using a venv, prefix all subsequent `python3` calls in this guide with `source ~/rade-venv/bin/activate` or use the full path `~/rade-venv/bin/python3`.

Verify

```
python3 -c "import websockets, matplotlib, torch, numpy, scipy; print('All OK')"
```

Expected output:

```
All OK
```

Step 3 — Clone and build the radae repository

The RADE decoder lives in a separate repository from codec2. `freedv_rx` from codec2 does **not** support RADE v1.

```
# Remove any previous incomplete clone
rm -rf ~/radae

# Clone
git clone https://github.com/drowe67/radae.git ~/radae
cd ~/radae

# Build
mkdir build && cd build
cmake ..
make -j$(nproc)
```

Verify lpcnet_demo was built

```
ls -la ~/radae/build/src/lpcnet_demo
```

Expected: the binary is present and executable.

Verify model weights are present

```
ls ~/radae/model19_check3/checkpoints/checkpoint_epoch_100.pth
```

Expected: the `.pth` file is listed. The weights are bundled in the repository — no separate download needed.

Step 4 — Verify the decode pipeline

This step confirms the full offline pipeline works before wiring it to the browser. Run from `~/radae`:

```
cd ~/radae
```

4a — Generate a RADE-encoded test signal

```
./inference.sh model19_check3/checkpoints/checkpoint_epoch_100.pth \
    wav/brian_g8sez.wav /dev/null \
    --rate_Fs --pilots --pilot_eq --eq_ls --cp 0.004 \
    --write_rx rx.f32 --auxdata
```

Wait for it to finish. The last lines printed should be:

```
loss: 0.741 Auxdata BER: 0.012
```

4b — Decode and play

```
cat rx.f32 \
    | python3 radae_rxe.py \
        --model_name model19_check3/checkpoints/checkpoint_epoch_100.pth \
    | ./build/src/lpcnet_demo -fargan-synthesis - - \
    | aplay -f S16_LE -r 16000
```

You should hear a voice. The output shows sync acquisition:

```
  1 state: search     ...
  5 state: sync       ... SNRdB:  4.03 uw_err: 0
Playing raw data 'stdin' : Signed 16 bit Little Endian, Rate 16000 Hz, Mono
```

> **`underrun!!!` messages during this test are expected and harmless.** They occur because `radae_rxe.py` processes slower than file I/O. They do not appear during live reception — the browser feeds audio at real-time rate.

4c — Verify the sidecar starts correctly

```
python3 ~/PhantomSDR-Plus/rade_helper.py
```

Expected output (no WARNING lines):

```
[RADE] helper starting on ws://0.0.0.0:8074
[RADE] radae_rx.py    : /home/<user>/radae/radae_rxe.py
[RADE] model          : /home/<user>/radae/model19_check3/checkpoints/checkpoint_epoch_100.pth
[RADE] lpcnet_demo    : /home/<user>/radae/build/src/lpcnet_demo
[RADE] auxdata        : ON (default)
[RADE] torch threads  : 1 per instance (RADE_TORCH_THREADS to override)
[RADE] architecture   : per-connection (each user tunes independently)
[RADE] listening — waiting for PhantomSDR-Plus clients
```

Press **Ctrl+C** to stop.

> `<user>` in the paths above is whichever account you are logged in as — on a stock Raspberry Pi OS image that makes them `/home/pi/radae/...`.

Step 5 — Build the PhantomSDR-Plus frontend

```
cd ~/PhantomSDR-Plus/frontend

# Only if node_modules is missing:
npm install

cd ~/PhantomSDR-Plus
./recompile.sh
```

Watch for Vite/acorn parser errors. The patched files deliberately avoid `?.`, `??`, and bare `catch {}` to comply with the acorn constraint.

Step 6 — Sidecar control

You normally do not need a separate RADE control script. The start scripts (`start-rx888mk2.sh`, `start-airspyhf.sh`, `start-rtl.sh`, `start-rsp1a.sh`, `start-fobos-hf.sh`, `start-fobos.sh`, `start-hackrf.sh`) start the sidecar themselves once the server is up, restart it if it exits, and report its state; `stop-websdr.sh` stops it along with the server. Go straight to Step 7.

`rade.sh` still ships for standalone use — running the sidecar without the PhantomSDR-Plus launcher, or testing it on its own. Its commands, its log, and the watchdog caveats that go with it are in Sidecar control.

Step 7 — Open port 8074

The sidecar listens on TCP port **8074**. The browser connects directly to this port. You must open it manually.

ufw (Ubuntu / Debian / Raspberry Pi OS)

```
sudo ufw allow 8074/tcp && sudo ufw reload
```

firewalld (Fedora / RHEL / Rocky)

```
sudo firewall-cmd --add-port=8074/tcp --permanent
sudo firewall-cmd --reload
```

iptables (any distro, permanent)

```
sudo iptables -I INPUT -p tcp --dport 8074 -j ACCEPT

# Ubuntu / Debian / Raspberry Pi OS — persist across reboots:
sudo apt install iptables-persistent
sudo netfilter-persistent save

# Fedora / RHEL — persist:
sudo service iptables save
```

Router

Add a NAT/port-forward rule: **TCP 8074 → server LAN IP : 8074**

Test from outside

Use **https://portchecker.co** and check port 8074 against your public hostname. Do **not** test with `curl` from the server itself — NAT hairpin gives false "Connection refused" results even when the port is open.

Alternative — Nginx proxy (if port 8074 is blocked by ISP)

Add inside your existing `server {}` block:

```
location /rade {
    proxy_pass         http://127.0.0.1:8074;
    proxy_http_version 1.1;
    proxy_set_header   Upgrade    $http_upgrade;
    proxy_set_header   Connection "upgrade";
    proxy_set_header   Host       $host;
    proxy_read_timeout 3600s;
}
```

```
sudo nginx -t && sudo nginx -s reload
```

Then edit `audio.js` inside `setRADEDecoding()`:

```
// Change:
var helperUri = uri || ('ws://' + window.location.hostname + ':8074');
// To:
var helperUri = uri || ('ws://' + window.location.host + '/rade');
```

Rebuild the frontend after this change (`./recompile.sh`).

Step 8 — Start the server

Start PhantomSDR-Plus as usual. The launcher starts RADE for you, once the receiver and `spectrumserver` are both confirmed running:

```
cd ~/PhantomSDR-Plus
./start-rx888mk2.sh      # or start-airspyhf.sh,
                         # start-rtl.sh, start-rsp1a.sh,
                         # start-fobos-hf.sh, start-fobos.sh,
                         # start-hackrf.sh
```

Use the launcher that matches your receiver. Each one includes its own watchdog and log; `./stop-websdr.sh` stops whichever is running, RADE included.

The launcher mirrors its log to the terminal and finishes with a summary line for the sidecar — `✔ RADE sidecar running`, or a `·` note explaining why it was not activated (not installed, `RADE_ENABLED=0`, or no `python3`). A missing sidecar never stops the server from starting.

Step 9 — Verify RADE

The sidecar was already started by Step 8. Confirm it:

```
cd ~/PhantomSDR-Plus
pgrep -af rade_helper.py     # should print one python3 process
tail -f rade.log
```

Expected log:

```
[RADE] sidecar starting at ...
[RADE] helper starting on ws://0.0.0.0:8074
[RADE] radae_rx.py    : /home/<user>/radae/radae_rxe.py
[RADE] model          : /home/<user>/radae/model19_check3/checkpoints/checkpoint_epoch_100.pth
[RADE] lpcnet_demo    : /home/<user>/radae/build/src/lpcnet_demo
[RADE] auxdata        : ON (default)
[RADE] torch threads  : 1 per instance
[RADE] architecture   : per-connection (each user tunes independently)
[RADE] listening — waiting for PhantomSDR-Plus clients
```

Step 10 — Using RADE in the browser

1. Open your PhantomSDR-Plus web interface
2. Find active stations at **https://qso.freedv.org**
3. Tune to the station's dial frequency
4. In **Decoder Options**, select:
5. **RADE v1 — RADEL (LSB)** for 40 m / 80 m / 160 m (≤ 10 MHz)
6. **RADE v1 — RADEU (USB)** for 20 m / 17 m / 15 m / 12 m / 10 m (> 10 MHz)
7. Click **Decoder: ON**

Steps 4 and 5 can be replaced by a single press: **RADE-v1L** / **RADE-v1U** buttons sit next to the **Modes** heading and inside the **Modes** and **Bands** popup windows. One press selects the decoder, switches it ON and scrolls the RADE panel into view; press the same button again to switch RADE off. See the RADE manual.

Indicator	Meaning
Red — "Connecting to sidecar…"	Port 8074 unreachable or sidecar not running
Yellow — "Searching for signal…"	Sidecar connected, no RADE frame detected yet (allow ~1.5 s)
Green — "Synced · SNR x.x dB"	Decoding — speech is playing

CPU note

Each RADE user uses ~8–10 % of one core (PyTorch thread count capped at 1 by the sidecar). If you hear audio dropouts, raise to 2 threads:

```
RADE_TORCH_THREADS=2 ./start-rx888mk2.sh     # or your receiver's launcher
```

The launcher passes its environment through to the sidecar, so setting the variable on the start command is enough. (Standalone: `RADE_TORCH_THREADS=2 ./rade.sh restart`.)

Simultaneous users	Approximate CPU
1	~9 %
5	~45 %
10	~90 %
20	~180 %

> These figures were measured on x86_64. A Raspberry Pi decodes RADE fine, but the per-user cost is higher and the table above does not transfer — measure your own board with `top` while a user is synced before advertising a user limit.

Optional — RADE V2 (beta) with rade_c

RADE V2 (beta) — the RADE-v2U button — is decoded by FreeDV's rade_c, which lives in its own folder and leaves `~/radae` alone. `install_rade.sh` builds it for you; by hand (on Ubuntu 22.04 first `sudo apt-get install autoconf automake libtool`):

```
cd ~
git clone https://github.com/freedv/rade_c.git
cd rade_c
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j4
```

`rade_helper.py` uses it from the next RADE2 connection — see the RADE README.

Updating RADE in future

```
cd ~/radae
git pull
cmake -S . -B build && make -C build -j$(nproc)

cd ~/PhantomSDR-Plus
./start-rx888mk2.sh          # or your receiver's launcher
```

Re-running the launcher restarts the sidecar along with the server. No frontend rebuild is needed unless `rade_helper.py` itself changed.

Tested on Ubuntu 24.04 (x86_64) and on Raspberry Pi 4 / Raspberry Pi OS Bookworm (ARM64, Python 3.11). PhantomSDR-Plus fork: sv1btl/PhantomSDR-Plus. RADE developed by David Rowe VK5DGR and the FreeDV team — https://freedv.org

CHAPTER 12

Editing Variants

EDITING_VARIANTS.md

PhantomSDR-Plus — How to Edit the Frontend Variants

(after the four `App__*_smeter_.svelte` files were merged into one `App.svelte`)

1. What the Variants Are

Two independent choices, giving four combinations:

```
smeter = "analog"     the moving-needle meter face
       = "digital"    the segmented bar-graph meter

layout = "v1"         Modes selector above,  Band selector below
       = "v2"         Band  selector above,  Modes selector below
                      (also moves the mobile fine-tuning button row)
```

All four live in **one build**, served at `/`. The ⚙selector at the top right switches between them inside the running page: it assigns `App.svelte`'s `smeter`/`layout` props, so nothing reloads and audio, the waterfall and the decoders keep running. The choice is stored per browser in `localStorage` under `phantom.variant`, and that stored value wins over the build default on the next visit.

The variant the sysop picks in `recompile.sh` is therefore the **starting** variant — what a first-time visitor sees before touching the menu.

URL	what it is
`/`	the desktop page, all four variants, switchable at runtime
`/mobile/`	*separate page, built by `build-mobile.sh` — not a variant*

> Until 2026-08-10 each variant was its own build under `/analog/`, `/digital/`, `/v2-analog/` and `/v2-digital/`, and the selector navigated between them — which is why switching used to reload the page and drop the audio. Those builds are gone; each of the four paths now holds only a small `index.html` that redirects to `/`, written by `frontend/make-redirect-stubs.sh` on every build so old bookmarks keep working.

The variant is **not** written into any source file. `src/main.js` is a fixed entry point that hands two build-time defines to the component:

```
const app = new App({
  target: document.getElementById('app'),
  props: { smeter: __PHANTOM_SMETER__, layout: __PHANTOM_LAYOUT__ }
})
```

`vite.config.js` fills those in at build time, taking the first of these that is set:

1. **`PHANTOM_SMETER` / `PHANTOM_LAYOUT`** — environment variables. Nothing sets them now that the per-variant builds are gone; they still work for a one-off build with a different starting variant.

2. **`frontend/variant.json`** — the site default, written by `recompile.sh`. The root (`/`) build passes no variant, so it lands here.
3. **`analog` / `v1`** — the fallback.

Those two values are only the **seed**. `App.svelte` overwrites them at init from `localStorage` when the visitor has already chosen a variant, and the ⚙selector assigns them again on every switch.

Inside `App.svelte` the props are turned into two flags at lines 20-21:

```
$: isAnalog = smeter !== "digital";
$: isV2     = layout === "v2";
```

Use those two flags — never `smeter`/`layout` directly — so the whole file reads the same way.

---

2. Which File Does What

`frontend/src/main.js` — 12 lines

The entry point. Nothing but the App instantiation and the two `__PHANTOM_*__` defines it passes as props. It is **constant** — neither `recompile.sh` nor the build scripts rewrite it any more, and keeping it that way is what lets the variant builds run at the same time.

> **Do not put application code here.** It is the entry file, not a place for logic.

`frontend/src/App.svelte` — ~17130 lines

The whole application: waterfall, tuning, decoders, bookmarks, panels, desktop layout AND the responsive phone layout. Almost every edit you will ever make goes here.

`frontend/src/lib/SMeterAnalog.svelte` — 824 lines

The moving-needle meter, complete: face drawing, needle, the light/dark face toggle and its localStorage key, and the needle smoothing.

- Input prop: `dbm` — the CALIBRATED power in dBm.
- Also takes: `mobile` — smaller canvas attributes for the phone layout.
- Tuning knob: `SMOOTH_TIME_MS` at line 55. Lower = faster needle, higher = smoother. Currently 16 (one frame at 60 fps, effectively the floor).

The three meter faces — and `smeter_theme.sh`

The analog meter has three background faces, all drawn by the same code:

```
dark      the original dark brushed metal
amber     a light, pale grey face
vintage   a warm aged-amber face
```

Do not confuse this with the `smeter = analog / digital` variant above. The variant chooses *which meter component* is shown; the face chooses *how the analog one is painted*. They are stored under different localStorage keys (`phantom.variant` vs `smeterTheme`) and are set in different places.

The visitor's side. The meter canvas is a button (`role="button"`, also Enter/Space from the keyboard, with a "click to switch style" hint under it). Each click cycles dark → amber → vintage and saves the choice in `localStorage.smeterTheme`. It is read back on every page load, so the visitor's pick survives

refreshes and browser restarts indefinitely. It is deliberately **not** a cookie: a cookie would be sent to the server on every waterfall and audio request for a value the server never reads.

The sysop's side. Two things in `SMeterAnalog.svelte` control what everyone else gets:

Line	What it does
`let smeterTheme = 'dark';`	The face a browser with **no saved choice** gets.
`const SMETER_PREF_VERSION = 2;`	A counter. When a browser's stored version is behind this, it forgets its saved face **once** and takes the default above. Bump it to move visitors who have already chosen.

Editing the default alone therefore only reaches brand-new visitors. Both edits together move everybody — that is what `smeter_theme.sh` in the repository root is for:

```
./smeter_theme.sh                # report the current default, then pick one from a menu
./smeter_theme.sh vintage        # set it straight away, then offer the rebuild
./smeter_theme.sh dark --build   # run frontend/build-all.sh without asking
./smeter_theme.sh amber --no-reset  # new visitors only — leave existing users alone
```

It reports the current default, sets the new one, bumps the reset counter (always upward — **never lower it**, a lower number simply stops the reset from firing), and then offers the same frontend build choice `recompile.sh` gives. It does **not** touch the backend and does **not** rewrite `variant.json`.

> A change reaches a connected listener only when their page reloads — the default is compiled into the JS bundle and the reset runs at load time. There is no live push. An ordinary reload is enough, since vite hashes the bundle filenames.

The counter's one-time wipe overrides everyone, including visitors who deliberately clicked their own face. That is intended: it is the only way to bring the whole site back onto one look. Use `--no-reset` when you would rather leave them alone.

`frontend/src/lib/SMeterDigital.svelte` — 184 lines

The segmented bar meter, complete.

- Input props: `rawDb` — the RAW `audio.getPowerDb()`
- `smeterOffset` — `audio.smeter_offset`
- `mobile` — smaller canvas attributes

> **NOTE:** this meter is deliberately driven from the RAW power plus the offset, NOT from the calibrated dBm the analog face uses. It has always done its own mapping. Feeding it the calibrated value would shift every reading on every digital site. Do not "unify" these two inputs.

`frontend/src/lib/BandSelector.svelte` — 69 lines

The band buttons grid. Rendered twice in `App.svelte` (once per layout).

`frontend/src/lib/ModesSelector.svelte` — 75 lines

The mode buttons grid. Rendered twice in `App.svelte`.

`frontend/src/lib/StatusIndicators.svelte` — 122 lines

The status lamps (mute, squelch, NR, NB, NS, AN, CTCSS). Takes a `wide` prop: the digital layout uses wider lamps than the analog one. Both Tailwind classes (`w-8` and `w-10`) are written out in full inside the

component — Tailwind's scanner only sees literal class names, so never build them by string concatenation.

All other files in `frontend/src/lib/` (Spectrogram, PassbandTuner, FrequencyInput, VersionSelector, ...) predate this work and are unchanged.

---

3. Making a Change That Affects Only Some Variants

Wrap it in the flag. In markup:

```
{#if isAnalog}
  ...only the analog sites see this...
{:else}
  ...only the digital sites see this...
{/if}

{#if isV2} ... {/if}          {#if !isV2} ... {/if}
```

In a class attribute:

```
class="p-4 {isAnalog ? 'text-xs' : 'text-sm'} rounded-md"
```

Write both alternatives out as whole literals, as above. Tailwind scans the source for complete class names; `text-{size}` or a concatenated name is purged from the CSS and the style silently vanishes.

In the `<script>` block: try NOT to branch at all. The script deliberately computes the inputs of BOTH meters on every tick —

```
smeterDbm    = powerDb;                    // analog face
smeterRawDb  = audio.getPowerDb();         // digital face
smeterOffset = audio.smeter_offset;        // digital face
```

— so only the markup has to choose. That is why there are just 15 variant switches in a 17000-line file. Keep it that way; script-level branching is what made the four old files drift apart.

---

4. Where the Variant Switches Are

Fifteen places in `App.svelte`. Line numbers drift when you edit — the reliable way to find them all is:

```
cd frontend/src && grep -n 'isAnalog\|isV2' App.svelte
```

As of this writing:

Line	Flag	What it switches
20		`isAnalog` declared
21		`isV2` declared
8062	`isV2`	desktop: Band selector first (v2 ordering)
8090	`isV2`	desktop: panel heading size
8686	`isV2`	desktop: Modes selector first (v1 ordering)
8717	`isAnalog`	meter panel min-width (needle is narrower)
8730	`isAnalog`	analog-only date/time line above the meter
8819	`isAnalog`	analog-only status lamps + divider
8834	`isAnalog`	digital-only time line, wide lamps, and the `<SMeterDigital>` / `<SMeterAnalog>` choice itself
8861	`isAnalog`	vertical spacing of the frequency block
8864	`isAnalog`	top margin of the fine-tuning block
12288	`isV2`	phone: fine-tuning row above (v2 ordering)
12347	`isAnalog`	phone: which meter to render
12433	`isV2`	phone: fine-tuning row below (v1 ordering)
13204	`isAnalog`	noise-gate button text size

Everything else in the file is shared by all four variants.

5. Other Knobs Worth Knowing

Location	Constant	Purpose
`App.svelte` line 5034	`const visualGain = 1.1;`	Analog meter calibration. Each 0.10 is roughly 5 dBm on the reading.
`SMeterAnalog.svelte` line 55	`const SMOOTH_TIME_MS = 16;`	Needle smoothing time constant, in milliseconds.
`SMeterAnalog.svelte` line 64	`let smeterTheme = 'dark';`	Default analog meter face for a browser with no saved choice. Set it with `./smeter_theme.sh`, not by hand.
`SMeterAnalog.svelte` line 69	`const SMETER_PREF_VERSION = 2;`	One-time reset counter for the saved face. Bump to move existing visitors onto the default; only ever goes up.
`SMeterDigital.svelte` line 149	`const DIGITAL_BAR_TRIM = 0;`	Shifts the bar by whole segments.
`SMeterDigital.svelte` line 31	`const numberOfDots = 35;`	Segment count of the bar.

6. After You Edit — Rebuilding

Easiest, from the repository root:

```
./recompile.sh
```

Menu **[2]** rebuilds the frontend and asks which variant the site root (`/`) should serve. That choice is recorded in `frontend/variant.json` — no source file is copied or rewritten. (It used to regenerate `VersionSelector.svelte` from a heredoc as well; that was removed when the selector stopped navigating, because it would have reverted the runtime switching on every run.) Menu **[1]** is backend only, **[3]** is both.

Or directly, from `frontend/`:

```
./build-all.sh          builds the desktop page AND /mobile/  &lt;-- normally use this
./build-default.sh      just the desktop page
./build-mobile.sh       just /mobile/
```

> `build-default.sh` lets vite empty `dist/`, which removes `dist/mobile/` until the next `build-all.sh` or `build-mobile.sh`. That is why `build-all.sh` is the normal choice.

`build-all.sh` has its own copy of the build logic (`build_version SMETER LAYOUT NAME OUTDIR BASE LOG`) rather than calling `build-default.sh`. If you change how the page is built, change it in BOTH places or the two paths will disagree.

A few details worth knowing:

- It sets `PHANTOM_KEEP_OUTDIR=1` so Vite does not empty the output directory, and cleans `dist/` itself instead — deleting the contents but keeping `users.json`, which the running `spectrumserver` rewrites on every listener connect and disconnect.
- It builds `/mobile` last, after the desktop output is complete.
- Its parallel-build machinery (`PHANTOM_BUILD_JOBS`, default 3) is left over from when there were five desktop builds. With one, nothing queues.

Executable bits: `recompile.sh` runs `chmod +x` on a script before running it, and `build-all.sh` deliberately invokes `bash build-mobile.sh` rather than `./build-mobile.sh`, so a missing bit no longer breaks a build. Uploading a script through the GitHub web interface still resets it to 644 though, so if you run one directly:

```
chmod +x frontend/build-*.sh
```

---

7. How to Check Your Edit Before Trusting It

A green `vite build` is **NOT** enough. The build cannot see an identifier that went missing when code moved between files — that is a runtime `ReferenceError`, and the page simply does not load. It has happened twice.

1) Undefined-identifier check (catches exactly that failure)

```
cd frontend
npx eslint --no-eslintrc \
    --parser svelte-eslint-parser \
    --rule '{"no-undef":"error"}' \
    --env browser,es2022 \
    src/App.svelte src/lib/SMeter*.svelte
```

One known-benign hit is expected:

```
'dBmCalOffset' is not defined
```

`typeof x === 'number'` on an undeclared name never throws, so that expression already evaluates to 0. Anything else is a real problem.

2) Build, then LOAD THE PAGE AND SWITCH THROUGH ALL FOUR VARIANTS

Not just the one you changed — an edit under `isAnalog`/`isV2` can break the other three while yours works. Open `/`, step through all four entries in the ⚙menu, then open `/mobile/`.

Open the browser console on each. A blank or half-drawn page with a `ReferenceError` in the console is the signature of the bug above.

3) Never leave `frontend/dist/` in a partial state

A bare `npx vite build` empties the output directory and takes `dist/mobile/` and the server's live `dist/users.json` with it — the mobile page 404s until the next successful build. Always build through the scripts: `build-all.sh` sets `PHANTOM_KEEP_OUTDIR=1` and does the cleaning itself, preserving `dist/users.json`. If in doubt, just run `./build-all.sh` again.

CHAPTER 13

User Guide

USER_GUIDE.md

PhantomSDR-Plus User Guide

Welcome to PhantomSDR-Plus! This guide will help you get the most out of your WebSDR listening experience.

Table of Contents

1. Introduction
2. Getting Started
3. Interface Overview
4. Basic Operations
5. Advanced Features
6. Demodulation Modes
7. Digital Decoders
8. Keyboard Shortcuts
9. Bookmarks
10. Mobile Usage
11. Tips and Best Practices
12. Troubleshooting
13. Frequently Asked Questions

Introduction

What is PhantomSDR-Plus?

PhantomSDR-Plus is a web-based Software Defined Radio (SDR) that allows you to listen to radio signals over the internet. No special software or hardware is required on your end—just a modern web browser!

What Can You Listen To?

Depending on the WebSDR's configuration, you can tune into:

- **Amateur Radio**: Ham radio operators worldwide
- **Broadcast Stations**: AM/FM radio, shortwave broadcasting
- **Aviation**: Air traffic control, aircraft communications
- **Marine**: Ship-to-shore, maritime weather
- **Weather Satellites**: NOAA, METEOR-M
- **Digital Modes**: FT8, JS8, RTTY, PSK31, and more
- **Utility Stations**: Time signals, military, government

System Requirements

- **Browser**: Chrome/Edge (recommended), Firefox, Safari
- **Connection**: Broadband internet (1+ Mbps recommended)
- **Audio**: Working speakers or headphones
- **Optional**: Mouse with scroll wheel for easier tuning

Getting Started

1. Access the WebSDR

Open your browser and navigate to the WebSDR URL provided by the operator.

Example: `http://websdr.example.com:9002`

2. Initial Page Load

When the page loads, you'll see:

- A colorful waterfall display showing radio activity
- Control panel with frequency display and buttons
- S-meter showing signal strength
- User count indicator

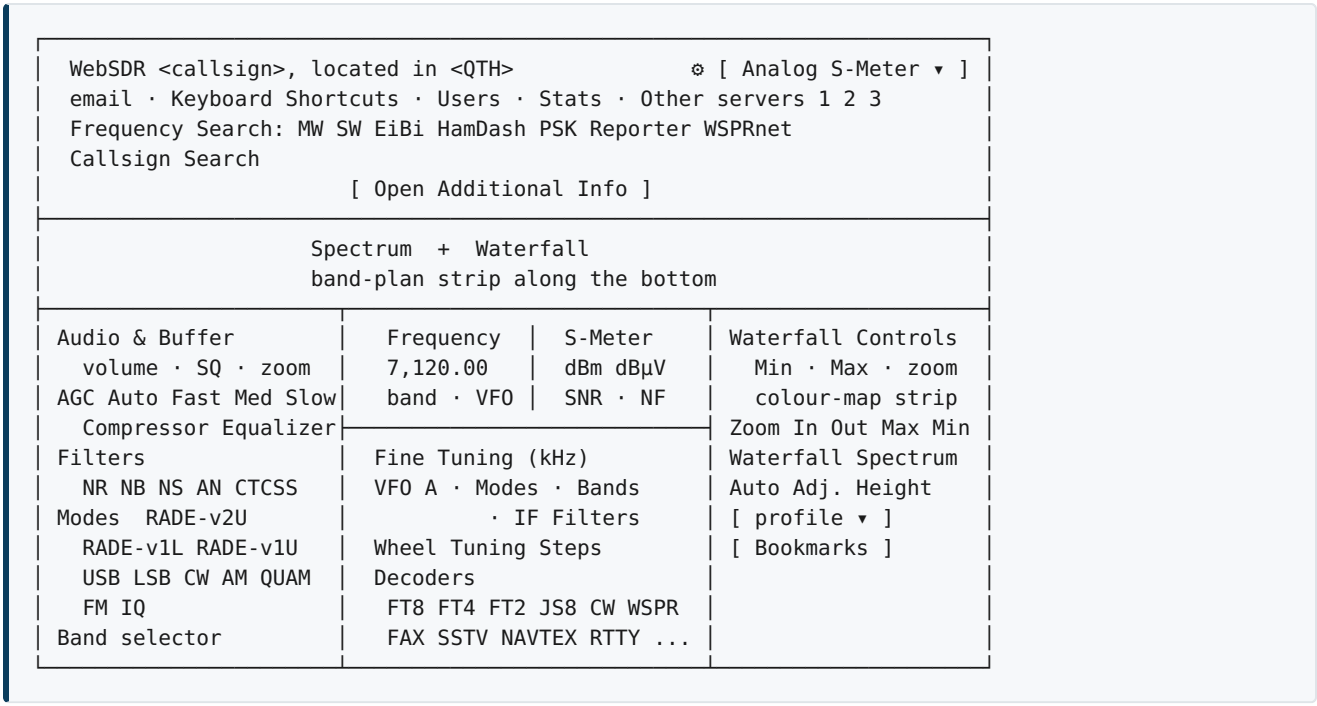
3. Start Listening

1. **Click on a signal** in the waterfall display
2. **Audio will automatically start** playing
3. **Adjust volume** using your browser's volume control or the on-screen slider
4. **Fine-tune frequency** by clicking precisely on the signal

---

Interface Overview

Main Components



The three panels below the waterfall scroll as one page; on a narrow window they stack instead of sitting side by side. **RADE-v2U**, **RADE-v1L** and **RADE-v1U** sit on the *Modes* heading line itself, to its right.

1. Waterfall Display

The waterfall is a visual representation of radio signals:

- **Horizontal axis**: Frequency
- **Vertical axis**: Time (scrolls downward)
- **Colors**: Signal strength
- **Dark blue/black**: No signal (noise floor)
- **Green/yellow**: Weak to moderate signals
- **Orange/red**: Strong signals
- **White**: Very strong signals

2. Frequency Display

Shows the currently tuned frequency in various formats:

- **MHz**: 7.100.000 MHz (HF bands)
- **kHz**: 14200 kHz
- **Hz**: 145500000 Hz (VHF/UHF)

3. S-Meter (analog or digital)

Displays signal strength:

- **S1-S9**: Standard signal strength scale
- **+dB**: Signals above S9 (e.g., S9+20dB)
- **Analog or Digital**: Depending on configuration

4. Mode Buttons

Select demodulation mode:

- **AM**: Amplitude Modulation
- **FM**: Frequency Modulation
- **USB**: Upper Sideband
- **LSB**: Lower Sideband
- **CW**: Morse Code (Continuous Wave)
- **WBFM**: Wideband FM (broadcast)
- **QUAM**: C-QUAM AM stereo — the label turns green when a stereo pilot is detected
- **IQ**: raw I/Q — 12 kHz of the band as it arrives, for recording or for another program (see the IQ section below)

Next to the **Modes** heading sit the **RADE-v2U**, **RADE-v1L** and **RADE-v1U** buttons, which start RADE digital voice in one press — see Digital Decoders. **RADE-v2U** starts RADE V2 (beta), always on USB; **RADE-v1L** and **RADE-v1U** start RADE v1.

5. Control Panel

Additional controls:

- **AGC**: Automatic Gain Control
- **NR**: Noise Reduction (spectral)
- **NB**: Noise Blanker (impulse)
- **NS**: Background Noise Suppression
- **AN**: Auto Notch
- **CTCSS**: Sub-audible tone squelch (FM). Pressing it opens a small window beside the button with *Any tone* and the 49 standard subtones (67.0–254.1 Hz); pick one and the button shows it (for example *88.5*, or *ANY*), and the audio then passes only while that tone is received. The window's *Heard:* line shows the tone the receiver hears on the channel right now and how strong it is, so an unknown repeater tone can be read off. While CTCSS is on, FM audio plays 0.4 s behind the receiver, so the squelch is already open when a station's first words arrive; with *Any tone* the start can still be clipped slightly, so pick the exact tone where you can. Press the button again to switch CTCSS off; Escape or a click outside closes the window without changing anything. Bookmarks keep the chosen tone.
- **SQL**: Squelch
- **AI**: AI noise reduction (voice modes)
- **Zoom**: Waterfall zoom level
- **Wheel Tuning Steps**: step size used by the mouse wheel, from 10 Hz to 12.5 kHz (12.5 kHz is the VHF/UHF channel spacing)
- **Decoders**: one-touch buttons that start and stop each decoder

6. Band Plan Overlay

Colored bars showing frequency allocations:

- Different colors for different services
- Helps identify what's allowed on each frequency

Basic Operations

Tuning to a Frequency

Method 1: Click on Waterfall

1. Click directly on a signal in the waterfall
2. The receiver will tune to that frequency
3. Audio will begin playing

Method 2: Type Frequency

1. Click anywhere in the black frequency / S-meter box — anywhere except on a digit, and except on the analog meter, whose click switches its style
2. A number pad opens; type the frequency on its keys or on the keyboard (`.` or `,` for the decimal point)
3. Press **kHz** or **MHz** to tune — the key you press sets the unit; on the keyboard, Enter means kHz and `M` means MHz

C clears the entry, ⌫ deletes the last digit, and Escape or a click outside closes the pad without tuning. In the extended mobile view, tapping the frequency field itself brings up the phone's own number keyboard.

Examples:

- `7100` + **kHz** → 7.100 MHz
- `14.2` + **MHz** → 14.200 MHz
- `145.5` + **MHz** → 145.500 MHz

A typed frequency brings its band. The receiver goes to the band you typed into the way its band button would: the waterfall zooms to the whole band, brightness and contrast take that band's settings, and the mode follows the band plan — but you stay exactly on the frequency you typed. Outside every defined band only the frequency changes. If the station has another receiver that covers the frequency — `145.5` **MHz** typed on an HF receiver, say — that receiver's page opens instead, already tuned and framed the same way (see *Choosing a receiver* below).

Method 3: Mouse on the Digits

1. Hover over a digit of the frequency display — it lights up
2. Scroll up or **left-click** to raise that digit by one; scroll down or **right-click** to lower it
3. The step is the digit's own place value — 1 kHz on the kHz digit, 100 Hz on the first decimal — and a digit that passes 9 carries into the one to its left
4. Anywhere else in the black frequency box — the band name, the mode line, the clock — the wheel tunes by the tuning step (**Shift** 1 kHz, **Alt** 10 kHz); only the analog S-meter is left out, so the wheel over it scrolls the page

Method 4: Step Buttons

1. Use the ▲ and ▼ buttons next to frequency
2. Step size varies by mode:
3. **AM/FM**: 1 kHz steps
4. **SSB**: 100 Hz steps
5. **CW**: 10 Hz steps

Method 5: Band Buttons

A band button frames its band — the waterfall zooms to the whole band and brightness and contrast take that band's settings — and tunes to the last frequency you used in that band, however you got there (a click on the waterfall, the digits, a bookmark, a chat link…). The first time, it goes to the band's default frequency. A second click on the band button that is already active (highlighted) forgets the remembered frequency and goes to the band's default. The memory is kept in your browser, so it survives a reload, and it is yours alone.

Scanner

The scanner walks the receiver across a range of channels and stops on the first one carrying a signal. It sits on the **Fine Tuning (kHz)** row, hard right:

```
7 152.0   ◂ ■ ▸ ⊘   30 dB · 4 ▾
```

Control	What it does
◂ ▸	Scan down / up. While parked on a signal, an arrow resumes the scan.
■	Stop.
⊘	Lock this channel out of the scan — for a birdie or a permanent carrier.
30 dB · 4 ▾	The threshold, with the level being measured right now beside it. Opens the settings.

The text to the left shows what the scanner is doing: `Scanner` when idle, the frequency it has reached while scanning, `◉ 3s` while counting down to resume, `◉ 30·22s` while the channel is still busy, `◉ hold` when it will stay put.

Settings

- **Range** — *Scan Band* sweeps the band the scan starts in; *Scan Visible* sweeps exactly what the waterfall shows, and follows it if you zoom or drag.
- **Scan** — *Every channel* tunes and listens to every channel in turn; *Skip empty* reads the spectrum and jumps straight to the signals.
- **Stop at** — how far above the band noise floor a channel must be to stop the scan, in dB. The floor is tracked continuously, so one setting works by day and by night. An empty channel does not read 0 dB — watch the live figure on the button for a moment and set the threshold above it.
- **Resume after** — how long a channel must stay quiet before the scan moves on. Pauses in speech do not restart it. *Hold* stays until you press a button.
- **Max stay** — moves on after this long even if the signal is still there, so a permanent carrier cannot hold the scan for ever.

The step follows the mode — 1 kHz in SSB, 0.1 kHz in CW, 5 kHz in AM, 9 or 10 kHz on medium wave and 9 kHz on long wave — and stops land on the channel grid. The scan stays inside its range and inside what the receiver can tune; at an edge it wraps round and carries on.

Settings and the locked-out channels are remembered in your browser.

Selecting Demodulation Mode

Choose the appropriate mode for the signal:

For Voice Communications:

- **AM**: Aviation, broadcast AM, some amateur
- **FM**: VHF/UHF repeaters, FM broadcast

- **USB**: HF ham radio (20m, 17m, 15m, 12m, 10m)
- **LSB**: HF ham radio (160m, 80m, 40m, 30m)

For Data/Digital:

- **USB**: Most digital modes (FT8, PSK31, RTTY)
- **LSB**: Some digital modes on lower HF bands

For Morse Code:

- **CW**: Telegraph/Morse code signals

Adjusting Volume

- **On-screen slider**: Drag the volume slider. 0 % is silence; below 50 % the level falls smoothly with fine steps, and from 50 % up each step is 0.4 dB. The page remembers where you left the slider, on the desktop page and on /mobile; on a first visit it starts where the sysop set it (50 % unless set otherwise).
- **Browser volume**: Use browser's media controls
- **System volume**: Adjust your computer's volume
- **Keyboard**: Use + and - keys (if supported)

Using the S-Meter

The S-meter shows signal strength:

- **S0-S3**: Very weak signal, difficult to copy
- **S4-S6**: Weak to fair signal
- **S7-S9**: Good to strong signal
- **S9+**: Extremely strong signal

Tip: For best audio, tune to signals showing S7 or higher.

Changing the meter face: on the analog (needle) meter, click the meter itself — or focus it and press Enter or Space — to cycle through three backgrounds: dark brushed metal, a light pale-grey face, and a warm vintage amber face. Your browser remembers the choice, so it is still there after a refresh or a restart. It is per browser and per address, so opening the receiver by hostname and by IP gives two separate settings, and a private window always starts from the site default.

Above 30 MHz the meter follows the VHF standard of IARU Region 1: S9 is −93 dBm there instead of −73 dBm on HF, still six dB per S-unit. The needle and the bar switch by themselves with the tuned frequency; the dBm and dBµV figures are always the measured level. Some stations also set their VHF/UHF receivers to rest at **0 on an empty channel**, like a VHF/UHF transceiver's meter: above 60 MHz the needle and the bar then come up only when a signal stands out of the noise. The dBm, dBµV, SNR and NF figures always show the real values.

Signal strength plot

Video under **Recording Options** records the spectrum and the waterfall together with the audio (**Area** chooses a part of them to record). A bar at the bottom of the picture shows the frequency, mode and time, and the tuned frequency is marked with the same yellow lines as the tuner on screen: dotted yellow lines at the frequency and at the passband edges, and the bracket across the top of the waterfall.

Under **Recording Options**, the **Plotter** button next to Audio, Video and Area draws the S-meter's reading as a running graph — handy for watching fading, comparing two antennas or stations, or seeing when a band opens. Each click goes **Off → Fast → Medium → Slow**: the plot shows the last 10, 30 or 60 seconds, the newest reading at the right, and your browser remembers the choice. It plots the same

dBm figure as the digital S-meter, 20 times a second; the left scale is in S-units (the VHF scale above 30 MHz) and the right one in dBm, and the range follows the signal by itself. The top line shows the reading now and the peak (**pk**) of the shown span, also drawn as a dashed red line; a yellow line with the new frequency marks every retune. In the top right corner ❙❙ pauses the plot (▶ carries on) and **PNG** saves it as a picture, `PhantomSDR_signal_<UTC>_<kHz>kHz.png`. Off, it costs nothing.

Choosing a receiver

Some stations run more than one receiver — for example one for HF and one for the 2 m band. They show a line *Receivers:* in the page header with one button per receiver; the one you are on is shown in yellow. Pressing another button opens that receiver's page, with its own waterfall, bands, chat and listener list. On the /mobile page the same buttons sit in the top bar. A station with a single receiver shows no such line.

Show All. A **Show All** checkbox follows the receiver buttons. Tick it and every other receiver of the station appears under your waterfall — its spectrum and waterfall, with its own markers and frequency scale and the newest line at the top. Hover over it to read a frequency and the nearest marker; click to open that receiver there, in the same tab: on a marker in the marker's mode, elsewhere in the mode the band plan gives. The checkbox is remembered in your browser and is off until you tick it, because each strip is one more waterfall stream (about 10 KB/s).

The view you left is kept. When you leave a receiver's page, its zoom, brightness, contrast, mode and frequency are remembered. Its strip on the other page then shows that same part of the band, with a small green ▼ where you were tuned; the button in the strip's header shows that range and **Full span** — press it for the receiver's whole range, and **Last view** to come back. Click a frequency inside that view and the receiver reopens on it, with the same zoom, levels and mode, instead of its whole range.

Choosing a band on the strip. When the other receiver covers more than one band — the HF receiver seen from the 2 m page — the strip's header has a dropdown in place of that button: **Last view**, **Full span** and each band of that receiver, the same bands its band buttons show. Pick a band and the strip shows it as that band button does, edge to edge with the band's brightness and contrast; a click on the strip then opens that receiver framed on the band, tuned where you clicked. Your choice is remembered in your browser, separately for each receiver. A receiver with a single band, such as 2 m, keeps the button.

The right receiver by itself. A frequency that this receiver does not cover but another of the station's receivers does — typed, from a link, a bookmark, a DX spot, a chat message or another listener's label — opens that receiver's page, tuned there. The mouse wheel and the digit steps never switch receivers, and neither does a transceiver connected over CAT.

EiBi schedule

The **EiBi** button in the Frequency Search row, after **SW**, opens a window that searches the EiBi schedule: the worldwide list of long wave, medium wave and shortwave broadcasts and utility stations kept by Eike Bierwirth (eibispace.de), the database shortwave.live is built on. The station keeps it up to date by itself.

Searching. Set any of the filters; the list follows as you change them.

- **Station**: type part of a name; names are suggested as you type.
- **Language**, **Target area**, **Station country**, **Transmitter in**: pick from the lists.
- **Band**: long wave, medium wave, all of shortwave, one broadcast band from 120 m to 11 m, or VLF/LF.
- **Frequency (kHz, ±2)**: everything within 2 kHz of it; **dial** fills in the frequency you are tuned to.
- **Time (UTC)**: *Any time*, *On air now*, or *On air at* a time you type (**now** fills in the current UTC time).
- **Today only** hides what is not scheduled for today's weekday or date. **Receivable here** hides frequencies this receiver does not cover.

Stations on the air right now are shown in green. Click a column header to sort by it, and hover over a row to see all its details: transmitter site, the dates it is valid, when it was last logged. **Reset** clears the filters. The window remembers your filters in your browser.

Tuning. Click a row. The receiver tunes the station in the mode of the **Mode** column: AM for broadcasting, USB or LSB for utility voice (Volmet, coast stations, navies), CW for Morse stations. A mode in amber (FAX, HFDL, DRM, NAVTEX, RTTY-WX and CW) also starts its decoder, the same way a decoder marker on the waterfall does; a CW row opens the CW decoder. The waterfall zooms to 100 kHz around the station, and a red marker with the station's name in yellow shows what you are listening to; it goes away when you tune more than 200 Hz from it. A station outside this receiver's range opens the receiver that covers it, when the station has one. The row you clicked turns red in the list for as long as you stay on that station.

Show on waterfall puts the results on the waterfall as markers with a thin green frame and green text (when zoomed in, like the other markers). They stay after the window closes, until you untick the box or reload the page. Clicking one tunes it like a row, and its row turns red in the list.

Hide HFCC markers hides the yellow markers that come from the HFCC broadcast schedule, so that only the EiBi ones show. Your own markers and the decoder markers stay. The setting is kept after the window closes and remembered in your browser; untick it to bring them back.

The window opens in the middle of the screen and can be dragged by its title. **Esc**, ×, the **EiBi** button or a click anywhere outside it closes it.

> The mode of each row is worked out from what EiBi writes about the station, so a few may be wrong. The yellow markers on the waterfall come from a separate list (HFCC); this window can hide them but never changes them.

Advanced Features

Auto Gain Control (AGC)

AGC keeps the audio at a steady level. The four buttons work like the AGC switch on a transceiver: all four give the same loudness, and they differ only in how quickly the gain comes back up when a signal gets weaker or stops.

- **Auto**: the receiver's own AGC, tuned separately for SSB, AM and C-QUAM. This is the default.
- **Fast**: no hang; the gain recovers in about 0.3 s. The noise comes up between words, and rapid fading is followed closely. Good for CW and for picking a weak station out between strong ones.
- **Medium**: holds for 0.25 s, then recovers in about 1.2 s. A good everyday setting for SSB.
- **Slow**: holds for 1 s, then recovers in about 3 s. The background stays down between words and between overs; a weak station right after a strong one sounds quieter for a few seconds.

While a decoder is open the AGC is held on Slow, whichever button is selected: the decoder hears the same audio as you and needs a steady gain. A small **Slow (decoder)** note appears next to the AGC heading, and your own choice returns as soon as the last decoder is closed.

Recommendation: Start with "Fast" for AM, "Slow" for SSB.

The four noise controls

NR, NB, NS and AN are separate on/off buttons that each attack a different kind of noise. They are independent — switching one on does not switch on another — and they can be combined freely.

None of them reach the decoders: FT8, CW, WSPR, SSTV, FAX, NAVTEX, RTTY/PSK31/Olivia/Packet/APRS and the QRSS grabber all read the audio *before* these filters, so you can set them purely by ear without affecting what decodes. See the Decoder manual.

Noise Reduction (NR)

Spectral noise reduction. It estimates the noise level in each part of the audio spectrum and turns those parts down, leaving what stands above the noise alone.

Use when: You hear steady hiss or white noise behind the signal.

Steady tones — a CW note, a carrier — are recognised as signal and protected, so NR does not eat a CW signal the way a naive filter would. Typical effect is a 10–12 dB drop in the noise between speech. The speech itself loses well under 1 dB on strong signals and a few dB on very weak ones, which still come through more clearly than without NR.

Noise Blanker (NB)

Removes impulse noise: clicks, pops, static crashes, ignition and power-line noise. It watches the audio envelope and silences only the samples that spike far above it — about a millisecond per crash, ramped in and out so the blanking does not itself click.

Use when: You hear clicks or crashes from power lines, motors, thunderstorms or vehicle ignition.

The 50 Hz and 60 Hz mains hum notches follow this button.

Background Noise Suppression (NS)

Measures the band's own noise floor over several seconds and applies a fixed cut to whatever sits at that floor, deeper at high audio frequencies than low. Where NR reacts moment to moment, NS is a slow, steady hand: it lowers the band hiss without changing how the signal itself sounds.

Use when: The band is quiet but hissy and you want the noise pushed down without the "underwater" quality of aggressive NR.

Active in USB, LSB and AM only — CW, FM and digital modes are left untouched. It re-measures the floor whenever you retune or change mode, and settles within a few seconds.

Auto Notch (AN)

Automatically finds and removes steady interfering tones — heterodynes, carriers, birdies — without you having to place a notch by hand. It adapts continuously, so several tones can be removed at once and a drifting one stays notched.

Use when: You hear a whistle or tone on top of the signal you want.

Bypassed in CW, where the wanted signal *is* a steady tone.

Delay note: NR and NS each add about 40 ms of audio delay while switched on (both together, about 80 ms). NB and AN add none. This affects listening only, never the decoders.

Auto Squelch (SQL)

Mutes audio when no signal is present:

- **Off**: Always playing (hear static)
- **Auto**: Automatically sets threshold
- **Manual**: Adjust threshold manually

Use when: Monitoring a frequency for activity.

While SQ is off, the threshold number beside the slider is shown in grey.

AI Noise Reduction (AI)

The **AI** button, with a strength slider beside it, sits in the Audio & Buffer panel right under SQ. On the / mobile page it is in the Audio tab, under Squelch. It removes band noise from speech with RNNoise, a small neural network trained on voice.

Everything happens in your own browser: nothing is sent to an outside server, there is no account, and the receiver does no extra work.

- **On/off**: Click **AI**; the button turns blue. The first click downloads the module (about 1.3 MB), and the button pulses while it loads.
- **Strength**: The slider sets how strongly it works while someone is talking — lower values keep more of the original sound. The top of the scale is deliberately softer than a full cut, so that speech is not damaged even at 100%, which still cleans more than 50% does. Between words it works harder on its own, so the hiss drops further there. The default is 50%. You can set it while AI is off.
- **Status label**: The AI label under the frequency display lights cyan while AI is working, and shows half-dimmed when AI is on but the current mode is not a voice mode.

Use when: Listening to SSB or AM voice on a noisy band. The hiss between words typically drops by about 10 dB at the default 50%, and more at 100%, up to about 20 dB, while the speech keeps its level.

Voice modes only — USB, LSB, AM and SAM. In CW, FM, the data modes and C-QUAM the audio passes through untouched, because the network treats a CW tone or music as noise. Like the four noise controls, it never reaches the decoders. On weak stations (below about +6 dB SNR) it measures the signal-to-noise ratio itself and smooths its decisions more, so speech does not take on the bubbling, "watery" sound this kind of filter can give; strong stations are processed exactly as before. If a very weak signal still sounds processed, lower the slider. It adds about 50 ms of audio delay.

Audio Buffer and Delay

Audio reaches the browser in small packets, 35 ms each on a typical receiver, and the browser keeps a little of it in hand so that a packet arriving late does not cause a gap. That reserve is the **buffer**, and it is most of the delay between the receiver and your loudspeaker.

- **Desktop**: in the Audio & Buffer panel, the button with the network icon moves the buffer one step up per click (it turns green above the lowest step), and the slider beside it picks a step from 1 to 6 directly. The lowest step suits a wired connection; phones start on step 2.
- **/mobile**: the **BUF** button and the **Buffer** slider in the Audio tab do the same.

The buffer adjusts itself. The page measures how irregularly your connection delivers the audio and keeps only as much in hand as the last minute showed was needed, plus the safety margin of the step you chose: 20 ms on step 1, 50 ms on step 2, more on the higher steps. After a moment of network trouble the extra delay drains away again within a minute or two, and a small difference between your sound card's clock and the receiver's no longer leads to a break every few minutes. The adjustments are a few milliseconds at a time, made where the sound repeats itself, and cannot be heard. The decoders and your recordings are never affected: they receive every sample.

If the audio breaks up, move the buffer one step higher. On a phone over home Wi-Fi the delay from the receiver to the loudspeaker is typically about 100–150 ms; the network between you and the receiver adds to it.

Your name or callsign

Next to **Open Additional Info** there is a box for your name or callsign: up to 10 characters and no spaces, as on a WebSDR. Type it and press **Save** (or Enter); **Edit** changes it later. Your browser remembers it, also when you switch to the station's other receivers. Leave it blank and you stay

anonymous: *anonymous* plus a number in the chat, and only your location everywhere else. On a phone the same box sits above the frequency display.

Your name appears in the **Users** window, on your chat lines and on the label the other listeners see under their waterfall — your name followed by your location, for example `SV1BTL-Athens`. Your own position is the yellow ★ with your name beside it. Hover over another listener's label to see the frequency and mode they are tuned to; click it to tune there.

As on a WebSDR, no listener's IP address is shown to the others — neither in the Users window nor in the User Statistics, which count visitors by an anonymous code.

The Users window also shows which decoder each listener is running — FT8, CW, DRM, HFDL and so on — as a small green tag after the mode, and the User Statistics have a **Decoder Popularity** panel: how many sessions ran each decoder. In the Users window opened from the receiver page, **Tune** beside a listener puts you on what they hear: the same frequency and mode, their filter bandwidth and their decoder — started for you, or yours stopped when they run none. The band is then framed as **To Band** does: the waterfall shows the whole band they are in, with that band's brightness, contrast and tuning step.

Band propagation

Under **Open Additional Info**, next to *Current band propagation and statistics:*, the **Propagation** button opens a page with the band conditions for this receiver's own location. It grades every band from 160 m to 6 m as Good, Fair, Poor or Closed for five distances (up to 300 km, 300–1000 km, 1000–2000 km, 2000–3000 km and DX beyond 3000 km). The grades come from the live measurements of the nearest ionosondes (foF2 and MUF), the position of the sun, and the solar and geomagnetic data (solar flux, Kp, X-rays, solar wind). The map is a globe centred on the station: drag to spin it, right-drag (or Ctrl + drag) to tilt it, and **2D / 3D** switches to a flat map. Its **band reach** rings show, for every open band, a dashed ring where its skip zone ends and a solid ring at its farthest reach; the legend under the map lists each band's span, and clicking a band shows its own open area, shaded by Good / Fair / Poor. The globe also shows the MUF and foF2 contours, day and night with the greyline, and every ionosonde reporting now, worldwide — bright for the ones used here. The layer switches in its corner turn each of these on or off. The rings follow the ionosphere direction by direction — where the path reflects, the sun there, the auroral oval — so a band usually reaches further one way than another; the table above is the view straight overhead. The globe needs a browser with WebGL2. The page refreshes itself every 10 minutes. It is an estimate for paths from this receiver, not a record of what is being heard.

Chat

When the operator has enabled the chat, it has its own panel on the desktop page and a **Chat** tab on / mobile. Type your message and send it: it goes out under the name in your name box (see above), or as *anonymous* plus a number.

To answer a particular message, click **↪ Reply** next to its time. A line above the input box shows who and what you are answering; type and send as usual, or press ✕ or Esc to cancel. The answer appears indented under the message it answers, with a short quote line. A reply to a reply joins the same thread. The server keeps the last 20 messages; when the original has already dropped out of that history, a reply shows only the name and time it answers.

A frequency in a message is a link: click it to tune there, in the mode given with it. The band is framed as **To Band** does — the waterfall shows the whole band, with that band's brightness and contrast — and the band button lights up.

Zoom Function

Magnifies the waterfall display:

The **Zoom** slider has 20 steps, shown as ×1 to ×20 (the step, not a magnification). ×1 shows the whole receiver; each step narrows the view around the dial by the same factor, so the zoom grows evenly, down to a few kHz at ×20. The button next to the slider goes one step further each click and returns to ×1 after ×20.

Use when: You need to see signals more clearly or tune precisely.

To Band shows the whole band of the frequency you are on, with 5 kHz to spare at each edge.

Auto Adj. (waterfall)

The **Auto Adj.** switch under Waterfall Controls sets the waterfall's brightness and contrast for you. It measures the noise floor across the visible span and turns it into an even, dark background — also where the noise differs from one part of the span to another — and stretches everything above it, so that weak signals stand out and the strongest reach the top colours. It works the same with every colour map, and only the waterfall changes: the spectrum trace keeps its true dB scale. While it is on, the Min and Max sliders are locked and show the measured noise floor and peak; switching it off returns them to the band's own settings.

3D spectrum

The **3D** button next to **Reverse** under Waterfall Controls turns the spectrum into a relief of the last few seconds: frequency across, time going back into the screen, signal strength as height. It uses the same colours, brightness (Min), contrast (Max) and Auto Adj. as the waterfall, and follows zoom and drag like the flat spectrum. Click on it to tune, drag to move along the band and turn the mouse wheel to zoom, exactly as on the flat spectrum. The relief narrows towards the back like a table seen at an angle, and the further back a row is, the lower its peaks. A click on an older row tunes to that row's frequency, and the frequency shown under the mouse then also gives the row's age (for example 7.074.000 · −6 s). The ⚙ button opens two sliders: **Angle** (how steeply the floor climbs towards the back) and **Depth** (how strongly the relief narrows and its rows get lower towards the back); **Reset** returns to the defaults, and your setting is remembered in your browser. The marks on the left show how many seconds back a row is, those on the right how many dB above the Min level a peak reaches (they are hidden while Auto Adj. is on). **× 2D**, or the 3D button again, returns to the flat spectrum, and your choice is remembered in your browser. The view needs WebGL2, which every current browser offers; without it the button does not appear. A video recording made while it is open records the 3D view in place of the flat spectrum. The 3D view is adapted from PhantomSDR-ORSAT by F1NSK.

Cursor line. Wherever the mouse is over the spectrum, the waterfall or the 3D view, a dashed yellow line marks that frequency on both the spectrum and the waterfall, and the spectrum shows it in MHz.

Audio analyser

The **Show** button in the Spectrogram section opens the audio analyser under the waterfall; **Hide** closes it. It shows the audio you hear, after demodulation, as a spectrum with a dB grid and a yellow peak-hold line. **Gain** raises or lowers the trace and **Color** picks its colours. The spectrum is drawn in relief, lit from the upper right. In the Rainbow colours each bar is coloured by how far it stands above the noise at its frequency — dark blue for the noise, also in speech pauses when the AGC lifts it, then blue, cyan, yellow, orange and red for ever stronger signals — while its height still reads on the dB scale. Like the main spectrum, the analyser and its reports are drawn on black, with grey grid lines.

- **Frequency axis.** It follows the mode: 0–4.5 kHz in SSB, 0–2 kHz in CW, 0–5.5 kHz in AM and FM. In LSB it is reversed (high tones on the left), so it reads in the same direction as the RF spectrum. A cyan line marks the dial frequency (in CW, the 600 Hz note), and under the mouse the analyser shows both the audio frequency and the RF frequency in kHz.
- **Bandwidth markers.** Coloured markers show the recommended widths for the mode — SSB standard 300–2700 Hz and ESSB 50–4000 Hz, CW 500 Hz and 250 Hz, AM broadcast and AM amateur

— and a dashed white box shows your current filter. The standard width is drawn as a solid cyan line, the alternative one (ESSB, CW 250 Hz, AM amateur) as a dashed light-blue line.

- **RF view in AM and FM.** Demodulated audio has no carrier, so in these modes the analyser shows the station as it is on the waterfall: the carrier in the centre, the sidebands either side. Its scale is in dB above the noise.
- **Occupied width.** The line under the spectrum gives the width the tuned station occupies (the −26 dB method, measured on the waterfall) with a verdict for SSB, AM and CW: green within the norm, amber wide (an ESSB station shows amber), red for likely splatter, which is also shaded on the spectrum. It needs about 2 s of measurements; static crashes are ignored.
- **No zoom needed.** While the analyser is open, the page keeps a second, hidden waterfall connection centred on the station, at the fine resolution the measurements need — so the RF view and every RF measurement work whatever the zoom of the waterfall. It closes with the analyser. It costs about 12–14 KB/s (some 45 MB an hour) of extra data while the analyser is open; on a metered connection, close the analyser when you do not need it.
- **Tuning from the analyser.** The dial frequency in the analyser's header tunes like the main dial: hover a digit and scroll, or left-click it to step up and right-click to step down; a click beside the digits opens the number pad. A click on the mode badge opens a menu of the modes.
- **Mute.** The Mute button in the analyser's header silences the receiver while the analysis goes on — for checking your own transmission without feeding it back into your microphone. Sound muted there comes back when the analyser closes.

SSB audio report. In USB and LSB a report panel opens under the analyser. Over a sliding 20 s window, and only while the station is talking, it judges the tonal balance — bass (100–300 Hz) and presence (1.5–2.7 kHz) against the mid band, compared with the reference you choose: **Standard SSB**, **DX / punch** or **ESSB / hi-fi** — then compression, mains hum (lines at 50/100/150 or 60/120/180 Hz), residual carrier and the opposite sideband (both measured on the waterfall) and the occupied width. Each row gets a green, amber or red dot; a row that cannot be measured says why, usually the receive filter (bass needs the filter's low edge at about 120 Hz or lower). Nothing above 2.7 kHz is judged, though the analyser still shows it. The average shape of the voice is drawn on the analyser as a blue curve. **Store A** keeps it as the dashed curve A, to compare after the other station changes a setting; **Clear A** removes it and **↺ Restart** starts the 20 s over. **Generate a listening report** downloads an HTML file to send to the other station, with the verdicts, a picture of the spectrum and the last 20 seconds of audio. The measurements are indicative: fading and the receiver's filter and AGC affect them, so prefer Slow AGC. The analyser and the report are adapted from PhantomSDR-ORSAT by F1NSK.

AM and FM report. In AM, C-QUAM and FM a report panel for these modes opens under the analyser instead. Here the receiver's audio says little about the transmitter — the AM carrier is removed from the audio and the AGC sets the level — so the report judges the station mostly on the RF spectrum, over a sliding 20 s window. In **AM**: the modulation depth (average and peak, from the power of the sidebands against the carrier), the balance of the two sidebands, the carrier's offset from the dial, the audio bandwidth, mains hum and the occupied width. In **FM**: the deviation (approximate, from the 99 % bandwidth by Carson's rule), the 99 % bandwidth with the channel it fits (12.5 or 25 kHz), the frequency offset, the audio bandwidth, the CTCSS tone if there is one, and mains hum. The waterfall averages each line over about 0.1 s, so the peak depth reads lower than on an oscilloscope; a carrier that reaches the top of the waterfall's scale is too strong to measure, and the report says so. The carrier offset includes the receiver's own frequency error. **↺ Restart** and **Generate a listening report** work as in the SSB report.

Transceiver control (CAT)

Your own transceiver and the receiver page can be kept on the same frequency: turn the rig's dial and the waterfall follows, or click a signal on the waterfall and the rig tunes to it. It only moves *your* listening session — nobody else on the receiver is affected.

With Desktop PhantomSDR+ (4.0 or later). The desktop application has a **Rig** menu. *Rig → Rig control...* opens a window where you choose your rig and how to reach it, and the menu itself switches the sync on and off. It syncs frequency, mode and filter width, in either direction or both, and can mute the receiver while you transmit. Besides PhantomSDR-Plus it drives **KiwiSDR, PA3FWM WebSDR and UberSDR** receivers the same way. It reaches the rig in one of four ways:

- **Built-in** — no other software: Icom (CI-V), Yaesu (new CAT and the FT-817/857/897), Kenwood, Elecraft, FlexRadio SmartSDR CAT, QRP Labs and other Kenwood-compatible rigs.
- **Hamlib** — every rig Hamlib knows, more than 300 of them, picked from a searchable list. The Windows installers include Hamlib; on Linux install `libhamlib-utils`.
- **rigctld on the network** — a `rigctld` that is already running.
- **flrig** — for a rig that flrig already shares with fldigi, WSJT-X or a logger.

The application, its installers and its full manual are at Desktop PhantomSDR+ (Dropbox).

With a web browser. The CATsync Tool for WebSDRs (Windows) couples a rig to the receiver page open in your browser. It syncs frequency and mode.

Filter width and mute on transmit work on KiwiSDR, WebSDR and UberSDR receivers, and on a PhantomSDR-Plus receiver running 4.1.0 or later. On an older PhantomSDR-Plus frequency and mode still sync; the filter does not.

The full manual — every setting, the supported rigs, how the two sides are kept from fighting, and troubleshooting — is **Rig Control**.

For developers. Every receiver page offers these functions on `window`, which is what both tools use:

Function	Does
`catsync_getFrequency()` / `catsync_setFrequency(hz)`	Tuned frequency, in Hz
`catsync_getMode()` / `catsync_setMode(mode)`	`USB`, `LSB`, `CW`, `CW-L`, `AM`, `FM`, `WBFM` …
`catsync_getBandwidth()` / `catsync_setBandwidth(hz)`	Whole passband width, in Hz. Set it after the mode: changing mode resets the passband
`catsync_getMute()` / `catsync_setMute(on)`	Mute, through the page's own mute button
`catsync_ready`	`true` once the functions are installed

The last three rows are new in 4.1.0; check that a function exists before calling it.

Demodulation Modes

AM (Amplitude Modulation)

Used for:

- Aviation communications
- AM broadcast radio
- Some amateur radio
- Maritime communications

Characteristics:

- Wide bandwidth (typically 10 kHz)
- Susceptible to noise

- Easy to tune (just click on signal)

Best Practices:

- Use Fast AGC
- Enable Noise Blanker if you hear clicks
- Tune to peak of signal on waterfall

QUAM (C-QUAM AM Stereo)

Used for:

- Medium-wave broadcasters transmitting AM stereo

Characteristics:

- 10 kHz wide, decoded as a stereo pair rather than mono AM
- The button label turns **green** on its own when the 25 Hz stereo pilot is present, so you can see which stations are actually in stereo before selecting it
- From AM, pressing the button again switches to synchronous AM on the carrier; the label turns **yellow** and reads `SAM`. A third press returns to plain AM
- While SAM runs, a line such as `SAM +3.2 Hz · 97%` appears under the status indicators (on the digital S-meter, under VFO | mode | bandwidth): how far the station's carrier is from the dial (+ = above it) and how well the detector is locked — yellow from 85 %, grey again below 60 %. Yellow only means a steady carrier is there; it does not tell the mode: an ordinary AM carrier on a frequency marked as DRM turns it yellow too
- C-QUAM audio is carried with Opus; every other mode uses FLAC

Best Practices:

- Look for the green label on strong medium-wave signals at night
- If stereo sounds unstable, `SAM` is the steadier choice on a weak carrier

FM (Frequency Modulation)

Used for:

- VHF/UHF amateur radio repeaters
- Public service (police, fire, EMS)
- Commercial two-way radio
- Some satellite communications

Characteristics:

- Narrow bandwidth (typically 12.5 or 25 kHz)
- Excellent noise immunity
- "Capture effect" (strongest signal wins)

Best Practices:

- Tune precisely to center of signal
- Use squelch to mute when idle
- Disable noise reduction (not needed)

USB (Upper Sideband)

Used for:

- HF amateur radio (above 10 MHz)
- Most HF digital modes
- Maritime communications (above 8 MHz)

Characteristics:

- Narrow bandwidth (typically 2.4 kHz)
- Efficient use of spectrum
- Requires precise tuning

Best Practices:

- Use Slow AGC
- Tune to lower edge of signal on waterfall
- Enable Noise Reduction if needed

LSB (Lower Sideband)

Used for:

- HF amateur radio (below 10 MHz)
- Some HF digital modes
- Maritime communications (below 8 MHz)

Characteristics:

- Same as USB but mirror image
- Convention: LSB on lower HF bands

Best Practices:

- Use Slow AGC
- Tune to upper edge of signal on waterfall
- Enable Noise Reduction if needed

CW (Continuous Wave / Morse Code)

Used for:

- Amateur radio telegraphy
- Navigational beacons
- Time signal stations

Characteristics:

- Very narrow bandwidth (100-500 Hz)
- High efficiency
- Requires learning Morse code to understand

Best Practices:

- Use narrow filter (400-500 Hz)
- Tune precisely to tone center
- Enable audio filter for better tone

CW-L (CW, Lower Sideband)

The same ±250 Hz Morse filter as **CW**, but the tone is taken from below the carrier instead of above it. Use it when a signal is easier to copy on the low side, or when an interfering carrier sits just above the one you want.

There is no CW-L button on the desktop page. Its mode row is `USB · LSB · CW · AM · QUAM · FM · IQ`. CW-L is offered on the simplified `/mobile` page, and on the desktop it can still be set by a link or a bookmark that names it.

Links written for KiwiSDR or UberSDR work too: `modulation=CWU` (or `CW-U`) opens in CW and `modulation=CWL` in CW-L.

WBFM (Wideband FM)

Used for:

- FM broadcast radio (88-108 MHz)
- Some satellite downlinks

Characteristics:

- Very wide bandwidth (200 kHz)
- Excellent audio quality
- High fidelity

Best Practices:

- Tune to exact center frequency
- No squelch needed for broadcast
- Enjoy high-quality audio!

IQ (raw I/Q)

Used for:

- Recording a piece of the band to play back later in HDSDR, SDR#, SDR++ or SDRuno
- Listening with another program — a KiwiSDR client can ask for it too

Characteristics:

- 12 kHz of the band centred on the dial, as I and Q — heard as stereo, I on the left and Q on the right, the way a KiwiSDR plays it. It sounds like hiss: it is the signal before any demodulation
- Sent lossless (FLAC), never with Opus, which would destroy the I/Q
- The level follows the band: noise stays quiet (about −45 dBFS) and a strong station is brought to about −20 dBFS. NR, NB, NS, AN, AI and the squelch are not applied, and choosing IQ switches off any decoder that is running
- **Audio** under *Recording Options* records the I/Q; when you stop, **Download IQ** saves it as a WAV file (16-bit stereo, 12 kHz) named for example `PhantomSDR_20261026_031500Z_6030000Hz_IQ.wav`. HDSDR, SDR# and SDRuno read the frequency and the time from the file, SDR++ from the name, so the recording opens already tuned
- The station decides: the button appears only where `[server] iq` is on (the default)

Best Practices:

- Put the station you want in the middle — the recording covers ±6 kHz around the dial (±12 or ±24 kHz in a wider IQ, below)

- To hear one station, a normal mode is simpler; IQ is for recording, for other software and for hearing several stations at once (below)

Wider IQ: 24 and 48 kHz

Where the station offers it, the IQ Demodulators panel has **Width: 12 kHz · 24 kHz · 48 kHz**. 24 kHz takes in DRM's wide modes or a station with both its neighbours, 48 kHz a whole group of broadcast stations or a CW/digital sub-band. The waterfall passband and the panel's picture follow the width, a recording is made at it (a WAV at 24 or 48 kHz), and your browser remembers the choice. A wider IQ costs the station more upload — about 0.7 Mbit/s per listener at 24 kHz and 1.4 Mbit/s at 48 kHz, against 0.35 at 12 kHz — so the station decides how many listeners may have it at once; when every place is taken you stay at 12 kHz (the panel says so) and the page asks again every 30 seconds. For Dream or another program on a virtual cable at 48 kHz, set the cable to 48 kHz as well. KiwiSDR clients always get 12 kHz.

For the sysop: two lines under `[server]` in the receiver's config turn it on, for any receiver; restart that receiver afterwards. The same settings are in the admin panel under **Receivers → ✎ CHANGE SETTINGS**, which also restarts the receiver.

```
iq_wide=48            # 12 (off, the default), 24 or 48 kHz
iq_wide_listeners=4   # listeners with a wide IQ at once, 0 = no limit
```

IQ demodulators

While the receiver is in IQ, an **IQ Demodulators** panel opens under the decoders. It runs up to six receivers on the I/Q in your browser — the receiver is not retuned and the station's computer does no extra work:

- The picture shows the whole IQ width (12, 24 or 48 kHz) around the dial with the real frequencies; each demodulator is a coloured band, its own dial a line inside it. Click the picture to move the selected demodulator there; the mouse wheel over it moves it by 10 Hz (Shift 100 Hz)
- Each row has the mode (USB, LSB, CW, AM, DSB, FM), the frequency with « ‹ › » buttons (100 and 10 Hz), a level bar, **M** to mute and ✕ to remove. The selected row also has **Width**, **Volume** and **Pan**
- **DSB** follows the carrier by itself (a carrier-tracking loop): put it within a few tens of Hz of the carrier and the audio stays clean and steady, for suppressed-carrier DSB as well as for AM, where it works as synchronous AM
- **+ Add** adds a demodulator at the dial, up to six. **Demodulators** / **Raw I/Q** chooses whether you hear them or the plain I/Q
- They are set relative to the dial, so tuning the receiver moves them all together; a station more than 6 kHz from the dial is not in the stream
- Example: a DX station working split — one LSB demodulator on the DX, panned left, and one on the callers a few kHz up, panned right
- Your demodulators are remembered in your browser. The noise controls (NR, NB, NS, AN, AI) do not act in IQ: narrow **Width** instead

---

Digital Decoders

PhantomSDR-Plus includes built-in decoders for digital modes. For a complete guide, see Decoders.

The quickest way in is the **Decoders** button row on the main panel, just below **Wheel Tuning Steps**: ten buttons — **FT8, FT4, FT2, JS8, CW, WSPR, FAX, SSTV, NAVTX, FLDIGI** — that each start their decoder, switch the master Decoder toggle ON and scroll the decoder's window into view in a single press. The button stays blue while the decoder runs; press it again to stop the decoder and close its

window. The **Decoder Options** dropdown still works exactly as before and stays in sync with the buttons.

RADE-v2U, **RADE-v1L** and **RADE-v1U** (RADE digital voice) have their own buttons next to the **Modes** heading and inside the **Modes** and **Bands** popup windows, and work the same press-on / press-off way. **RADE-v2U** starts RADE V2 (beta), always on USB; **RADE-v1L** and **RADE-v1U** start RADE v1.

The SSTV, HF FAX, NAVTEX, FSK/RTTY and CW decoders each run on their own background thread, so switching a decoder on or off never interrupts the audio, and the waterfall stays smooth while decoding. They are fed audio taken before the noise reduction and mute of your browser — you can mute the receiver and decoding continues unaffected. While a decoder is open the AGC is held on Slow, so the gain stays steady for it.

While a decoder runs it holds the mode and the passband. Normally the mode follows the band plan — tune into a segment marked LSB or AM and the receiver switches to it. A running decoder overrides that, keeping the mode and the narrow passband it needs even if you tune to another band. The band's own mode returns as soon as you switch the decoder off, and the mode buttons still work at any time if you want to take over. The CW decoder is the exception: it decodes in whatever mode you are listening in.

FT8, FT4 Decoder

What is FT8, FT4?

- Popular amateur radio digital mode
- Weak signal communication
- 15-second transmissions for FT8, 7.5-second transmissions for FT4

How to use:

1. Tune to FT8 or FT4 frequencies
2. Select USB mode
3. Enable the FT8 decoder — press the **FT8** button, or pick it from the menu
4. Watch decoded messages appear

Common FT8 Frequencies (USB):

- 160 meters: 1.840 MHz
- 80 meters: 3.573 MHz
- 60 meters: 5.357 MHz
- 40 meters: 7.074 MHz
- 30 meters: 10.136 MHz
- 20 meters: 14.074 MHz
- 17 meters: 18.100 MHz
- 15 meters: 21.074 MHz
- 12 meters: 24.915 MHz
- 10 meters: 28.074 MHz
- 6 meters: 50.313 MHz (50.323 MHz for DX)

Common FT4 Frequencies (USB):

- 80m: 3.575 MHz
- 40m: 7.0475 MHz
- 30m: 10.140 MHz

- 20m: 14.080 MHz
- 17m: 18.104 MHz
- 15m: 21.140 MHz
- 12m: 24.919 MHz
- 10m: 28.180 MHz
- 6m: 50.318 MHz

JS8 Decoder

What is JS8?

- FT8's weak-signal engine used for keyboard-to-keyboard conversation
- Free text instead of fixed exchanges, so long messages arrive over several cycles
- Five speeds; **Normal** (15 s) is the calling speed and carries almost all traffic

How to use:

1. Tune to a JS8 frequency
2. Select USB mode
3. Enable the JS8 decoder — press the **JS8** button, or pick it from the menu
4. Leave **Speed** on *Normal* and leave the sync offset on **Auto**

Reading the panel:

- Messages still arriving show at the top in green with a blinking cursor — a message can take a full minute to complete on Normal
- Completed messages list below as `Mode | Hz | dB | Message`, with **callsigns in green**
- A dimmed, italic row timed out before its last frame arrived: the text is real but may be cut short

Common JS8 Frequencies (USB):

- 160m: 1.842 MHz
- 80m: 3.578 MHz
- 40m: 7.078 MHz
- 30m: 10.130 MHz
- 20m: 14.078 MHz
- 17m: 18.104 MHz
- 15m: 21.078 MHz
- 12m: 24.922 MHz
- 10m: 28.078 MHz

JS8 is far quieter than FT8 — one transmission every few minutes is normal, and quiet spells are not a fault. See Decoders for the full guide.

CW Decoder

Just press the CW button and the decoded text will appear. Press the CW button once more to clear the window and restart the decoder.

WSPR

WSPR (Weak Signal Propagation Reporter, pronounced "whisper") is an ultra-weak-signal beacon mode that maps HF propagation paths worldwide. Each transmission takes approximately 110 seconds and fits inside a 200 Hz-wide slot. The decoder waits for a complete 2-minute UTC-aligned slot before decoding. Enable the decoder and select **WSPR** from the dropdown. Or simply press the **WSPR** button.

HF FAX / WEFAX

HF Radiofax (also known as WEFAX) is used by coast guard and meteorological services worldwide to broadcast weather maps, sea-state charts, and surface analyses over shortwave. The decoder reconstructs the image line by line as it is received.

NAVTEX

NAVTEX is the international maritime broadcast system for coastal safety information — navigational warnings, weather forecasts, and search-and-rescue notices.

FLDIGI — ALE 2G, RTTY, PSK31, Olivia, FST4W, MFSK, MSK144, DominoEX, THOR, THROB, Hellschreiber, MT63, Packet and APRS

One window for the narrow-band keyboard modes fldigi made popular, with fourteen variants: Amateur RTTY, **PSK31/63/125** (phase-shift keying, BPSK/QPSK, 31.25–125 baud), **Olivia**, **Contestia** and **MFSK16/32/64** (multi-tone with forward error correction), **DominoEX** and **THOR** (incremental frequency keying, THOR with error correction), **THROB**, **Hellschreiber** (painted as a picture), **MT63** (64 carriers), **Packet (AX.25)** and **APRS**. It opens from the **FLDIGI** button, or from **FLDIGI** in the decoder dropdown; fldigi itself is not needed. The panel adapts to the variant — the shift, baud and framing controls disappear for everything but Amateur RTTY, and the other modes add a Mode selector and, except Hellschreiber, a squelch slider. Three variants list what they decode one line at a time instead of text: **ALE 2G HAM**, just before Packet in the list (HF link establishment, MIL-STD-188-141, on the HFLINK amateur channels), and **FST4W** and **MSK144**, the WSJT-X modes for LF/MF beacons and for meteor scatter. **PSK Browser**, the **Browser — all signals** knob under the PSK31 **Mode** list, decodes every BPSK31 signal in the passband at once, one line per signal; click a line to follow that one. See the Decoder manual. **Browser — all signals** under the Olivia **Mode** list does the same for Olivia and Contestia, for the configuration chosen there. Maritime FSK (SITOR), Weather RTTY and the government, military and utility ALE networks (**ALE 2G**) are in the **SERVICES** window, with the same decoders.

CW Browser: switch on the **Browser — all signals** knob in the CW window to decode every CW signal in 200–3000 Hz at once, one line each, with the station's callsign and CQ calls marked; the receiver goes to USB 200–3000 Hz while it runs.

Amateur RTTY and Weather RTTY have a **Squelch (SNR)** slider that stops noise from printing as random letters, and every variant except Olivia shows a real SNR on the same 3 kHz scale. Two things to know: PSK31 corrects its own tuning error over about ±25 Hz, so you only need to get close; Olivia needs its **Mode** (tones / bandwidth) set to exactly match the transmission, sends no preamble, and therefore takes a few seconds to synchronise before any text appears. The panel opens on Olivia **8 / 250**. For the other fldigi modes, put the signal on the centre frequency or press ⟳ **Auto-tune Center** and pick the matching **Mode**: THROB needs tuning within ±3 Hz, DominoEX, THOR and MT63 forgive much more, and MFSK, THOR and MT63 print a few seconds behind the signal. Hellschreiber shows a picture, not text.

Packet prints every AX.25 frame it hears, like a TNC monitor; **APRS** turns APRS packets into plain text — position with locator, speed, altitude, weather, messages, objects and status. Choose the **Speed**: **1200 Bd** for VHF/UHF, which switches the receiver to **FM** on the channel (144.800 MHz in Europe is in the frequency list), or **300 Bd** for HF, in USB. Only packets with a correct checksum are printed, so nothing

appears on a quiet channel. Under the text, a map shows every station heard with a position, with tracks for moving ones and your receiver in amber; it can be minimised, and clicking a decoded line zooms to that station. See the Decoder manual.

QRSS Grabber

QRSS is CW sent so slowly that a single dot lasts seconds, and it is watched rather than heard — the grabber paints the trace on its own display. It is not in the Decoder dropdown; it has its own **QRSS** section and can run at the same time as a decoder.

Press **Show**, then pick a window from the **Band** list and press **Tune**: the receiver goes to that frequency in **CW** with a passband sized for the mode, and the trace lands on the centre line of the display. 30 m (10,140.00 kHz) is the busiest window. Set **Speed** to match the beacon — **QRSS 10** if you do not know — and be patient: a callsign can take ten minutes to cross the screen. Full details in Decoders.

SSTV

Slow-Scan Television sends still pictures over a normal SSB channel, one line at a time. Enable the decoder, select **SSTV**, and tune to an SSTV frequency — 14.230 MHz is the main international calling frequency. Leave **Mode** on **Auto**: the decoder reads the transmission's VIS header, and falls back to identifying the mode from the sync timing if you tuned in after the header. Martin, Scottie and Robot modes are supported, and the picture builds line by line as it arrives. Or simply press the **SSTV** button.

Automatic spot reporting and server graphs

FT8, FT4 and WSPR decodes can also be uploaded automatically by the server itself — FT8/FT4 to PSK Reporter, WSPR to WSPRnet — by an autorun daemon the sysop starts from the admin panel. It is independent of the decoders in your browser: it keeps running whether or not anyone is listening, and nothing you decode in the browser is reported.

The sysop follows it with two counters, which are easy to mistake for each other: the per-decoder tiles count the spots uploaded since the daemon last started, while the number beside each band/mode checkbox is that slot's all-time total and survives restarts. The same panel has a **Graphs** page plotting CPU frequency, CPU load, CPU temperature and users online over the last 15 minutes to 24 hours. Both are described in the Admin Panel guide.

Keyboard Shortcuts

Keyboard shortcuts for faster operation.

Frequency Control

Arrow keys tune from anywhere on the page:

- **Arrow Up / Down**: ±10 Hz; hold **Shift** for ±1 kHz, **Ctrl** (or **Cmd**) for ±100 kHz, **Shift + Ctrl** for ±1 MHz
- **Arrow Left / Right**: ±100 Hz; hold **Shift** for ±10 kHz, **Ctrl** (or **Cmd**) for ±500 kHz, **Shift + Ctrl** for ±10 MHz
- **Right Ctrl**: drops the fraction, tuning to the whole kHz at or below the current frequency

In the chat box, Left / Right move the cursor instead.

Mouse on the frequency display: hover a digit, then scroll up or left-click to raise it and scroll down or right-click to lower it, each time by the digit's own place value (nine digits, 100 MHz down to 1 Hz). Between the digits the wheel steps by the band's tuning step (1 kHz unless the band plan sets another); hold **Shift** for 1 kHz, **Alt** for 10 kHz. A click anywhere else in the black frequency / S-meter box opens the number pad (Method 2 under *Tuning to a Frequency*).

Mouse wheel over the waterfall: zooms. Hold **Ctrl** (or **Cmd**) or **Shift** to tune instead of zooming, and **Shift + Ctrl** together to snap to the nearest whole kHz.

> There are no Page Up / Page Down shortcuts.

Bookmarks

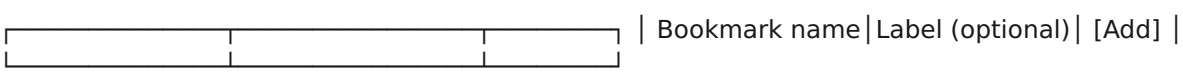
Save your favorite frequencies for quick access. You can also export the list of bookmarks and save it locally, then import the list into any other PhantomSDR.

When bookmarks and markers overlap:

Bookmarks (thin cyan frame, yellow text) appear on top
Yellow markers appear underneath
    Bookmark clicks take priority

Adding a Bookmark

1. Tune to desired frequency
2. Click "Bookmarks" button
3. Click "Add Bookmark"
4. Enter description
5. Click "Save"



How to Use:

Add bookmark:

- Name: "Local News Station"
- Label: "NEWS"
- Click Add

View on waterfall:

- Zoom in until markers appear
- See marine blue box with "NEWS" in bold yellow

Click bookmark:

- Tunes to frequency
- Sets demodulation mode
- Works exactly like clicking a marker

Managing Bookmarks

- **Edit**: Click pencil icon next to bookmark

- **Delete**: Click trash icon next to bookmark
- **Export**: Download bookmarks as JSON file
- **Import**: Upload bookmarks from JSON file

Sharing Bookmarks

1. Click "Export Bookmarks"
2. Share the JSON file with others
3. Recipients click "Import Bookmarks"
4. Select your file

Mobile Usage

PhantomSDR-Plus works great on mobile devices!

Two mobile views

There are two ways to use the receiver on a phone:

- `http://your_server:PORT/mobile` — the simplified page. No waterfall, so it uses roughly half the data. Frequency entry, tuning steps, modes, S-meter, bands, bookmarks, users and chat.
- **The extended view** — the main interface's own phone layout, with the waterfall and the full set of controls.

Switch between them with the buttons at the bottom of `/mobile` (**Mobile extended view**, **Full desktop view**) and the **Simplified mobile** button in the extended layout.

Your frequency follows you. Switching view keeps you on the same signal — the frequency and mode travel in the link, so you no longer land back on the receiver's default frequency. The address bar tracks your tuning as well, which means reloading the page, bookmarking it, or sending the link to someone else all return to that exact frequency.

The mode follows the band plan on both pages. Tune into a segment marked AM, LSB, USB or CW — by typing a frequency, stepping to it or pressing a band button — and the receiver switches to that mode, on the simplified page as well as the full interface. A mode you pick by hand stays put while you move around inside the same segment, and outside the defined bands your mode is left alone. While RADE (RADEL/RADEU) is running it keeps the receiver, so tuning does not interrupt it.

When you switch between the two views your current mode travels with you, but if the view you are switching to has no equivalent for it, the mode for that frequency is taken from the band plan — a broadcast frequency arrives in AM, 40 m in LSB, a CW segment in CW. `SAM` on the simplified page becomes AM with the synchronous detector in the extended view, and back again. `RADEL` and `RADEU` exist only on the simplified page, so switching away from one of those keeps your frequency and lets the band plan pick the mode.

A frequency outside the receiver's coverage is pulled back to the nearest edge, so an old link can never strand you off-band.

Mobile-Specific Features

- **Touch-friendly controls**: Large buttons and sliders
- **Swipe to tune**: Swipe left/right on waterfall
- **Pinch to zoom**: Pinch waterfall to zoom in/out
- **Landscape mode**: Rotate for better view

- **Spectrum and waterfall sized for a phone**: in the extended view both are taller than the desktop proportions would make them

Mobile Tips

1. **Use WiFi**: Streaming audio uses data
2. **Landscape orientation**: Better waterfall view
3. **Headphones**: Better audio quality
4. **Bookmark favorites**: Easier to revisit stations
5. **Close other apps**: Ensure smooth performance

Mobile Browser Recommendations

- **Android**: Chrome or Samsung Internet
- **iOS**: Mozilla
- **Both**: Ensure browser is up to date

Tips and Best Practices

For Best Reception

1. **Choose strong signals**: Look for orange/red on waterfall
2. **Tune precisely**: Click directly on signal center
3. **Select correct mode**: Match the signal type
4. **Adjust AGC**: Fast for AM, Slow for SSB
5. **Use NR/NB**: Help with noisy conditions

Finding Activity

1. **Watch the waterfall**: Colors show signal strength
2. **Listen on popular frequencies**:
3. 40m: 7.100-7.300 MHz (LSB)
4. 20m: 14.200-14.350 MHz (USB)
5. 2m: 145.200-145.600 MHz (FM)
6. **Check band plan overlay**: Shows frequency allocations
7. **Use bookmarks**: Quick access to active frequencies

Understanding Band Conditions

Daytime HF (High Frequency):

- Higher bands work better (20m, 15m, 10m)
- Long-distance (DX) communication possible
- Broadcast stations audible

Nighttime HF:

- Lower bands work better (80m, 40m)
- Different propagation patterns
- Different stations audible

VHF/UHF:

- Mostly line-of-sight
- Local communications
- More consistent conditions

Etiquette

1. **Don't tie up the receiver**: Others want to listen too
2. **Use chat respectfully**: Be courteous to other users
3. **Report problems**: Help the operator maintain the station
4. **Don't ask for tech support**: This is a listening platform. Send a message to the Sysop for help.

Troubleshooting

No Audio

Possible causes:

1. Browser muted → Check browser volume controls
2. System muted → Check computer volume
3. Weak signal → Tune to stronger signal (S7+)
4. Wrong mode → Try different demodulation modes

Solutions:

1. Click a strong signal (orange/red on waterfall)
2. Check browser isn't muted (look for mute icon in tab)
3. Try a different frequency
4. Reload the page (F5)

Distorted Audio

Possible causes:

1. Overdriven signal → Signal too strong
2. Wrong mode → AM signal on SSB mode, etc.
3. Interference → Adjacent signals bleeding over

Solutions:

1. Reduce volume
2. Try different demodulation mode
3. Use narrower filter bandwidth
4. Tune away from interfering signals

Audio Breaks Up or Lags Behind

Possible causes:

1. Irregular connection → packets arrive late and the buffer runs dry
2. Busy phone or computer → the browser cannot keep up

Solutions:

1. Move the buffer one step higher (see *Audio Buffer and Delay*)
2. Prefer Wi-Fi or a cable to mobile data
3. Close other tabs and apps

For a closer look, open the page with `?diag=1` added to its address (for example `http://your_server:PORT/mobile?diag=1`). The /mobile page then shows the delay, how irregular the connection is and how the buffer is doing under its controls; the desktop page writes the same line to the browser console (F12) every ten seconds. Useful when reporting a problem to the station operator.

Waterfall Not Updating

Possible causes:

1. Network issue → Slow or interrupted connection
2. Browser performance → Too many tabs open
3. Server overload → Too many users

Solutions:

1. Check internet connection
2. Close unnecessary browser tabs
3. Reload page (F5)
4. Try again later when fewer users online

Can't Tune to Frequency

Possible causes:

1. Frequency out of range → SDR doesn't cover that frequency
2. Typing wrong format → Use correct format (e.g., "14200" not "14.200.000")

Solutions:

1. Check SDR's frequency coverage (shown on page)
2. Use frequency format examples provided
3. Click on waterfall instead

Stuttering/Choppy Audio

Possible causes:

1. Slow internet connection
2. High server load
3. Browser performance issues

Solutions:

1. Close other applications using bandwidth
2. Try again during off-peak hours
3. Close unnecessary browser tabs
4. Use wired connection instead of WiFi

Frequently Asked Questions

General Questions

Q: Do I need special equipment to use WebSDR? A: No! Just a computer or mobile device with internet access.

Q: Is WebSDR free to use? A: Yes, most WebSDRs are free. They're operated by volunteers.

Q: Can I transmit with WebSDR? A: No, WebSDR is receive-only. You cannot transmit.

Q: What frequencies can I listen to? A: Depends on the WebSDR's configuration. Check the station info.

Q: Can I record audio? A: Some browsers allow recording. Check your browser's features.

Technical Questions

Q: What sample rate does the SDR use? A: Varies by station. Check the station information page.

Q: What's the latency? A: Usually a few tenths of a second between the radio signal and your speakers: the receiver's processing, the network, and the browser's playback buffer, which adjusts itself to your connection (about 100–150 ms on a phone over home Wi-Fi). See *Audio Buffer and Delay*.

Q: Can I use multiple instances? A: Usually yes, but it may strain the server. Be considerate.

Q: Does it work offline? A: No, WebSDR requires internet connection.

Q: What browsers are supported? A: Chrome, Firefox, Edge, Safari (all recent versions)

Usage Questions

Q: How many people can listen at once? A: Depends on server capacity. Often 50-200+ users.

Q: Can I see what others are listening to? A: If enabled, yes. Look for "other users" indicators.

Q: Can I chat with other listeners? A: If enabled by the operator. Look for the chat box; **↪ Reply** answers a particular message (see *Chat* above).

Q: Why do some frequencies show nothing? A: No signals on that frequency at the moment. Try others!

Q: What are the colored bands on the waterfall? A: Band plan overlay showing frequency allocations.

Resources

Learning More About Radio

- **Band Plans**: Search "amateur radio band plan" + your region
- **Propagation**: Learn about HF radio wave propagation
- **Digital Modes**: Research FT8, PSK31, RTTY
- **Ham Radio**: Consider getting an amateur radio license!

Finding More WebSDRs

- **WebSDR Directory**: http://sdr-list.xyz
- **WebSDR.org**: http://websdr.org
- **KiwiSDR**: http://kiwisdr.com/public/

Getting Help

1. **Station operator**: Check contact info on page
2. **User chat**: Ask other listeners (if available)
3. **Online forums**: Search for WebSDR communities
4. **Documentation**: Refer to this guide!

Appendix: Common Frequencies

HF Amateur Radio Bands

Band	Frequency Range	Mode	Activity
160m	1.800-2.000 MHz	LSB	Night/Local
80m	3.500-4.000 MHz	LSB	Night/Regional
40m	7.000-7.300 MHz	LSB	Day/Night/DX
30m	10.100-10.150 MHz	USB	Data/CW only
20m	14.000-14.350 MHz	USB	Daytime/DX
17m	18.068-18.168 MHz	USB	Daytime/DX
15m	21.000-21.450 MHz	USB	Daytime/DX
12m	24.890-24.990 MHz	USB	Daytime/DX
10m	28.000-29.700 MHz	USB	Sporadic/DX

VHF/UHF Amateur Bands

Band	Frequency Range	Mode	Activity
6m	50.000-54.000 MHz	USB/FM	Sporadic
2m	144.000-148.000 MHz	FM	Very Active
70cm	420.000-450.000 MHz	FM	Active

Broadcast Bands

Service	Frequency Range	Mode
AM Radio	530-1710 kHz	AM
Shortwave	2.3-26.1 MHz	AM
FM Radio	88-108 MHz	WBFM

Aviation

Service	Frequency Range	Mode
Air Traffic Control	118-137 MHz	AM
ACARS (data)	130-136 MHz	Data

Maritime

Service	Frequency Range	Mode
Marine VHF	156-162 MHz	FM
Marine HF	2-22 MHz	USB

Glossary

AGC: Automatic Gain Control - adjusts audio levels automatically

AM: Amplitude Modulation - voice mode used for aviation and broadcast

Bandwidth: The frequency range of a signal

CW: Continuous Wave - Morse code signals

DX: Long distance communication

FFT: Fast Fourier Transform - converts time to frequency domain

FLDIGI: the decoder window for ALE 2G, RTTY, PSK31, Olivia, FST4W, MFSK, MSK144, DominoEX, THOR, THROB, Hellschreiber, MT63, Packet and APRS, named after the fldigi program that made these modes popular

FM: Frequency Modulation - voice mode for VHF/UHF

HF: High Frequency (3-30 MHz) - long distance bands

kHz: Kilohertz (1,000 Hz)

LSB: Lower Sideband - voice mode for lower HF bands

MHz: Megahertz (1,000,000 Hz)

NB: Noise Blanker - removes impulse noise

NR: Noise Reduction - reduces background noise

PSK: Phase Shift Keying - digital mode

RTTY: Radio Teletype - digital text mode

S-meter: Signal strength meter

SDR: Software Defined Radio

SQL: Squelch - mutes audio when no signal present

SSB: Single Sideband (USB or LSB)

USB: Upper Sideband - voice mode for higher HF bands

VHF: Very High Frequency (30-300 MHz) - line-of-sight

UHF: Ultra High Frequency (300-3000 MHz) - line-of-sight

Waterfall: Visual display of radio spectrum over time

Enjoy exploring the radio spectrum with PhantomSDR-Plus!

73 (Best regards) de SV1BTL, 9A7AOF, F1NSK & SV2AMK

For installation instructions, see INSTALLATION.md. For technical details, see PROJECT_STRUCTURE.md.

CHAPTER 14

KiwiSDR Client Emulation

Aether_config.md

KiwiSDR Client Emulation

Connecting AetherSDR, kiwiclient and other KiwiSDR software to PhantomSDR-Plus

Since v4.1.0 PhantomSDR-Plus can also answer the **KiwiSDR protocol**, so software written for a KiwiSDR — **AetherSDR**, `kiwiclient` and the rest — connects to your receiver directly. It is a bridge inside the same server process, on the same host and the same port you already publish: no second daemon, no second port, no proxy.

It is off until you enable it. Nothing about the bridge runs — not a socket, not a check — while `[kiwi_emulation]` is missing or `false`, which is the state a fresh tree is in.

Table of Contents

1. What a Kiwi client gets
2. Installing the bridge
3. Switching it on
4. Configuration reference
5. Connecting a client
6. Audio level
7. The S-meter
8. Waterfall and spectrum
9. If something looks wrong
10. How far this has been verified

1. What a Kiwi client gets

Audio	`ws://<host>:<port>/kiwi/<id>/SND` (also `ws://<host>:<port>/<id>/SND`) — demodulated audio in the Kiwi's own SND framing
Waterfall	`ws://<host>:<port>/kiwi/<id>/W/F` (also `ws://<host>:<port>/<id>/W/F`) — the spectrum, in Kiwi W/F frames
Retuning	real, not cosmetic: `SET mod=…` from the client changes frequency, sideband and passband on the PhantomSDR side, exactly as a browser listener would
S-meter	driven from the demodulator's own per-block power, on the same scale as the web S-meter
IQ	`SET mod=iq` gives 12 kHz of I/Q in the Kiwi's stereo frames, as `kiwirecorder.py -m iq` expects. No GPS timestamps, so not for TDoA. Only while `[server] iq` is on (the default)

Retuning is supported for a **real** input (`signal = "real"` in your `.toml`). For an IQ input the bin-to-frequency relation differs and is not implemented, so an IQ receiver will serve audio and waterfall but will not follow a Kiwi client's tuning.

2. Installing the bridge

The bridge is a set of patches to the backend sources plus one new header, `src/kiwi_bridge.h`. There are two ways to get them in.

With the installer. `install.sh` (and `install_arch.sh`, `install_fedora.sh`, `install_opensuse.sh`) have a step of their own for it — step 17, *Kiwi client emulation*. It is offered, not forced: answer `n` and nothing is patched. For an unattended run, `PHANTOM_KIWI=y|n` decides it, and the default is yes.

On a tree that is already installed, run the installer script that ships in the repository root:

```
cd ~/PhantomSDR-Plus
./kiwi_install.sh
./recompile.sh          # choose [1] Backend only
```

`kiwi_install.sh` backs every file it touches up into `backup_kiwi_bridge_<timestamp>/` first, and applies each patch by exact text match — if an anchor is not where it expects it (a locally modified tree, a different version), it stops there and then, names the file, and leaves that file untouched. It is idempotent: run it on a tree that already has the bridge and it reports each patch as already applied and changes nothing.

> **If you have just applied an update archive that already contains the bridge, you do not need to run it.** The patched sources are in the drop; `kiwi_install.sh` would only tell you every patch is already in place.

The files it patches are `src/client.h`, `src/signal.cpp`, `src/waterfall.cpp`, `src/spectrumserver.h`, `src/spectrumserver.cpp`, `src/websocket.cpp` and `src/http.cpp`, and it copies `kiwi_bridge.h` into `src/`.

> **It installs the bridge; it does not upgrade one.** If a patch ever *fails* on a tree where the bridge already works, the source has moved ahead of the script and it is the script that needs updating — not your tree.

---

3. Switching it on

Add this to the `.toml` your receiver actually runs with — the same file you pass to `spectrumserver`, the one named in your `start-<radio>.sh`. Not `config.example.*`:

```
[kiwi_emulation]
enabled = true
```

Then restart the receiver:

```
./stop-websdr.sh
./start-<your-radio>.sh
```

`kiwi_install.sh` adds the block for you, with `enabled = true`, to whichever of `config.toml`, `config-rx888mk2.toml`, `config-airspyhf.toml`, `config-rtl.toml`, `config-rsp1a.toml`, `config-fobos.toml`, `config-fobos-hf.toml` and `config-hackrf.toml` exist in the repository root. It cannot know about a config you keep elsewhere or under another name, and **update archives never ship a `.toml`**, so after an update the block is yours to add by hand. The shipped `config.example.*.toml` files carry it documented and set to `false`.

---

4. Configuration reference

Everything the bridge reads lives in one `[kiwi_emulation]` section. Only `enabled` is required; the four trims all default to the calibrated value, so a block with nothing but `enabled = true` is already correct.

Key	Default	What it does
`enabled`	`false`	Answer the KiwiSDR protocol at all. Off = completely inert.
`audio_gain`	`0`	dB. Output gain for Kiwi clients only. See Audio level.
`smeter_offset`	`input.analog_smeter_offset`	dB. Overrides the S-meter offset for Kiwi clients only. See The S-meter.
`wf_cal`	`0`	dB. Display trim for the waterfall and spectrum. See Waterfall and spectrum.
`wf_fps_max`	`23`	Waterfall frames per second. See Waterfall and spectrum.

A fully populated block, with the values this project's own receiver runs:

```
[kiwi_emulation]
enabled       = true
audio_gain    = 60.0    # dB, Kiwi clients only. 0 = untouched.
wf_cal        = -10.0   # dB, display taste. 0 = agrees with the S-meter.
wf_fps_max    = 28      # frames/s. 23 is the protocol maximum.
# smeter_offset = 5.0   # dB. Defaults to input.analog_smeter_offset.
```

> **Every key here is read at startup.** Changing any of them is a **restart**, not a rebuild — `./stop-websdr.sh` then your start script. You never need to recompile to retune these.

Integer and decimal forms are both accepted: `28` and `28.0` are read identically. Only a key that is genuinely absent falls back to its default.

---

5. Connecting a client

Point the client at the same address and port your listeners use — no path, no prefix. AetherSDR asks for a host and a port; `kiwiclient` takes `-s` and `-p`:

```
# kiwiclient, recording 30 seconds of 7100 kHz LSB
python3 kiwirecorder.py -s your.receiver.example -p 8073 -f 7100 -m lsb --tlimit=30

# 30 seconds of I/Q around 6030 kHz, saved as a stereo WAV
python3 kiwirecorder.py -s your.receiver.example -p 8073 -f 6030 -m iq --tlimit=30
```

The client tunes, the waterfall fills, and the S-meter reads. If nothing happens at all, the bridge is almost certainly still disabled — see section 3.

---

6. Audio level

The first thing most people notice is that a Kiwi client sounds thinner than the PhantomSDR web page. That is not the bridge losing anything: one buffer feeds every encoder, and measuring the same frequency and passband through the browser's own audio path and through `/SND` gives identical samples. The web page is loud because `audio.js` rebuilds the sound in the browser — bass boost,

bandpass, a presence lift, a compressor with makeup gain and the volume slider. A Kiwi client has none of that and cannot be given it.

`audio_gain` closes the gap, for Kiwi clients only:

```
[kiwi_emulation]
enabled    = true
audio_gain = 55.0       # dB, Kiwi clients only. 0 = untouched.
```

Start around 55–60 dB. 55 dB is not arithmetic: it is where a Kiwi client and the PhantomSDR web page were measured level with each other, listening to a strong local signal on 729 kHz. In practice a little more is often preferred — this receiver settled on 60 — so pick it by ear.

You cannot break the audio with it. A **look-ahead peak limiter** sits between the gain and the 16-bit clamp: every sample is delayed by 4 ms while the gain is worked out from audio that has not been sent yet, so a loud passage arrives to a gain that has already come down for it, and peaks fold instead of being flattened. Driven to 60 and even 70 dB — far past the headroom the raw gain has — not one sample in a quarter of a million was clipped. Below the limiter's threshold the gain is exactly 1.0, so a receiver that is not driving the audio hard gets the samples untouched.

What a too-high value costs is loudness, not damage. Above about 60 dB the extra gain is simply limited away: measured here, the level rose about 1 dB from 55 to 60 and hardly moved from 60 to 70. Past that point you are buying compression rather than volume.

None of this touches the web page, and none of it moves the S-meter — that is derived from the demodulator's power, not from the audio samples, so turning the volume up cannot make the meter lie.

---

7. The S-meter

The Kiwi S-meter is not calibrated separately: it reproduces what your web page displays, stage for stage. The demodulator's per-block power is the starting point for both, and the page then applies `input.analog_smeter_offset`, expands the result about −130 dBm, and adds its own display offset. The bridge does the same, so a Kiwi client and the page show the same signal at the same strength — verified on a strong local station, where both read −37 dBm.

If, and only if, you want Kiwi clients to read differently from your page, give the bridge an offset of its own:

```
[kiwi_emulation]
enabled       = true
smeter_offset = 5.0     # replaces input.analog_smeter_offset for Kiwi clients only
```

> **Compare the two meters on the same filter, or not at all.** The reading comes from the power in the passband, so a wider filter collects more noise and reads higher — on a quiet band the same frequency measured 5.6 dB stronger at 9 kHz wide than at 2.4 kHz. A Kiwi client with a 6 kHz filter against a web page on 2.4 kHz will disagree by several dB no matter how either is calibrated. Match the mode and the bandwidth first.
>
> **This is the page's scale, not a physical one.** The web meter expands its range for readability, so 10 dB of real signal change is displayed as about 11. Matching it means Kiwi clients agree with your receiver and differ from a genuine KiwiSDR by an amount that grows with signal strength. That is the right trade when your own page is the reference everyone compares against; if you would rather the bridge stayed physically honest, this is the section to change.

---

8. Waterfall and spectrum

The dB scale — `wf_cal`

Each waterfall bin is converted to a real dBm before it is sent, on the same scale the S-meter is calibrated to. A carrier therefore reads the **same level at every zoom**, which is what a genuine KiwiSDR gives you, and the spectrum agrees with the web page's own S-meter over the same passband.

```
[kiwi_emulation]
enabled = true
wf_cal  = 0.0           # dB, Kiwi clients only
```

> **`wf_cal` is not a calibration.** `0` is the value at which the spectrum agrees with your S-meter, and that is what to leave it at if you want the numbers to mean something. It exists because how *strong* a spectrum should look is a matter of taste and of what your client does with the range — this project's receiver runs `-10` simply because that is what looks right in AetherSDR. One value moves the spectrum and the waterfall together: they are the same bytes on the wire, and the Kiwi protocol cannot scale them apart. If you want them to differ, that has to come from the client's own display controls.

To check the scale, compare the waterfall summed over the passband against the S-meter on a **quiet** frequency. On a carrier an SSB passband excludes the carrier itself and the two are not comparable.

The frame rate — `wf_fps_max`

A Kiwi client asks for the fastest waterfall it can have (`SET wf_speed=4`) and paces its scroll — and its spectrum averaging — on the rate the server says it will deliver. If the two disagree the display looks sluggish even though nothing is actually late.

The receiver produces `2 × sps / fft_size` spectra per second. Kiwi clients are served from every one of them and thinned to the rate they asked for; the web page keeps its own slower cadence and is unaffected.

```
[kiwi_emulation]
enabled    = true
wf_fps_max = 23         # frames per second
```

`23` is the KiwiSDR protocol's own declared maximum and the safe default. **The lower of `wf_fps_max` and the receiver's own rate always wins**, so raising it does something only if your FFT is fast enough to have frames to spare. Worked example, for a receiver at 60 Msps with a 4194304-point FFT — `2 × 60000000 / 4194304 = 28.6` frames a second:

`wf_fps_max`	delivered
absent (default 23)	23.0 fps
`28`	28.0 fps
`40`	28.6 fps — the receiver's own ceiling wins

`SET wf_speed` from the client is honoured as well: `0` off, `1` = 1 fps, `2` a quarter of the maximum, `3` a half, `4` the maximum. A client on a thin link can therefore ask for less.

> **Whether raising it helps is the client's decision, not the server's.** A client that draws each frame as it arrives gives you a smoother scroll; a client that paces itself at the 23 it expects will simply queue the extra frames, and the latency will grow. Try it, and if the waterfall feels laggier rather than smoother, put it back to `23`.

What sets the rest of the delay

If you are chasing latency, most of it is not in the bridge. The analysis window is `fft_size / sps` wide — 70 ms on a 4194304-point FFT at 60 Msps — and a sample also waits up to half of that for the hop to fill. Halving `fft_size` halves both and doubles the frame rate, but it also halves your frequency resolution, which is what the narrowband decoders (WSPR, FT8, CW) live on. On most receivers that is a poor trade for a few tens of milliseconds. It is a configuration decision in `[input]`, not something the bridge can do anything about.

---

9. If something looks wrong

Every command a Kiwi client sends is written to **`/tmp/kiwi_retune.log`** — the tuning requests, the mode changes, the authentication, and anything the bridge did not recognise. It costs nothing measurable and is meant to be left in place. When a client behaves oddly, that file shows what it actually asked for, which is usually the whole answer.

Symptom	Look at
Client connects and immediately drops	`[kiwi_emulation] enabled = true` missing from the config the server was **launched with**
No audio, no waterfall, no log lines	the backend was not rebuilt after `kiwi_install.sh` — run `./recompile.sh`, option `[1]`
`kiwi_install.sh` stops on a patch	the message names the file and the anchor; the tree has diverged from what the patch expects, and that file was left untouched
Audio present but thin and quiet	expected — set `audio_gain`, see section 6
Waterfall scrolls sluggishly	`wf_fps_max`, see section 8
Tuning in the client does nothing	an IQ input (`signal = "iq"`); retuning is implemented for real inputs only
A changed setting had no effect	these keys are read at startup — restart the receiver

> **AGC is client-side.** `SET agc=` from a Kiwi client is accepted and ignored — the audio arrives with PhantomSDR's own AGC applied, and the client's AGC controls act on what it receives.

---

10. How far this has been verified

The wire format has been checked against a live receiver and read line by line against the source of both clients it targets — AetherSDR's `KiwiSdrProtocol.cpp` and kiwiclient's `kiwi/client.py`. Frame layout and size, sequence numbers, byte order, the S-meter scale, retuning and the waterfall all agree. The bridge has also been **confirmed working with the AetherSDR desktop application** itself, by the maintainer, against this receiver.

Measured on that receiver, with a test client on the live sockets: retuning lands in 35–50 ms, audio runs at real time with no drift, and the waterfall delivers the configured rate with no stalls or dropped frames.

If something does look wrong on your own installation, `/tmp/kiwi_retune.log` is the place to look first, and a report either way is welcome.

See also: Installation Guide · Project Structure · User Guide

CHAPTER 15

Receive Diversity

RECEIVE_DIVERSITY.md

PhantomSDR-Plus — Receive Diversity

> **On a station set up with the station questions (`configure-station.sh`)** the WebSDR relay listens only inside the computer, on `PORT_RELAY` (9013), and the page reaches it as `/relay` on the public port through `proxy.py` — there is no relay port to forward, and the frontend needs no rebuild for it. On a station set up before the wizard, the port settings below apply as written.

Combine your receiver with a **second receiver somewhere else** and listen to whichever of the two currently has the better signal. When one site drops into a fade, the other usually has not, and you keep copy through it.

The second receiver can be another PhantomSDR-Plus, a KiwiSDR, an UberSDR or a WebSDR. For the first three, everything runs in the browser: no server changes, no configuration files, nothing to install, and no special access to the other receiver. A WebSDR is the exception and needs one small piece of software on your own server — see Using a WebSDR below.

What it does, and what it does not

Diversity here means **selection**: at any moment you are listening to one site or the other, with a short crossfade when it switches. It is not a phased array and it does not add the two signals together.

That is a deliberate choice, and the reason is physics rather than software. Two receivers hundreds of kilometres apart hear the same transmission over different ionospheric paths. The two audio waveforms arrive with unrelated phase and slightly different Doppler, so adding them sounds hollow and comb-filtered — the classic "phasey" artefact. True coherent combining needs two receivers on a **common clock**, sample-aligned; two independent receivers connected over the internet can never provide that.

So what you gain is **continuity, not raw signal strength**:

- On a stable path where one receiver is simply better, you will hear that receiver and gain nothing. This is normal.
- On a fluttery path with deep QSB, fades at two distant sites are largely uncorrelated. Site A drops out for three seconds, site B does not, and the audio carries on. Expect "*I stopped losing words*", not "*S3 became S7*".
- The second win is often **local noise and QRM**. Two sites have different neighbours, different power lines and different broadcast splatter. On a noisy band this can matter more than the fading.

There is a cost: you listen **one to three seconds behind real time**, because the earlier stream has to be delayed to match the later one. That is irrelevant for listening and decoding, and unhelpful if you are trying to work a station.

> **NOTE**
> The digital decoders (FT8, JS8, WSPR, RADE and the rest) deliberately keep using your **local** receiver, not the combined audio. Those modes integrate coherently across a whole transmission slot, and switching sites in the middle of a slot is a phase discontinuity that can cost you the very decode diversity was meant to save. Diversity serves the speaker; the decoders keep the continuous local stream.

Quick start

1. Open the **Receive Diversity** panel — it sits just below the decoder windows, collapsed, showing `Receive Diversity — off`. Click it to expand.
2. Choose what the second receiver is: **PhantomSDR+**, **KiwiSDR**, **UberSDR** or **WebSDR**.
3. Type its address — or pick one you have saved — and press **Start**.
4. Tune your own receiver normally. The second receiver **follows automatically** — frequency, mode and passband — every time you retune.

That is the whole setup. There is nothing to configure on the other receiver, and no separate tuning control: it always tracks you.

Addresses

Second receiver	What to type	Notes
PhantomSDR+	`host:8900`	The receiver's normal port
KiwiSDR	`host:8073`	The KiwiSDR's own port
UberSDR	`host`	Its **normal web port**, not 8073
WebSDR	`host:8901`	The site's own port. Needs the relay — see below

A bare hostname, an `http://` URL or a full `ws://` URL are all accepted. If the other receiver is served over HTTPS, type the hostname and it will use a secure connection.

If **your own page** is opened over https://, the browser does not let it open plain `ws://` connections, so partners that offer only http — most PhantomSDR+ stations and every KiwiSDR — are reached through your station's relay instead (the relay is described under *Using a WebSDR* below). The panel does this by itself; the relay only has to be running. See Secure Access — https://.

Saved receivers

Every address you connect to is remembered, in a list of its own for each receiver type — a KiwiSDR address is never a useful suggestion when the selector says WebSDR. The lists live in your own browser: they are not sent anywhere, and every listener has their own.

The ☰ button beside the address box opens the list.

- **Names.** Any entry can be given a name — `Twente` reads better than `websdr.ewi.utwente.nl:8901`. Named entries appear in green above their address, and the name shows beside the address in the drop-down as you type. An entry is identified by its address, so naming one never puts the same station in the list twice. While diversity is running, the receiver it is running against shows its name **bold and slowly blinking**, so the list also says which one you are actually hearing.
- **add** enters a receiver by hand, without connecting to it first. ✎ corrects a name and address in place, × removes one entry, and **clear all** empties the list for the type on screen. Enter saves, Escape cancels.

- **** opens that receiver's own web page in a new tab. The address stored is the one the software dials, so it is turned back into a browsable one first — `ws://` becomes `http://`, and the `/audio`, `/ws` or KiwiSDR `/kiwi/…/SND` path each kind of receiver needs is trimmed away, since the site's own page is at the root. It is a real link, so a middle click or a long press behaves as usual.
- **The order is yours.** Move a row with ▲▼, or drag it and drop it where it belongs. That is the order in which the drop-down offers the addresses.
- **export and import.** The lists are held in the browser's own storage, which is thrown away when you clear the site's data — this is what protects against that. Export writes all four lists, names included, to a JSON file you can keep or carry to another browser or machine. Import reads it back and asks whether to **merge** it with what is saved or **replace** everything.
- ▦ **QR.** Draws every list as a QR code on the screen, for carrying it to a phone — see "On a phone" below. A list too large to scan falls back to the text beside it, which can be copied and pasted anywhere.

There is no limit on how many receivers you keep.

On a phone

The `/mobile` page has the same feature in a **Div** tab, beside Audio, Bands, Marks, Users and Chat: the receiver type, the address, Start and Stop, the saved receivers as a tap-to-pick list, the live figures and the SNR trim. Each row also carries the **** link to that receiver's own page. The second receiver follows the phone's tuning by itself, exactly as on the desktop page. What it leaves out is the editing — renaming, reordering, deleting — which stays on the desktop page, where there is room for it.

The saved list belongs to one browser, so a phone starts with an empty one however many receivers are saved on the desktop. That is what the QR code is for: press ☰ then ▦ **QR** on the desktop, scan the code with the phone's camera, and paste the text into **Import** on the Div tab. It asks whether to merge or replace, and Export on the phone sends a list back the other way.

Using a WebSDR

A WebSDR — the software written by Pieter-Tjerk de Boer, PA3FWM, that runs at Twente and several hundred other sites — can be used as the second receiver, but not directly from the browser.

The reason is a deliberate check on their side. A WebSDR refuses its audio connection unless the `Origin` header names its own site, and `Origin` is a *forbidden header name*: the browser sets it from the page you are on and no script is permitted to change it. There is no client-side way around this, and there should not be.

So the connection is made by a small program on your own server instead, `websdr_relay.py`. Your browser talks to the relay, and the relay talks to the WebSDR.

Installing the relay

The four distribution installers offer it as an optional step. To add it to an existing installation:

```
cd ~/PhantomSDR-Plus
./setup_websdr_relay.sh
```

On a station set up with the station questions (`configure-station.sh`) it sees that the station sits behind the proxy: the relay then listens on this computer only and is reached as `/relay` on your public address, so it asks no port. On an older station it asks for one. Either way it takes your callsign and site address from `frontend/site_information.json` and offers to install a systemd service so the relay starts at boot.

Settings live in `websdr_relay.json` and can be changed at any time. Answering yes to the relay question in `bash configure-station.sh` installs and starts it as well.

Router port. Behind the proxy there is none to open: the page reaches the relay as `/relay` through the address listeners already use, over http and https alike. Only an older station, set up before the station questions, runs the relay on a port of its own; visitors' browsers then connect to it directly, so that port must be forwarded on your router, or WebSDR diversity works only from inside your own network. The other three receiver types do not need the relay — except from a page served over https, where it also carries http-only PhantomSDR+ and KiwiSDR partners (see above).

If the admin panel is installed, its dashboard shows a **WebSDR Diversity Relay** card with the relay's state, the port, how many sessions are running and to which sites. Above it, a **Receive Diversity** card lists every listener who is combining your receiver with another one — of any type, not only WebSDR — with the receiver, their name and city, frequency and mode, and the partner's type and address.

Being a good guest

The relay connects to other people's receivers from your server rather than from each listener's own address, so it is built to behave:

- **A cap of ten concurrent sessions to any one WebSDR**, and sixty overall. This is what stops a busy day at your site from looking like an attack on someone else's.
- **A `User-Agent` naming your station and its operator**, so an operator who would rather not be used this way knows exactly who to write to. Please fill it in honestly.
- **Only tuning commands are forwarded.** The connection cannot be used to send anything else.
- **Private, loopback and carrier-NAT addresses are refused**, so a visitor cannot aim the relay at something inside your machine or your LAN.

> **IMPORTANT**
> The `Origin` check exists because the WebSDR authors did not want their receivers driven from other people's pages. Using the relay is a considered decision, not an oversight on their part. Keep the cap where it is, keep the User-Agent honest, and stop if an operator asks you to.

What is different about a WebSDR

- **Many sites cover a few narrow slices of spectrum**, not a continuous range — a 256 kHz window on 40 m, for example. Diversity engages only inside them, and the panel reads the site's own band list to know where those are.
- **The audio sample rate varies** with the filter width and sits further from nominal than the other receiver types, so alignment can take a little longer.
- **CW is not adjusted** for WebSDR's convention, which places the passband entirely below the carrier. SSB and AM are correct.

Reading the panel

Once running, the panel shows a small table:

Row	Meaning
link	The connection to the second receiver. `ready` is what you want. A failure shows the server's own error text and the WebSocket close code
aligned	The measured delay between the two streams once they are locked together, or `searching`
corr	How strongly the two streams correlate. Only meaningful once aligned
remote audio	Decoded audio arriving from the second receiver. `none` in amber means nothing is being decoded
SNR local / remote	The two signal-to-noise estimates being compared
switches	How many times it has changed site

Beside the title, one word tells you where you stand:

`off` (grey)	not running
`Please wait…` (amber)	connecting, or connected and still aligning
`Ready` (green)	linked, locked and following the better site
the receiver's own error (red)	it has failed and will not recover on its own

The distinction between amber and red is the one that saves time: amber means keep waiting, red means stop waiting and read the message.

The coloured dot adds which site you are actually hearing — green for your own receiver, cyan for the remote.

`Ready (unaligned)` (green, too) means it is already choosing between the sites while the alignment search goes on — see the next section.

Alignment takes about 15 seconds

This is normal and worth expecting. The two streams are aligned by correlating their **audio envelopes**, and that needs a window of audio to work with. The search runs in two stages: a narrow ±1.5 second search can start once about 9 seconds of audio has been gathered, and a second, independent measurement five seconds later has to **agree** with the first before the alignment is trusted. When the two sites are further apart in time than that — a second or two of extra buffering somewhere along the path — the full ±4 second search takes over at about 14 seconds and confirmation follows at 19.

The confirmation step matters. When two sites fade in antiphase — never both carrying the signal at the same moment — a single correlation will happily lock onto a confident, completely wrong delay, which sounds like an echo. Requiring two independent measurements to agree rejects that.

A second thing has to be right before a lock can hold: the two receivers' sample rates. Two receivers are two clocks and two decimation chains, so their audio streams do **not** arrive at quite the same rate even when both declare 12 kHz — 2% apart is normal, which is 240 samples a second of creep, far more than the alignment tolerance. The remote stream is therefore continuously resampled to match your own receiver's rate. That ratio is measured by **counting samples**, not by correlating, so it needs no lock of its own and is settled within the first few seconds — before the first alignment search even runs. A weak or intermittent signal can still take two or three tries to align, so 30 seconds is not a cause for concern; a minute with nothing to show is.

While the search runs, diversity does not wait for it. The two streams are paired by when their audio arrives — or by the last delay that was found — and the better site is chosen by SNR as usual; rate

correction, level matching and the SNR trim all apply. That is what lets a station heard **only** by the remote come through: the case where diversity helps most, and where no alignment is possible at all. The panel then says `Ready (unaligned)` and the **aligned** row reads `by arrival, searching`. While both sites hear the station but are not aligned yet, a switch between them can repeat or skip up to the real delay between the two, usually under 1.5 s; once the search locks, it moves to the exact delay.

Remote SNR trim

This slider biases the choice between the two sites. It is added to the remote receiver's measured SNR before the comparison — positive favours the remote, negative favours your own. It changes **nothing else**: not the audio level, not the combining, only which site wins.

It exists because the two SNR figures are not always comparable. Both are measured the same way — a percentile spread of the audio — but a receiver whose codec floors its own noise reads lower than it deserves. A KiwiSDR applies strong gain and a limiter, which compresses the spread; a lossy codec sets a noise floor the signal never gets below.

How to set it:

1. Tune both receivers to a signal that is reliably present, and wait for `aligned`.
2. Watch the two SNR figures for half a minute. Note the typical gap.
3. Use the extremes as a listening test: **+15** forces the remote site live, **−15** forces your own. Listen to each for a few seconds. This is the only way to hear one site in isolation.
4. If the remote reads, say, 5 dB lower but sounds just as good, set **+5**. You are making the two readings agree when the two sites sound equal.
5. Watch the **switches** counter over ten minutes of listening. Constant flipping means the trim is too close to a tie; never switching even when your own receiver audibly fades means it is too far the other way.

Starting points: **0** for another PhantomSDR-Plus, which uses an identical audio path and is comparable by construction; a small **positive** value for a KiwiSDR or UberSDR.

> **TIP**
> If you need more than about ±8 dB, stop trimming. At that point the honest explanation is usually that one receiver really is worse for that path, and biasing past a real difference just means listening to the weaker site.

The setting applies immediately and is remembered in your browser.

Choosing a second receiver

Distance matters. Fades decorrelate with distance — a few hundred kilometres on HF is a good target. Two receivers in the same city fade together and buy you nothing. Too far apart and the second receiver may not hear your signal at all.

Both must actually hear the signal. This is the requirement no software can work around. If the second receiver cannot hear what you are listening to, the correlation stays low and it will refuse to lock — deliberately, because a wrong alignment sounds worse than no diversity at all.

Coverage. A second PhantomSDR-Plus or a KiwiSDR announces the frequency range it covers, and the panel warns you if you tune outside it. A WebSDR announces its bands, which are often narrow slices

rather than a continuous range. UberSDR does not announce coverage, so there the refusal to lock is your only signal.

> **IMPORTANT**
> Public receivers are run by volunteers and have a limited number of listener slots. A diversity session occupies one of them for as long as it runs, just as a human listener would. Please be considerate about leaving one connected indefinitely, and ask the operator if you intend to use their receiver heavily.

Troubleshooting

`link` never leaves `connecting`, or cycles connect/close The address or port is probably wrong. Check the table above — in particular, UberSDR uses its normal web port, not 8073. The panel shows the WebSocket close code, and the browser console (F12) shows the underlying error.

`link` shows an error with text after it That text comes from the other receiver. It is usually specific: a refused session, a full server, or a receiver that does not accept outside clients.

KiwiSDR refuses the connection Some KiwiSDRs require a password, and on an UberSDR the KiwiSDR-compatible endpoint is switched off by default. For an UberSDR, use the **UberSDR** source type instead — that path is always available.

WebSDR: `the WebSDR relay is not reachable` The relay is not running, or your browser cannot reach it. On a station set up with the station questions it is reached as `/relay` on the page's own address: open `http://<hostname>:<public port>/relay/status`, or `https://<hostname>/relay/status`, in the same browser. If nothing answers, the relay is not running — see `sudo systemctl status phantomsdr-websdr-relay`, or install it with `bash setup_websdr_relay.sh`. On an older station the relay has a port of its own and must be reachable at the **same hostname the receiver page is served from**, because the browser connects to it directly: open `http://<that hostname>:<relay port>/status`. If it answers on your LAN but not from outside, the port is not forwarded on the router.

WebSDR: an error mentioning a refused connection That site has locked things down further than the standard `Origin` check, or its operator has blocked this station.

`link` is `ready` but `remote audio` shows `none` Audio is not being decoded. Reload the page; if it persists, the browser console will show which decoder failed.

Audio arrives but it never aligns The two receivers are not hearing the same thing. Try a strong broadcast station, or a band where both sites have good propagation. A quiet band with only noise at both ends will never correlate — and should not.

It switches back and forth constantly The two sites are too close in SNR. Adjust the trim slightly so one is consistently preferred.

It never uses the second receiver Check `remote audio` is climbing and that the SNR figures look sensible. If the remote reads much lower than it sounds, that is what the trim is for.

Limitations

- **Not coherent.** No phasing, no nulling, no direction finding. Cancelling a noise source needs two antennas on one clock at one site; this is a different technique for a different problem.
- **One to three seconds of delay**, unavoidably.
- **Two receivers only.**

- **Mono.** If your receiver is in a stereo mode such as C-QUAM, the audio passes through untouched and diversity does not engage.
- **Alignment needs signal.** Under about 10 dB SNR at one of the two sites, expect it to sit at `searching`.
- **A WebSDR needs the relay** on your own server, and its port forwarded. The other three receiver types need neither.

CHAPTER 16

Rig Control

RIG_CONTROL.md

PhantomSDR-Plus — Rig Control (CAT)

Keep **your own transceiver** and a **PhantomSDR-Plus receiver** on the same frequency, mode and filter. Turn the rig's dial and the waterfall follows; click a signal on the waterfall and the rig tunes to it. Key the rig, and the receiver can fall silent so it does not play your own signal back to you.

It works with **PhantomSDR-Plus, KiwiSDR, PA3FWM WebSDR and UberSDR** receivers, yours or someone else's, and only ever moves *your* listening session — nobody else on the receiver hears or sees a thing. There is nothing for the receiver's operator to install or configure.

What you need

Two ways to connect a rig, and one requirement on the receiver:

Piece	What it is	Syncs
Desktop PhantomSDR+ 4.0 or later	The desktop application, with a **Rig** menu. Linux (PC and Raspberry Pi) and Windows.	Frequency, mode, filter width, mute on transmit — in either direction or both
CATsync Tool for WebSDRs	A separate Windows program that couples a rig to the receiver page in your browser.	Frequency and mode
TCI-CAT on the receiver page	A button on the PhantomSDR-Plus page itself, in any browser. Talks TCI to ExpertSDR, AetherSDR or Thetis, or to any Hamlib rig through a small bridge. Only on PhantomSDR-Plus receivers running 4.1.0 or later.	Frequency, mode and filter width both ways; mute on transmit
The receiver	PhantomSDR-Plus, KiwiSDR, PA3FWM WebSDR or UberSDR. A PhantomSDR-Plus receiver needs **4.1.0** or later for filter width and mute.	An older PhantomSDR-Plus still syncs frequency and mode

Most of this manual describes Desktop PhantomSDR+. The TCI-CAT button has its own section, TCI-CAT on the receiver page. The CATsync Tool has its own documentation on its website.

Supported receivers

The app recognises the kind of receiver in a station window and drives it through that page's own controls. The Rig control window shows which kind it found next to the station's name.

Receiver	Frequency and mode	Filter width	Mute on transmit
PhantomSDR-Plus	Yes	With 4.1.0 or later	With 4.1.0 or later
KiwiSDR (including Web-888)	Yes	Yes	Yes
PA3FWM WebSDR	Yes, switching band on a multi-band site	Yes	Yes
UberSDR	Yes	Yes	Yes

A web receiver has fewer modes than most rigs, so some rig modes share one receiver mode: KiwiSDR and WebSDR have a single CW for both CW and CW-R. The receiver modes named in this manual are PhantomSDR-Plus's; the other receivers use their nearest equivalent. A WebSDR keeps a CW filter under 1 kHz and other filters at 1 kHz or wider, because that is how the page itself tells CW apart, and a frequency outside every band a WebSDR site covers is left alone. UberSDR sets its filter in slider steps, so a width can land up to about 50 Hz from the rig's. Any other kind of page — OpenWebRX, for example — shows *not a receiver this app can drive*, and nothing is synced.

What it does, and what it does not

- It syncs **one rig with one receiver window** at a time.
- It reads both sides several times a second and, when they disagree, sets one to match the other. It does **not** transmit, key the rig, or send audio anywhere.
- Moving the receiver changes only your own session. Other listeners on the same receiver are not affected, and the receiver's operator does not have to allow it.
- A serial port can be opened by **one program at a time**. If WSJT-X, a logger or a manufacturer's utility already has the rig's port, use the **flrig** or **rigctld on network** choice to share the rig instead of fighting over the port.

Quick start

1. Open a station in Desktop PhantomSDR+ as usual.
2. **Rig → Rig control...**
3. Under **Connection**, choose **Built-in** if your rig is in the list, otherwise **Hamlib (all rigs)**.
4. Pick your rig, the serial port, and the speed your rig's own CAT or CI-V menu is set to.
5. Under **Sync**, leave **Both directions** selected.
6. Press **Connect**. The two readouts at the top — transceiver and receiver — should show the same frequency within a second.

Every setting is saved as you change it. Next time, **Rig → Connect** is enough, or tick **Connect when the app starts**.

Choosing how to reach the rig

Choice	Use it when	Needs
Built-in	Your rig is in the list below.	Nothing else
Hamlib (all rigs)	Your rig is anything else — Hamlib knows more than 300 rigs. The app starts Hamlib's `rigctld` for you, on a private local port, and stops it when you disconnect.	Windows: nothing, Hamlib is included. Linux: `sudo apt install libhamlib-utils`
rigctld on network	A `rigctld` is already running, on this computer or another one on your network.	The host and port (4532 by default)
flrig	flrig already controls the rig for fldigi, WSJT-X or a logger.	flrig running, with its XML-RPC port (12345 by default)

Rigs with a built-in driver

The speed and CI-V address shown are the factory defaults the app fills in. **They are only a starting point — set them to what your rig's menu says.**

Family	Rigs	Default speed	Notes
Icom CI-V	IC-7300, IC-7610, IC-705, IC-9700, IC-905, IC-7760, IC-7851, IC-7100, IC-7410, IC-9100, IC-7600, IC-7200, IC-7700, IC-7000, IC-7800, IC-756PROIII, IC-756PROII, IC-R8600, and any other CI-V rig	19200	CI-V address filled in per model (IC-7300 `94`, IC-705 `A4`, IC-9700 `A2`, IC-7610 `98` …)
	IC-746PRO, IC-718, IC-R75	9600	
Xiegu (CI-V)	G90, X6100	19200	Address `70`; check the rig's menu
Yaesu new CAT	FTDX101D/MP, FTDX10, FT-710, FT-991/A, FT-891, FTDX5000, FTDX3000, FTDX1200, FT-950, FT-2000, FT-450/450D	38400	
Yaesu classic CAT	FT-817/818, FT-857/857D, FT-897/897D	38400	2 stop bits; tunes in 10 Hz steps
Kenwood	TS-990S, TS-890S, TS-590S/SG	115200	
	TS-480, TS-2000, TS-870S	57600	
Elecraft	K4, K3/K3S, KX3, KX2	38400	Filter width syncs
Kenwood-compatible	FlexRadio SmartSDR CAT (virtual port), QRP Labs QMX/QMX+/QDX, (tr)uSDX, Lab599 Discovery TX-500, other Kenwood-compatible rigs	9600–38400	

A rig that ought to be compatible but will not talk to a built-in driver usually works with **Hamlib**, which copes with far more variations.

Serial settings

Setting	What to put there
Serial port	The rig's port. USB adapters and rigs with a USB port are listed first. **Other / network address…** takes a port the list does not show — `COM7`, `/dev/ttyUSB1` — or `tcp://host:port` for a serial port served over the network by ser2net or similar.
Speed (baud)	Exactly what the rig's CAT / CI-V baud-rate menu says. A mismatch looks like a rig that never answers.
Stop bits	1 for almost everything; 2 for the FT-817/857/897 family.
CI-V address	Icom only, in hexadecimal (`94`, not `148`). Must match the rig's CI-V address menu.
DTR / RTS	Leave **off** unless your interface needs them. Many CAT cables key the transmitter, or reset the rig, on one of those lines.
Hardware flow control	Leave off unless the rig's manual asks for RTS/CTS.

For **Hamlib** the same settings are passed on to `rigctld`. Stop bits there has a *Rig default* choice, and **Extra rigctld options** takes anything else `rigctld` accepts, for example `--set-`

`conf=post_write_delay=10`. **rigctld program** lets you point at a particular `rigctld` if you have more than one installed.

Sync

Direction

Choice	What happens
Rig → receiver	The receiver window follows the rig. A change made on the waterfall is put back to the rig's frequency.
Receiver → rig	The rig follows the receiver window. Turning the rig's dial is undone.
Both directions	Whichever side you touched **last** wins. At the moment you connect, before either has been touched, the rig wins.

The direction can also be changed from the **Rig** menu while connected.

How the two sides are kept from fighting

Every value the app writes shows up a moment later as a change on the other side. If that were taken at face value, the rig and the receiver would chase each other for ever. The app avoids it in three ways:

- The receiver page applies a change at once, so it is read back straight after writing, and that reading becomes the new starting point.
- A rig applies a change a little later, so each value sent to it is remembered. When the rig reports that value, it is recognised as the app's own write rather than a hand on the dial.
- A value the rig refuses — wide FM on an HF rig, for example — is sent **twice** and then left alone until the source side changes, instead of being repeated several times a second.

Retuning the receiver page can make it choose its band's default mode (LSB below 10 MHz, for instance). When the rig is leading, the app puts the rig's own mode back straight away, so a rig in USB on 40 m keeps the receiver in USB.

Which receiver window

Receiver window chooses which station follows the rig:

- **The station window last in front** (default) — with two stations open, click into one and the rig follows that one.
- **A particular station** — pin the rig to it, whether or not it is in front. If that station is not open, nothing is synced until it is.

Update rate

Update every sets how often both sides are read: 150 ms, 300 ms (default), 500 ms or 1 s. Faster feels more immediate on the dial; slower is kinder to an old rig at 4800 or 9600 baud, where each read takes real time on the wire.

Modes

Rig mode	Receiver listens in
USB, LSB	USB, LSB
CW	CW
CW-R (reverse)	CW-L
AM, synchronous AM, DSB	AM
FM, narrow FM	FM
Wide FM	WBFM
RTTY / FSK	LSB
RTTY-R / FSK-R	USB
Data modes (USB-D, DATA-U, PKTUSB, DIG)	USB
Data LSB, data FM	LSB, FM

Receiver mode	Rig is set to
USB, LSB	USB, LSB
CW	CW
CW-L	CW-R
AM, QUAM	AM
FM	FM
WBFM	WFM — most HF rigs refuse it, and are left alone after two tries
RADE (upper / lower)	USB / LSB

A rig in a **data mode** stays in it: the receiver's USB is treated as agreeing with the rig's USB-D, so the receiver never knocks the rig out of data mode.

Filter width

Tick **Sync filter width** to keep the passbands matched. Two differences of less than 60 Hz are treated as equal, since no two filters step alike.

Rig connection	Filter width
Hamlib	Yes, where Hamlib supports it for that rig
flrig	Yes
Built-in Icom CI-V	Yes — 50 Hz steps up to 500 Hz, then 100 Hz steps up to 3.6 kHz; AM in 200 Hz steps up to 10 kHz; not in FM
Built-in Elecraft	Yes, in 10 Hz steps
Built-in Kenwood, Yaesu, FT-817 family	No — these rigs select filters from per-model tables. Use Hamlib if you need the filter

KiwiSDR, WebSDR and UberSDR receivers always have the control. A PhantomSDR-Plus receiver must run **4.1.0** or later; on an older one frequency and mode still sync, and the Rig control window says why the filter does not.

Mute while transmitting

Tick **Mute receiver while transmitting**. While the rig is keyed the receiver window is muted, and when it is unkeyed the sound comes back. The receiver's own mute button shows it, and you can still unmute by hand.

If you had already muted the receiver yourself, it is left muted afterwards too.

It needs a rig connection that reports transmit state — all the built-in drivers, flrig, and Hamlib for most rigs — and, on a PhantomSDR-Plus receiver, 4.1.0 or later.

Frequency offset

Frequency offset is added to the rig's frequency to get the receiver's:

> receiver frequency = rig frequency + offset

Setup	Offset
2 m transverter on a 10 m rig (144.100 MHz shows as 28.100 MHz)	`116000000`
70 cm transverter on a 2 m rig (432 → 144)	`288000000`
No transverter	`0`

The Rig menu

Item	Does
Rig control...	Opens the Rig control window
Connect / **Disconnect** *rig name*	Starts or stops the sync; for Hamlib this also starts or stops `rigctld`
Rig to receiver / **Receiver to rig** / **Both directions**	Sync direction
Sync filter width	On / off
Mute receiver while transmitting	On / off
Status line	*Not connected*, *Connecting...*, *Connected: rig name*, or the last error

The live frequency readout is in the Rig control window rather than in the menu, which would close itself every time it changed.

Linux

Serial port permission. Serial ports belong to the `dialout` group. A user outside it gets *Could not open ttyUSB0*. Add yourself once, then log out and back in:

```
sudo usermod -aG dialout $USER
```

Hamlib. Install it from your distribution:

```
sudo apt install libhamlib-utils
```

The `.deb` package of Desktop PhantomSDR+ recommends it, so `sudo apt install ./phantomsdr-plus-desktop_4.0.0_amd64.deb` brings it along; `dpkg -i` does not install recommended packages. The built-in drivers and flrig need no Hamlib.

Windows

Hamlib's own `rigctld.exe` is included in both the 64-bit and the 32-bit installer. COM ports appear in the port list by name (`COM3`). If a rig's USB driver is needed, install the manufacturer's driver first — the port does not exist until it is.

TCI-CAT on the receiver page

A PhantomSDR-Plus receiver page can also follow a transceiver **by itself**, with no desktop application: the browser talks straight to a **TCI** server on your own computer. TCI is the WebSocket control protocol of ExpertSDR2/ExpertSDR3 (SunSDR), AetherSDR (FlexRadio) and Thetis (Apache Labs ANAN, Hermes). For a rig without TCI, a small bridge included with PhantomSDR-Plus makes any Hamlib rig look like a TCI server.

It needs a PhantomSDR-Plus receiver running **4.1.0 or later**. KiwiSDR, WebSDR and UberSDR pages do not have it.

The TCI-CAT button

The button sits in the row with **VFO**, **Modes**, **Bands** and **IF Filters** — **TCI-CAT** in the wide layout, **CAT** in the compact one. Its dot is **green** while a TCI server is connected and **grey** otherwise. It opens a window with: Until 4.2.0 the button was labelled **QRG Sync**.

Control	What it does
Status	*Active — TCI (port 50001)* when connected; *Inactive* with *Wait* while it searches
CAT Sync	Frequency, mode and filter sync on or off. The mute on transmit works either way
Host	The computer running the TCI server: `localhost` for this computer, or its address on your network, e.g. `192.168.1.42`. Press Enter or click away to reconnect

Nothing needs choosing beyond that: ports **50001** (ExpertSDR3, AetherSDR) and **40001** (ExpertSDR2, Thetis) are tried at the same time and retried every few seconds, so the page connects as soon as the software starts. **CAT Sync** and **Host** are remembered by the browser.

Browsers that ask before a web page may reach your local network — recent Chrome and Edge do — ask once, the first time the page connects. Allow it, or the dot stays grey.

What is kept in step

	Behaviour
Rig → receiver: frequency	Only the dial moves. Zoom, brightness and contrast stay as you left them; the waterfall scrolls only when the frequency is off screen, and the band plan does not change the mode
Rig → receiver: mode	Follows when the rig's mode changes — USB, LSB, CW, AM, FM; data modes as USB or LSB. A mode the receiver lacks leaves it alone. A running decoder keeps its own mode
Rig → receiver: filter width	The receiver's passband takes the rig's filter width: turning the rig's filter, or pressing its **FIL** button, changes the passband. A running decoder keeps its own passband
Receiver → rig: frequency	Typing a frequency, a label or bookmark, a click on the waterfall and a passband drag all move the rig's VFO. A drag sends only changed frequencies
Receiver → rig: mode	Every mode change on the page — a mode button, the band plan, a decoder, a bookmark — sets the rig's mode (USB, LSB, CW, AM, FM; RADE as USB or LSB). A rig in a data mode (USB-D) stays in it
Receiver → rig: filter width	The **IF Filters** buttons, the IF slider, a passband drag and a mode change all set the rig's filter; a drag sends only the width it settles on. On Icom rigs the bridge selects FIL1, FIL2 or FIL3 instead — see Icom filters
No echo	A change that came from the rig is never sent back to it, and a report of the rig's previous mode or filter that arrives just after the page changed it is ignored
Mute on transmit	Always on, even with CAT Sync off: the page falls silent while the rig transmits. Moving the volume slider during transmit keeps it silent; on receive the sound returns at the slider's current level
Not handled	Split, VFO B, RIT/XIT and transverter offsets. Only VFO A of the first receiver is followed

Mode and filter width travel as TCI's own commands (`modulation`, `rx_filter_band`), so ExpertSDR, AetherSDR and Thetis should follow them too; only the Hamlib bridge has been tried. For a transverter offset, use Desktop PhantomSDR+ instead. Do not use both on the same rig at once — two controllers fight over the dial.

Which transceivers it works with

Transceiver	Works	How
SunSDR (ExpertSDR2/3), FlexRadio (AetherSDR), ANAN/Hermes (Thetis)	Yes	Directly — enable the TCI server in the software. Not yet tried with these programs; tested against a simulated TCI server
A radio with a CAT port that Hamlib supports — most Icom, Yaesu, Kenwood, Elecraft, Xiegu, QRP Labs	Yes	Through the Hamlib bridge below. Tried on an **Icom IC-7300**, on Linux and Windows; other drivers may differ in mode names or in reporting PTT
A radio with no CAT port, or one Hamlib does not support	No	There is nothing to read the frequency from

The Hamlib bridge (IC-7300 and other rigs without TCI)

`tci-bridge/tci-rigctld.mjs` in the PhantomSDR-Plus tree turns any radio that Hamlib can control into a TCI server for the page. It runs on **your** computer — the one wired to the radio, next to the browser — not on the receiver. Four times a second it asks Hamlib for frequency, mode, filter width and transmit state, sends the page only what changed, and passes the page's frequency, mode and filter changes to the rig.

It reaches Hamlib in one of two ways:

Way	Chain	Use it when
`--rigctl` (recommended)	transceiver → `rigctl` → `tci-rigctld.mjs` → page	Normally. The bridge starts Hamlib's `rigctl` itself: one window, no network port. **Use this on Windows**, where `rigctld.exe` is often refused with *Access is denied*
`rigctld`	transceiver → `rigctld` → `tci-rigctld.mjs` → page	Another program, such as WSJT-X, must use the radio at the same time — see Sharing the radio

What it needs

	Linux	Windows
Hamlib	`sudo apt install libhamlib-utils`, or your distribution's hamlib package	`hamlib-w64-….zip` from github.com/Hamlib/Hamlib/releases, extracted to `C:\hamlib` — `rigctl.exe` is in `C:\hamlib\bin`
Node.js	18 or newer	The LTS *Windows Installer (.msi)* from nodejs.org, with the default options
The `ws` package	Found automatically inside the PhantomSDR-Plus tree	Copy `tci-rigctld.mjs` to a folder such as `C:\tci-bridge` and run `npm install ws` there once
The radio's port	`ls /dev/serial/by-id/`. Join the `dialout` group once: `sudo usermod -aG dialout $USER`, then log out and back in	Install the radio's USB driver from the manufacturer; the COM number is in **Device Manager → Ports (COM & LPT)**

Only one program can hold the rig's CAT port. Close WSJT-X, flrig, JS8Call, RS-BA1 or Desktop PhantomSDR+'s rig connection before starting the bridge.

Example: Icom IC-7300

On the radio: **MENU → SET → Connectors → CI-V** — **CI-V USB Baud Rate** `115200`, **CI-V Transceive** `ON`. Hamlib's model number for the IC-7300 is `3073`.

Linux. The radio is the line containing `IC-7300` in `ls /dev/serial/by-id/`, usually also `/dev/ttyUSB0`.

1. Check that Hamlib reaches the radio — it must print the frequency, e.g. `14280000`: `bash rigctl -m 3073 -r /dev/ttyUSB0 -s 115200 f`
2. Start the bridge and leave it running: `bash cd ~/PhantomSDR-Plus/tci-bridge node tci-rigctld.mjs --rigctl rigctl -m 3073 -r /dev/ttyUSB0 -s 115200`

Windows. The radio appears in Device Manager as *Silicon Labs CP210x USB to UART Bridge (COM4)* — use your own COM number. In a Command Prompt:

1. Check that Hamlib reaches the radio — it must print the frequency: `bat C:\hamlib\bin\rigctl.exe -m 3073 -r COM4 -s 115200 f`
2. Start the bridge and leave the window open: `bat cd C:\tci-bridge node tci-rigctld.mjs --rigctl C:\hamlib\bin\rigctl.exe -m 3073 -r COM4 -s 115200` The line is long: make sure it really ends in `-s 115200`, or `rigctl` stops with *Type: rigctl --help*.

On both systems the bridge prints, within a second:

```
rig answered through rigctl
rig -> page 14280000 Hz
rig -> page mode USB
rig -> page RX
```

Then open the receiver page in a browser on the same computer, open **TCI-CAT** and switch **CAT Sync** on. The dot turns green and the bridge prints `page connected`. Turn the VFO and the receiver follows; click the waterfall and the radio follows (`page -> rig ... Hz`); key the radio and the page goes silent. **Ctrl+C** stops the bridge, and `rigctl` with it.

Everything after `--rigctl` is the `rigctl` program and its own options — exactly the ones that worked in the check. That is the rule for every rig: **when `rigctl … f` prints the frequency, the bridge works with the same options.**

Icom filters (FIL1, FIL2, FIL3)

Icom rigs such as the IC-7300 do not take just any filter width: they have three filters, **FIL1**, **FIL2** and **FIL3**, each with a width set in the rig's menu. Given a width, Hamlib would select one of them and also overwrite its width — and on the IC-7300 that width can land on the filter selected before, scrambling the rig's settings. So for Icom rigs the bridge never sends a width: it picks the filter whose reference width is nearest to the page's passband and only **selects** it, with the rig's own CI-V command. The widths set on the rig are never changed.

Mode	FIL1	FIL2	FIL3
USB, LSB (and their data modes)	2700 Hz	2400 Hz	1800 Hz
CW	1200 Hz	500 Hz	250 Hz
AM	9000 Hz	6000 Hz	3000 Hz
FM	15000 Hz	10000 Hz	7000 Hz

On the IC-7300 (`-m 3073`) this is automatic. Set the rig's SSB filters to match — FIL1 2.7 kHz, FIL2 2.4 kHz, FIL3 1.8 kHz (hold **FIL** on the radio) — and choosing 2.7 / 2.4 / 1.8 kHz in **IF Filters** selects FIL1 / FIL2 / FIL3, while pressing **FIL** on the radio sets the page's passband to that filter's width. The bridge prints, for example, `page -> rig filter 2398 Hz LSB → FIL2 sent`.

Option	Use
`--filters 3000,2400,1800`	Other SSB reference widths, in the order FIL1,FIL2,FIL3; also switches filter selection on for another Icom rig
`--civ A4`	The rig's CI-V address in hex, when it is not `94` (IC-705 `A4`, IC-9700 `A2`, IC-7610 `98`)
`--filters off`	Send widths through Hamlib instead, as for other makes

Both options go before `--rigctl`, e.g. `node tci-rigctld.mjs --filters 3000,2400,1800 --civ A4 --rigctl rigctl -m 3085 -r /dev/ttyACM0 -s 19200`. Tried on an IC-7300 only. Other makes — Yaesu, Kenwood, Elecraft and the rest — receive the page's width through Hamlib, which sets it as closely as the rig allows.

Other rigs

The same steps work for any rig Hamlib supports; only the options change.

1. **Prepare the rig.** In its menu, note the CAT (or CI-V) speed, and switch on any *CAT over USB* or *CI-V transceive* setting it has. On Windows, install the manufacturer's USB driver.
2. **Find the Hamlib model number** in the list `rigctl -l` prints:
3. Linux: `rigctl -l | grep -i 991`
4. Windows: `C:\hamlib\bin\rigctl.exe -l | findstr /i 991`
5. **Find the port.** Linux: `ls /dev/serial/by-id/`. Windows: Device Manager. Some rigs create **two** ports — Yaesu's FT-991A, FTDX10 and FT-710 name them *Enhanced* and *Standard*; CAT is on the **Enhanced** one.

6. **Check, then start the bridge** with `-m <model> -r <port> -s <speed>`, as in the IC-7300 example: first `rigctl -m … -r … -s … f`, then the same options after `--rigctl`.

Some model numbers, from Hamlib 4.5 — confirm them with `rigctl -l`, as another Hamlib version may number a rig differently:

Rig	-m	Rig	-m
Icom IC-7300	3073	Yaesu FT-991 / FT-991A	1035
Icom IC-705	3085	Yaesu FTDX10	1042
Icom IC-7610	3078	Yaesu FT-710	1049
Icom IC-9700	3081	Yaesu FT-891	1036
Xiegu G90	3088	Yaesu FT-817	1020
Xiegu X6100	3087	Kenwood TS-590SG	2037
Elecraft K3 / K3S	2029	Kenwood TS-890S	2041
Elecraft KX3	2045	Kenwood TS-2000	2014
Elecraft K4	2047	QRP Labs QCX / QDX	2052

Extra options, only when needed:

- **An Icom rig with a changed CI-V address:** add `-c` and the address in *decimal* — `94h` is `-c 148`.
- **A setting Hamlib offers for that rig:** `rigctl -m <model> -L` lists them; set one with `-C name=value`, e.g. `-C post_write_delay=10` for a slow interface.
- **A rig that keys up or resets when the port opens:** some CAT cables use DTR or RTS for PTT; add `-C dtr_state=OFF -C rts_state=OFF`.

Example: Yaesu FT-991A

Not tried on a real FT-991A — it follows Hamlib's settings for the rig; the `rigctl … f` check tells you at once whether it works.

On the radio, set **CAT RATE** (menu 031) to `38400`. Hamlib's model number for the FT-991 and FT-991A is `1035`.

The FT-991A's USB cable creates **two** serial ports. CAT is on the **Enhanced** one:

- **Linux:** `ls /dev/serial/by-id/` shows two lines for the radio; the one ending in `-if00-port0` is Enhanced, usually `/dev/ttyUSB0`.
- **Windows:** Device Manager shows *Silicon Labs Dual CP2105 USB to UART Bridge: Enhanced COM Port (COM5)* and a *Standard COM Port*; use the Enhanced one's number.

Linux:

```
rigctl -m 1035 -r /dev/ttyUSB0 -s 38400 f
cd ~/PhantomSDR-Plus/tci-bridge
node tci-rigctld.mjs --rigctl rigctl -m 1035 -r /dev/ttyUSB0 -s 38400
```

Windows:

```
C:\hamlib\bin\rigctl.exe -m 1035 -r COM5 -s 38400 f
cd C:\tci-bridge
node tci-rigctld.mjs --rigctl C:\hamlib\bin\rigctl.exe -m 1035 -r COM5 -s 38400
```

If the check prints nothing useful, the usual cause is the Standard port instead of the Enhanced one, or a CAT RATE that differs from `-s`.

Example: Kenwood TS-590SG

Not tried on a real TS-590SG — it follows Hamlib's settings for the rig; the `rigctl … f` check tells you at once whether it works.

In the radio's menu, set the baud rate of the **USB** port to `115200` (the menu number is in the TS-590SG manual). Hamlib's model number is `2037`; the older TS-590S is `2031`. On Windows, install Kenwood's virtual COM port driver for the USB port first.

The USB cable creates **one** serial port:

- **Linux:** the radio's line in `ls /dev/serial/by-id/`, usually `/dev/ttyUSB0`.
- **Windows:** Device Manager → Ports (COM & LPT), for example `COM6`.

Linux:

```
rigctl -m 2037 -r /dev/ttyUSB0 -s 115200 f
cd ~/PhantomSDR-Plus/tci-bridge
node tci-rigctld.mjs --rigctl rigctl -m 2037 -r /dev/ttyUSB0 -s 115200
```

Windows:

```
C:\hamlib\bin\rigctl.exe -m 2037 -r COM6 -s 115200 f
cd C:\tci-bridge
node tci-rigctld.mjs --rigctl C:\hamlib\bin\rigctl.exe -m 2037 -r COM6 -s 115200
```

If the check fails, the usual cause is a USB baud rate that differs from `-s`, or a cable in the rig's RS-232 (COM) socket while `-r` names the USB port.

More examples

None of these has been tried on real hardware. Each row gives the options to put after `rigctl` for the check and after `--rigctl rigctl` for the bridge; set the rig's menu to the same speed. The Linux port is the usual one — confirm it with `ls /dev/serial/by-id/`. On Windows, replace the port with the COM number from Device Manager and `rigctl` with `C:\hamlib\bin\rigctl.exe`.

Rig	Rig's menu	Options (Linux)	Notes
Icom IC-705 (USB)	CI-V USB Baud Rate `19200`	`-m 3085 -r /dev/ttyACM0 -s 19200`	Two ports appear; CI-V is the first (`ttyACM0`)
Icom IC-7100 (USB)	CI-V USB Baud Rate `19200`	`-m 3070 -r /dev/ttyUSB0 -s 19200`	Hamlib's highest speed for this rig is 19200
Icom IC-7610	CI-V USB Baud Rate `115200`	`-m 3078 -r /dev/ttyUSB0 -s 115200`	
Icom IC-9700	CI-V USB Baud Rate `38400`	`-m 3081 -r /dev/ttyUSB0 -s 38400`	
Yaesu FTDX101D / FTDX101MP	CAT RATE `38400`	`-m 1040 -r /dev/ttyUSB0 -s 38400`	`-m 1044` for the FTDX101MP. Two ports; use Enhanced
Yaesu FTDX10	CAT RATE `38400`	`-m 1042 -r /dev/ttyUSB0 -s 38400`	Two ports; use Enhanced, as on the FT-991A
Yaesu FT-710	CAT RATE `38400`	`-m 1049 -r /dev/ttyUSB0 -s 38400`	Two ports; use Enhanced, as on the FT-991A
Yaesu FT-891	CAT RATE `38400`	`-m 1036 -r /dev/ttyUSB0 -s 38400`	Two ports; use Enhanced, as on the FT-991A
Yaesu FT-450D	CAT RATE `38400`	`-m 1046 -r /dev/ttyUSB0 -s 38400`	RS-232 socket: use a USB-serial adapter
Yaesu FT-817 / FT-818	CAT RATE `38400`	`-m 1020 -r /dev/ttyUSB0 -s 38400`	`-m 1041` for the FT-818. Needs a CAT cable on the ACC socket
Yaesu FT-857 / FT-897	CAT RATE `38400`	`-m 1022 -r /dev/ttyUSB0 -s 38400`	`-m 1023` for the FT-897. Needs a CAT cable
Kenwood TS-890S (USB)	USB baud rate `115200`	`-m 2041 -r /dev/ttyUSB0 -s 115200`	
Kenwood TS-990S (USB)	USB baud rate `115200`	`-m 2039 -r /dev/ttyUSB0 -s 115200`	
Kenwood TS-590SG / TS-590S (USB)	USB baud rate `115200`	`-m 2037 -r /dev/ttyUSB0 -s 115200`	Worked example above
Kenwood TS-480	COM port baud rate `57600`	`-m 2028 -r /dev/ttyUSB0 -s 57600`	RS-232 socket: use a USB-serial adapter
Kenwood TS-2000	COM port baud rate `57600`	`-m 2014 -r /dev/ttyUSB0 -s 57600`	RS-232 socket; Hamlib's highest speed for this rig is 57600
Elecraft K4 (USB)	RS232 speed `115200`	`-m 2047 -r /dev/ttyUSB0 -s 115200`	
Elecraft K3 / K3S	RS232 speed `38400`	`-m 2029 -r /dev/ttyUSB0 -s 38400`	K3S: USB; K3: serial port or KUSB cable
Elecraft KX3	RS232 speed `38400`	`-m 2045 -r /dev/ttyUSB0 -s 38400`	KXUSB cable
Elecraft KX2	RS232 speed `38400`	`-m 2044 -r /dev/ttyUSB0 -s 38400`	KXUSB cable
Xiegu G90	CI-V baud rate `19200`	`-m 3088 -r /dev/ttyUSB0 -s 19200`	Hamlib uses the G90's default CI-V address

Rig	Rig's menu	Options (Linux)	Notes
Xiegu X6100	CI-V baud rate `19200`	`-m 3087 -r /dev/ttyUSB0 -s 19200`	Hamlib's highest speed for this rig is 19200
Lab599 TX-500	—	`-m 2050 -r /dev/ttyUSB0 -s 9600`	Hamlib uses 9600 only
ELAD FDM-DUO	—	`-m 33001 -r /dev/ttyUSB0 -s 115200`	
QRP Labs QDX	—	`-m 2052 -r /dev/ttyACM0 -s 9600`	A USB serial port; the speed does not matter but must be given

Complete Linux command for the IC-705, as an example of reading a row:

```
rigctl -m 3085 -r /dev/ttyACM0 -s 19200 f
node tci-rigctld.mjs --rigctl rigctl -m 3085 -r /dev/ttyACM0 -s 19200
```

Rigs run by other software

Some radios are already run by a program that can do the job — sometimes with no bridge at all. None of these has been tried.

Radio and program	What to do
SunSDR with ExpertSDR2/3, **FlexRadio** with AetherSDR, **Apache Labs ANAN** / **Hermes** with Thetis	No bridge. Switch on the program's **TCI server** and use **TCI-CAT** directly — ports 50001 and 40001 are found automatically
FlexRadio with SmartSDR (Windows)	Add a port in **SmartSDR CAT**. It speaks Kenwood CAT, so a serial port there works as a TS-2000: `-m 2014 -r COM8 -s 57600`. A TCP port works with Hamlib's FlexRadio model: `-m 2036 -r 127.0.0.1:<port>`
Any rig controlled by flrig	Leave flrig running and use Hamlib's flrig model; the rig stays shared with fldigi, WSJT-X and loggers: `-m 4 -r 127.0.0.1:12345`
A running rigctld, or a program offering a *Hamlib NET rigctl* server	Start the bridge without `--rigctl`, with that address: `node tci-rigctld.mjs 50001 127.0.0.1:4532`

As everywhere, the options go after `rigctl` for the check and after `--rigctl rigctl` for the bridge. A network address such as `127.0.0.1:12345` needs no `-s`.

Sharing the radio: the rigctld way

When WSJT-X or a logger must use the radio while the bridge runs, start Hamlib's server `rigctld` with the same options, leave it running, and connect both programs to it:

```
rigctld -m 3073 -r /dev/ttyUSB0 -s 115200
node tci-rigctld.mjs
```

Without `--rigctl` the bridge looks for `rigctld` at `127.0.0.1:4532` and prints `rigctld connected`; `node tci-rigctld.mjs 50001 192.168.1.50:4532` uses a `rigctld` on another computer. In WSJT-X choose rig *Hamlib NET rigctl* at the same address. On Windows the server is `C:\hamlib\bin\rigctld.exe` with the same options; if Windows answers *Access is denied*, use `--rigctl` and close the other program while the bridge runs.

Ports. The bridge serves TCI on port 50001; another port goes first on the command line, e.g. `node tci-rigctld.mjs 40001 --rigctl …`. If the bridge and the browser are on different computers, put the bridge computer's address in **Host**.

TCI-CAT troubleshooting

Symptom	Likely cause	What to do
Dot stays grey	TCI server or bridge not running; wrong **Host**; the browser's local-network permission was refused	Enable TCI in ExpertSDR/Thetis/ AetherSDR, or start the bridge; check **Host**; allow local network access in the site settings
`rigctl … f` gives an error or waits	Wrong port or speed; no `dialout` permission (Linux) or no USB driver (Windows); another program has the port	Match `-s` to the rig's menu; check the port (`ls /dev/serial/by-id/`, Device Manager); close other rig programs
Bridge prints *rigctl stopped ... Type: rigctl --help*	The command is incomplete — usually the speed after `-s` is missing	Type the whole line again
Bridge prints *The value after -s is missing*	The command line was cut short when it was pasted	Type the end of the line again, e.g. `-s 115200`
The filter does not follow in one direction or the other	**CAT Sync** is off, a decoder is running on the page, or (IC-7300) the rig's FIL widths differ from 2700 / 2400 / 1800 Hz	Switch CAT Sync on; stop the decoder; set the FIL widths on the rig, or give `--filters` with the rig's own widths
Bridge prints *rigctl stopped ... rig_open: error*	`rigctl` cannot open the port	As for `rigctl … f` above
Bridge prints *the rig is not answering*	The port opens but the rig does not reply: wrong speed or model, CAT switched off in the rig's menu, or a changed Icom CI-V address	Repeat the `rigctl … f` check; add `-c` for a changed CI-V address
Bridge prints *rigctld not reachable*	Started without `--rigctl`, and no `rigctld` is running	Add `--rigctl …`, or start `rigctld` first
`rigctld.exe` says *Access is denied* (Windows)	Windows refuses to run it	Use `--rigctl` — it needs only `rigctl.exe`
Connected, but the receiver does not move	**CAT Sync** is off	Switch it on — the mute on transmit works without it
Rig mode does not follow	A mode the receiver has no equivalent for, or the Hamlib driver reports an unusual name	Frequency still syncs; set the mode on the page
No mute on transmit	The rig or its Hamlib driver does not report transmit state	Nothing to set on the page
The dial jumps back and forth	Desktop PhantomSDR+ or another program is also syncing the rig	Use one controller at a time

For receiver operators

Nothing to configure. Rig control uses a small JavaScript interface every PhantomSDR-Plus page already carries; it involves no server setting, no open port and no admin permission. The filter and mute functions came with version 4.1.0 — after applying it, rebuild the frontend (`./recompile.sh`, option 2); the receiver does not need to be stopped. KiwiSDR, WebSDR and UberSDR receivers need nothing either: the app uses the controls their pages already have.

The **TCI-CAT** button came with 4.1.0 too, again a frontend rebuild only. It opens no port on the receiver: the connection runs from each listener's browser to their own computer. The Hamlib bridge, `tci-bridge/tci-rigctld.mjs`, is for listeners to run at home; the receiver does not use it. The browser opens that connection only after the listener opens **TCI-CAT** or turns **CAT Sync** on (remembered per browser), so over https visitors who never use it see no local-network permission prompt.

For developers: the page interface

Both Desktop PhantomSDR+ and the CATsync Tool use these functions, which every PhantomSDR-Plus page puts on `window` once it has loaded (KiwiSDR, WebSDR and UberSDR pages are driven through their own, different controls):

Function	Returns / does
`catsync_ready`	`true` once the functions below are installed
`catsync_getFrequency()`	Tuned frequency, Hz
`catsync_setFrequency(hz)`	Tune to `hz`
`catsync_getMode()`	`USB`, `LSB`, `CW`, `CW-L`, `AM`, `QUAM`, `FM`, `WBFM`, `RADEU`, `RADEL`
`catsync_setMode(mode)`	Set the mode; resets the passband to the mode's default
`catsync_getBandwidth()`	Whole passband width, Hz
`catsync_setBandwidth(hz)`	Set the width — grows upwards in USB, downwards in LSB, evenly otherwise. Call it **after** `catsync_setMode`
`catsync_getMute()`	`true` when muted
`catsync_setMute(on)`	Mute or unmute, through the page's mute button

The last four arrived with 4.1.0, so test before calling:

```
if (window.catsync_ready) {
  window.catsync_setFrequency(7074000)
  window.catsync_setMode('USB')
  if (typeof window.catsync_setBandwidth === 'function') window.catsync_setBandwidth(2400)
}
```

Setting the frequency retunes audio, so call a setter only when the value has actually changed — polling a setter with the same value is audible. The older KiwiSDR/WebSDR-style entry points (`setfreq`, `set_mode`, `freqset_complete`) are still there for tools that expect them.

Troubleshooting

Symptom	Likely cause	What to do
Could not open ttyUSB0 (Linux)	Not in the `dialout` group, or another program has the port	`sudo usermod -aG dialout $USER`, log out and in; close WSJT-X, loggers, rig utilities
The receiver readout says *not a receiver this app can drive*	Another kind of web receiver, or the page is still loading	Supported are PhantomSDR-Plus, KiwiSDR, PA3FWM WebSDR and UberSDR; give a slow page a few seconds
The rig did not answer	Wrong speed, wrong rig type, wrong CI-V address, rig off	Match the speed to the rig's menu; check the CI-V address; try Hamlib
Hamlib is not installed	No `rigctld` found	Linux: `sudo apt install libhamlib-utils`. Or set **rigctld program** to its path
rigctld stopped: ...	Hamlib could not open the rig — its own message follows	Usually the port or speed; the text after the colon is Hamlib's reason
flrig is not running at ...	flrig closed, or its XML-RPC port differs	Start flrig; check its port in flrig's configuration
Connected, but the receiver does not move	No station window open, or **Receiver window** pinned to a station that is closed	Open the station, or choose *The station window last in front*
The rig transmits when connecting	DTR or RTS keys the rig through your interface	Untick **DTR on** and **RTS on**
Filter does not follow	Receiver older than 4.1.0, or a built-in Kenwood/Yaesu driver	Frequency and mode still sync; use Hamlib for the filter on Kenwood/Yaesu
Mute on transmit does nothing	Receiver without the update, or the rig does not report transmit state	As above
Lost the rig ... reconnecting	The cable was pulled, the rig was switched off, or rigctld died	Nothing — it retries every 3 seconds and carries on when the rig is back
The two sides keep jumping	Two programs are controlling the rig at once	Let only one program set the rig, or share it through flrig

Known limitations

- One rig, one receiver window at a time.
- Receivers other than PhantomSDR-Plus, KiwiSDR, PA3FWM WebSDR and UberSDR — OpenWebRX, for example — are not supported.
- The built-in drivers follow the manufacturers' published protocols and were tested against simulated rigs and real Hamlib; for a rig that behaves differently, Hamlib is the fallback.
- Split operation, VFO B, RIT/XIT and memory channels are not synced — only the frequency of the active VFO.
- TCI-CAT on the receiver page does not sync split, VFO B, RIT/XIT or transverter offsets, and was tried only on an IC-7300 through the Hamlib bridge.
- On Linux the packages use the distribution's Hamlib; none is bundled.

CHAPTER 17

Project Structure

PROJECT_STRUCTURE.md

PhantomSDR-Plus Project Structure

This document provides a comprehensive overview of the PhantomSDR-Plus directory structure, file organization, and component relationships.

Table of Contents

1. Directory Tree
2. Root Directory
3. Source Code (`src/`)
4. Frontend (`frontend/`)
5. Frequency Lists (`frequencylist/`)
6. Configuration Files
7. Build System

Directory Tree

```
PhantomSDR-Plus
├── add-receiver.sh            # adds one more receiver: driver, instances/<name>/, receivers.toml
– see docs/MULTI_RECEIVER.md
├── ADDITIONAL_TERMS.md        # GPL v3 section 7 additional terms: the "original from" line must
stay
├── admin_config.json
├── admin_server.py
├── autorun
│   ├── audiotap.js
│   ├── bandplan.js
│   ├── decodeworker.js
│   ├── drm-helper.js          # DRM for listeners: one ubersdr-drm per session, on an IQ tap
│   ├── drmheard.js            # DRM services decoded here ("heard here"), merged into the schedule
│   ├── drmschedule.js         # DRM schedule: drmrx.org + KiwiSDR confirmations + HFCC
│   ├── index.js
│   ├── iqtap.js               # AudioTap in the server's IQ mode
│   ├── manager.js
│   ├── package.json
│   ├── pool.js
│   ├── probe-ft8.js
│   ├── probe-js8.js
│   ├── pskreporter.js
│   ├── serverport.js          # where the taps find spectrumserver (receivers.toml aware)
│   ├── spotparse.js
│   ├── wasm-shim.js
│   └── wsprnet.js
├── cf32_to_real.c             # Fobos HF: keeps I from rx_sdr's CF32, scaled to s16 (built by
setup-fobos.sh)
├── chat_history.txt
├── check-go.sh                # legacy: watchdog for the go.sh chain
├── config-airspyhf.toml
├── config.example.hackrf.toml
├── config.example.rtlsdr.toml
├── config-fobos-hf.toml
├── config-fobos.toml
├── config-hackrf.toml
├── config-rsp1a.toml
├── config-rtl.toml
├── config-rx888mk2.toml
├── config.toml
├── configure-station.sh       # the station wizard: asks once, writes station.conf, the .toml and
site_information.json
├── connection_impl.hpp
├── docs
│   ├── ADMIN_PANEL_SETUP.md
│   ├── Aether_config.md
│   ├── Appendix.md
│   ├── CONNECTION_LIMITS.md
│   ├── de
│   │   ├── ADMIN_PANEL_SETUP.md
│   │   ├── Aether_config.md
│   │   ├── Appendix.md
│   │   ├── CONNECTION_LIMITS.md
│   │   ├── DECODERS.md
│   │   ├── EDITING_VARIANTS.md
│   │   ├── HTTPS.md
│   │   ├── INSTALLATION.md
│   │   ├── MULTI_RECEIVER.md
│   │   ├── PhantomSDR-Plus-Documentation-DE.pdf
│   │   ├── PROJECT_STRUCTURE.md
│   │   ├── RADE_General_INSTALL_MANUAL_LINUX.md
│   │   ├── RADE_README.md
│   │   ├── README.md
│   │   ├── RECEIVE_DIVERSITY.md
```

```
│   │   ├── RIG_CONTROL.md
│   │   ├── THERMAL_GUARD.md
│   │   └── USER_GUIDE.md
│   ├── DECODERS.md
│   ├── EDITING_VARIANTS.md
│   ├── el
│   │   ├── ADMIN_PANEL_SETUP.md
│   │   ├── Aether_config.md
│   │   ├── Appendix.md
│   │   ├── CONNECTION_LIMITS.md
│   │   ├── DECODERS.md
│   │   ├── EDITING_VARIANTS.md
│   │   ├── HTTPS.md
│   │   ├── INSTALLATION.md
│   │   ├── MULTI_RECEIVER.md
│   │   ├── PhantomSDR-Plus-Documentation-EL.pdf
│   │   ├── PROJECT_STRUCTURE.md
│   │   ├── RADE_General_INSTALL_MANUAL_LINUX.md
│   │   ├── RADE_README.md
│   │   ├── README.md
│   │   ├── RECEIVE_DIVERSITY.md
│   │   ├── RIG_CONTROL.md
│   │   ├── THERMAL_GUARD.md
│   │   └── USER_GUIDE.md
│   ├── es
│   │   ├── ADMIN_PANEL_SETUP.md
│   │   ├── Aether_config.md
│   │   ├── Appendix.md
│   │   ├── CONNECTION_LIMITS.md
│   │   ├── DECODERS.md
│   │   ├── EDITING_VARIANTS.md
│   │   ├── HTTPS.md
│   │   ├── INSTALLATION.md
│   │   ├── MULTI_RECEIVER.md
│   │   ├── PhantomSDR-Plus-Documentation-ES.pdf
│   │   ├── PROJECT_STRUCTURE.md
│   │   ├── RADE_General_INSTALL_MANUAL_LINUX.md
│   │   ├── RADE_README.md
│   │   ├── README.md
│   │   ├── RECEIVE_DIVERSITY.md
│   │   ├── RIG_CONTROL.md
│   │   ├── THERMAL_GUARD.md
│   │   └── USER_GUIDE.md
│   ├── fr
│   │   ├── ADMIN_PANEL_SETUP.md
│   │   ├── Aether_config.md
│   │   ├── Appendix.md
│   │   ├── CONNECTION_LIMITS.md
│   │   ├── DECODERS.md
│   │   ├── EDITING_VARIANTS.md
│   │   ├── HTTPS.md
│   │   ├── INSTALLATION.md
│   │   ├── MULTI_RECEIVER.md
│   │   ├── PhantomSDR-Plus-Documentation-FR.pdf
│   │   ├── PROJECT_STRUCTURE.md
│   │   ├── RADE_General_INSTALL_MANUAL_LINUX.md
│   │   ├── RADE_README.md
│   │   ├── README.md
│   │   ├── RECEIVE_DIVERSITY.md
│   │   ├── RIG_CONTROL.md
│   │   ├── THERMAL_GUARD.md
│   │   └── USER_GUIDE.md
│   ├── hr
│   │   ├── ADMIN_PANEL_SETUP.md
│   │   ├── Aether_config.md
```

```
│   │   ├── Appendix.md
│   │   ├── CONNECTION_LIMITS.md
│   │   ├── DECODERS.md
│   │   ├── EDITING_VARIANTS.md
│   │   ├── HTTPS.md
│   │   ├── INSTALLATION.md
│   │   ├── MULTI_RECEIVER.md
│   │   ├── PhantomSDR-Plus-Documentation-HR.pdf
│   │   ├── PROJECT_STRUCTURE.md
│   │   ├── RADE_General_INSTALL_MANUAL_LINUX.md
│   │   ├── RADE_README.md
│   │   ├── README.md
│   │   ├── RECEIVE_DIVERSITY.md
│   │   ├── RIG_CONTROL.md
│   │   ├── THERMAL_GUARD.md
│   │   └── USER_GUIDE.md
│   ├── HTTPS.md
│   ├── INSTALLATION.md
│   ├── MULTI_RECEIVER.md
│   ├── PhantomSDR-Plus-Documentation-EN.pdf
│   ├── PROJECT_STRUCTURE.md
│   ├── RADE_General_INSTALL_MANUAL_LINUX.md
│   ├── RADE_README.md
│   ├── README.md
│   ├── RECEIVE_DIVERSITY.md
│   ├── RIG_CONTROL.md
│   ├── ru
│   │   ├── ADMIN_PANEL_SETUP.md
│   │   ├── Aether_config.md
│   │   ├── Appendix.md
│   │   ├── CONNECTION_LIMITS.md
│   │   ├── DECODERS.md
│   │   ├── EDITING_VARIANTS.md
│   │   ├── HTTPS.md
│   │   ├── INSTALLATION.md
│   │   ├── MULTI_RECEIVER.md
│   │   ├── PhantomSDR-Plus-Documentation-RU.pdf
│   │   ├── PROJECT_STRUCTURE.md
│   │   ├── RADE_General_INSTALL_MANUAL_LINUX.md
│   │   ├── RADE_README.md
│   │   ├── README.md
│   │   ├── RECEIVE_DIVERSITY.md
│   │   ├── RIG_CONTROL.md
│   │   ├── THERMAL_GUARD.md
│   │   └── USER_GUIDE.md
│   ├── sdr-stats
│   │   ├── package.json
│   │   ├── readme_de.md
│   │   ├── readme_el.md
│   │   ├── readme_es.md
│   │   ├── readme_fr.md
│   │   ├── readme_hr.md
│   │   ├── README.md
│   │   └── readme_ru.md
│   ├── THERMAL_GUARD.md
│   ├── USER_GUIDE.md
│   ├── websdr2.png
│   ├── websdr3.png
│   └── websdr.png
├── drm                         # built by setup-drm.sh: src/ and bin/ (not in git)
├── favicon.ico
├── fftw_wisdom
├── fix_local_geo.py
├── frequencylist
│   ├── 0.TXT
```

```
│   ├── A26all00.TXT          # not in the repo: unzipped from a26allx2.zip by update-markers.sh
│   ├── a26allx2.zip
│   ├── admin.txt
│   ├── antenna.txt
│   ├── broadcas.txt
│   ├── curl-output.txt
│   ├── currentUpdateFile.txt
│   ├── eibi-README.TXT  # not in the repo: EiBi's code tables, fetched by update-eibi.sh
│   ├── fmorg.txt
│   ├── generate-current-shortwave.py
│   ├── generate-eibi.py
│   ├── language.txt
│   ├── mymarkers.json
│   ├── README.md
│   ├── shortwavestations.json
│   ├── site.txt
│   ├── sked-b26.csv  # not in the repo: the EiBi season schedule, fetched by update-eibi.sh
│   ├── update-eibi.sh
│   └── update-markers.sh
├── frontend
│   ├── build-all.sh
│   ├── build-default.sh
│   ├── build-mobile.sh
│   ├── debug-title.sh
│   ├── favicon.ico
│   ├── fix-title-python.py
│   ├── index.html
│   ├── jsconfig.json
│   ├── LICENSE
│   ├── make-redirect-stubs.sh
│   ├── mobile
│   │   └── index.html
│   ├── package.json
│   ├── package-lock.json
│   ├── pnpm-lock.yaml
│   ├── postcss.config.cjs
│   ├── .prettierrc.json      # Prettier formatting rules for the frontend sources
│   ├── public
│   │   ├── analyze_users.py
│   │   ├── decoders
│   │   │   ├── ft8_lib.wasm
│   │   │   ├── hfdl_systable.conf # HFDL ground stations (dumphfdl system table)
│   │   │   ├── hfdl.wasm      # HFDL decoder, built from jsdsp/hfdl_wasm
│   │   │   ├── js8_dict.bin   # JS8 word dictionary, fetched only when needed
│   │   │   └── js8.wasm       # JS8 decoder, built from jsdsp/js8_wasm
│   │   ├── eibi.json  # not in the repo: the EiBi schedule, written by frequencylist/update-
eibi.sh
│   │   ├── logo.jpg
│   │   ├── stats.html
│   │   ├── users.html
│   │   └── wf-message.json
│   ├── README.md
│   ├── site_information.example.json # the generic template, safe to ship; copy it to
site_information.json
│   ├── site_information.json
│   ├── src
│   │   ├── app.css
│   │   ├── App.svelte
│   │   ├── assets
│   │   │   ├── amateurfrequencies.json
│   │   │   ├── background.jpg
│   │   │   ├── shortwavestations.json
│   │   │   ├── SSTV.png
│   │   │   ├── SSTV.svg
│   │   │   └── svelte.png
```

```
│   │   ├── ale.js                          # ALE 2G (FLDIGI, SERVICES) — gr-ale / LinuxALE
│   │   ├── audio.js
│   │   ├── audio-stream-worklet.js
│   │   ├── ax25.js
│   │   ├── bands-config.js
│   │   ├── broadcastSchedules.js
│   │   ├── clientVersion.js
│   │   ├── cwDecoder.js
│   │   ├── cwBrowser.js                    # CW Browser (CW, all signals)
│   │   ├── cw.worker.js
│   │   ├── cwWorkerProxy.js
│   │   ├── decoder.worker.js
│   │   ├── diversity.js
│   │   ├── diversityList.js
│   │   ├── dsc.js                          # DSC + HF Selcall (SERVICES)
│   │   ├── eventBus.js
│   │   ├── events.js
│   │   ├── fax.js
│   │   ├── fax.worker.js
│   │   ├── faxWorkerProxy.js
│   │   ├── fft.js
│   │   ├── flacLowLatency.js         # FLAC decoder that plays each packet as it arrives
│   │   ├── fldigiFec.js              # Viterbi decoder and helpers shared by the fldigi-family
decoders
│   │   ├── fsk.js
│   │   ├── fsk.worker.js
│   │   ├── fskWorkerProxy.js
│   │   ├── fst4w.js                        # FST4W (FLDIGI) — WSJT-X
│   │   ├── hell.js
│   │   ├── hfdl.worker.js
│   │   ├── hfdlWorkerProxy.js
│   │   ├── ifk.js                    # DominoEX and THOR
│   │   ├── kiwiSource.js
│   │   ├── lib
│   │   │   ├── AlePanel.svelte             # ALE 2G (FLDIGI, SERVICES)
│   │   │   ├── aleNets.js                  # ALE: KiwiSDR ALE_nets + HFLINK
│   │   │   ├── aprsMap.js
│   │   │   ├── backend.js
│   │   │   ├── BandSelector.svelte
│   │   │   ├── catsync.js
│   │   │   ├── chatReply.js               # chat reply marker and threading
│   │   │   ├── CheckButton.svelte
│   │   │   ├── colormaps.js
│   │   │   ├── Counter.svelte
│   │   │   ├── DiversityPanel.svelte
│   │   │   ├── DrmPanel.svelte # DRM panel: lamps, figures, station, schedule selector
│   │   │   ├── DscPanel.svelte             # DSC (SERVICES)
│   │   │   ├── drmSchedule.js # DRM schedule snapshot + day-aware "on air now"
│   │   │   ├── fftRadix2.js
│   │   │   ├── fftMixed.js                 # FFT, any N (FST4W)
│   │   │   ├── fst4wTables.js              # LDPC (240,74) — WSJT-X
│   │   │   ├── freedv-reporter.js
│   │   │   ├── FreeDVReporter.svelte
│   │   │   ├── FreqKeypad.svelte      # number pad for typing a frequency
│   │   │   ├── FrequencyInput.svelte
│   │   │   ├── FrequencyMarkers.svelte
│   │   │   ├── FtxSpectrum.svelte
│   │   │   ├── hammeractions.js
│   │   │   ├── HfdlPanel.svelte # HFDL panel and aircraft map (from PhantomSDR-ORSAT)
│   │   │   ├── hfdlStations.js # HFDL ground stations and frequencies
│   │   │   ├── identity.js                # listener name/callsign: box, storage, URL carry-over
│   │   │   ├── LineThroughButton.svelte
│   │   │   ├── Logger.svelte
│   │   │   ├── MagicEyeIndicator.svelte
│   │   │   ├── ModeIdChip.svelte
```

```
│   │   │   ├── ModesSelector.svelte
│   │   │   ├── msk144Tables.js            # LDPC (128,90) — WSJT-X
│   │   │   ├── opusMlDecoder.js
│   │   │   ├── pack77.js                  # WSJT-X 77-bit messages (FST4W, MSK144)
│   │   │   ├── PassbandTuner.svelte
│   │   │   ├── Popover.svelte
│   │   │   ├── QrssPanel.svelte
│   │   │   ├── receivers.js           # receiver picker data (/receivers.json), ranges, links
between receivers
│   │   │   ├── rnnoise.js     # loads RNNoise for the AI noise reduction
│   │   │   ├── rx.js                  # which receiver a page belongs to (?rx=)
│   │   │   ├── RxStrip.svelte         # “Show All”: live strip of another receiver
│   │   │   ├── SelcallPanel.svelte        # HF Selcall (SERVICES)
│   │   │   ├── ServiceSelect.svelte       # SERVICES: Service
│   │   │   ├── sideService.js         # where the page finds RADE and the relay (siteRade /
siteRelay)
│   │   │   ├── SMeterAnalog.svelte
│   │   │   ├── SMeterDigital.svelte
│   │   │   ├── Spectrogram.svelte     # audio analyser (Show)
│   │   │   ├── SsbReport.svelte       # SSB audio report under the analyser
│   │   │   ├── AmFmReport.svelte      # AM/FM report under the analyser
│   │   │   ├── StatusIndicators.svelte
│   │   │   ├── SwSchedule.svelte          # the EiBi window
│   │   │   ├── TimeSignalPanel.svelte     # Time Signals (SERVICES)
│   │   │   ├── storage.js
│   │   │   ├── sUnits.js              # S-unit scale by frequency (S9 = −93 dBm above 30 MHz)
│   │   │   ├── Tooltip.svelte
│   │   │   ├── tzLookup.js                # tz-lookup 6.1.25 (CC0)
│   │   │   ├── version.js             # VERSION + ORIGIN_URL for the receiver page and /mobile
│   │   │   ├── VersionSelector.svelte
│   │   │   ├── VideoAreaSelector.svelte
│   │   │   ├── volume.js              # volume slider curve, start volume, remembered setting
│   │   │   ├── Waterfall3D.svelte     # 3D spectrum view (WebGL2)
│   │   │   ├── wfAutoLevel.js         # Auto Adj.: waterfall brightness and contrast
│   │   │   └── wrappers.js
│   │   ├── main.js
│   │   ├── mobile
│   │   │   ├── backend.js
│   │   │   ├── bookmarks.js
│   │   │   ├── main.js
│   │   │   ├── Mobile.svelte
│   │   │   └── tuning.js
│   │   ├── modeId.js
│   │   ├── modeId.worker.js
│   │   ├── modeIdWorkerProxy.js
│   │   ├── modePriors.js
│   │   ├── modules
│   │   │   ├── decode.wasm
│   │   │   ├── encode.wasm
│   │   │   ├── ft4.js
│   │   │   ├── ft8.js
│   │   │   ├── hfdl.mjs
│   │   │   ├── js8-decoder.js
│   │   │   ├── js8-format.js
│   │   │   ├── js8.js
│   │   │   ├── js8-reassembler.js
│   │   │   ├── js8-slots.js
│   │   │   ├── js8-tables.js
│   │   │   ├── package.json
│   │   │   ├── phantomsdrdsp_bg_fallback.js
│   │   │   ├── phantomsdrdsp_bg.js
│   │   │   ├── phantomsdrdsp_bg.wasm
│   │   │   ├── phantomsdrdsp_bg.wasm.d.ts
│   │   │   ├── phantomsdrdsp.d.ts
│   │   │   ├── phantomsdrdsp.js
```

```
│   │   │   ├── phantomsdrdsp_router.js
│   │   │   └── wspr.js
│   │   ├── mfsk.js
│   │   ├── msk144.js                    # MSK144 (FLDIGI) — WSJT-X
│   │   ├── mt63.js
│   │   ├── olivia.js
│   │   ├── oliviaBrowser.js             # Olivia Browser (Olivia / Contestia)
│   │   ├── playoutControl.js         # measures the connection, keeps the playback buffer small
│   │   ├── psk31.js
│   │   ├── pskBrowser.js                # PSK Browser (BPSK31)
│   │   ├── refused.js
│   │   ├── remoteSource.js
│   │   ├── scanner.js
│   │   ├── services.worker.js           # SERVICES: DSC, Time Signals, HF Selcall
│   │   ├── servicesWorkerProxy.js       # SERVICES worker proxy
│   │   ├── siteInfo.js               # site_information.json, overlaid with the receiver's own
copy
│   │   ├── sstv.js
│   │   ├── sstv.worker.js
│   │   ├── sstvWorkerProxy.js
│   │   ├── timesig.js                   # Time Signals: DCF77, MSF, RWM, WWV/WWVH
│   │   ├── throb.js
│   │   ├── uberSource.js
│   │   ├── unused
│   │   │   ├── AudioProcessor.js
│   │   │   ├── decoder.js
│   │   │   ├── decoding.js
│   │   │   ├── modules-emscripten
│   │   │   │   ├── dav1d.js
│   │   │   │   ├── dav1dnoWasm.js
│   │   │   │   ├── dav1dnoWasm.js.mem
│   │   │   │   ├── dav1d.wasm
│   │   │   │   ├── decode_ft8.js
│   │   │   │   ├── decode_ft8.wasm
│   │   │   │   ├── FoxenFlac.js
│   │   │   │   ├── jsDSP.js
│   │   │   │   ├── jsDSPnoWasm.js
│   │   │   │   ├── jsDSPnoWasm.js.mem
│   │   │   │   ├── jsDSPnoWasm.wasm
│   │   │   │   ├── jsDSP.wasm
│   │   │   │   ├── libzstd.js
│   │   │   │   ├── LiquidDSP.js
│   │   │   │   ├── opus.js
│   │   │   │   ├── opusnoWasm.js
│   │   │   │   ├── opusnoWasm.js.mem
│   │   │   │   ├── opus.wasm
│   │   │   │   ├── redsea.js
│   │   │   │   └── redsea.wasm
│   │   │   ├── unused.js
│   │   │   └── wrappers.js
│   │   ├── videoRecorder.js
│   │   ├── vite-env.d.ts
│   │   ├── waterfall.js
│   │   ├── webSdrCodec.js
│   │   └── webSdrSource.js
│   ├── stats.html
│   ├── svelte.config.js
│   ├── tailwind.config.cjs
│   └── vite.config.js
├── go.sh                        # legacy launcher chain, superseded by start-<radio>.sh
├── install_arch.sh
├── install_fedora.sh
├── install_opensuse.sh
├── install_rade.sh
├── install_rade_ubuntu22.sh
```

```
├── install.sh
├── install-stats-server.sh
├── instances                  # extra receivers, one folder each (not in git) — see docs/
MULTI_RECEIVER.md
├── instructions-for-airspy
├── instructions-for-rsp1a
├── jsdsp
│   ├── compilejs.sh
│   ├── configureredsea.sh
│   ├── extract_EXPORTED_FUNCTIONS.js
│   ├── ft8_wasm
│   │   ├── build_ft8_wasm.sh
│   │   ├── README.md
│   │   └── wasm_wrapper.c
│   ├── include
│   │   ├── avif
│   │   │   ├── avif.h
│   │   │   └── internal.h
│   │   └── liquid
│   │       └── liquid.h
│   ├── hfdl_wasm              # HFDL decoder: dumphfdl + libacars in WebAssembly — see its README
│   ├── js8_wasm               # JS8 decoder in C, built to WebAssembly, with its tests and
reference harness — see its README
│   │   ├── build_js8_wasm.sh
│   │   ├── check_against_reference.sh
│   │   ├── extract_js8_constants.py
│   │   ├── extract_js8_varicode.py
│   │   ├── js8_ab_test.mjs
│   │   ├── js8_autorun_test.mjs
│   │   ├── js8_chain_test.mjs
│   │   ├── js8_constants.c
│   │   ├── js8_constants.h
│   │   ├── js8_decode.c
│   │   ├── js8_decode.h
│   │   ├── js8_decode_test.c
│   │   ├── js8_encode.c
│   │   ├── js8_encode.h
│   │   ├── js8_frames_test.mjs
│   │   ├── js8_osd.c
│   │   ├── js8_osd.h
│   │   ├── js8_reassembly_test.mjs
│   │   ├── js8_roundtrip.c
│   │   ├── js8_slots_test.mjs
│   │   ├── js8_subtract.c
│   │   ├── js8_subtract.h
│   │   ├── js8_wasm_test.mjs
│   │   ├── js8_wasm_wrapper.c
│   │   ├── README.md
│   │   ├── reference
│   │   │   ├── build_in_container.sh
│   │   │   ├── build_reference_decoder.sh
│   │   │   ├── build_reference_frames.sh
│   │   │   ├── Containerfile
│   │   │   ├── Containerfile.decoder
│   │   │   ├── ref_decode.f90
│   │   │   ├── ref_decode.txt
│   │   │   ├── ref_frames.cpp
│   │   │   └── ref_frames.txt
│   │   ├── ref_gen.f90
│   │   └── run_tests.sh
│   ├── lib
│   │   ├── ANR.c
│   │   ├── arm_funcs.h
│   │   ├── CMSIS_DSP
│   │   │   ├── BUILDING.txt
```

```
│   │   │   └── LICENSE.txt
│   │   ├── dav1d.cpp
│   │   ├── NB.c
│   │   ├── NR_spectral.c
│   │   └── types.h
│   ├── redsea.js
│   ├── redsea.wasm
│   └── src
│       ├── index.js
│       ├── libzstd.js
│       ├── LiquidDSP.js
│       ├── NoiseProcessing.js
│       └── wbfmpll.cpp
├── kill.sh                     # legacy: kills the server processes, called by go.sh
├── kiwi_install.sh             # installs the KiwiSDR bridge into an existing tree
├── LICENSE
├── logproxy                    # rotated copies of the panel / proxy / autorun logs
├── logrotate
│   └── phantomsdr              # logrotate config for proxy.log + admin.log (edit paths before
installing)
├── manage_admin.sh
├── markers.json
├── meson.build
├── meson_options.txt
├── phantom_fftw_wisdom
├── phantomsdr-admin.service    # sample systemd unit for the admin panel (starts at boot, restarts
after a crash)
├── phantomsdr-proxy.service    # sample systemd unit for the proxy, installed alongside the panel
unit
├── phantomsdr-websdr-relay.service  # sample systemd unit for the WebSDR diversity relay
├── proxy.py                    # the one public port: the page, /admin, /rade, /stats, /relay,
and ?rx= receivers
├── rade_helper.py              # the RADE sidecar: V1 through ~/radae, V2 (beta) through ~/rade_c
├── rade_loadtest.csv           # output of rade_loadtest.py (one row per ramp step)
├── rade_loadtest.py            # RADE concurrency knee test — see docs/RADE_README.md
├── rade.sh
├── README.md
├── recompile.sh                # rebuild; --backend / --frontend / --both ask nothing
├── receivers.toml.example      # receiver list for proxy.py — copy to receivers.toml
├── _relaunch.sh                # legacy: delayed re-launch helper for the go.sh chain
├── request.hpp
├── setup_admin.sh
├── setup-airspyhf.sh           # Airspy HF+ driver chain: libairspyhf + SoapyAirspyHF + rx_sdr +
udev
├── setup-autostart.sh          # start at boot: phantomsdr-receiver.service for the start script in
station.conf
├── setup-cpufreq-perms.sh      # grants group write on scaling_max_freq so the guard can throttle
without root
├── setup-drm.sh                # builds the DRM decoder (ubersdr-drm + fdk-aac 2.0.3) into drm/bin
├── setup-firewall.sh           # optional nftables flood guard — see docs/CONNECTION_LIMITS.md
├── setup-fobos.sh              # Fobos driver chain: libfobos + SoapyFobosSDR + rx_sdr +
cf32_to_real + udev
├── setup-hackrf.sh             # HackRF: the distribution's hackrf package + udev (nothing built)
├── setup-https.sh              # https:// in front: Caddy on 443 with a Let's Encrypt certificate —
see docs/HTTPS.md
├── setup-rsp1a.sh              # RSP1A driver chain: libmirisdr-5 + SoapyMiri + rx_sdr + msi2500
blacklist + udev
├── setup-rtlsdr.sh             # RTL-SDR driver: the distribution's package, or the Blog V4 driver
from source + DVB blacklist + udev
├── setup-rx888-udev.sh
├── setup-sdr-common.sh         # shared helpers sourced by the three setup scripts above
├── setup_websdr_relay.sh       # installs the WebSDR diversity relay (port, identity, systemd)
├── smeter_theme.sh
├── src
│   ├── audio.cpp
```

```
│   ├── audio.h
│   ├── chat.cpp
│   ├── chat.h
│   ├── client.cpp
│   ├── client.h
│   ├── compression.cpp
│   ├── compression.h
│   ├── crash_handler.cpp
│   ├── crash_handler.h
│   ├── events.cpp
│   ├── events.h
│   ├── fft.cpp
│   ├── fft_cuda.cu
│   ├── fft.h
│   ├── fft_impl.cpp
│   ├── fft_mkl.cpp
│   ├── http.cpp
│   ├── kiwi_bridge.h          # KiwiSDR protocol bridge — see docs/Aether_config.md
│   ├── listing
│   │   ├── software_info.cpp
│   │   └── software_info.h
│   ├── samplereader.cpp
│   ├── samplereader.h
│   ├── signal.cpp
│   ├── signal.h
│   ├── spectrumserver.cpp
│   ├── spectrumserver.h
│   ├── utils
│   │   ├── audioprocessing.cpp
│   │   ├── audioprocessing.h
│   │   ├── dsp.cpp
│   │   └── dsp.h
│   ├── utils.cpp
│   ├── utils.h
│   ├── waterfallcompression.cpp
│   ├── waterfallcompression.h
│   ├── waterfall.cpp
│   ├── waterfall.h
│   ├── websocket.cpp
│   └── websocket.h
├── start-airspyhf.sh
├── start-all.sh               # starts every receiver in receivers.toml
├── start-fobos-hf.sh
├── start-fobos.sh
├── start-hackrf.sh
├── start-rsp1a.sh
├── start-rtl.sh
├── start-rx888mk2.sh
├── station.conf               # this station's answers to the wizard (not in git, never touched by
update.sh)
├── main.env                   # the main receiver's gain, set in the admin panel (not in git,
never touched by update.sh)
├── stop-websdr.sh
├── subprojects
│   ├── fftw3.wrap
│   ├── flac.wrap
│   ├── glaze.wrap
│   ├── libcds.wrap
│   ├── libflac.wrap
│   ├── libvolk.wrap
│   ├── ogg.wrap
│   ├── opus.wrap
│   ├── tomlplusplus-3.4.0
│   │   ├── CHANGELOG.md
│   │   ├── cmake
```

```
│   │   │   ├── install-rules.cmake
│   │   │   ├── project-is-top-level.cmake
│   │   │   ├── tomlplusplusConfig.cmake
│   │   │   ├── tomlplusplusConfig.cmake.meson.in
│   │   │   ├── tomlplusplusConfigVersion.cmake.meson.in
│   │   │   └── variables.cmake
│   │   ├── CMakeLists.txt
│   │   ├── CODE_OF_CONDUCT.md
│   │   ├── CONTRIBUTING.md
│   │   ├── cpp.hint
│   │   ├── docs
│   │   │   ├── images
│   │   │   │   ├── badge-awesome.svg
│   │   │   │   ├── badge-C++17.svg
│   │   │   │   ├── badge-gitter.svg
│   │   │   │   ├── badge-license-MIT.svg
│   │   │   │   ├── badge-TOML.svg
│   │   │   │   ├── banner.ai
│   │   │   │   ├── banner.png
│   │   │   │   ├── banner.svg
│   │   │   │   ├── favicon.ico
│   │   │   │   ├── logo.ai
│   │   │   │   └── logo.svg
│   │   │   ├── pages
│   │   │   │   └── main_page.md
│   │   │   └── poxy.toml
│   │   ├── examples
│   │   │   ├── benchmark_data.toml
│   │   │   ├── CMakeLists.txt
│   │   │   ├── error_printer.cpp
│   │   │   ├── error_printer.vcxproj
│   │   │   ├── examples.hpp
│   │   │   ├── example.toml
│   │   │   ├── merge_base.toml
│   │   │   ├── merge_overrides.toml
│   │   │   ├── meson.build
│   │   │   ├── parse_benchmark.cpp
│   │   │   ├── parse_benchmark.vcxproj
│   │   │   ├── simple_parser.cpp
│   │   │   ├── simple_parser.vcxproj
│   │   │   ├── toml_generator.cpp
│   │   │   ├── toml_generator.vcxproj
│   │   │   ├── toml_merger.cpp
│   │   │   ├── toml_merger.vcxproj
│   │   │   ├── toml_to_json_transcoder.cpp
│   │   │   └── toml_to_json_transcoder.vcxproj
│   │   ├── include
│   │   │   ├── meson.build
│   │   │   └── toml++
│   │   │       ├── impl
│   │   │       │   ├── array.hpp
│   │   │       │   ├── array.inl
│   │   │       │   ├── at_path.hpp
│   │   │       │   ├── at_path.inl
│   │   │       │   ├── date_time.hpp
│   │   │       │   ├── formatter.hpp
│   │   │       │   ├── formatter.inl
│   │   │       │   ├── forward_declarations.hpp
│   │   │       │   ├── header_end.hpp
│   │   │       │   ├── header_start.hpp
│   │   │       │   ├── json_formatter.hpp
│   │   │       │   ├── json_formatter.inl
│   │   │       │   ├── key.hpp
│   │   │       │   ├── make_node.hpp
│   │   │       │   ├── node.hpp
```

```
│   │   │       │   ├── node.inl
│   │   │       │   ├── node_view.hpp
│   │   │       │   ├── parse_error.hpp
│   │   │       │   ├── parse_result.hpp
│   │   │       │   ├── parser.hpp
│   │   │       │   ├── parser.inl
│   │   │       │   ├── path.hpp
│   │   │       │   ├── path.inl
│   │   │       │   ├── preprocessor.hpp
│   │   │       │   ├── print_to_stream.hpp
│   │   │       │   ├── print_to_stream.inl
│   │   │       │   ├── simd.hpp
│   │   │       │   ├── source_region.hpp
│   │   │       │   ├── std_except.hpp
│   │   │       │   ├── std_initializer_list.hpp
│   │   │       │   ├── std_map.hpp
│   │   │       │   ├── std_new.hpp
│   │   │       │   ├── std_optional.hpp
│   │   │       │   ├── std_string.hpp
│   │   │       │   ├── std_string.inl
│   │   │       │   ├── std_utility.hpp
│   │   │       │   ├── std_variant.hpp
│   │   │       │   ├── std_vector.hpp
│   │   │       │   ├── table.hpp
│   │   │       │   ├── table.inl
│   │   │       │   ├── toml_formatter.hpp
│   │   │       │   ├── toml_formatter.inl
│   │   │       │   ├── unicode_autogenerated.hpp
│   │   │       │   ├── unicode.hpp
│   │   │       │   ├── unicode.inl
│   │   │       │   ├── value.hpp
│   │   │       │   ├── version.hpp
│   │   │       │   ├── yaml_formatter.hpp
│   │   │       │   └── yaml_formatter.inl
│   │   │       ├── toml.h
│   │   │       └── toml.hpp
│   │   ├── LICENSE
│   │   ├── meson.build
│   │   ├── meson_options.txt
│   │   ├── README.md
│   │   ├── src
│   │   │   ├── meson.build
│   │   │   └── toml.cpp
│   │   ├── tests
│   │   │   ├── at_path.cpp
│   │   │   ├── conformance_burntsushi_invalid.cpp
│   │   │   ├── conformance_burntsushi_valid.cpp
│   │   │   ├── conformance_iarna_invalid.cpp
│   │   │   ├── conformance_iarna_valid.cpp
│   │   │   ├── cpp.hint
│   │   │   ├── for_each.cpp
│   │   │   ├── formatters.cpp
│   │   │   ├── impl_toml.cpp
│   │   │   ├── leakproof.hpp
│   │   │   ├── lib_catch2.hpp
│   │   │   ├── main.cpp
│   │   │   ├── manipulating_arrays.cpp
│   │   │   ├── manipulating_parse_result.cpp
│   │   │   ├── manipulating_tables.cpp
│   │   │   ├── manipulating_values.cpp
│   │   │   ├── meson.build
│   │   │   ├── odr_test_1.cpp
│   │   │   ├── odr_test_2.cpp
│   │   │   ├── parsing_arrays.cpp
│   │   │   ├── parsing_booleans.cpp
```

```
│   │   │   ├── parsing_comments.cpp
│   │   │   ├── parsing_dates_and_times.cpp
│   │   │   ├── parsing_floats.cpp
│   │   │   ├── parsing_integers.cpp
│   │   │   ├── parsing_key_value_pairs.cpp
│   │   │   ├── parsing_spec_example.cpp
│   │   │   ├── parsing_strings.cpp
│   │   │   ├── parsing_tables.cpp
│   │   │   ├── path.cpp
│   │   │   ├── settings.hpp
│   │   │   ├── tests.cpp
│   │   │   ├── tests.hpp
│   │   │   ├── user_feedback.cpp
│   │   │   ├── using_iterators.cpp
│   │   │   ├── visit.cpp
│   │   │   ├── vs
│   │   │   │   ├── odr_test.vcxproj
│   │   │   │   ├── test_debug_x64_cpplatest_noexcept_unrel.vcxproj
│   │   │   │   ├── test_debug_x64_cpplatest_noexcept.vcxproj
│   │   │   │   ├── test_debug_x64_cpplatest_unrel.vcxproj
│   │   │   │   ├── test_debug_x64_cpplatest.vcxproj
│   │   │   │   ├── test_debug_x64_noexcept_unrel.vcxproj
│   │   │   │   ├── test_debug_x64_noexcept.vcxproj
│   │   │   │   ├── test_debug_x64_unrel.vcxproj
│   │   │   │   ├── test_debug_x64.vcxproj
│   │   │   │   ├── test_debug_x86_cpplatest_noexcept_unrel.vcxproj
│   │   │   │   ├── test_debug_x86_cpplatest_noexcept.vcxproj
│   │   │   │   ├── test_debug_x86_cpplatest_unrel.vcxproj
│   │   │   │   ├── test_debug_x86_cpplatest.vcxproj
│   │   │   │   ├── test_debug_x86_noexcept_unrel.vcxproj
│   │   │   │   ├── test_debug_x86_noexcept.vcxproj
│   │   │   │   ├── test_debug_x86_unrel.vcxproj
│   │   │   │   ├── test_debug_x86.vcxproj
│   │   │   │   ├── test_release_x64_cpplatest_noexcept_unrel.vcxproj
│   │   │   │   ├── test_release_x64_cpplatest_noexcept.vcxproj
│   │   │   │   ├── test_release_x64_cpplatest_unrel.vcxproj
│   │   │   │   ├── test_release_x64_cpplatest.vcxproj
│   │   │   │   ├── test_release_x64_noexcept_unrel.vcxproj
│   │   │   │   ├── test_release_x64_noexcept.vcxproj
│   │   │   │   ├── test_release_x64_unrel.vcxproj
│   │   │   │   ├── test_release_x64.vcxproj
│   │   │   │   ├── test_release_x86_cpplatest_noexcept_unrel.vcxproj
│   │   │   │   ├── test_release_x86_cpplatest_noexcept.vcxproj
│   │   │   │   ├── test_release_x86_cpplatest_unrel.vcxproj
│   │   │   │   ├── test_release_x86_cpplatest.vcxproj
│   │   │   │   ├── test_release_x86_noexcept_unrel.vcxproj
│   │   │   │   ├── test_release_x86_noexcept.vcxproj
│   │   │   │   ├── test_release_x86_unrel.vcxproj
│   │   │   │   └── test_release_x86.vcxproj
│   │   │   └── windows_compat.cpp
│   │   ├── toml++.code-workspace
│   │   ├── toml.hpp
│   │   ├── toml++.natvis
│   │   ├── toml++.props
│   │   ├── toml++.sln
│   │   ├── toml-test
│   │   │   ├── meson.build
│   │   │   ├── README.md
│   │   │   ├── tt_decoder.cpp
│   │   │   ├── tt_decoder.vcxproj
│   │   │   ├── tt_encoder.cpp
│   │   │   ├── tt_encoder.vcxproj
│   │   │   └── tt.hpp
│   │   ├── toml++.vcxproj
│   │   ├── toml++.vcxproj.filters
```

```
│   │   ├── tools
│   │   │   ├── ci_single_header_check.py
│   │   │   ├── clang_format.bat
│   │   │   ├── generate_conformance_tests.py
│   │   │   ├── generate_single_header.bat
│   │   │   ├── generate_single_header.py
│   │   │   ├── generate_windows_test_targets.py
│   │   │   ├── requirements.txt
│   │   │   ├── utils.py
│   │   │   └── version.py
│   │   └── vendor
│   │       ├── catch.hpp
│   │       ├── json.hpp
│   │       └── README.md
│   ├── tomlplusplus.wrap
│   ├── websocketpp.wrap
│   ├── zlib.wrap
│   └── zstd.wrap
├── tci-bridge
│   └── tci-rigctld.mjs        # TCI server for Hamlib rigs (IC-7300…), run on the listener's PC —
see docs/RIG_CONTROL.md
├── thermal_guard.py           # CPU over-temperature guard used by the admin panel (also runs
standalone)
├── thermal-guard.service      # sample systemd unit for the guard, for installs without the admin
panel
├── tmpfiles
│   └── phantomsdr-logs.conf   # keeps admin.log + proxy.log owned by the panel's user (edit paths
before installing)
├── update-known.txt           # fingerprints of every published version — lets update.sh tell old
files from edited ones
├── update.sh                  # updater: report, then "update now?"; --check, --from, --make-known
├── waterfall.sh
├── websdr_relay.json.example  # its config template (port, caps, station identity)
├── websdr_relay.py            # the relay itself — see docs/RECEIVE_DIVERSITY.md
├── websocketpp_asio_connection.hpp  # Boost >= 1.87 shim: ws_post instead of io_service::post
├── websocketpp_asio_endpoint.hpp    # Boost >= 1.87 shim: ws_work / ws_restart,
max_listen_connections
├── websocketpp_asio.hpp        # Boost >= 1.87 shim for websocketpp (io_context,
executor_work_guard)
└── xgo.sh                     # legacy: starts spectrumserver, called by check-go.sh
```

Root Directory

Configuration Files

File	Purpose	When to Modify
`config.toml`	Default configuration	Initial setup, testing
`config-rtl.toml`	RTL-SDR specific config	Using RTL-SDR device
`config-rsp1a.toml`	SDRplay RSP1A config	Using RSP1A device
`config-airspyhf.toml`	Airspy HF+ config	Using Airspy device
`config-fobos-hf.toml`	RigExpert Fobos SDR, HF1/HF2 direct sampling (0-25 MHz)	Using a Fobos on HF
`config-fobos.toml`	RigExpert Fobos SDR, RF path (25-6000 MHz)	Using a Fobos above 25 MHz
`config-hackrf.toml`	HackRF One config	Using a HackRF
`config-rx888mk2.toml`	RX888 MK2 config	Using RX888 device
`config.example.hackrf.toml`	HackRF One example	Using HackRF device
`station.conf`	The station wizard's answers — receiver, band, station, ports, extras. Written by `configure-station.sh`; the start scripts and `proxy.py` read it. Never in git, never touched by `update.sh`	Through `bash configure-station.sh`
`main.env`	The main receiver's own `instance.env`: its argument line (`RX888_ARGS` / `RX_ARGS`) with the gain set in Admin → Receivers → ✎ CHANGE SETTINGS. Read last by every start script; absent until a gain is saved. Never in git, never touched by `update.sh`	Admin → Receivers, or by hand

Start/Stop and Maintenance Scripts

Each `start-*.sh` below is a **self-contained launcher + watchdog + logger**: it stops any running instance, brings up the receiver + `spectrumserver`, detaches into the background, auto-restarts the chain if it dies, and logs to `logwebsdr.txt`. They share one stop script and a single `flock` lock (only one receiver runs at a time). Edit only the **RECEIVER CONFIGURATION** block at the top of each (receiver args / config / process name). The six launchers built from the same template (`start-airspyhf.sh`, `start-rtl.sh`, `start-rsp1a.sh`, `start-fobos.sh`, `start-fobos-hf.sh`, `start-hackrf.sh`) are identical below that block; `start-rx888mk2.sh` is a script of its own. Since v5.0.0 several receivers can run side by side: start a launcher with `INSTANCE=<name>` and that receiver gets its own folder in `instances/<name>/`, its own log, lock and FIFO — see Several Receivers.

Script	Purpose
`install.sh`	Automated installation and build — the station questions first, then everything unattended (also `install_fedora.sh`, `install_arch.sh`, `install_opensuse.sh`)
`configure-station.sh`	The station wizard: receiver, band, station, internet, ports and extras, asked once and saved in `station.conf`; from it writes the receiver's `.toml`, `site_information.json` and the relay settings, changing only the keys it owns and backing each file up first. `--show` prints the answers, `--apply` rewrites the files from them — see Installation Guide
`setup-autostart.sh`	Start at boot: installs `phantomsdr-receiver.service`, which runs the start script from `station.conf` (or the one named) at boot and `stop-websdr.sh` at shutdown, with the `plugdev` group; `--remove` undoes it
`setup-https.sh`	Serves the receiver over https:// as well as http://: installs Caddy on port 443 with a Let's Encrypt certificate for the station's DNS name (renewed by itself) and points it at `proxy.py`'s loopback port `PORT_TLS`, where the proxy takes each visitor's real address from Caddy. `--lan` for the local network with Caddy's own certificate, `--status`, `--remove`; the station questions run it when https is chosen — see Secure Access
`add-receiver.sh`	Adds one more receiver to this computer: asks which receiver and what it covers, installs its driver, creates `instances/<name>/` and the `receivers.toml` entry, and offers to start it and restart the proxy. The installers offer it at the end — see Several Receivers
`start-rtl.sh`	Launch + watchdog server with RTL-SDR (`rtl_sdr`)
`start-rsp1a.sh`	Launch + watchdog server with SDRplay RSP1A (`rx_sdr`); uses libmirisdr-5, or SDRplay's API when that is installed (`RX_DRIVER` forces one)
`start-airspyhf.sh`	Launch + watchdog server with Airspy HF+ (`rx_sdr`)
`start-fobos-hf.sh`	Launch + watchdog server with RigExpert Fobos SDR, HF1/HF2 direct sampling (`rx_sdr | cf32_to_real`)
`start-fobos.sh`	Launch + watchdog server with RigExpert Fobos SDR, RF path (`rx_sdr`)
`start-hackrf.sh`	Launch + watchdog server with HackRF One (`hackrf_transfer`)
`start-rx888mk2.sh`	Launch + watchdog server with RX888 MK2 (`rx888_stream`)
`start-all.sh`	Starts every receiver listed in `receivers.toml` through its own launcher — set it as the admin panel's start script on a station with several receivers; see Several Receivers
`stop-websdr.sh`	Stop the server + its watchdog — shared by all receivers
`stop-websdr.sh <instance>`	Stops one receiver only (`main` for the main one); without an argument every receiver stops, as before
`setup-rx888-udev.sh`	Install udev rules so RX-888 `rx888_stream` runs without sudo
`setup-rsp1a.sh` / `setup-fobos.sh` / `setup-airspyhf.sh`	Install the SoapySDR driver chain for the RSP1A, Fobos or Airspy HF+ — SoapySDR from the distribution, the driver and rx_tools built into `sdr_drivers/`, a udev rule — on any of the four supported distributions. Installer options 3, 5 and 6 run them; they also run on their own. Shared code in `setup-sdr-common.sh` — see Installation Guide
`setup-hackrf.sh`	Installs the distribution's `hackrf` package and a udev rule for the HackRF One, on any of the four supported distributions; installer option 7 runs it
`setup-rtlsdr.sh`	Installs the RTL-SDR driver on any of the four supported distributions — the distribution's `rtl-sdr` package, or with `RTL_V4=y` the RTL-SDR Blog V4 driver built from source — blacklists the DVB-T driver and adds a udev rule; `add-receiver.sh` runs it
`setup-firewall.sh`	Optional kernel-level flood guard: loads an nftables table with a per-source connection ceiling and rate on the receiver ports, an SSH brute-force brake, and Windows file sharing closed outside private ranges. Needs root, cannot lock you out (policy accept,

Script	Purpose
	established accepted first) and `--apply` rolls itself back unless confirmed within 60 s — see Connection Limits
`setup-cpufreq-perms.sh`	Grants a `cpufreq` group write access to the per-CPU frequency limit, so the thermal guard's throttle stage works without running the panel as root. Installs a `tmpfiles.d` rule so it survives a reboot; `--revoke` undoes it
`update.sh`	Update the installation from the published tree, leaving your configuration, markers, frequency list and local edits alone: it reports, asks "Update these files now?" (ENTER = no), backs up, applies and rebuilds only what changed. `--check` for cron, `--from` a ZIP, tarball or folder without network — see Installation Guide
`update-known.txt`	The SHA-256 of every version of every file ever published, shipped with each release, so `update.sh` updates a file that is merely old without asking. Written by `bash update.sh --make-known` in the source tree
`recompile.sh`	Rebuild the backend and/or the frontend, and pick the variant served at `/`. With `--backend`, `--frontend` or `--both` it asks nothing and keeps the current variant — how `install.sh` and `update.sh` call it
`smeter_theme.sh`	Set the default analog S-meter face (dark / amber / vintage) for all users, and offer the frontend rebuild — see Editing Variants
`waterfall.sh`	Change the default minimum waterfall level (dB) in `waterfall.js` + `App.svelte` — see README
`kiwi_install.sh`	Install the KiwiSDR client-emulation bridge into a tree that has not got it: patches the backend sources, copies `src/kiwi_bridge.h`, and adds a documented `[kiwi_emulation]` block to the config files in the repository root. Idempotent, and backs up every file it touches — see KiwiSDR Client Emulation
`tci-bridge/tci-rigctld.mjs`	Not used by the receiver. A small Node.js program a listener runs next to their own transceiver: it reads frequency, mode and transmit state from Hamlib's `rigctld` and serves them as TCI on port 50001, so the page's **TCI-CAT** button can drive a rig with no TCI of its own, such as the IC-7300 — see Rig Control

Legacy launcher chain. `go.sh`, `xgo.sh`, `check-go.sh`, `kill.sh` and `_relaunch.sh` are the previous generation of launcher, watchdog and stop scripts. Everything they did is now inside each `start-<radio>.sh`, which is what you should use. They are kept on disk because existing installations still reference them, and are not maintained.

Data Files

File	Purpose	Format
`markers.json`	Frequency bookmarks and markers	JSON
`chat_history.txt`	User chat messages	Plain text
`favicon.ico`	Website icon	ICO image
`fftw_wisdom`	FFT optimization data	FFTW binary
`phantom_fftw_wisdom`	Additional FFT optimization	FFTW binary

Admin Panel and Spot Reporting

File	Purpose
`admin_server.py`	The admin panel itself. Besides the management pages it runs the **Graphs** sampler: a background thread takes CPU frequency, load, temperature and users online every 2 seconds and keeps them in memory only — 1 hour at full resolution plus 24 hours of 30-second averages. Nothing is written to disk, so the history is lost on restart.
`admin_config.json`	Admin panel settings (password hash, ports, options, thermal guard thresholds)
`proxy.py`	The public port. Serves the page and carries `/admin` to the panel and — on a station set up with the wizard — `/rade`, `/stats` and `/relay` to the services behind it, so only one port is forwarded. Reads its ports from `admin_config.json`, or from `station.conf` when there is no panel (`setup_admin.sh --proxy-only`)
`thermal_guard.py`	CPU over-temperature guard. Stops the server when the CPU gets too hot and restarts it once cool, deriving its thresholds from the critical trip point your own CPU publishes instead of a fixed number. It never identifies what supervises the server — while over-temp it re-issues the stop every 2 s, so a watchdog, systemd unit or cron job that revives it is undone until the machine cools. Stdlib-only; imported by `admin_server.py` (ticked from the Graphs sampler) and also runnable standalone for installations without the panel. Defaults to log-only, so it acts on nothing until enabled — see the Thermal Guard manual
`autorun/`	The spot-reporting daemon — see Autorun Spot Reporter
`autorun.json`	Which bands/modes to decode, identity and destinations
`autorun-status.json`	Live state of the running daemon — feeds the **per-run** spot counters (the per-decoder tiles), which reset on Stop/Start
`autorun-totals.json`	**All-time** uploaded spots per band+mode — the number beside each checkbox; written by the daemon so the counts survive restarts

Build System Files

File	Purpose
`meson.build`	Main build configuration
`meson_options.txt`	Configurable build options
`.gitattributes`	Git repository attributes

Source Code (`src/`)

The `src/` directory contains the C++ backend implementation.

Key Components

1. Main Application (`main.cpp`)

- Parses command-line arguments
- Loads configuration file
- Initializes server components
- Starts event loop

2. Spectrum Server (`spectrumserver.cpp`)

- Coordinates all components

- Manages user connections
- Distributes spectrum data
- Handles user requests

3. SDR Drivers (`drivers/`)

- Abstract interface for SDR hardware
- Read and format sample data
- Handle device-specific features

4. DSP Engine (`dsp/`)

- FFT calculation (CPU/GPU accelerated)
- Demodulation (AM, FM, SSB, CW, etc.)
- Audio filtering and resampling
- AGC per listener — Auto (tuned per mode) or transceiver-style Fast / Medium / Slow, chosen with the `agc_speed` command (`utils/audioprocessing.cpp`) — and noise reduction

5. Web Server (`server/`)

- WebSocket communication
- HTTP static file serving
- User session management
- Real-time data streaming

6. Audio Encoding (`audio/`)

- FLAC compression
- Opus compression
- Streaming optimization

Frontend (`frontend/`)

The web-based user interface built with Svelte and Vite.

1. Main App (`App.svelte`)

- Top-level component
- Layout structure
- Component orchestration
- **Decoders button row** — one button per decoder, on the main panel directly under the Modes; press to start the decoder and open its window, press again to stop. It replaced the former Bandwidth row. RADE-v2U/RADE-v1L/RADE-v1U live in `lib/ModesSelector.svelte` instead, beside the Modes and inside the Modes and Bands pop-ups.
- **Version line** — "PhantomSDR+ | v5.5.0 original from github.com/sv1btl/PhantomSDR-Plus", with the address as a link; the same line is on the /mobile page. The version number and the address live in `lib/version.js`; the line is an author attribution covered by `ADDITIONAL_TERMS.md`.

2. Waterfall Display (`waterfall.js` + `lib/`)

- Canvas-based spectrum and waterfall rendering and colormaps in `waterfall.js` (plain JS, not a component); the **Auto Adj.** brightness/contrast remap is `lib/wfAutoLevel.js`
- Interactive tuning through `lib/PassbandTuner.svelte`
- Band plan and marker overlays through `lib/FrequencyMarkers.svelte`

- The audio analyser is `lib/Spectrogram.svelte`, with the SSB audio report in `lib/SsbReport.svelte` and the AM/FM report in `lib/AmFmReport.svelte`; their RF measurements (occupied width, the SSB carrier and opposite sideband, AM depth and FM deviation, the AM/FM RF view) come from `measureOccupied`, `measureSsb`, `measureAmFm` and `getRfSlice` in `waterfall.js`, which while the analyser is open keeps a hidden fine-detail waterfall connection centred on the dial (`detailFollow`), so they work at any zoom; adapted from PhantomSDR-ORSAT by F1NSK
- The 3D spectrum is `lib/Waterfall3D.svelte` (WebGL2): while it is open, `waterfall.js` hands it every line with the waterfall's own levels (Min/Max and Auto Adj. applied), the zoom range and the colour map through `waterfall.view3d`; adapted from PhantomSDR-ORSAT by F1NSK

3. Controls (`App.svelte` + `lib/`)

- Frequency input/display — `lib/FrequencyInput.svelte`
- Number pad for typing a frequency — `lib/FreqKeypad.svelte`; the typed frequency is tuned with its band's zoom, Min/Max and mode (`tuneTypedFrequency()` in `App.svelte`), and one another receiver covers opens that receiver
- **Show All** strips of the other receivers — `lib/RxStrip.svelte`: one `/waterfall?rx=<id>` socket each, every frame decoded and drawn, in the other receiver's own palette and levels, newest line at the top; the receiver ranges and the links between receivers come from `lib/receivers.js`; with two or more bands in the other receiver's range, a band dropdown in its header (choice in `localStorage` `rxStripView:<id>`, click arrives with `&band=1`)
- Mode selection (AM/FM/SSB/CW) — `lib/ModesSelector.svelte`, band switching — `lib/BandSelector.svelte`
- EiBi window — `lib/SwSchedule.svelte`: searches `/eibi.json` with shortwave.live's criteria; a row tunes through the marker-click path (`handleFrequencyMarkerClick()` in `App.svelte`, so decoder modes start their decoder), and its green results and the red "now listening" marker are drawn by `lib/FrequencyMarkers.svelte`, and **Hide HFCC markers** hides the `markers.json` entries that `update-markers.sh` tags `"source": "hfcc"`; CW rows are tuned as the marker mode `MORSE`, so they open the CW decoder
- AGC/NR/NB and the remaining controls live in `App.svelte` itself; there is no separate `Controls.svelte`

3a. Scanner (`scanner.js`)

- Channel scanner: walks the receiver across a range and stops on the first channel carrying a signal. Plain JS, not a component — `App.svelte` supplies the VFO, the mode, the band plan and the tuning call, and receives the UI state back through one callback
- Threshold is dB over the band noise floor rather than an absolute level, reusing the floor `waterfall.js` already tracks (`snrNoiseDb`), so it needs no calibration of its own
- Two ways to cross the band: tune and listen to every channel, or screen the spectrum first and only tune to what it cannot rule out. The spectrum may skip a channel but never stop on one — every stop comes from a real dwell
- Range is either the band plan's band or exactly what the waterfall shows; auto-resume, a max-stay timeout and a lockout list live here too, persisted in `localStorage`

4. Audio System (`audio.js`)

- WebSocket audio stream
- FLAC/Opus decoding. FLAC goes through `flacLowLatency.js`, which decodes each packet the moment it arrives and corrects the stream's true rate (for example 12016.3 Hz) to 12000 Hz with a 64-tap resampler (2.7 ms of delay). The WASM decoder in `modules/` released audio in fixed 1024-sample chunks and held back about 160 ms; it is still used for Opus, for diversity sources, and for FLAC when the page is opened with `?flac=wasm`
- Audio playback control

- Playback buffer control (`playoutControl.js`): `ArrivalMeter` measures how irregularly packets arrive against the stream's own timeline, and the drift between the receiver's sample clock and the listener's sound card; `CushionController` keeps the smallest buffer seen in the last minute close to the buffer preset's margin, by cutting or repeating one pitch-matched period (`spliceOut` / `spliceIn`) of the audio going to the loudspeaker. Decoders and recordings get the untouched stream. Applies to the scheduled-buffer path that every listener on a plain-http station uses; `?drain=0` switches it off, `?diag=1` shows its figures
- Distributes raw PCM (taken before the browser's noise reduction and mute; the server AGC is held on Slow while a decoder is open) to the mode decoders
- AI noise reduction: `lib/rnnoise.js` loads the RNNoise WebAssembly module (`@jitsi/rnnoise-wasm`) on first use; `audio.js` runs it on voice modes only, after the decoder tap, so the decoders never hear it

4a. Mode decoders and their workers

Each of the heavy mode decoders runs in its own Web Worker so that decoding never blocks audio playback or the waterfall. They follow one common pattern — three files per decoder:

Decoder	Engine	Worker	Main-thread proxy
SSTV	`sstv.js`	`sstv.worker.js`	`sstvWorkerProxy.js`
HF FAX	`fax.js`	`fax.worker.js`	`faxWorkerProxy.js`
NAVTEX + FLDIGI (ALE 2G HAM, FSK/RTTY, PSK31/63/125, QPSK, Olivia, Contestia, PSK Browser, Olivia Browser, FST4W, MFSK, MSK144, DominoEX, THOR, THROB, Hellschreiber, MT63, Packet/APRS) + SERVICES (Maritime FSK / SITOR, Weather RTTY, ALE 2G)	`fsk.js`, `psk31.js`, `olivia.js`, `mfsk.js`, `ifk.js`, `throb.js`, `hell.js`, `mt63.js`, `fldigiFec.js`, `ax25.js`, `ale.js`, `fst4w.js`, `msk144.js`, `pskBrowser.js`, `oliviaBrowser.js`	`fsk.worker.js`	`fskWorkerProxy.js`
SERVICES (DSC, Time Signals, HF Selcall)	`dsc.js`, `timesig.js`	`services.worker.js`	`servicesWorkerProxy.js`
CW, CW Browser	`cwDecoder.js`, `cwBrowser.js`	`cw.worker.js`	`cwWorkerProxy.js`

- The **engine** is plain DSP code with no knowledge of workers, so it can also be run directly (unit tests, or the in-thread fallback).
- The **worker** holds one engine instance and forwards its events verbatim.
- The **proxy** mirrors the engine's method surface, so `audio.js` calls it exactly as it would call the decoder. It creates the worker lazily on first enable and falls back to running in-thread if workers are unavailable.

Two details are load-bearing: PCM is **copied** into a fresh buffer before being transferred to the worker (transferring a view into the audio accumulator would detach it and kill playback), and the worker's `init` message re-applies configuration to an already-running engine rather than assuming a fresh one.

`fsk.js` serves both NAVTEX and FSK/RTTY from one engine, selected per instance by a `role` field; each instance owns its own state, so the two can run independently.

The `fsk` role additionally hosts decoders that do not use its FSK chain at all. Selecting the `psk31`, `olivia`, `packet` or `aprs` variant, or one of the fldigi-family modems (`mfsk`, `dominoex`, `thor`, `throb`, `hell`, `mt63`, listed in `FLDIGI_MODEMS`), makes `fsk.js` hand the audio to `psk31.js`, `olivia.js`, `ax25.js`, `mfsk.js`,

`ifk.js`, `throb.js`, `hell.js` or `mt63.js` instead of its own discriminator chain, while still borrowing its configuration, worker and event plumbing — so `fsk.worker.js` and `fskWorkerProxy.js` need no knowledge of these modes, and the UI consumes the same `char`/`status`/`metrics` events throughout, plus a `line` event that carries one whole decoded packet and a `hell` event that carries one painted Hellschreiber column. The modems all take the same config — a sub-mode (`modemMode`), a squelch (`modemSquelch`) and a centre — and a squelch-only change retunes the running decoder instead of rebuilding it.

- `psk31.js` — BPSK31/63/125, QPSK31/63/125 (K=5 Viterbi): complex baseband, matched filter, differential detection and varicode, with a spectral coarse acquisition plus a fine AFC covering about ±25 Hz.
- `olivia.js` — Olivia/Contestia MFSK: a port of Pawel Jalocha's MFSK receiver from fldigi (`pj_mfsk.h`, GPL-3, as is this project), including the Walsh/Hadamard FEC and the blind synchronisation search over block phase and frequency offset.
- `mfsk.js` — MFSK16/32/64: a port of fldigi's IZ8BLY MFSK receiver (`mfsk.cxx`, GPL-3) — sliding DFT, depth-10 interleaver, K=7 Viterbi decoder, IZ8BLY varicode, symbol-timing and AFC loops. fldigi names its two Gray-code functions the other way round from the textbook; `mfsk.js` follows fldigi, because that is what is on the air.
- `ifk.js` — DominoEX and THOR: a port of fldigi's IFK+ receiver (`dominoex.cxx`, `thor.cxx`, GPL-3) — three sliding DFTs a third of a bin apart, symbol timing from the previous tone's peak, the DominoEX nibble varicode, and for THOR soft decisions, preamble detection, the interleaver and a K=7 or K=15 Viterbi decoder. Its squelch measures the tone against the other bins rather than fldigi's figure, which reads low on some clean signals.
- `throb.js` — THROB and THROBX: fldigi's `throb.cxx` — pulse-shaped tone references correlated against the decimated signal, the strongest tone pair looked up, envelope-peak symbol timing and AFC.
- `hell.js` — Hellschreiber (Feld, Slow, X5, X9, FSK Hell 245/105, Hell 80): fldigi's `feld.cxx` receive side; it emits 2 x 20-pixel columns that `App.svelte` paints on a canvas.
- `mt63.js` — MT63-500/1000/2000, short and long interleave: Pawel Jalocha's MT63 receiver as carried in fldigi (`mt63base.cxx`, `dsp.cxx`) — I/Q split filter, a 512-point FFT probed four times per symbol, the correlation synchroniser for time and frequency, differential demodulation and the Walsh-function FEC with its +/-8 carrier scan.
- `fldigiFec.js` — what these share: a soft-decision Viterbi decoder for any constraint length (Int32 metrics, so K=15 fits in 24 MB), moving averages and the resampler.
- `ax25.js` — Packet radio and APRS: an AFSK demodulator for 1200 Bd (Bell 202, received in FM) and 300 Bd HF packet (received in USB), with five slicers at different tone balances and a clock-recovery loop each, then NRZI/HDLC deframing, the CRC-16 frame check, AX.25 address and control-field parsing, and an APRS parser (positions including compressed and Mic-E, weather, messages, objects, items, status, telemetry). Only frames that pass the checksum and carry a well-formed address are reported. `packet` prints every frame like a TNC monitor; `aprs` prints APRS frames decoded into plain text.
- `ale.js` — ALE 2G (MIL-STD-188-141): a port of gr-ale (Milen Rangelov, GPL-3) as carried in KiwiSDR's ALE 2G extension, itself built on LinuxALE — a 64-point sliding DFT at 8 kHz, 17 decoders on 125 Hz offsets, triple-redundancy vote and Golay (24,12). Its panel is `lib/AlePanel.svelte`, its networks `lib/aleNets.js`.
- `fst4w.js` and `msk144.js` — FST4W and MSK144, ports of WSJT-X (GPL-3): LDPC (240,74) with OSD and CRC-24, and LDPC (128,90) with CRC-13. FST4W records a whole UTC period on a narrow complex baseband and FFTs it with `lib/fftMixed.js`; both unpack their messages with `lib/pack77.js`.

- `pskBrowser.js` — PSK Browser: finds every BPSK31 signal between 300 and 2700 Hz once a second and runs one `psk31.js` demodulator per signal.
- `lib/aprsMap.js` — the APRS map: keeps every station with a decoded position (objects and items under their own name, killed ones removed, a track for moving ones) and draws them with Leaflet on OpenStreetMap tiles, with the receiver's own position taken from `siteGridSquare`. Leaflet is imported only when the map is first shown, so it costs nothing to listeners who never open APRS. Text received over the air reaches the page only through `textContent` or escaping. `App.svelte` places the map under the APRS text pane and opens and closes it with the pane.
- `broadcastSchedules.js` — the UTC timetables the FAX, NAVTEX and RTTY decoders offer as presets, taken from the NOAA/NWS marine radiofacsimile schedules and the published NAVTEX station lists

4b. Receive Diversity (`diversity.js`)

- Combines the local receiver with a second one elsewhere and follows whichever site currently has the better signal. Plain JS, not a component — it sits on one seam in `audio.js`, which hands it the local PCM and plays back what it returns
- **Selection, not summing.** Two sites hear the same transmission over different ionospheric paths, so their waveforms have unrelated phase; adding them sounds comb-filtered. Coherent combining would need a common clock, which two receivers over the internet cannot share. The two streams are only ever mixed during a 30 ms crossfade
- Alignment correlates the two **audio envelopes** (log power at 100 Hz), never the waveforms — the envelope survives both the path and any codec. A lock is only trusted once a second, independent search agrees with it, which rejects the confident-but-wrong delay two antiphase-fading sites would otherwise produce
- The remote is **rate-locked** to the local stream first. Two receivers are two clocks and two decimation chains, so their audio arrives up to 2% apart even when both declare 12 kHz — 240 samples a second of creep, which no correlation can hold. The ratio is measured from how many samples each side actually delivers and applied by a resampler that carries its fractional phase across blocks, so an arbitrary ratio holds indefinitely
- Site choice uses a percentile SNR measured on **content-aligned** samples, with hysteresis, a dwell timer and a fast escape hatch for a collapsed incumbent. Levels are matched noise-to-noise, so a switch does not change the background hiss
- Decoders keep the **local** stream: FT8, JS8, WSPR and RADE integrate coherently across a slot, and a mid-slot switch is a phase discontinuity that can cost the decode

4c. Diversity sources (`remoteSource.js`, `kiwiSource.js`, `uberSource.js`, `webSdrSource.js`)

- One contract — `onPcm` / `onState` / `tune` / `canReceive` — so `diversity.js` never learns what is on the other end. Adding a receiver type is one new file
- `remoteSource.js` — another PhantomSDR-Plus over `/audio` (cbor + FLAC), reusing `createDecoder()` from `lib/wrappers.js`
- `kiwiSource.js` — a KiwiSDR: `SND` frames, big-endian PCM, with the 10-byte GPS timestamp a stereo packet inserts before the audio
- `uberSource.js` — an UberSDR over its native `/ws`: Opus in a 21-byte header, retuning over the open socket. The session id must be registered with `POST /connection` first and must be a UUID
- `webSdrSource.js` — a WebSDR, through `websdr_relay.py` on this server: the browser cannot connect directly, because WebSDR checks the `Origin` header and no script may change it. Tuning is a text frame on the same socket; band coverage comes from the relay
- `webSdrCodec.js` — the WebSDR audio format: a tagged byte stream whose compressed blocks drive a 20-tap leaky-LMS predictor. Ported from the WebSDR client and verified sample-for-sample against it
- `diversityList.js` — the saved-receiver list and the address rules for the four types, shared by the desktop panel and the mobile page so one format serves both. Also the transfer — a compact blob,

its QR code, and a parser that accepts every shape the two pages have ever written — and `browseUrl()`, which walks a dialled address back into one a browser can open

- `lib/DiversityPanel.svelte` — the UI: endpoint, source type, SNR trim and live status, plus the saved receivers — named, reorderable, and exported to and imported from a JSON file, kept per source type in `localStorage`. Everything is edited inside the panel: `prompt()` and `confirm()` block the main thread, which is where audio frames are pushed into the playback worklet. A ▦ **QR** button draws the list as a scannable code, because `localStorage` is per browser and a phone starts empty. `mobile/Mobile.svelte` carries the same feature in a **Div** tab, in that page's own plain CSS. See Receive Diversity

4d. Mode identifier (`modeId.js`, `modePriors.js`)

- Answers "what am I listening to?". It reads the same raw PCM tap as the decoders and ranks the likely modes, so the operator can pick the right decoder instead of trying all ten. It never decodes anything: it measures physical properties of the signal and scores them against a table of known modes
- Occupied bandwidth is the contiguous −15 dB width around the peak, deliberately not a 99 %-power figure: keying splatter puts long tails on the power integral and made every narrow mode read several times too wide. Symbol rate comes from the **instantaneous frequency** rather than tone energies, because no integration length can both resolve a 170 Hz shift and a 100 Bd symbol. Keying rate comes from the on/off envelope, and doubles as a CW speed readout
- `modePriors.js` adds the one clue the audio cannot carry — where you are tuned. A 100 Bd / 170 Hz signal on 518 kHz is NAVTEX; the identical signal on 14.070 MHz is not. It only reweights what the signal already supported, and never invents a candidate
- FT8, JS8 and FT2 are reported as one group on purpose: they are not separable on bandwidth and tone spacing alone, and pretending otherwise would be a confident wrong answer
- Below roughly 10 dB SNR it stays quiet rather than guessing
- Runs in its own Web Worker (`modeId.worker.js` + `modeIdWorkerProxy.js`), following the same engine/worker/proxy pattern as the decoders above; the result is the chip in `lib/ModeIdChip.svelte`

4e. Connection refusals (`refused.js`, `clientVersion.js`)

- `refused.js` is the shared vocabulary for a connection the server turns away: close code **4003** (over a per-IP limit, or the page is older than `[server] min_client_version`) and **4001** (a sysop kick). Both are final. `audio.js`, `waterfall.js` and `events.js` all import `isRefusal()` from it, because all three open a socket that can be refused and all three must settle their init promise when it is — otherwise the page waits forever on a connection that is never coming
- Nothing retries. A dropped `/audio` socket ends the session by design: `/waterfall` and `/events` never came back with a reconnect, so a retried session was live audio attached to a frozen waterfall, and against a rate limit each retry would extend the very refusal it was working around. The desktop page explains a refusal that happens at page load and simply stops when one happens mid-session; `/mobile` shows one line asking the listener to reload
- `clientVersion.js` holds a single integer, `CLIENT_VERSION`, which the page appends to its audio socket as `/audio?v=N`. The server refuses anything below `[server] min_client_version`, which is how a station forces tabs still running an older build to reload — the only lever there is, since the server cannot reach JavaScript already running in a browser. Bump it when a frontend change must not keep being ignored by open tabs
- Full reference: Connection Limits

4f. Chat replies (`lib/chatReply.js`)

- A reply is an ordinary chat message that starts with a marker naming the message it answers, `[RE:<timestamp>|<name>]`; the server stores and relays it like any other line and needs no change

- `chatReply.js` builds and reads that marker and groups replies under their original, one level deep. `App.svelte` and `mobile/Mobile.svelte` both use it, so the two pages thread the chat the same way
- When the original has already left the server's 20-message history, the reply stays where it arrived and shows only the name and time it answers

4g. Listener names and privacy (`lib/identity.js`)

- The name box next to **Open Additional Info** keeps the name or callsign in `localStorage` (`chatusername`) and sends it on `/audio` as `{"cmd":"identity","name":…}`; `audio.js` sends it again whenever its socket opens. A link to a receiver on another address carries it as `?idname=`, read once and removed from the address bar
- The server cleans it (`clean_listener_name` in `utils.cpp`): at most 10 characters and no spaces as on a WebSDR, at least one letter or digit; the chat's reserved names and the `anonymous1234` placeholders count as none. `/users` gives it as `name`; the waterfall labels show it followed by the city
- The public `/users`, `users.json` and the `/logs/users_*.jsonl` that `stats.html` reads carry no IP address: the log copy has an opaque `visitor` code instead (keyed hash, key in `logs/.visitor_salt`); the files on disk keep the addresses. The admin panel reads `/~~users`, answered only to a loopback peer without `X-Forwarded-For`
- The running decoder goes the same way: `App.svelte` sends `{"cmd":"decoder","name":"FT8"}` (empty when none) whenever it changes, and `audio.js` sends it again on a new socket. The server keeps a short label (letters, digits, space, `-+/`, at most 12) and gives it as `decoder` in `/users` and in each `users_*.jsonl` line; a change is logged like a retune, so `stats.html` (Decoder Popularity) and the admin Visitors page can count it
- The admin panel's 8/16/24-hour bans live in `admin_bans.json` (never distributed) and are re-sent to every receiver once a minute; `/~~unban` lifts one

5. State Management (`stores/`)

- Reactive data stores
- Shared application state
- Event handling

Key Features

- Real-time audio processing
- Digital mode decoding (FT8, RTTY, etc.)
- Audio filtering
- Spectrum analysis

Frequency Lists (`frequencylist/`)

The on-waterfall frequency markers. `mymarkers.json` is the list the receiver actually shows; the rest is the raw material `update-markers.sh` turns into it, refreshed from the online schedules. `README.md` in that directory explains the update in all seven languages.

```
frequencylist/
├── mymarkers.json            # the markers the receiver shows — hand-edited, survives an update
├── shortwavestations.json    # shortwave broadcasters, generated by generate-current-shortwave.py
├── 0.TXT                     # the A26 global HF schedule, unpacked and parsed by update-
markers.sh
├── a26allx2.zip              # source archive downloaded by update-markers.sh
├── admin.txt · antenna.txt · broadcas.txt · fmorg.txt · language.txt · site.txt
│                             # supporting lists from the same source (sites, languages, admins,
antennas)
├── generate-current-shortwave.py
├── update-eibi.sh            # fetches the EiBi schedule for the EiBi window, at most once a day
├── generate-eibi.py          # turns it into frontend/public/eibi.json and frontend/dist/eibi.json
├── update-markers.sh         # refreshes everything above from the online schedules
└── README.md
```

The EiBi window (the **EiBi** button) uses a second, separate list. `update-markers.sh` first runs `update-eibi.sh`, which checks the EiBi schedule (http://www.eibispace.de/dx/) at most once a day and downloads it only when EiBi has published a new one, switching to the next season's list (summer A, winter B) by itself; `generate-eibi.py` turns it into `eibi.json` for the page and works out each station's receiver mode. Run `./frequencylist/update-eibi.sh --force` to fetch it at once. A failed download never stops the markers update. `update.sh` updates both scripts and asks first if you edited them. `update-markers.sh` also tags every HFCC entry it writes to `markers.json` with `"source": "hfcc"`; the EiBi window's **Hide HFCC markers** hides exactly those.

Format (`mymarkers.json`, `shortwavestations.json`)

```
[
    { "frequency": 77500,   "name": "DCF77", "mode": "CW" },
    { "frequency": 2485000, "name": "Vanuatu Broadcasting", "mode": "AM" }
]
```

`mode` is a receiver mode (`USB`, `LSB`, `AM`, `FM`, `CW`, …) or the name of a digital mode — `FT8`, `FT4`, `FT2`, `JS8`, `WSPR`, `SSTV`, `NAVTEX`, `FAX`, `RTTY`, `RTTY-WX`, `SITOR`, `PSK31`, `PSK63`, `PSK125`, `BPSK31/63/125`, `QPSK31/63/125`, `OLIVIA`, `CONTESTIA`, `MFSK`, `MFSK16`, `MFSK32`, `MFSK64`, `DOMINOEX`, `THOR`, `THROB`, `HELL`, `FELDHELL`, `MT63`, `PACKET`, `APRS`, `RADEU`, `RADEL`, `RADE` or `FREEDV` — in which case a click on the marker starts that decoder (details).

Configuration Files

Server Configuration (`.toml` files)

Structure of configuration files:

```
[server]
# Web server settings
port = 9002
html_root = "frontend/dist/"
threads = 2
otherusers = 1
iq = true

[limits]
# Per-IP connection limits — all default to off or to a sensible value, so a
# config without them behaves as it always did. See CONNECTION_LIMITS.md.
per_ip = 3              # simultaneous listeners from one address
per_ip_rate = 40        # new sockets per minute from one address

[websdr]
# Online registration
register_online = true
name = "WebSDR Name"
antenna = "Antenna Type"
grid_locator = "AB12cd"
hostname = "domain.com"

[input]
# SDR input settings
sps = 2048000           # Sample rate
fft_size = 131072       # FFT size
frequency = 145000000   # Base frequency
signal = "iq"           # Signal type: "iq" or "real"
audio_sps = 12000       # Audio sample rate
audio_compression = "opus"  # "flac" or "opus"
accelerator = "opencl"  # "none", "cuda", "opencl"

[input.driver]
# Driver settings
name = "stdin"
format = "u8"           # Sample format

[input.defaults]
# User interface defaults
frequency = 145500000
modulation = "FM"
```

Site Information (`site_information.json`)

```
{
  "siteSysop": "Operator Callsign",
  "siteSysopEmailAddress": "email@example.com",
  "siteGridSquare": "AB12cd",
  "siteCity": "City, Country",
  "siteInformation": "https://github.com/sv1btl/PhantomSDR-Plus",
  "siteHardware": "Hardware specs",
  "siteSoftware": "Software version",
  "siteReceiver": "SDR model",
  "siteAntenna": "Antenna description",
  "siteNote": "Additional notes",
  "siteIP": "http://domain.com:9002",
  "siteSDRBaseFrequency": 0,
  "siteSDRBandwidth": 2048000,
  "siteRegion": 1,
  "siteChatEnabled": true
}
```

Build System

Meson Build Configuration

`meson.build` (Root)

Defines:

- Project metadata
- Dependencies
- Compiler options
- Source file lists
- Build targets

`meson_options.txt`

Available options:

```
option('opencl', type: 'boolean', value: false, description: 'Enable OpenCL support')
option('cuda', type: 'boolean', value: false, description: 'Enable CUDA support')
option('optimization', type: 'string', value: '3', description: 'Optimization level')
```

File Dependencies

Backend Build Dependencies

```
spectrumserver binary depends on:
├── C++ source files (src/**/*.cpp)
├── External libraries:
│   ├── FFTW3
│   ├── WebSocket++
│   ├── FLAC
│   ├── Opus
│   ├── Liquid-DSP
│   ├── Boost
│   ├── zlib
│   ├── zstd
│   └── OpenCL/CUDA (optional)
└── Subproject headers:
    ├── nlohmann/json
    └── toml11
```

Frontend Build Dependencies

```
frontend/dist/ depends on:
├── Source files (frontend/src/**)
├── npm packages (node_modules/):
│   ├── Svelte
│   ├── Vite
│   ├── @wasm-audio-decoders/opus-ml
│   └── ...
└── Static assets (frontend/public/)
```

Data Flow

Server Operation Flow

```
1. SDR Hardware → rtl_sdr/hackrf_transfer/etc.
                ↓
2. Sample Stream → stdin → spectrumserver
                ↓
3. spectrumserver:
   - FFT calculation (waterfall)
   - Demodulation (audio)
   - Compression (FLAC/Opus)
                ↓
4. WebSocket → Browser Client
                ↓
5. Browser:
   - Render waterfall
   - Decode and play audio
   - Display controls
```

User Interaction Flow

```
1. User clicks on waterfall
                ↓
2. JavaScript sends frequency change request
                ↓
3. WebSocket → spectrumserver
                ↓
4. spectrumserver:
   - Updates demodulator frequency
   - Sends new audio stream
                ↓
5. Browser receives and plays new audio
```

File Modification Guide

When you modify backend code (`src/**`):

```
cd PhantomSDR-Plus
meson compile -C build
# Server restart required
```

When you modify frontend code (`frontend/src/**`):

```
cd PhantomSDR-Plus/frontend
npm run build
cd ..
# Server restart required (for static files)
```

When you modify configuration (`.toml`, `.json`):

```
# Restart server
./stop-websdr.sh
./start-rtl.sh  # (or appropriate start script)
```

When you modify markers (`markers.json`):

```
# Reload page in browser
# No server restart needed
```

Important Paths

Runtime Paths

- **Configuration**: `./config-*.toml`
- **HTML Root**: `./frontend/dist/`
- **Markers**: `./markers.json`
- **Chat History**: `./chat_history.txt`
- **FFTW Wisdom**: `./fftw_wisdom`, `./phantom_fftw_wisdom`

Build Paths

- **Binary Output**: `./build/spectrumserver`
- **Frontend Output**: `./frontend/dist/`
- **Node Modules**: `./frontend/node_modules/`

Source Paths

- **Backend Source**: `./src/`
- **Frontend Source**: `./frontend/src/`
- **DSP Libraries**: `./jsdsp/`

Common File Operations

Adding a New SDR Configuration

1. Copy existing config: `cp config-rtl.toml config-mydevice.toml`
2. Edit parameters: `nano config-mydevice.toml`
3. Create start script: `cp start-rtl.sh start-mydevice.sh`
4. Edit start script: `nano start-mydevice.sh` — change only the **RECEIVER CONFIGURATION** block at the top (`RX_LABEL`, `RX_COMM` = the receiver's process name, `RX_ARGS`, `CONFIG`, `FIFO`, and the `prestart` hook if the device needs one). The launcher/watchdog/logging logic below it is generic and needs no changes.
5. Make executable: `chmod +x start-mydevice.sh`

> `stop-websdr.sh` already stops any `start-*.sh --watchdog`; if your receiver uses a process name other than `rx888_stream`/`rx_sdr`/`rtl_sdr`/`hackrf_transfer`/`cf32_to_real`, add a `killall -9 <name>` line for it there too.

Customizing the Frontend

1. Modify source: `nano frontend/src/App.svelte`
2. Rebuild: `cd frontend && npm run build && cd ..`
3. Restart server: `./stop-websdr.sh && ./start-rtl.sh`

Adding Custom Markers

1. Edit markers file: `nano markers.json`
2. Format:
   `json { "markers": [ { "frequency": 145500000, "label": "2m Calling", "mode": "FM" } ] }`
3. Reload browser (no server restart needed)

Version Control

Files to Track in Git

- Source code (`src/`, `frontend/src/`, `jsdsp/`)
- Configuration examples (`config.example.*.toml`)
- Build system (`meson.build`, `meson_options.txt`)
- Documentation (`*.md`, `docs/`)
- Scripts (`*.sh`)

Publishing a release (maintainer)

Before the files are uploaded, refresh the fingerprint list, so that `update.sh` on every station recognises the versions it has as published ones:

```
# 1. copy the changed files to the upload copy, then:
bash update.sh --make-known /path/to/the/upload/copy
# 2. commit update-known.txt and copy it to the upload copy as well
```

The list only grows: entries from earlier releases are kept, so a public copy published once stays recognisable in every later release.

Files to Ignore (`.gitignore`)

- Build output (`build/`, `frontend/dist/`)
- Dependencies (`frontend/node_modules/`)
- User data (`chat_history.txt`)
- Personal configs (`config-rtl.toml` if customized)
- Binary data (`*.o`, `*.so`)

This structure documentation should help you navigate and understand the PhantomSDR-Plus codebase.

For setup instructions, see INSTALLATION.md. For usage information, see USER_GUIDE.md. For the KiwiSDR client bridge, see Aether_config.md.

73 de SV1BTL, 9A7AOF, F1NSK & SV2AMK

CHAPTER 18

Appendix

Appendix.md

Appendix — Command Reference

Every command a sysop uses to run a PhantomSDR-Plus station, grouped by task, each with a one-line explanation. Run them from inside the `PhantomSDR-Plus` directory unless a line says otherwise. The step-by-step manual installation (dependencies, Node.js, OpenCL, building by hand) is not repeated here — see the Installation Guide. The last column links to the section that explains the command; in the PDF the page number follows the link.

Installing and setting up the station

Command	What it does	See
`git clone https://github.com/sv1btl/PhantomSDR-Plus`	Downloads the project.	README · p. 11
`bash install.sh`	Full installer for Ubuntu/Debian: asks the station questions, installs, builds.	Installation Guide · p. 25
`bash install_fedora.sh`	The same installer for Fedora.	Installation Guide · p. 25
`bash install_arch.sh`	The same installer for Arch Linux.	Installation Guide · p. 25
`bash install_opensuse.sh`	The same installer for openSUSE.	Installation Guide · p. 25
`PHANTOM_NONINTERACTIVE=1 PHANTOM_SDR=1 ./install.sh`	Unattended install (here for an RX888 MkII); every question takes its default.	Installation Guide · p. 29
`chmod +x *.sh`	Makes the scripts executable again (for example after unpacking a zip).	Installation Guide · p. 52
`bash configure-station.sh`	Station wizard: asks the questions, saves `station.conf`, writes every config file from it.	Installation Guide · p. 27
`bash configure-station.sh --ask`	Asks and saves `station.conf` only.	Installation Guide · p. 27
`bash configure-station.sh --apply`	Rewrites the config files from `station.conf` without asking.	Installation Guide · p. 27
`bash configure-station.sh --show`	Prints the current answers.	Installation Guide · p. 27
`./add-receiver.sh`	Adds one more receiver (for example an RTL-SDR for 2 m) next to the main one.	Admin Panel Setup · p. 77

Receiver drivers

Command	What it does	See
`./setup-rx888-udev.sh`	Installs the RX888 udev rules so it runs without sudo.	Installation Guide · p. 39
`./setup-rtlsdr.sh`	Installs the RTL-SDR driver; asks whether the stick is a Blog V4.	Installation Guide · p. 40
`RTL_V4=y ./setup-rtlsdr.sh`	Installs the RTL-SDR Blog V4 driver without asking.	Installation Guide · p. 40
`./setup-rsp1a.sh`	Installs the open driver chain for the SDRplay RSP1A.	Installation Guide · p. 37
`./setup-airspyhf.sh`	Installs the Airspy HF+ driver chain.	Installation Guide · p. 37
`./setup-fobos.sh`	Installs the RigExpert Fobos driver chain.	Installation Guide · p. 37
`./setup-hackrf.sh`	Installs the HackRF package and its udev rule.	Installation Guide · p. 40
`./setup-drm.sh`	Builds the DRM decoder (ubersdr-drm with fdk-aac 2.0.3) into `drm/bin/`.	Decoders · p. 165
`rtl_test`	Checks that an RTL-SDR stick is seen.	Installation Guide · p. 37
`SoapySDRUtil --find="driver=soapyMiri"`	Checks that the RSP1A is seen (`driver=fobos`, `driver=airspyhf` for the others).	Installation Guide · p. 37
`SoapySDRUtil --info`	Lists the SoapySDR drivers that are installed.	Installation Guide · p. 49

Starting and stopping the receiver

Command	What it does	See
`./start-rx888mk2.sh`	Starts (or restarts) the RX888 MkII receiver with its watchdog and shows the log until it is up.	README · p. 12
`./start-rtl.sh`	The same for an RTL-SDR.	Installation Guide · p. 40
`./start-rsp1a.sh`	The same for an SDRplay RSP1A.	README · p. 12
`./start-airspyhf.sh`	The same for an Airspy HF+.	README · p. 12
`./start-hackrf.sh`	The same for a HackRF One.	README · p. 12
`./start-fobos.sh`	The same for a Fobos, RF input (25–6000 MHz).	README · p. 12
`./start-fobos-hf.sh`	The same for a Fobos, HF direct sampling (0–25 MHz).	README · p. 12
`./start-all.sh`	Starts every receiver listed in `receivers.toml`.	Several Receivers · p. 109
`./start-rx888mk2.sh -q`	Any start script with `-q`: two lines of output instead of the live log.	README · p. 12
`INSTANCE=vhf ./start-rtl.sh`	Starts a second receiver set up in `instances/vhf/`.	Several Receivers · p. 107
`SPECTRUM_CORES=0-3 ./start-rx888mk2.sh`	Pins the server to these CPU cores (`none` = no pinning).	README · p. 12
`RADE_ENABLED=0 ./start-rx888mk2.sh`	Starts without the RADE sidecar.	RADE README · p. 172
`DRM_ENABLED=0 ./start-rx888mk2.sh`	Starts without the DRM helper.	Decoders · p. 165
`./stop-websdr.sh`	Stops every receiver of this installation and its watchdog.	Installation Guide · p. 41
`./stop-websdr.sh main`	Stops only the main receiver.	Several Receivers · p. 109
`./stop-websdr.sh vhf`	Stops only the receiver started with `INSTANCE=vhf`.	Several Receivers · p. 109
`tail -f logwebsdr.txt`	Follows the receiver's live log.	Installation Guide · p. 40

Starting at boot

Command	What it does	See
`bash setup-autostart.sh`	Starts the receiver at boot (the start script named in `station.conf`).	Installation Guide · p. 41
`bash setup-autostart.sh start-rtl.sh`	The same, for a particular start script.	Installation Guide · p. 41
`bash setup-autostart.sh start-all.sh`	The same, for every receiver in `receivers.toml`.	Installation Guide · p. 41
`bash setup-autostart.sh --status`	Shows whether it is installed, and for which script.	Installation Guide · p. 41
`bash setup-autostart.sh --remove`	Stops starting at boot (the receiver keeps running now).	Installation Guide · p. 41

Building and updating

Command	What it does	See
`./recompile.sh`	Rebuilds the server and/or the web page; asks what to build.	Installation Guide · p. 52
`./recompile.sh --backend`	Rebuilds the server only.	Installation Guide · p. 52
`./recompile.sh --frontend`	Rebuilds the web page only (desktop and /mobile).	Admin Panel Setup · p. 80
`./recompile.sh --both`	Rebuilds both.	Installation Guide · p. 52
`cd frontend && ./build-all.sh`	Builds the desktop page and /mobile.	Editing Variants · p. 198
`cd frontend && ./build-default.sh`	Builds just the desktop page.	Editing Variants · p. 198
`cd frontend && ./build-mobile.sh`	Builds just /mobile.	Editing Variants · p. 198
`bash update.sh`	Reports what a new version would change, then asks whether to update.	Installation Guide · p. 50
`./update.sh --check`	Reports only, never asks (for cron; exit code 10 = an update is waiting).	Installation Guide · p. 51
`./update.sh --apply`	Updates, asking about the files you edited yourself.	Installation Guide · p. 51
`./update.sh --apply --yes`	Updates unattended; every file you edited is kept.	Installation Guide · p. 51
`./update.sh --apply --prune`	Also offers to delete files that were removed upstream.	Installation Guide · p. 50
`./update.sh --from FILE`	Takes the new version from a `.zip`/`.tar.gz` or a folder — no network.	Installation Guide · p. 51
`./update.sh --ref v5.5.0`	Updates to a tag, branch or commit instead of the current tree.	Installation Guide · p. 51
`./update.sh --list-excludes`	Prints the files the updater never touches.	Installation Guide · p. 51
`./update.sh --verbose`	Lists every file, not only the first 40.	Installation Guide · p. 51
`./update.sh --restore LAST`	Puts back the files the last update overwrote.	Installation Guide · p. 51
`./update.sh --restore 20260923-164530`	Puts back the files from that particular run.	Installation Guide · p. 51
`./frequencylist/update-eibi.sh --force`	Fetches the EiBi schedule for the EiBi window now instead of waiting for the daily check.	Project Structure · p. 290

Look of the page

Command	What it does	See
`./waterfall.sh`	Changes the default minimum waterfall level; shows the current value and asks.	README · p. 13
`./waterfall.sh -v -15`	Sets it to −15 dB without asking (`-y` also skips the confirmations).	README · p. 13
`./waterfall.sh -s`	Shows the current values only.	README · p. 13
`./smeter_theme.sh`	Chooses the default S-meter theme from a menu.	Editing Variants · p. 195
`./smeter_theme.sh vintage`	Sets that theme straight away, then offers the rebuild.	Editing Variants · p. 195
`./smeter_theme.sh dark --build`	Sets it and rebuilds without asking (`--no-build` skips the rebuild).	Editing Variants · p. 195
`./smeter_theme.sh amber --no-reset`	Applies to new visitors only; existing users keep theirs.	Editing Variants · p. 195

Admin panel and proxy

Command	What it does	See
`./setup_admin.sh`	Interactive setup of the admin panel: ports, scripts, thermal guard, systemd units.	Admin Panel Setup · p. 58
`./setup_admin.sh --sudoers`	Installs only the sudoers rule so the two units restart without a password.	Admin Panel Setup · p. 58
`./setup_admin.sh --proxy-only`	Installs only the proxy on the public port, for a station without the panel.	Admin Panel Setup · p. 58
`./manage_admin.sh start`	Starts the panel and the proxy (when they are not under systemd).	Admin Panel Setup · p. 59
`./manage_admin.sh stop`	Stops both.	Admin Panel Setup · p. 59
`./manage_admin.sh status`	Shows whether they run, with their PIDs.	Admin Panel Setup · p. 59
`sudo systemctl restart phantomsdr-admin phantomsdr-proxy`	Restarts panel and proxy under systemd (the receiver keeps running).	Admin Panel Setup · p. 60
`sudo systemctl enable --now phantomsdr-admin`	Starts the panel now and at every boot (`phantomsdr-proxy` likewise).	Admin Panel Setup · p. 59
`sudo systemctl disable --now phantomsdr-admin`	Stops the panel and no longer starts it at boot.	Admin Panel Setup · p. 59
`systemctl status phantomsdr-admin`	Shows the panel's state (no sudo needed).	Admin Panel Setup · p. 59
`sudo journalctl -u phantomsdr-admin -f`	Follows the panel's log.	Admin Panel Setup · p. 59
`AUTORUN_CORES=2-3 ./manage_admin.sh start`	Pins the autorun decoder daemon to these CPU cores.	Admin Panel Setup · p. 74
`sudo cp logrotate/phantomsdr /etc/logrotate.d/phantomsdr`	Installs log rotation for the panel, proxy and autorun logs.	Admin Panel Setup · p. 65
`sudo logrotate -d /etc/logrotate.d/phantomsdr`	Dry run of the log rotation.	Admin Panel Setup · p. 65

Thermal guard

Command	What it does	See
`python3 thermal_guard.py --once`	Prints the sensor, trip point and thresholds; takes no action.	Thermal Guard · p. 87
`python3 thermal_guard.py --mode log`	Watches the temperature live, logging only (Ctrl-C to quit).	Thermal Guard · p. 91
`python3 thermal_guard.py --mode stop+restart`	Runs it armed: stops the receiver when too hot, restarts it when cool.	Thermal Guard · p. 91
`python3 thermal_guard.py --config FILE`	Uses another configuration file.	Thermal Guard · p. 91
`./setup-cpufreq-perms.sh`	Lets the guard lower the CPU frequency without running as root.	Thermal Guard · p. 92
`sudo systemctl enable --now thermal-guard`	Runs the guard as its own service (only when the admin panel is not used).	Thermal Guard · p. 91

Connections, firewall and HTTPS

Command	What it does	See
`./setup-firewall.sh --show`	Prints the kernel connection-limit rules; changes nothing.	Connection Limits · p. 100
`sudo ./setup-firewall.sh --check`	Checks the rules against your kernel.	Connection Limits · p. 100
`sudo ./setup-firewall.sh --apply`	Loads the rules, with a 60-second automatic rollback.	Connection Limits · p. 100
`sudo ./setup-firewall.sh --persist`	Reloads the rules at every boot.	Connection Limits · p. 100
`sudo ./setup-firewall.sh --status`	Shows the per-rule packet counters.	Connection Limits · p. 100
`sudo ./setup-firewall.sh --remove`	Removes all of the rules.	Connection Limits · p. 100
`sudo ufw allow 9000/tcp`	Opens the public port in the ufw firewall.	Installation Guide · p. 48
`bash setup-https.sh`	Turns on HTTPS with a free Let's Encrypt certificate; HTTP keeps working.	Secure Access (HTTPS) · p. 112
`bash setup-https.sh --lan`	HTTPS inside the local network only (browsers warn about the certificate).	Secure Access (HTTPS) · p. 114
`bash setup-https.sh --status`	Shows whether HTTPS is on and the certificate answers.	Secure Access (HTTPS) · p. 113
`bash setup-https.sh --remove`	Turns HTTPS off; plain HTTP stays as it is.	Secure Access (HTTPS) · p. 113

Optional services

Command	What it does	See
`./install-stats-server.sh`	Installs the statistics server (CPU, temperature, users).	Statistics Server · p. 117
`sudo systemctl restart sdr-stats.service`	Restarts it (`start`, `stop`, `status` likewise).	Statistics Server · p. 122
`sudo journalctl -u sdr-stats.service -f`	Follows its log.	Statistics Server · p. 122
`curl http://localhost:3001/api/system-stats`	Checks that it answers.	Statistics Server · p. 121
`./install_rade.sh`	Installs the RADE / FreeDV decoder sidecar (`install_rade_ubuntu22.sh` on Ubuntu 22.04).	RADE README · p. 167
`cd ~/rade_c && git pull && cmake --build build`	Updates rade_c, the RADE V2 (beta) decoder; new RADE2 connections use the new build.	RADE README · p. 169
`./rade.sh start`	Starts the RADE sidecar with its watchdog.	RADE README · p. 172
`./rade.sh stop`	Stops it, its watchdog and its decoder processes.	RADE README · p. 172
`./rade.sh restart`	Stops and starts it cleanly.	RADE README · p. 172
`./rade.sh status`	Shows whether it runs.	RADE README · p. 172
`RADE_CORES_PER_CLIENT=3 ./rade.sh restart`	Restarts it giving each listener three CPU cores.	RADE README · p. 182
`tail -f rade.log`	Follows the RADE log.	RADE README · p. 172
`python3 rade_loadtest.py`	Measures how many RADE listeners this machine can carry.	RADE README · p. 180
`./kiwi_install.sh`	Installs the KiwiSDR client emulation (for Kiwi apps such as AetherSDR).	KiwiSDR Client Emulation · p. 233
`./setup_websdr_relay.sh`	Installs the relay that lets receive diversity use a WebSDR as the second receiver.	Receive Diversity · p. 241

Rig control (TCI bridge)

Command	What it does	See
`sudo usermod -aG dialout $USER`	Gives your user access to the rig's serial port (log in again afterwards).	Rig Control · p. 253
`rigctl -m 3073 -r /dev/ttyUSB0 -s 115200 f`	Reads the rig's frequency through Hamlib — checks the CAT link (model, port, speed for your rig).	Rig Control · p. 255
`cd tci-bridge && node tci-rigctld.mjs`	Runs the bridge against a `rigctld` already running.	Rig Control · p. 257
`node tci-rigctld.mjs --rigctl rigctl -m 3073 -r /dev/ttyUSB0 -s 115200`	Runs the bridge driving the rig through `rigctl` directly (the way to use on Windows).	Rig Control · p. 257

Checking and troubleshooting

Command	What it does	See
`ss -tlnp`	Lists the listening ports and the programs behind them.	Admin Panel Setup · p. 64
`sudo lsof -i :9002`	Shows which program holds a port.	Installation Guide · p. 47
`pkill -f admin_server.py`	Ends a panel that was started by hand (`proxy.py` likewise).	Admin Panel Setup · p. 64
`sudo fuser -k 3000/tcp`	Frees a port held by a stuck program.	Admin Panel Setup · p. 64
`lsusb`	Lists the USB devices — is the receiver seen?	Installation Guide · p. 48
`sudo timedatectl set-ntp true`	Keeps the clock in sync — the FT8/FT4/WSPR decoders need it.	Installation Guide · p. 46

Internal scripts (not run by hand)

Script	What it does	See
`start-*.sh --watchdog`	The watchdog a start script launches for itself.	Installation Guide · p. 42
`setup-sdr-common.sh`	Shared helpers sourced by the `setup-*.sh` driver installers.	Project Structure · p. 265
`go.sh`, `xgo.sh`, `check-go.sh`, `kill.sh`, `_relaunch.sh`	The older launcher chain, kept for existing stations; the `start-*.sh` scripts replace it.	Project Structure · p. 265